

内 容 简 介

本书根据作者利用多媒体设备从事计算方法课教学的经验编写而成,新增了常用函数值计算和函数增量计算等重要内容,体系更趋合理。书中对所论及的各种典型数学问题,通过分析问题的特点,寻找求解的思路,形成相应的算法,指导编写程序这样一条线索展开研究;通过简明的理论分析和具体的案例分析来说明算法的性能;而且给出了进一步改进算法和改写源程序代码的建议。书中对基础性的问题采用列表计算的模式编写程序;对复杂的数学问题,则采用模块化程序设计方法编写程序。

本书可作为一般理工科大学各专业 3~4 学分计算方法课教材或教学参考书,也可作为广大科技人员的参考书。

版权所有,翻印必究。举报电话:010-62782989 13501256678 13801310933

本书封面贴有清华大学出版社防伪标签,无标签者不得销售。

本书防伪标签采用特殊防伪技术,用户可通过在图案表面涂抹清水,图案消失,水干后图案复现;或将表面膜揭下,放在白纸上用彩笔涂抹,图案在白纸上再现的方法识别真伪。

图书在版编目(CIP)数据

实用数值计算方法/甄西丰编著. —北京:清华大学出版社,2006.1

(重点大学计算机专业系列教材)

ISBN 7-302-11853-1

I. 实… II. 甄… III. 数值计算—计算方法—高等学校—教材 IV. O241

中国版本图书馆 CIP 数据核字(2005)第 109319 号

出 版 者:清华大学出版社

<http://www.tup.com.cn>

社 总 机:010-62770175

地 址:北京清华大学学研大厦

邮 编:100084

客户服务:010-62776969

组稿编辑:丁 岭

文稿编辑:咏 鹏 闫红梅

印 装 者:北京鑫海金澳胶印有限公司

发 行 者:新华书店总店北京发行所

开 本:185×260 印张:26.25 字数:621 千字

版 次:2006 年 1 月第 1 版 2006 年 1 月第 1 次印刷

书 号:ISBN 7-302-11853-1/TP·7701

印 数:1~4000

定 价:35.00 元

出版说明

随着国家信息化步伐的加快和高等教育规模的扩大，社会对计算机专业人才的需求不仅体现在数量的增加上，而且体现在质量要求的提高上，培养具有研究和实践能力的高层次的计算机专业人才已成为许多重点大学计算机专业教育的主要目标。目前，我国共有 16 个国家重点学科、20 个博士点一级学科、28 个博士点二级学科集中在教育部部属重点大学，这些高校在计算机教学和科研方面具有一定优势，并且大多以国际著名大学计算机教育为参照系，具有系统完善的教学课程体系、教学实验体系、教学质量保证体系和人才培养评估体系等综合体系，形成了培养一流人才的教学和科研环境。

重点大学计算机学科的教学与科研氛围是培养一流计算机人才的基础，其中专业教材的使用和建设则是这种氛围的重要组成部分，一批具有学科方向特色优势的计算机专业教材作为各重点大学的重点建设项目成果得到肯定。为了展示和发扬各重点大学在计算机专业教育上的优势，特别是专业教材建设上的优势，同时配合各重点大学的计算机学科建设和专业课程教学需要，在教育部相关教学指导委员会专家的建议和各重点大学的大力支持下，清华大学出版社规划并出版本系列教材。本系列教材的建设旨在“汇聚学科精英、引领学科建设、培育专业英才”，同时以教材示范各重点大学的优秀教学理念、教学方法、教学手段和教学内容等。

本系列教材在规划过程中体现了如下一些基本组织原则和特点。

1. 面向学科发展的前沿，适应当前社会对计算机专业高级人才的培养需求。教材内容以基本理论为基础，反映基本理论和原理的综合应用，重视实践和应用环节。

2. 反映教学需要，促进教学发展。教材要能适应多样化的教学需要，正确把握教学内容和课程体系的改革方向。在选择教材内容和编写体系时注意体现素质教育、创新能力与实践能力的培养，为学生知识、能力、素质协调发展创造条件。

3. 实施精品战略，突出重点，保证质量。规划教材建设的重点依然是专业基础课和专业主干课；特别注意选择并安排了一部分原来基础比较好的优秀教材或讲义修订再版，逐步形成精品教材；提倡并鼓励编写体现重点大学计算机专业教学内容和课程体系改革成果的教材。

4. 主张一纲多本，合理配套。专业基础课和专业主干课教材要配套，同一门课程可以有多本具有不同内容特点的教材。处理好教材统一性与多样化的关系；基本教材与辅助教材以及教学参考书的关系；文字教材与软件教材的关系，实现教材系列资源配套。

5. 依靠专家，择优落实。在制订教材规划时要依靠各课程专家在调查研究本课程教材建设现状的基础上提出规划选题。在落实主编人选时，要引入竞争机制，通过申报、评审确定主编。书稿完成后要认真实行审稿程序，确保出书质量。

繁荣教材出版事业，提高教材质量的关键是教师。建立一支高水平的以老带新的教材编写队伍才能保证教材的编写质量，希望有志于教材建设的教师能够加入到我们的编写队伍中来。

教材编委会

前言

17 世纪是数学发展史上一个划时代的时期，当我们今天享受着高科技成果所带来的各种便利条件时，应该意识到笛卡儿 (Descartes, 1596—1650) 和牛顿 (Newton, 1642—1727) 这两位卓越的先驱人物所作出的具有划时代意义的贡献。

首先是笛卡儿创立平面解析几何，为我们开创了利用数学研究变量的新时代，使我们能用数学形式描述动态变化着的客观对象。从此生活在这个星球上的人们可以利用数学形式来描述天体运动规律以及一些非常复杂的系统的运动规律。接下来牛顿与莱布尼兹 (Leibniz) 等人创立微积分学，为我们研究连续变量的变化规律给出了完整的方法体系。随后，又在这个基础上产生了更多的数学分支以及相关的一些交叉学科分支。

由于数学研究的范围在不断扩大，而且研究的对象更为复杂，一个伴随的问题就是相应的数值计算更加困难。如果不解决与理论方法平行的数值计算问题，再好的数学理论也难以发挥应有的作用。牛顿的天才之处还在于，他在创立微积分学的同时，在数值计算领域也取得了具有历史意义的成就，实际上已经开始了这个领域中的奠基性的工作，这一点由数值积分的牛顿-柯特斯 (Newton-Cotes) 公式可以明显地看出来。

计算方法，过去有不少人称之为数值分析，现在更多的人称之为科学计算，其核心思想就是通过有限步的加、减、乘、除四则运算得到某个连续变量的近似值。如果我们进一步追根溯源，它应该是微积分学孕育出来的一个数学分支。当然，在计算机问世以前也只能是一个数学分支。极限论为近似计算提供了理论依据，如果一个无限序列 $\{x_n\}$ 收敛于某个极限值 x^* ，那么我们可用这个序列中的某个元素 x_N 作为 x^* 的近似值。只要序号 N 取得足够大，那么 x_N 与 x^* 的差值 $|x_N - x^*|$ 就不会超过某个预先给定的充分小的正数 ϵ 。利用泰勒 (Taylor) 展式，可以把某个充分光滑的实函数 $f(x)$ 表示为一个项数为无限多的多项式，再利用极限理

论,我们可以取这个多项式的前 N 项近似表示 $f(x)$,从而可以通过有限步四则运算得到 $f(x)$ 的近似值。

如果说极限理论和泰勒展式为我们在更大范围内进行近似计算提供了理论依据,那么插值法则进一步为近似计算提供了实践指南。对于实际问题来说,通常需要计算某个适当光滑的实函数 $f(x)$ 在某个适宜的区间 $[a, b]$ 内的任一点处的函数值,而且不希望花费太多的计算时间。只要这个区间的长度不太大,可以事先采用其他方法计算出区间内的少量几个离散点 $x_0, x_1, \dots, x_n$ 处的函数值 $f(x_0), f(x_1), \dots, f(x_n)$,接下来再构造相应的 n 次或低于 n 次的插值多项式 $p(x)$,并利用 $p(x)$ 进行相应的计算。实际上,我们还可以利用 $p(x)$ 完成关于 $f(x)$ 的更多的计算,比如微分和积分,加快一个序列的收敛速度等。当我们利用函数 $f(x)$ 的插值多项式 $p(x)$ 求 $f(x)$ 在某个点处的微分或某个区间上的积分时,相应的计算又可表示为求 $f(x)$ 在一些点处的函数值的线性组合。所以只要找到这一组神奇的组合系数,接下来的具体计算就可以按机械操作的方式完成。正是因为一些看似十分复杂的数学问题,却蕴含着如此简单的求数值解方法,才使得计算方法成为一个魅力无穷的研究领域。

数值计算的重要性一方面促进了计算方法的研究,另一方面也促进了计算工具的发展。随着 20 世纪 40 年代中期人类第一台电子数字计算机的问世,数值计算终于有了理想的支撑工具。坚实的数学理论,科学的计算方法以及先进的计算工具的有机结合,可以为我们打造了一个无比坚实的工作平台。正是因为有了这样一个平台,才有了 20 世纪 60 年代人类登上月球的壮举以及今天许多国家雄心勃勃的太空开发计划。完全可以这么讲,今天几乎所有的高科技成果都是在这个平台上产生的。正是因为有了这样一个平台,从而使得我们这个时代的科学技术能够飞速发展。

今天,我们正处在一个非常有利的历史时期:过去,以牛顿为代表的一大批先驱人物在当时的计算工具极为落后的条件下进行了大量的研究,取得了丰硕的成果,为我们留下了宝贵的精神财富;今天,电子计算机性能不断提升而价格又不断下跌为我们的科学计算提供了难得的物质基础;再者,当代科学技术的不断进步和生产力的迅速发展又为高水平的科学计算提供了用武之地。利用现有的计算机条件,再现以牛顿为代表的这些优秀人物的思想和方法,进一步发掘潜藏在计算方法领域中的科学真谛,完全可以期待着更加辉煌的成就。

作者在华中科技大学长期从事与数值计算密切相关的计算方法和运筹学等课程的教学,感触甚多。近几年来,坚持利用多媒体设备从事计算方法课的教学,又进行了不懈的探索,深刻领悟到现代计算方法课的教学对于培养大学生和研究生的综合能力以及科学研究的兴趣来说处于得天独厚的地位。通过计算方法的教学,可以帮助学生早期实现利用所学的数学基础知识应用于比较复杂的数学模型求数值解的算法研究,进而利用计算机求解相应的问题。写一本与培养目标相适应,与教学手段相适应,而且为学生留有一定的自学余地和超前引导内容的教材,既是学生的企盼,也是教师的责任。本书就是在这种背景下写成的,全书共有 13 章,适当选取,可适应多层次的教学需要。

下面简要介绍一下本书的主要内容以及写作的经过。

第 1 章相当于概论,主要论述利用计算机进行数值计算的重要意义和基本方法,结

合本书的特点介绍了列表计算的重要意义,结合简单的数值微分算例给出了完整的列表计算源程序。鼓励读者从一开始阅读本书就与上机操作联系起来。

第2章在介绍了误差的来源和误差的基本概念后,着重介绍了利用微分估算误差和利用条件数估算误差,最后结合算例介绍了科学计算的基本规则,这也是作者编程的心得体会。

第3章主要利用基本初等函数的泰勒展式求函数值,并根据极限原理来控制误差:对于适当给定的 $\epsilon > 0$,根据泰勒展式的余项估计公式利用计算机程序计算出相应的项数 n ,按照科学计算的规则求相应的 n 次多项式的值。所有的计算实际上都得到了利用双精度数计算所能得到的最精确的结果。这些结果一方面证实了所采用的方法的科学性,反过来又为科学计算奠定了基础。长期以来,人们总是面临这样的尴尬局面:科学计算离不开软件开发商提供的基本初等函数计算程序,而相应的源代码和算法却是商业机密。现在,科学计算终于有了属于自己的基础领地。如果说极限论中的 ϵ - N 规则奠定了连续变量数学的基础,那么 ϵ - N 规则在计算机上的实现,将会使得计算机在解决数学问题领域发挥更大的作用。

第4章函数增量的计算方法,主要解决科学计算领域最令人头疼的问题:求连续函数 $y=f(x)$ 在 x_0 处的 Δx 增量 $\Delta y=f(x_0+\Delta x)-f(x_0)$ 和增量比 $\Delta y/\Delta x=[f(x_0+\Delta x)-f(x_0)]/\Delta x$ 。这里所得到的结果从哲学的角度讲也是很有意义的。以计算

$$\Delta y = \arctan(x_0 + \Delta x) - \arctan x_0 \quad (1)$$

为例,首先对 Δy 取正切,利用公式

$$\tan(a-b) = (\tan a - \tan b)/(1 + \tan a \tan b)$$

可以得到

$$\tan(\Delta y) = \Delta x/[1 + x_0(x_0 + \Delta x)]$$

接下来再对 $\tan(\Delta y)$ 取反正切,又可以得到 Δy ,从而有

$$\Delta y = \arctan(\Delta x/[1 + x_0(x_0 + \Delta x)]) \quad (2)$$

通过这么一轮否定之否定,比起(1)式来说,利用(2)式计算 Δy ,计算量可以大大减少,计算精度可以大大提高。如果说计算量的减少属于量变,那么计算精度的提高应该属于质变。如果进一步利用(2)式并借助泰勒展式来计算 $\Delta y/\Delta x$,还可以得到更加美妙的结果。

在第4章中,我们正是利用基本初等函数各自特有的性质,借鉴第3章利用泰勒展式进行精确计算的方法,解决了所有基本初等函数以及多项式函数的增量和增量比的计算方法,所产生的误差主要由机器的原因和相应的数学问题的条件数所决定,与我们的方法几乎没有关系,并且在计算函数的增量比 $\Delta y/\Delta x$ 时, Δx 可以取零值。在这一章中,还进一步讨论了复合函数以及函数的和、差、积、商的增量和增量比的计算方法,从而解决了初等函数的这一类计算问题。作者深信,这一章的成果对读者将来从事高级应用领域的科学计算一定能够产生重大的影响。

第5章解决函数的零点和极值点的求法问题,由于求函数零点的二分法与求函数的极值点的黄金分割法都属于按比例缩小区间的搜索方法,所以放在一起学习更容易理解算法的共性。牛顿法既可以求函数的零点,也可以求函数的极值点,所以从学习效果

看,把这两个问题放在一起讨论也是合适的。在国内外同类型的文献中,对牛顿法的性能分析似乎都处理得不太好,缺少灵气,极大地影响了牛顿法的应用。在这一章中,我们特别结合凸函数的性质对牛顿法的性能进行了深入研究,在适当的假设下得出了牛顿法具有单调收敛性以及二次终结性质,这方面的一些结论比参考文献[11]中的相应的结果更具说服力。有了这样的理论基础,相信广大读者会让牛顿法在现代计算机条件下发挥更大的作用。关于非线性方程组求解,我们采用的是循序渐进的策略,在这一章中只作最基本的介绍,接下来在第6章中再给出一个模块程序,在第7章中利用所得到的模块程序简化了Bairstow方法求多项式复根的程序设计。

第6章主要介绍两个变元的无约束极值问题的数值解方法。对于这类问题来说,最速下降法总是研究的出发点,然后再根据最速下降法的锯齿现象寻找克服的措施,从而得到改进的方法。所给出的三部曲算法可以说是作者送给读者的一份价廉物美的礼品。价廉是指方法特别简单,大学生也看得懂,用得上;物美指的是这个方法特别好。在运筹学领域,对无约束极值问题差不多进行了拉网式的研究,偏偏把这个方法给漏掉了,作者也是借此机会了却一个心愿。原来的写作计划只是在前面一章中有这样一节,只是编程时突然感觉到应该换一种编程的风格,也是对“实用”的承诺,所以就有了这一章。同时也就有了模块化程序设计方法;有了采用三部曲算法的最优化模块程序;有了牛顿-拉弗桑(Newton-Raphson)方法解非线性方程组的模块程序,有了用最优化法解非线性方程组的程序;有了Bairstow方法求多项式的复根的模块程序;也有了用最优化法求多项式的复根的模块程序。在后面(实际上在前面也有一些)的许多章节中,读者可以体验到模块化程序设计方法所带来的各种方便。

第7章多项式计算的完成看来要感谢上帝的指点。无论从哪个意义上讲,多项式计算应该是科学计算的重要组成部分。作者当然希望在这方面有所突破,为此也阅读了不少文献,但总觉得不够理想。一个偶然的机会在参考文献[9]中读到了多项式的Sturm定理,兴奋不已:对一般的多项式,判定所有实根所在的区间并不困难,利用Sturm定理可以确定任意区间内的实根个数,所以能够利用Sturm定理确定多项式实根的个数以及所在的区间,在此基础上可以进一步设计一个算法,还是利用Sturm定理,把多项式的所有实根所在的区间划分为若干个小区间,使得每个区间内恰有一个实根,所有这些工作都可以交由计算机以高效率的方式完成。应用Bairstow方法求多项式的复根比求实根的效果要好一些,这样一来多项式求根的问题大体上形成了解决方案。Bairstow方法涉及到所谓的二阶综合除法,所以必须介绍原始的综合除法,现在只好称为一阶综合除法。对一阶综合除法的研究终于发现它原来还有很多应用:可以求多项式的值,可以求多项式的导数值,可以对多项式作平移变换,可以用来精确计算多项式函数的增量及增量比,从而可以与第4章中解决同类问题的方法互相印证。对于求多项式的实根来说,还有一个很大的麻烦就是重根的负面影响,如果能有效地消去多项式的重根,应用Sturm定理编程也简单得多。无论是消除重根,还是求多项式的Sturm序列,都需要用到欧几里得(Euclid)算法,所以在介绍Sturm定理之前介绍欧几里得算法,并应用于消除多项式的重根也就顺理成章。第7章的构思就这样在不知不觉中达到了理想境界,将古今中外在这个领域中的重要成果有机地融于一体。

第8章介绍线性方程组求解方法,对于一般的应用而言,采用直接方法也就可以了。这一章重点介绍了高斯(Gauss)消去法,实际上已经引导读者首先开发出一个学习、研究高斯消去法和选列主元的高斯消去法的演示系统。由于借用系统仿真的方法随机生成线性方程组,所以学习起来有点像卡拉OK厅里面的自娱自乐。为了强调实用性,另外还给出了应用高斯消去法解线性方程组的模块程序,并通过与第6章牛顿-拉弗桑方法解非线性方程组的模块程序连接而证实了实用性。由于高斯-约当消去法相对来说要简单得多,所以只给出两个模块程序,如果需要,读者完全能够按照学习高斯消去法的模式完成相应的演示系统的编写。

曲线拟合与多项式插值虽然同属逼近理论的应用,但两者的应用领域却大相径庭,由此导致应用程序的风格也相距甚远,所以在第9章专门介绍最小二乘法和曲线拟合,并给出了几种不同工作方式的模块程序,基本上可以应用于解决实际问题。

第10章插值方法,这部分内容在数值计算领域相对来说比较成熟,但是插值方法的应用领域正在悄悄地发生变化。在过去计算工具十分落后的条件下,插值方法主要应用于求函数的近似值,应用于制作数学用表。在现代条件下,插值方法主要应用于加速序列的收敛速度。为此,本章特别增加了灵活应用插值方法举例一节,具体介绍如何集成插值方法与二分法解非线性方程;集成抛物线插值法与黄金分割法求函数的极小值点;并通过案例的对比分析说明采用插值方法加速的意义。

第11章介绍的数值微分与外推加速方法也有多重意义。在普遍应用高性能计算机的今天,数值微分的价值在不断上升。比如说,把一个程序的输出看成是输入的函数,如果这个函数从数学的意义上讲也是可微的,那么寻找一种有效的方法直接利用这个程序求相应的数值微分很能体现数学理论和计算工具有机结合的美学价值。在这一章中,首先具体介绍了利用差商或中心差商作为微商的近似值;利用不同步长的差商产生微商的近似值序列;直接构造外推方法加速近似值序列的收敛。为了得到简单、高效而且具有普遍实用意义的数值微分方法,一方面利用泰勒展式构造高阶近似公式从而导出理查逊(Richardson)外推加速算法;另一方面又把涅维尔(Neville)的递推式插值公式改造成更为简单的通用外推格式。实际上,外推方法在数值计算领域有着广泛的应用。毫不夸张地说,现代科学计算领域的许多新成就很大程度上源自外推方法的应用。

第12章从实用性考虑,详细介绍了变步长复化型自动调整精度的系列方法:梯形系列、辛普森(Simpson)序列、柯特斯(Cotes)序列以及龙贝格(Romberg)序列。由于辛普森序列、柯特斯序列和龙贝格序列可以看成是梯形系列经过一次、两次和三次外推所形成的序列,所以这一章的内容实际上特别少:除了一些基本概念外,基本方法就是变步长复化梯形公式方法和理查逊(Richardson)外推加速方法。利用复化梯形公式和复化中点公式的关系,可以使得复化梯形公式的计算量节省一半,再加之外推方法的优越性,所以数值积分的一些其他方法的性能很难超过这里介绍的序列方法。

第13章常微分方程数值解法,这部分内容在数值计算领域相对来说比较丰富。本章前面几节结合案例计算介绍了一些实用的经典的方法如欧拉(Euler)方法,龙格-库塔(Runge-Kutta)法,阿达姆斯(Adams)外推与内插方法,可以满足一般应用的需要。由于常微分方程的数值解方法都是按步进的方式计算,所以为了消除误差的积累还需要研

究高精度或者自动控制精度的方法。在一般情况下,提高精度的途径是适当减小步长,所付出的代价是增加了计算量。为了两者兼顾,特别介绍了自适应步长的龙格-库塔方法和大步长问题的外推加速方法。这里的程序都以模块的形式给出,有利于读者结合实际问题的灵活应用。对于求常微分方程数值解来说,刚性问题总是一个严重的挑战,为此我们结合刚性问题的特点,介绍了 Bader 和 Deufhard 的基于修正中点公式的半隐式外推方法和我们给出的利用隐式梯形公式结合牛顿法求解的隐式外推方法,并结合案例进行了对比分析,使得针对这类问题的特殊方法更趋完善。在本章最后,我们还根据问题的特点,对各种方法的选用给出了指导性意见。

为了让本书更好地适应教学目标和教学手段,也便于广大科技人员自学,本书在写作手法方面凸现了如下一些特点。

拓宽了计算方法的基础知识,为计算方法正名。纯数学界很难接受利用计算机求极限的观点。如果真是这样的话,理论上就没有计算方法存在的合法性,因为计算方法在很大程度上都使用计算机求极限的近似值,虽然不是真值,但总比人工做出来的结果更精确。第1章、第3章和第4章使得计算方法与高等数学建立了密切的联系。这样,读者一翻开本书,即有似曾相识的亲近感觉。

博众家之长,让读者站在巨人的肩上。本书虽然篇幅不长,但极尽全力介绍这个领域中做出过创造性贡献的人们的工作。集中介绍他们处理问题的思想和方法,引导读者在今天的条件下如何在计算机上再现他们的思想和方法,并结合案例分析而形成自己独立的判断。既能虚心学习,又不盲目崇拜。所花笔墨不多,但吸取的精华却不少。

尽可能把方法处理得简单。华罗庚教授曾在一首小诗中写过,“神奇化易是坦途,易化神奇不足提”。对于数值计算来说,这无疑是金玉良言。在本书中,或者借助图形解释,或者借助生活中的常识见解把算法原理解释得简单、再简单、尽可能简单。这样的例子随处可见,从而形成了风格。对于一些典型的问题求解,采用列表计算的固定格式编程,对算法熟悉了,对程序格式熟悉了,编程也就简单了。对于复杂的问题,采用模块化程序设计方法,每个模块相对简单,而且可以反复使用。

循序渐进,步步为营。书中的内容,从所涉及的数学内容看,大体保持了数学知识的逻辑顺序关系,从所求解的典型问题看,大体保持了由易到难的顺序关系;从采用某个方法,调用某个程序的依赖关系看,也大体上是被利用的方法或程序在前,使用者在后。这样的安排,减少了学习困难,有利于循序渐进、巩固提高和综合应用。

把读者扶上马,送一程。本书采用手把手的方式教读者如何编程实现相应的算法,并给出了主要代码以及案例的计算结果。除此之外,还明确地告诉读者如何改写书中的哪个程序以适应什么样的问题,鼓励读者举一反三。

简言之,作者在本书所追求的目标就是引导读者充分利用自己学过的基础知识,创造性地借鉴牛顿等先驱人物解决复杂计算问题的思维方式以及所形成的科学方法,充分利用现有的计算机资源,去解决一个又一个复杂的数值计算问题,并在这个过程中快速成长起来。作者深信,广大理工科大学生和工程技术人员阅读了本书之后,对科学计算的领悟会上升一个台阶,通过亲自动手编程,直接利用计算机求解复杂的工程问题的兴趣、勇气和能力将会大大提高,将来面对一些复杂的数值计算问题时,肯定能够少走

弯路。

前面提到过，本书是根据作者在华中科技大学多年讲授计算方法，特别是近几年采用多媒体教学的经验编写而成，可作为大专院校一般理工科各专业 3~4 学分的计算方法课的教材或教学参考书使用。本书致力于“实用性”并不意味着一定要给出许多工程应用的案例，而是创造尽可能好的条件，帮助读者解决好从学习计算方法的算法原理到解决实际问题的程序设计这中间可能遇到的诸多问题，然后鼓励读者自由发挥，独立地解决问题。这个目标看来是可以实现的，作者从学生们的试验报告来看，大都能够应用课堂学习的方法解决他们专业课程中的、科技文献中的、个人感兴趣的、家长请帮忙的各种计算问题，思路之宽阔，想象力之丰富，情绪之高昂已超出作者的预期，对本书的形成也产生了一定的影响。如果作为 3 学分的教材使用，可以略去第 4、6、7 这三章，并不影响其余各章的学习，实际上也不会影响大部分学生自学这几章的兴趣。即使作为 4 学分的教材使用，也要适当留下一些内容（通常是比较容易的和少量有一定难度但不作具体要求的内容）让学生自学。

在一本教科书中把数学分析、算法说明、程序说明、利用计算机程序解数学问题非常密切地联系起来，难免出现术语的混乱，主要有下面几个原因：变量既是计算机词汇，又是数学词汇，但在不同领域中的含义是不同的；数组是个计算机词汇，一般指存放数据的地方，在本书中，又必须用数组来表示（存放）数学中的许多不同对象，比如向量、序列、集合、一个变量的不同的取值的集合等；C 语言的指针变量的作用相当于数组；函数在 C 语言程序中是一段代码，作为数学概念表示两个变量之间的依存关系，本书中有许多同名的函数既是 C 语言函数，又是数学函数。为了尽可能避免一些不必要的混乱，本书中通常结合算法和程序的说明对一些重要的变量和数组作出相应的解释；函数作为 C 语言函数引用时，一般采用函数名连同一对空括号“()”的格式，如果一定带上自变量，则一定有相应的类型说明；作为数学函数引用时，则按数学的习惯使用，如果有同名的 C 语言函数，则蕴含相应的函数调用，并返回所得到的结果，这与函数的数学定义并不矛盾。采用这样一些约定，再加之表述时适当使用修饰成分，基本能够保持术语的规范。在极个别情况下，读者可以根据上下文来理解。

关于数学术语和数学符号的使用，与一般数学教科书中的用法基本相同。在本书中，我们采用加粗的斜体字母表示向量，向量的上标用于标示不同的向量，而向量的下标用来标示向量的分量。如果表示某个分量的指数幂，则用括号连同幂指数的形式给出。

关于引用问题，文中的定义、定理、公式、图、表都用“节. 序号”的格式编号，程序则按“章. 序号”的格式编号。如果本章引用，都直接通过编号引用；如果跨章引用，则在编号前指出所在的章，比如“见第 10 章(3.2)式”，指的就是第 10 章第 3 节公式(3.2)。

关于程序以及编号，书中所有的程序虽然没有按软件工程的规则检测，但都通过了 Turbo C 2.0 的编译、调试以及简单的试运行。为了说明算法，需要把一个较大的程序分割成几块分别介绍，而调试程序时只作为一个程序调试，对于这样的程序，我们采用程序编号后面附加字母 A、B、C、D... 的方式介绍。所以如果不计编号后面的附加字

母,那么每个程序都对应着一个调试程序。书中有一些是完整的源程序,读者可以直接输入到计算机中编译运行,首先对照书上显示的结果检测无误后,可以大胆改写,让它发挥更大的作用。为了节省篇幅,有些程序中略去了一些前面出现过的相同的或类似的代码,读者可以自己补充完整。如果读者消化这些程序有困难,可以访问清华大学出版社的网站: <http://www.tup.com.cn>,以寻找帮助,也可以(最好以团体的方式)直接向清华大学出版社索取完整的源代码。练习题中有些编程的问题可模仿书上的程序编写,也有一些则超过了本书作为教材使用的范围,作者也将在网上陆续作答,敬请读者关注。

与本书配套的教学课件内容包括:PowerPoint 文档给出的详细教学大纲,HTML 文档连同脚本程序给出的结合案例分析的算法演示系统,Word 文档给出的各种专题讨论。选用本书作教材的学校和教师本人如果感兴趣,可以直接与清华大学出版社联系。如果对课件有任何意见、建议、或具体要求,也可直接与作者本人联系。

尽管作者主观上想为读者提供一本好书,也为此付出了辛勤的劳动,但客观效果应由广大读者评价,另外书中也难免有缺点和错误。恳请广大读者和同仁不惜赐教,提出批评意见。

作者的 E-mail 地址: hustxfzhen@163.com

甄西丰

2005 年 8 月于华中科技大学

第 1 章 数值计算问题概述	1
1.1 数值计算问题的提出	1
1.2 计算机能够完成的工作	2
1.3 计算方法研究的主要问题	3
1.4 利用机器计算的基本方法	4
1.5 计算方法与计算机算法	8
1.6 关于算法的评价	9
1.7 列表计算的优越性	11
1.8 一个完整的列表计算程序	14
练习题	17
第 2 章 误差分析	19
2.1 误差的来源与分类	19
2.2 误差的基本概念	23
2.3 有效数字	25
2.4 利用微分估算误差	28
2.5 利用条件数估算误差	32
2.6 近似计算的基本规则	36
练习题	43
第 3 章 常用函数值计算方法	44
3.1 引言	44
3.2 多项式与有理函数值计算方法	45
3.3 数的开平方与开立方	47
3.4 一元二次方程求根方法	51
3.5 三角函数值计算方法	53

3.6 对数函数值计算方法	60
3.7 指数函数值与幂函数值计算方法	65
3.8 反正弦和反余弦函数值计算方法	70
3.9 反正切和反余切函数值计算方法	76
练习题	81
第4章 函数增量的计算方法	82
4.1 引言	82
4.2 二次根式函数增量的计算方法	83
4.3 三角函数增量的计算方法	87
4.4 对数函数增量的计算方法	93
4.5 指数函数增量的计算方法	95
4.6 反正弦与反余弦函数增量的计算方法	99
4.7 反正切与反余切函数增量的计算方法	102
4.8 整数幂函数与多项式函数增量的计算方法	105
4.9 一般初等函数增量的计算方法	108
练习题	111
第5章 求函数的零点与极值点问题	112
5.1 函数的零点与极值点问题概述	112
5.2 区间对分法	115
5.3 黄金分割法	119
5.4 牛顿(Newton)迭代法	125
5.5 凸函数的性质与牛顿迭代法的性能分析	129
5.6 基于插值的方法	137
5.7 压缩映像原理与不动点算法	142
5.8 简单的非线性方程组求解	150
练习题	158
第6章 简单的无约束极值问题	159
6.1 问题的提法与算法框架	159
6.2 模块化程序设计方法	162
6.3 最速下降法	168
6.4 三部曲算法	171
6.5 解非线性方程组的模块化程序设计方法	174
6.6 最优化方法解非线性方程组	177
练习题	180

第 7 章 多项式计算	181
7.1 引言	181
7.2 多项式的基本运算	182
7.3 余数定理与综合除法	186
7.4 多项式的平移变换与泰勒(Taylor)展开	189
7.5 多项式的重根问题与欧几里得(Euclid)算法	192
7.6 Sturm 定理与多项式实根的隔离方法	196
7.7 多项式实根的求法	208
7.8 求多项式复根的 Bairstow 方法	210
练习题.....	218
第 8 章 线性方程组求解	219
8.1 线性方程组概述	219
8.2 高斯(Gauss)消去法	224
8.3 选主元高斯消去法	231
8.4 高斯消去法应用模块的编写	235
8.5 高斯-约当(Gauss-Jordan)消去法	241
8.6 追赶法解三对角线性方程组	244
练习题.....	248
第 9 章 最小二乘法与曲线拟合	250
9.1 线性函数拟合方法	250
9.2 最小二乘法	254
9.3 多项式拟合方法	259
9.4 一般基函数的曲线拟合方法	265
练习题.....	266
第 10 章 插值方法	268
10.1 引言.....	268
10.2 一般的多项式插值问题.....	269
10.3 拉格朗日(Lagrange)插值方法.....	272
10.4 差商与牛顿插值公式.....	277
10.5 插值余项与差商的性质讨论.....	285
10.6 灵活应用插值方法举例.....	287
10.7 分段埃尔米特(Hermit)插值	294
10.8 三次样条插值方法简介.....	299
练习题.....	301

第 11 章 数值微分与外推加速方法	303
11.1 利用一阶差商外推加速	303
11.2 利用中心差商外推加速	306
11.3 理查逊(Richardson)外推加速方法	308
11.4 涅维尔(Neville)插值方法与通用外推格式	312
11.5 实用外推加速方法	316
11.6 二阶导数的计算方法	324
练习题	327
第 12 章 数值积分	328
12.1 求积公式与代数精度的概念	328
12.2 牛顿-柯特斯(Newton-Cotes)求积公式	331
12.3 变步长复化梯形公式方法	335
12.4 变步长复化辛普森(Simpson)公式方法	339
12.5 龙贝格(Romberg)求积算法	342
12.6 复化梯形公式的精度分析	345
12.7 关于数值积分方法的注释	346
练习题	347
第 13 章 常微分方程的数值解法	349
13.1 常微分方程初值问题数值解方法概述	349
13.2 欧拉(Euler)方法及其改进形式	354
13.3 龙格-库塔(Runge-Kutta)方法	359
13.4 线性多步法	365
13.5 求常微分方程数值解的模块化程序设计方法	370
13.6 自适应步长的龙格-库塔方法	374
13.7 大步长问题与外推加速方法	384
13.8 刚性方程与隐式外推方法	391
13.9 常微分方程数值解方法小结	401
练习题	402
参考文献	404

线性方程组求解

第 8 章

许多工程问题的求解都归结于解一个线性方程组,比如在一个电网中计算各区段上的电位差和电流,利用多级水库进行跨流域调水等。科学计算领域中的许多典型问题求解也涉及到求解线性方程组:第 5 章和第 6 章用牛顿-拉弗桑方法解非线性方程组采用线性化近似方法而得到线性方程组;第 9 章曲线拟合所得到的正规方程;第 10 章的三次样条插值也需要解一个三对角线性方程组,利用差分方法求解二阶常微分方程的两点边值问题也需要求解一个带状线性方程组等。正因为线性方程组有着广泛的应用,所以对它求数值解方法的研究一直都没有停止。

线性方程组求解的方法可以分为三大类:直接方法,也可以称为代数方法,是中学里学习的方法的伸延;迭代方法,实际也是不动点方法的应用,由于涉及到依距离收敛的概念,也可以理解为几何的方法;再就是近代方法,通常与求解大规模最优化问题联系在一起,人们还在为之奋斗。

对于大部分应用问题来说,目前的主流还是直接方法:主要是高斯消去法和它的各种变化形式。

8.1 线性方程组概述

8.1.1 术语和记号

由 n 个变元, m 个线性方程组成的线性方程组的一般形式为:

$$\left. \begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1j}x_j + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2j}x_j + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{ij}x_j + \cdots + a_{in}x_n &= b_i \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mj}x_j + \cdots + a_{mn}x_n &= b_m \end{aligned} \right\} \quad (1.1)$$

对于上面给出的一般形式的线性方程组(1.1),可以利用矩阵和向量的记号来表示。记

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

则(1.1)式可简写为 $Ax=b$, 其中 A, x, b 分别称为系数矩阵, 未知向量, 以及常数向量。

8.1.2 基本假设及主要结论

为了简化讨论, 对于(1.1)式给出 n 个变元, m 个方程组成的一般形式的线性方程组, 记 A 为方程组的系数矩阵。作为研究数值解方法来说, 总可以假定 A 或者满行秩, 或者满列秩。在绝大多数情况下, 这种假设都可以得到满足或者不难得到满足。

在上面的假定下, 利用代数学中已有的结论, 有:

如果 $m=n$, 则 A 是满秩的 n 阶方阵, 那么方程组有惟一解。线性方程组求解主要研究这种情形的问题;

如果 $m < n$, 那么方程组有无数组解, 在这种情况下, 通常希望寻找某种意义下的最优解, 这是运筹学中的线性规划部分所研究的问题;

如果 $m > n$, 那么方程组无解, 但是作为实际问题的数学模型还是有意义的, 解决的办法是转向寻求它的最小二乘解, 这个问题将在第 9 章专门讨论。

所以, 如果线性方程组的系数矩阵是满行秩或者满列秩, 总有相应的求数值解方法来处理。如果情形不是这样, 可以利用计算机程序发现这个问题并报告出来, 然后再想其他办法来处理。

8.1.3 线性方程组的增广矩阵

对于求线性方程组(1.1)的数值解来说, 首先要解决好的问题是如何把所有“原始数据”按一定的“结构”输入到计算机中。一种考虑是让程序更方便地应用于求解实际问题, 这时, 可以在程序中说明一个二维数组和一个一维数组分别存放线性方程组的系数矩阵 A 和常数向量 b 。如果是这样的话, 编程会稍微麻烦一些, 目前不提倡这样处理。另一种考虑是把常数向量 b 直接写到系数矩阵 A 的后面从而得到相应的增广矩阵 (A, b) , 这时, 只需在程序中说明一个二维数组来存放相应的增广矩阵即可。由于线性方程组的增广矩阵也包含了线性方程组的全部原始数据, 所以也能直接根据增广矩阵编程, 而且相应的程序还简单得多。

如果单纯研究线性方程组的数值解方法, 采用线性方程组的增广矩阵来编程是合理的, 而且本身也有一定的实用性。掌握了线性方程组求解的基本原理和方法后, 再来编写便于应用的程序也就容易了。

作为编程的习惯, 我们把线性方程组的增广矩阵还是记为 A , 对于线性方程组(1.1)来说, 有

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n}b_1 \\ a_{21} & a_{22} & \cdots & a_{2n}b_2 \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn}b_m \end{pmatrix}$$

8.1.4 线性方程组的解

设 $Ax=b$ 是给定的线性方程组,如果向量 x^* 满足条件 $Ax^*=b$,则 x^* 称为问题的解。现在假定是利用计算机求解,而且输入到计算机中的原始数据只是方程组的增广矩阵,所谓解线性方程组,其实也就是对相应的增广矩阵进行一系列行“变换”,所以问题便转化为在什么条件下可以停止进一步的计算,如何得到所需要的结果。显然,如果线性方程组的增广矩阵的形式为

$$A = \begin{pmatrix} 1 & 0 & \cdots & 0 & a_{1n+1} \\ 0 & 1 & \cdots & \cdots & a_{2n+1} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \cdots & \cdots & 1 & a_{nn+1} \end{pmatrix}$$

则 $x_j = a_{jn+1}, j=1,2,\cdots,n$ 即为问题的解。所以,如果线性方程组的增广矩阵的左边 n 列是一个单位矩阵,则问题已经得到解决。

8.1.5 编程的准备工作

在接下来的几节中,我们将结合相应的 C 语言程序设计,结合案例计算介绍线性方程组求解的消元方法,所有的操作归根结底还是对存放在计算机内的增广矩阵进行变换,所以我们应首先学会如何得到一个线性方程组的增广矩阵,怎样在计算机屏幕上显示的方法。

一般地,可以在程序的前面部分说明一个二维数组 A 存放线性方程组的增广矩阵,说明两个整形变量 M, N 表示增广矩阵 A 的行数和列数。对于一个具体的 n 个变元 n 个方程的线性方程组来说,矩阵的行数为 $M=n$,列数为 $N=n+1$ 。

接下来可以编写一个 C 语言函数

```
int FormProblem(){...}
```

来完成数组 A 的初始化工作。由于实际数据也可能来自数据文件,也可能来自键盘输入,也可以在程序中直接赋值,所以函数 $\text{FormProblem}()$ 最好不带参数,程序中所用到的数据都由它来安排。对于初学者来说,首先不宜把问题搞得很复杂,可以采用最简单的办法直接在程序中用赋值语句来完成。假如利用计算机求解线性方程组

$$\left. \begin{aligned} x+y+z &= 6 \\ 2x \quad + z &= 5 \\ y+2z &= 8 \end{aligned} \right\} \quad (1.2)$$

相应的变量说明和 $\text{FormProblem}()$ 函数可以写为

```
double A[3][4];
```

```

int M=3,N=4;
int FormProblem()
{ A[0][0]=1;A[0][1]=1;A[0][2]=1;A[0][3]=6;
  A[1][0]=2;A[1][1]=0;A[1][2]=1;A[1][3]=5;
  A[2][0]=0;A[2][1]=1;A[2][2]=2;A[2][3]=8;
  return;
}

```

实际上,由于解线性方程组的用途非常广泛,而我们学习的目的就是编程计算,所以以一种超然的态度来构造案例更有利于专心研究数值解方法。我们可以利用计算机生成随机数来给线性方程组的增广矩阵赋初值,一方面所得到的线性方程组有更好的代表性,另外利用计算机随机地生成线性方程组(的增广矩阵)可以免去大量的手工劳动。

任何一门高级语言都会提供随机数生成函数,也称为随机数发生器,所以利用随机数发生器来生成线性方程组也是很方便的。C 语言的随机数发生器的相关说明在头文件“stdlib. h”中,下面是对有用的几个相关函数的简单介绍。

(1) 设置随机数种子

函数原型 void srand(unsigned int seed)

计算机产生随机数的算法实际上就是迭代法,所以需要给出迭代的初值,人们习惯上把这个初值称为“种子”(seed)。尽管系统已经设置了迭代的初值,但由于相同的初值会产生相同的随机数序列,这对于应用来说显然不好,所以系统允许用户通过接口函数 srand(seed)来重新设定种子 seed 的值。对于构造线性方程组来说,不同的种子可以得到不同的线性方程组,在不同的机器上采用相同的种子又能够得到相同的线性方程组,这显然是一件很有意义的事。

(2) 设置一个随机数作为种子值

函数原型 void randomize(void)

无论是由系统设置的还是由程序员临时设置的 seed 的值,总令人觉得计算机输出的随机数还是受到“人”的操纵,并不是真正的随机数,在一些特殊的应用中难免有失“公平”。为了避免这个弊端,函数 randomize()通过取计算机上的系统时间作为 seed 的值,其数值为格林威治时间 1970 年元月 1 日 0 时 0 分 0 秒到执行 randomize()时刻所经历的秒数。所以如果要调用 randomize()函数,还需要在程序前面加上预编译行

```
#include<time. h>
```

目前我们不必想得这么多,也可以不用这个函数。

(3) 输出一个随机数

函数原型 int rand(void)

调用一次函数 rand()一方面返回一个 0 到 RAND_MAX 之间的伪随机数;另外又用这个随机数替代 seed,从而体现了迭代的方法。这里 RAND_MAX 是由系统定义的常数,在 Turbo C 2.0 中的实际值是 32 767。

在程序设计时,可以通过强制类型转换把 rand()返回的 int 型随机数转换成 double 型浮点数,再除以 RAND_MAX 即可得到 $[0,1]$ 内的实数,最后可以根据需要进行简单的

数学变换而得到某个给定的区间内的实数。利用随机的方法生成线性方程组的增广矩阵的源代码见程序 8.01, 所得到的增广矩阵中的数都在区间 $[-10, 10]$ 内。为了便于显示, 我们约定 A 只用于存放 4 行 5 列的矩阵, 并给出相应的显示函数 PrintMatrix()。

程序 8.01 随机生成线性方程组的增广矩阵

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#ifndef ZERO
#define ZERO 1.0E-20
#endif
#define M 4
#define N 5
unsigned int SEED=8000;
double A[M][N];
int SetPrintMatrix=0;
int SetSelectMain =1;
int RM=M,RN=N;
void FormProblem()
{int i,j,k;
 double x;
 srand(SEED);
 for(i=0;i<M;i++)
 for(j=0;j<N;j++)
 {k=rand();
 x=(double)k/RAND_MAX;
 /* RAND_MAX=32767,by system */
 A[i][j]=x*20.0-10.0;
 }
 return;
}
PrintMatrix()
{int i,j;
 printf("\n=====");
 printf("=====");
 printf("\nA[i][j]");
 for(j=0;j<RN;j++) printf(" [%1d] ",j);
 printf("\n-----");
 printf("-----");
 for(i=0;i<RM;i++)
 {printf("\n [%1d] ",i);
 for(j=0;j<RN;j++)
 printf("%13.5f",A[i][j]);
 }
```



```

printf("\n=====");
printf("=====");
getch();
}
main()
{FormProblem();
 PrintMatrix();
}

```

8.1.6 关于术语和记号的特别约定

在接下来的几节中,大量的程序都是在程序 8.01 的基础上增加相应的 C 语言函数并修改 main() 函数而得到的。为了避免一些累赘的表述方式,我们约定术语“利用程序 8.01 生成的线性方程组”的含义是指相关的变量名与程序 8.01 中的完全相同,并且已经利用 FormProblem() 函数对相关的变量赋初值。作为编写源程序文件来说,也就是把程序 8.01 复制一份,然后再作相应的修改和增补,从而可以减少编辑源程序文件的工作量。程序 8.01 中的两个整型变量 SetPrintMatrix 和 SetSelectMain 也将在后面的应用中发挥很有意义的作用。

8.2 高斯(Gauss)消去法

高斯消去法解线性方程组其实就是中学里学过的加减消元法。它包括两个过程:首先把线性方程组化为上三角形线性方程组,接下来再用一个回代过程求这个上三角形线性方程组的解。

在所有算法中,高斯消去法的计算量最小,所以是手工求解或利用计算器求解的首选方法,事实上也是求解一般问题的主流方法。

8.2.1 消元法的 C 语言代码

假设有两个 n 个变量的方程所组成的线性方程组

$$\left. \begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1j}x_j + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2j}x_j + \cdots + a_{2n}x_n &= b_2 \end{aligned} \right\} \quad (2.1)$$

如果 $a_{11} \neq 0$, 那么可以从第一个方程中解出 x_1 , 得

$$x_1 = \frac{b_1}{a_{11}} - \left(\frac{a_{12}}{a_{11}}x_2 + \frac{a_{13}}{a_{11}}x_3 + \cdots + \frac{a_{1n}}{a_{11}}x_n \right) \quad (2.2)$$

再把上面的(2.2)式代入(2.1)式的第二个方程中,整理后即得

$$\left(a_{22} - a_{21} \frac{a_{12}}{a_{11}} \right) x_2 + \left(a_{23} - a_{21} \frac{a_{13}}{a_{11}} \right) x_3 + \cdots + \left(a_{2n} - a_{21} \frac{a_{1n}}{a_{11}} \right) x_n = b_2 - a_{21} \frac{b_1}{a_{11}} \quad (2.3)$$

从而得到与(2.1)式完全同解的线性方程组

$$\left. \begin{aligned} x_1 + \frac{a_{12}}{a_{11}}x_2 + \frac{a_{13}}{a_{11}}x_3 + \cdots + \frac{a_{1n}}{a_{11}}x_n &= \frac{b_1}{a_{11}} \\ \left(a_{22} - a_{21} \frac{a_{12}}{a_{11}} \right) x_2 + \left(a_{23} - a_{21} \frac{a_{13}}{a_{11}} \right) x_3 + \cdots + \left(a_{2n} - a_{21} \frac{a_{1n}}{a_{11}} \right) x_n &= b_2 - a_{21} \frac{b_1}{a_{11}} \end{aligned} \right\} \quad (2.4)$$

(2.4)式看起来虽然有些复杂,但是利用C语言程序却不难把(2.1)式给出的增广矩阵改写为(2.4)式给出的增广矩阵。下面通过一个例子说明相应的编程方法。

例 2.1 随机生成2个方程,4个变元的线性方程组的增广矩阵,然后利用消元法消去第二个方程中的第一个变量。

利用程序8.01中的代码,改写其中的main()函数即可完成相关操作。相应的源代码见程序8.02。

程序 8.02 简单的消元法举例

```
main()
{int j;
 double temp;
 RM=2;
 SEED=8000;
 FormProblem();
 PrintMatrix();/* NO1 */
 temp=A[0][0];
 for(j=0;j<N;j++)A[0][j]/=temp;
 PrintMatrix();/* NO2 */
 temp=A[1][0];
 for(j=0;j<N;j++)A[1][j]-=temp*A[0][j];
 PrintMatrix();/* NO3 */
 return;
}
```

程序的运行结果如表2.1A~2.1C所示,它们分别是执行程序8.02中注释NO1、NO2、NO3所标示的3条“PrintMatrix();”语句所显示的结果,读者不难跟踪程序的流程来阅读它们,也容易利用手工验算来体验程序的执行流程。

表 2.1A 程序 8.02 执行第一个 PrintMatrix()所显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	0.944 55	-2.817 16	-2.258 06	-2.892 24	1.511 58
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52

表 2.1B 程序 8.02 执行第二个 PrintMatrix()所显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52

表 2.1C 程序 8.02 执行第三个 PrintMatrix()所显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	-22.364 92	-5.061 20	-15.509 43	3.469 95

8.2.2 化为上三角形线性方程组

程序 8.02 中已经体现了高斯消元法的基本技巧,下面进一步讨论如何用高斯消去法把一个线性方程组化为上三角形线性方程组,这也是高斯消去法的关键所在。

定义 2.1 设 A 是一个 $n \times n$ 矩阵,如果对任意 $1 \leq j < i \leq n$ 均有 $a_{ij} = 0$,则称 A 是一个上三角形的矩阵。

定义 2.2 如果线性方程组 $Ax=b$ 的系数矩阵 A 是上三角形矩阵,则称为是上三角形线性方程组。

显然,上三角形线性方程组的一般形式为:

$$\left. \begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1j}x_j + \cdots + a_{1n}x_n &= b_1 \\ a_{22}x_2 + \cdots + a_{2j}x_j + \cdots + a_{2n}x_n &= b_2 \\ \vdots & \\ a_{kk}x_j + \cdots + a_{kn}x_n &= b_k \\ \vdots & \\ a_{nn}x_n &= b_n \end{aligned} \right\} \quad (2.5)$$

从而得到相应的增广矩阵的形式为

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ 0 & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & a_{nn} & b_n \end{pmatrix} \quad (2.6)$$

不难看出,上三角形线性方程组 $Ax=b$ 有惟一解的充要条件是 $a_{kk} \neq 0, k=1, 2, \dots, n$,这个条件其实也是上三角形系数矩阵 A 可逆的充要条件。

对一般的 n 个变元的线性方程组来说,可以反复利用程序 8.02 中的代码把一个一般形式的增广矩阵化为(2.6)式的形式。方法是,首先利用 $a_{11} \neq 0$ 依次把 $a_{21}, a_{31}, \dots, a_{n1}$ 都化为零;接下来利用 $a_{22} \neq 0$ 又依次把 $a_{32}, \dots, a_{n2}$ 都化为零;由此类推,直到得到(2.6)式的形式为止。在一般情况下,如果所给的线性方程组有惟一解,这个过程是可以完成的。把一般形式的线性方程组化为上三角形线性方程组的算法说明如下,其中数组 A 用于存放线性方程组 $Ax=b$ 的增广矩阵。

```
double A[M][N];
void ToTriangle()
{ int i,j,k;
  double temp;
  for(k=0;k<M;k++)
  { temp= A[k][k];
    for(j=k;j<N;j++) A[k][j]/=temp;
    for(i=k+1;i<M;i++)
    { temp= A[i][k];
      for(j=k;j<N;j++) A[i][j] -= temp * A[k][j];
    }
  }
```

```

    }
    return;
}

```

对于利用程序 8.01 所生成的线性方程组,我们可以接着编写一个专门的 C 语言函数 ToTriangle()来实现上面的算法 ToTriangle,相应的源代码见程序 8.03A。为了便于理解程序的执行流程,我们在程序中专门增加了显示功能。

程序 8.03A 把给定的线性方程组化为上三角形形式

```

int ToTriangle()
{
    int i,j,k;
    double x;
    if(SetPrintMatrix) PrintMatrix();
    for(k=0;k<M;k++)
    {
        x = A[k][k];
        for(j=k;j<N;j++) A[k][j] /= x;
        if(SetPrintMatrix) PrintMatrix(); /* NO1 */
        if(k==M-1) break;
        for(i=k+1;i<M;i++)
        {
            x=A[i][k];
            for(j=k;j<N;j++)
                A[i][j] -= x * A[k][j];
        }
        if(SetPrintMatrix) PrintMatrix(); /* NO2 */
    }
    return 0;
}

```

下面通过一个完整的程序说明函数 ToTriangle()的作用及用法。利用程序 8.01 和程序 8.03A 中的代码,只需给出 main()函数即可,相应的源代码见程序 8.03B。

程序 8.03B 专门调试 ToTriangle()的主程序

```

main()
{
    SEED=8000;
    FormProblem();
    SetPrintMatrix=1;
    ToTriangle();
    PrintMatrix();
}

```

执行程序 8.03B 过程中所显示的中间结果见表 2.2~2.6,它们中标有 A 的表格是程序 8.03A 中循环体内第 1 条 PrintMatrix()所显示的结果,也是程序执行算法 2.1 循环体内第 1 步的结果,标有 B 的则是第 2 条 PrintMatrix()所显示的结果,也是程序执行算法 2.1 循环体内第 2 步的结果。读者不难通过对这些表格的分析来理解程序的执行流程。

表 2.2 程序 8.03 所产生的初始增广矩阵

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	0.944 55	-2.817 16	-2.258 06	-2.892 24	1.511 58
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52
[2]	-7.561 57	-4.036 68	9.384 75	-8.644 98	1.232 64
[3]	-8.944 67	-5.311 14	-0.986 66	-4.109 32	0.610 68

表 2.3A 程序 8.03 循环体内 $k=0$ 时第 1 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52
[2]	-7.561 57	-4.036 68	9.384 75	-8.644 98	1.232 64
[3]	-8.944 67	-5.311 14	-0.986 66	-4.109 32	0.610 68

表 2.3B 程序 8.03 循环体内 $k=0$ 时第 2 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	-22.364 92	-5.061 20	-15.509 43	3.469 95
[2]	0.000 00	-26.589 47	-8.692 17	-31.798 78	13.333 60
[3]	0.000 00	-31.989 08	-22.370 06	-31.498 21	14.925 04

表 2.4A 程序 8.03 循环体内 $k=1$ 时第 1 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	-26.589 47	-8.692 17	-31.798 78	13.333 60
[3]	0.000 00	-31.989 08	-22.370 06	-31.498 21	14.925 04

表 2.4B 程序 8.03 循环体内 $k=1$ 时第 2 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	0.000 00	-2.674 96	-13.359 75	9.208 21
[3]	0.000 00	0.000 00	-15.130 91	-9.314 71	9.961 89

表 2.5A 程序 8.03 循环体内 $k=2$ 时第 1 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	0.000 00	1.000 00	4.994 38	-3.442 38
[3]	0.000 00	0.000 00	-15.130 91	-9.314 71	9.961 89

表 2.5B 程序 8.03 循环体内 $k=2$ 时第 2 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	0.000 00	1.000 00	4.994 38	-3.442 38
[3]	0.000 00	0.000 00	0.000 00	66.254 77	-42.124 39

表 2.6 程序 8.03 循环体内 $k=3$ 时第 1 条语句显示的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	0.000 00	1.000 00	4.994 38	-3.442 38
[3]	0.000 00	0.000 00	0.000 00	1.000 00	-0.635 79

评估程序 8.03A 的计算量也是十分容易的,注意到主循环体内的计算量实际上与增广矩阵中 a_{kk} 右下方的元素的个数成正比,而 a_{kk} 右下方的元素的个数为 $(n-k)(n+1-k)$,所以总计算量为

$$\sum_{k=0}^{n-1} (n-k)(n+1-k) \approx \sum_{k=0}^{n-1} (n-k)^2 = O(n^3/3) \quad (2.7)$$

8.2.3 求解上三角形线性方程组

对于求解形如(2.5)式的上三角形线性方程组来说,无论是手工计算还是编程计算,都不再是困难事情,按照逆推的顺序,对 $k=n, n-1, \dots, 2$,首先由第 k 个方程解出 x_k ,并从第一个方程到第 $k-1$ 个方程中立即消去 x_k ,从而问题得到解。

由于我们总是利用函数 ToTriangle()把线性方程组化为上三角形的,而且主对角线上的元素都是 1,可以称之为标准化的上三角形线性方程组,相应的算法可以说明如下。完整的源代码见程序 8.04A。

```
double A[M][N];
void SolutionTriangle()
{ int i,k;
  for(k=M;k>0;k--)
  { temp=A[k][M];
    for(i=0;i<k;i++) A[i][M]-=temp*A[i][k];
  }
  return;
}
```

程序 8.04A 求解标准化的上三角形线性方程组

```
void SolutionTriangle()
{ int i,j,k;
```

```

double x;
if(SetPrintMatrix)PrintMatrix();
for(k=M-1;k>0;k--)
{ for(i=k-1;i>=0;i--)
  { x= A[k][N-1];
    A[i][N-1]-=A[i][k]*x;
    A[i][k]=0.0;
  }
  if(SetPrintMatrix)PrintMatrix();
}
return;
}

```

程序 8.04A 的循环体内增加了一条语句“ $A[i][k]=0.0;$ ”从解方程组的角度看并无实际意义,不过它可使得调试程序时所显示的界面更漂亮些,即使对于实际应用的程序来说,增加这一条语句对计算时间也没有实质性的影响。

把程序 8.04A 和 8.03A 合在一起即可形成一个完整的求解线性方程组的方法,也就是高斯消去法。

随机产生一个 4 个变元的线性方程组,用高斯消去法求数值解的完整例程见程序 8.04B。为简单起见,我们只给出相应的 `main()` 函数,读者不难利用程序 8.01、8.03A 和 8.04A 得到一个完整的程序(源程序文件)。

程序 8.04B 完整的高斯消去法解线性方程组程序

```

main()
{ system("cls");
  SEED=8000;
  FormProblem();
  SetPrintMatrix=0;
  ToTriangle();
  SetPrintMatrix=1;
  SolutionTriangle();
  PrintMatrix();
}

```

程序 8.04B 前面部分与程序 8.03B 基本相同,所以执行 `SolutionTriangle()` 相当于在程序 8.03B 的基础上求解上三角形线性方程组。所显示的计算结果见表 2.7~2.10,表 2.7 与表 2.6 实际上完全相同。表 2.10 最后一列即为表 2.2 所表示的线性方程组的解。

表 2.7 程序 8.03B 所得到的上三角形线性方程组

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	-3.062 04	1.600 32
[1]	0.000 00	1.000 00	0.226 30	0.693 47	-0.155 15
[2]	0.000 00	0.000 00	1.000 00	4.994 38	-3.442 38
[3]	0.000 00	0.000 00	0.000 00	1.000 00	-0.635 79

表 2.8 消去 x_3 后所得到的增广矩阵

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	-2.390 63	0.000 00	-0.346 50
[1]	0.000 00	1.000 00	0.226 30	0.000 00	0.285 75
[2]	0.000 00	0.000 00	1.000 00	0.000 00	-0.266 98
[3]	0.000 00	0.000 00	0.000 00	1.000 00	-0.635 79

表 2.9 消去 x_2 后所得到的增广矩阵

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	-2.982 55	0.000 00	0.000 00	-0.984 75
[1]	0.000 00	1.000 00	0.000 00	0.000 00	0.346 17
[2]	0.000 00	0.000 00	1.000 00	0.000 00	-0.266 98
[3]	0.000 00	0.000 00	0.000 00	1.000 00	-0.635 79

表 2.10 消去 x_1 后所得到的增广矩阵

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	0.000 00	0.000 00	0.000 00	0.047 72
[1]	0.000 00	1.000 00	0.000 00	0.000 00	0.346 17
[2]	0.000 00	0.000 00	1.000 00	0.000 00	-0.266 98
[3]	0.000 00	0.000 00	0.000 00	1.000 00	-0.635 79

对于求解一般的 n 个变元的上三角形线性方程组来说,注意到计算量大体等于非零元素的个数,所以计算量为 $O(n^2/2)$,相对于化为上三角形的计算量而言由于低一个数量级,因而可以忽略不计。所以用高斯消去法求解 n 个变元的线性方程组的计算量为 $O(n^3/3)$ 。可以说,在所有求解线性方程组的方法中,高斯消去法的计算量是最小的,不过在计算机的速度越来越快的今天,高斯消去法的这种优势地位会逐步下降。

8.3 选主元高斯消去法

对于求解 n 个变元, n 个线性方程所组成的线性方程组 $Ax=b$ 来说,高斯消去法的基本思路是对 $k=1,2,\dots,n$, 首先将第 k 个方程两边同时除以 x_k 的系数 a_{kk} , 接下来再进一步把 $a_{kk+1}, \dots, a_{nk+1}$ 化为零, 从而把方程组 $Ax=b$ 化为等价的上三角形线性方程组。由此引出的问题是如果有某个 a_{kk} 为零或绝对值充分小, 那么计算过程可能被迫终止, 或者是相应的数值性能会变坏, 因此需要一种解决的办法, 这就是选主元的来由。

作为改进高斯消去法的一个技巧, 我们只推荐选列主元的方法, 它既能明显改善高斯消去法的性能, 而且实现的方法相对说来也显得特别简单。对一般性的应用来说, 利用双精度数计算, 再结合选列主元的方法, 总的说来效果都还是很不错的。

8.3.1 选列主元的方法

由线性方程组的性质可知,互换方程组中任意两个方程的位置,所得到的新方程组的解不变。选列主元的思想就是依次对 $k=1,2,\dots,n$,通过有选择地互换方程组中两个方程的位置而把 $a_{kk}, a_{(k+1)k}, \dots, a_{nk}$ 中绝对值最大的元素移到主对角线上来,从而改进高斯消去法的性能。

对于程序 8.01 所生成的线性方程组来说,假如对任意给定的 k 值选列主元,相应的 C 语言函数可以说明为

```
double GetMainElement(int k)
```

所要完成的操作为:首先找到主元所在的行;接下来把主元所在的行与第 k 行的元素互换。为了提高代码的效率,函数的返回值为该主元的绝对值是合理的。

选列主元的方法还是很容易理解的,只是编程有些小麻烦,为此可以定义两个特殊变量:用一个标志旗 FLAG 指示到目前为止所找到的最好的主元的位置;用一个特别的变量 MEAV 记住到目前为止所找到的最好的主元的绝对值,程序中只需让每个主元的候选者与 MEAV 比较绝对值的大小即可。这个方法有点像擂台赛,FLAG 相当于擂主,最后持有 FLAG 者即为最终的优胜者,也就是我们要找的主元。找到主元后,互换增广矩阵相应的行就很容易了。完整的源代码见程序 8.05A。

程序 8.05A 选主元方法

```
double GetMainElement(int k)
{ int i,j,FLAG;
  double temp,MEAV;
  FLAG=k;
  MEAV=fabs(A[k][k]);
  for(i=k+1;i<M;i++)
  { temp=fabs(A[i][k])
    if(temp<=MEAV)continue;
    FLAG=i;
    MEAV=temp;
  }
  for(j=0;j<N;j++)
  { temp=A[k][j];
    A[k][j]=A[FLAG][j];
    A[FLAG][j]=temp;
  }
  return MEAV;
}
```

GetMainElement() 函数把主元的绝对值作为返回值是很有意义的,如果函数的返回值充分小,说明所讨论的方程组是病态的,或者用高斯消去法不能解决问题,或者所得到的结果有较大的误差,所以调用程序可以根据返回值的大小来决定程序的流向。由于这

种情况一般不会出现,目前可以只给一个错误信息,如果在求解具体问题时真的遇到这种情况,也可以由人工结合实际问题的特点寻找具体的解决办法。

8.3.2 集成选主元方法的高斯消去法

在高斯消去法中集成选主元的方法还是很容易做到的,由于选主元的操作只是在化为上三角形的过程中采用,所以只要改写高斯消去法中的 ToTriangle() 函数即可,其实也就是在适当位置调用选主元的函数 GetMainElement()。

既然函数 GetMainElement() 除了完成选主元的功能外还能返回主元的信息,如果主元的值太小或者为零,那么就不需要再接着往下计算,而是把错误信息返回给更高层的调用程序,所以为了充分利用选主元所得到的结果,有必要对程序 8.03A 中的 ToTriangle() 作更多一点的修改。函数名还是保持不变,完整的源代码见程序 8.05A。

对于返回“错误信息”的函数来说,一般可说明为整型,程序中可以说一个特殊变量“ErrCode”(错误代码),用以记录程序的运行状况,最后返回给调用程序。一般的说,如果程序运行正常,则把返回的 ErrCode 值设为零;如果程序运行不正常,则根据错误的类型给 ErrCode 赋不同的值。程序中的函数 Gauss() 可用来形成一个完整的高斯消去法子程序,目前虽然意义不是很大,但对于下一节编写应用模块来说,还是很有必要的。另外,作为编程技巧,程序根据变量 SetSelectMain 的值决定是否执行选主元的操作,这样更容易对比分析选主元的结果之间的差异。

程序 8.05B 集成选主元的高斯消去法

```
int ToTriangle()
{ int i,j,k;
  int ErrCode=0;
  double x,test=0;
  if(SetPrintMatrix) PrintMatrix();
  for(k=0;k<RM;k++)
  { if(SetSelectMain)
    { test=GetMainElement(k);
      if(SetPrintMatrix) PrintMatrix();
      if(test<ZERO){ErrCode=k+1;break;}
    }
    x = A[k][k];
    for(j=k;j<RN;j++) A[k][j] /= x;
    if(SetPrintMatrix) PrintMatrix();
    if(k==RM-1)break;
    for(i=k+1;i<RM;i++)
    { x=A[i][k];
      for(j=k;j<RN;j++)
        A[i][j] -= x * A[k][j];
    }
    if(SetPrintMatrix) PrintMatrix();
  }
}
```

```

        if(ErrCode>0)PrintMatrix();
        return ErrCode;
    }

    int Gauss()
    { int ErrCode=0;
      ErrCode=ToTriangle();
      if(!ErrCode)SolutionTriangle();
      return ErrCode;
    }

```

8.3.3 一个完整的选主元高斯消去法计算案例

利用程序 8.01、8.04A、8.05A、8.05B 即可形成一个完整的选主元高斯消去法解线性方程组程序,相应的 main()函数见程序 8.05C。程序首先随机生成 4 个变元的线性方程组的增广矩阵,把变量 SetPrintMatrix 和 SetSelectMain 的值都设置为 1,接下来调用函数 Gauss()求解所生成的线性方程组。运行程序 8.05C 所显示的前 4 个表格见表 3.1~3.4,它们已经体现了选主元的执行流程。

程序 8.05C 选主元高斯消去法例程

```

main()
{ int ErrCode=0;
  system("cls");
  SEED=8000;
  FormProblem();
  SetPrintMatrix=1;
  SetSelectMain=1;
  ErrCode=Gauss();
  printf("\nErrCode=%2d",ErrCode);
  getchar();
}

```

表 3.1 初始增广矩阵

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	0.944 55	-2.817 16	-2.258 06	-2.892 24	1.511 58
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52
[2]	-7.561 57	-4.036 68	9.384 75	-8.644 98	1.232 64
[3]	-8.944 67	-5.311 14	-0.986 66	-4.109 32	0.610 68

表 3.2 执行选主元的结果

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	-8.944 67	-5.311 14	-0.986 66	-4.109 32	0.610 68
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52
[2]	-7.561 57	-4.036 68	9.384 75	-8.644 98	1.232 64
[3]	0.944 55	-2.817 16	-2.258 06	-2.892 24	1.511 58

表 3.3 高斯消去法(1)

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	0.593 78	0.110 31	0.459 42	-0.068 27
[1]	-6.207 16	-3.851 74	9.777 83	3.497 12	-6.463 52
[2]	-7.561 57	-4.036 68	9.384 75	-8.644 98	1.232 64
[3]	0.944 55	-2.817 16	-2.258 06	-2.892 24	1.511 58

表 3.4 高斯消去法(2)

$A[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	1.000 00	0.593 78	0.110 31	0.459 42	-0.068 27
[1]	0.000 00	-0.166 07	10.462 52	6.348 78	-6.887 29
[2]	0.000 00	0.453 20	10.218 84	-5.171 08	0.716 39
[3]	0.000 00	-3.378 01	-2.362 26	-3.326 18	1.576 07

8.4 高斯消去法应用模块的编写

我们已经掌握了用高斯消去法解线性方程组,并且通过案例计算验证了方法和程序的正确性。接下来要进一步作好的工作是利用高斯消去法编写模块程序,这样,“别的”应用程序可以通过与模块的连接而让高斯消去法在应用程序中发挥作用。

编写模块程序的一个基本规则是问题的定义方式由用户给出,比如有的问题很容易直接给出线性方程组的增广矩阵,有的问题是把线性方程组的系数矩阵和常数向量分开给出的,所以我们还应学会根据原始数据的不同的组织形式编写不同的应用程序。另外有些问题所提供的原始数据需要保留下来,这也会对模块程序提出不同的要求。

由于线性方程组的用途非常广泛,一下子完成对所有不同的需求编写相应的模块也不太现实。在这一节中,只讨论最一般形式的模块程序的编写方法;在下一章中,还将结合曲线拟合介绍相应的模块程序的编写方法。有了这些范例,读者也能够模仿着编写具有自己风格的模块程序来。

8.4.1 二维数组的再现方式

对于编写求解线性方程组的模块程序来说,调用程序通常只提供存放线性方程组的系数矩阵或增广矩阵的二维数组的首地址以及矩阵的行数(以及列数,如果需要的话)。高级的程序员当然会利用这些信息在程序中动态地计算他所需要的某个元素的位置。按这种方式编写程序一是编写比较麻烦,二是编出来的程序的可读性很差。其实现在的计算机资源已经非常丰富了,完全没有必要把编程搞得那么辛苦,那么枯燥。

还是通过一个例子来说明如何在子程序中轻松地再现主程序传送过来的二维数组,

相应的源代码见程序 8.06A。程序中说明一个长度为 50^① 的 double 型指针数组 PA, 两个整型全局变量 PM, PN; 函数

```
int ShowMatrix(double * A, int M, int N)
```

巧妙地利用指针把主程序中的二维数组 $A[M][N]$ 等价地再现成本模块中的二维数组 $PA[PM][PN]$ 。方法是把 M, N 的值赋给本模块中的变量 PM, PN , 并令指针 $PA[0], PA[1], \dots, PA[PM-1]$ 依次指向数组 A 的元素 $A[0][0], A[1][0], \dots, A[M-1][0]$, A 实际上是调用程序中的二维数组 A 的第一个元素 $A[0][0]$ 的地址。

函数 ShowMatrix() 通过调用本模块中的显示子程序 PrintMatrixPA() 来显示所形成的二维数组 PA , 如果所显示的 PA 与主程序中显示的 A 完全一致, 说明我们在子程序中再现了主程序中的二维数组 A 。程序中的 PrintMatrixPA() 函数只是把程序 8.01 中的 PrintMatrix() 函数中的数组名 A , 变量名 RM, RN 分别改写为本模块中的数组名 PA 和变量名 PM, PN 而已, 这正好表明 PA 可以看成是与主程序中的 A 完全相同的二维数组。

程序 8.06A 显示一个二维数组的模块程序

```
#include <stdio.h>
double * PA[50];
int PM, PN;
void PrintMatrixPA()
{ int i, j;
  printf("\n=====");
  printf("=====");
  printf("\nPA[i][j]");
  for(j=0; j<PN; j++) printf(" [%1d] ", j);
  printf("\n-----");
  printf("-----");
  for(i=0; i<PM; i++)
  { printf("\n [%1d] ", i);
    for(j=0; j<PN; j++)
      printf("%13.5f", PA[i][j]);
  }
  printf("\n=====");
  printf("=====");
  getch();
}
int ShowMatrix(double * A, int M, int N)
```

① 对一般的应用问题来说, 确定这里的指针数组的长度可以处理得宽松一些。对于 C 语言编程来说, 一般可以解不超过 50 个变量的线性方程组, 此时指针数组占用的存储空间也就 100 字节, 相对于 64KB 来说, 还是微不足道的。如果用 C++ 编程, 即使把指针数组的长度说明为 1 000 甚至更多, 也只占用几个 KB 的存储空间, 相对于配备数百 MB 内存的计算机来说, 仍然是微不足道的。而采用指针数组来编程, 相应的程序易编、易读、易理解, 由此换来的好处显而易见。现在学习计算方法不必把节省计算机的内存过于看重; 相反, 应该充分利用计算机提供的条件把事情做得更好些。


```

{ int k, ErrCode=0;
  if(M>50) return -1;
  PM=M;PN=N;
  /* Following is The Most Important Line */
  for(k=0;k<PM;k++)PA[k]=A+k*PN;
  PrintMatrixPA();
  return ErrCode;
}

```

程序 8.06A 可以专门作为一个模块保存,模块(文件)名可采用“Matrix. C”,用户的应用程序即可通过文件包含而调用这里的显示矩阵的程序。为了说明程序 8.06A 的用法和效果,可以利用程序 8.01 的代码,在最前面增加一行

```
#include "Matrix. C"
```

改写其中的 main() 函数,相应的源代码见程序 8.06B(这里只列出改写的部分)。

程序 8.06B 矩阵显示模块的测试程序

```

#include "MATRIX. C"
:
main()
{ FormProblem();
  PrintMatrix();
  ShowMatrix(&A[0][0],RM,RN);
}

```

运行程序 8.06B 时,屏幕上的显示结果如图 4.1 所示,图中的两个表格分别为主程序中的 PrintMatrix() 函数和模块文件“Matrix. C”中的 PrintMatrixPA() 函数所显示的结果,两者完全相同说明我们的目的达到了。

A[i][j]	[0]	[1]	[2]	[3]	[4]
[0]	0.94455	-2.81716	-2.25806	-2.89224	1.51158
[1]	-6.20716	-3.85174	9.77783	3.49712	-6.46352
[2]	-7.56157	-4.03668	9.38475	-8.64498	1.23264
[3]	-8.94467	-5.31114	-0.98666	-4.10932	0.61068
PA[i][j]	[0]	[1]	[2]	[3]	[4]
[0]	0.94455	-2.81716	-2.25806	-2.89224	1.51158
[1]	-6.20716	-3.85174	9.77783	3.49712	-6.46352
[2]	-7.56157	-4.03668	9.38475	-8.64498	1.23264
[3]	-8.94467	-5.31114	-0.98666	-4.10932	0.61068

图 4.1 程序 8.06B 的运行结果

8.4.2 利用指针给出增广矩阵的高斯消去法

利用 8.4.1 小节中给出的再现二维数组的技巧,我们可以利用指针数组连同相应的表示(向量的)维数的变量来确认给定的线性方程组的增广矩阵,从而可以直接模仿前面两节介绍的方法编写应用程序。

对于编写应用程序来说,把高斯消去法的两个过程合并在同一个 C 语言函数中是合理的,函数可以说明为 int 型,函数名可以用 Gauss()。

由于选列主元的算法并不复杂,但可以明显改善程序的性能。更重要的是选列主元可以避免方程本身有解,而高斯消去法可能失败的尴尬。

利用指针数组实现高斯消去法解线性方程组的源代码见程序 8.07A,作为同时兼具带有研究性质的程序,我们还是让程序具有自由选择是否显示详细计算过程的功能。

程序 8.07A 利用指针数组实现高斯消去法

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#ifndef ZERO
    #define ZERO 1.0E-30
#endif
double * PA[50];
int PM,PN;
int SetPrintMatrix=0;
PrintMatrix(){...}
double GetMainElement(int K){...}
int Gauss()
{ int i,j,k;
  double x,test=0;
  if(SetPrintMatrix) PrintMatrix();
  for(k=0;k<PM;k++)
  { test=GetMainElement(k);
    if(SetPrintMatrix) PrintMatrix();
    if(test<ZERO){return k+1;}
    x = PA[k][k];
    for(j=k;j<PN;j++)PA[k][j] /= x;
    if(SetPrintMatrix) PrintMatrix();
    if(k==PM-1)break;
    for(i=k+1;i<PM;i++)
    { x=PA[i][k];
      for(j=k;j<PN;j++)
        PA[i][j] -= x * PA[k][j];
    }
  }
  if(SetPrintMatrix)PrintMatrix();
  for(k=PM-1;k>0;k--)
  { for(i=k-1;i>=0;i--)
    { PA[i][PN-1]-=PA[i][k] * PA[k][PN-1];
      PA[i][k]=0.0;
    }
    if(SetPrintMatrix)PrintMatrix();
  }
  return 0;
}
```

程序中的 PrintMatrix()函数和 GetMainElement()函数可由程序 8.01 和 8.05A 中的同名函数改写而来,也只是把数组名 A 和变量名 M, N 相应地改写为模块中的数组名 PA 和变量名 PM,PN,读者可以模仿着完成改写工作。

在程序 8.07A 中的全局变量 PA,PM,PN 还没有赋值之前,还不能调用 Gauss()函数,这个问题下面将接着讨论。

8.4.3 解线性方程组的 C 语言函数

我们在第 6 章 6.5 节中给出了用 Newton_Raphson 方法解非线性方程组的模块化程序设计方法,其中有一个专门的模块用于解线性方程组,相应的 C 语言函数的原型为

```
double SolutionLEG(double * A,double * X,int n)
```

其中二维数组 A 为存放线性方程组的增广矩阵 A ,数组 X 用于存放线性方程组的解向量 x , n 为向量 x 的维数,函数的返回值为 x 的模。现在,我们可以利用程序 8.07A 给出的高斯消去法完成这个模块的编写。

函数 SolutionLEG()要完成的任务其实已经不多了,根据主程序传过来的参数给程序 8.07A 中的全局变量赋值、调用 Gauss()函数、根据函数的原型返回结果即可。相应的源代码见程序 8.07B。

程序 8.07B 高斯消去法解线性方程组的“接口”程序

```
double SolutionLEG(double * A,double * X,int m)
{ double norm=0.0;
  int k,ErrCode=0;
  if(m>50) return -1;
  PM=m;PN=PM+1;
  for(k=0;k<PM;k++)PA[k]=A+k*PN;
  ErrCode=Gauss();
  if(ErrCode>0) return -2;
  for(k=0;k<PM;k++)
  { X[k]=PA[k][PN-1];
    norm+=X[k]*X[k];
  }
  return sqrt(norm);
}
```

把程序 8.07B 和 8.07A 写在同一个源程序文件中,用文件名“LEG_GAUS.C”保存即可完成高斯消去法解线性方程组的模块程序的编写,应用程序通过文件包含即可利用函数 SolutionLEG()解线性方程组。作为一个直接应用,读者可以把这里得到的模块文件“LEG_GAUS.C”复制一份放在程序 6.03D 所在的目录中(不必更改文件名),并把程序 6.03D 中的预处理行

```
#include "NLEG_LEG.C"
```

改写为

```
#include "LEG_Gaus.C"
```

即可得到正式的 LEG 求解模块。

8.4.4 模块文件“LEG_Gaus.C”应用举例

程序 8.07C 通过简单的文件包含实现与应用模块 LEG_Gaus.C 的连接。Main() 函数首先调用函数 FormProblem() 生成 4 个变元的线性方程组的增广矩阵；置 SetPrintMatrix 的值为 1，以便程序运行时动态跟踪程序的执行流程；接下来调用 SolutionLEG() 函数求解相应的线性方程组；最后显示结果。执行程序 SolutionLEG() 函数所显示的过程与程序 8.05C 的完全相同，这也是预料之中的。

程序 8.07C 调用 Gauss() 函数求解线性方程组的例程

```
#include "LEG_Gaus.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MM 4
#define MN 5
unsigned SEED=8000;
unsigned int RM=MM;
double A[MM][MN],x[MM];
void FormProblem(){...}
main()
{ double norm;
  int k;
  SEED=8000;
  FormProblem();
  SetPrintMatrix=1;
  norm=SolutionLEG(&A[0][0],x,RM);
  printf("\nnorm= %12.4e",norm);
  getchar();
  for(k=0;k<RM;k++)printf("\nx[%2d] = %24.16E",k,x[k]);
  getchar();
}
```

运行程序 8.07C 最后显示的结果见图 4.2 所示，图中所显示的求解结果与表 2.10 最后一列大体相同(差别是由于不同的显示格式引起的)。至此，我们对高斯消去法的介绍划上了一个圆满的句号。

```
norm= 7.731e-01
x[ 0] = 4.772231548793160E-02
x[ 1] = 3.461713817314431E-01
x[ 2] = -2.669803414684183E-01
x[ 3] = -6.357940183394505E-01
```

图 4.2 运行程序 8.07C 最后显示的结果

8.5 高斯-约当(Gauss-Jordan)消去法

对于求解 n 个变元, n 个线性方程所组成的线性方程组 $Ax=b$ 来说, 高斯消去法的关键是首先把线性方程组化为上三角形线性方程组, 也就是对 $k=1, 2, \dots, n$, 利用 a_{kk} 不为零来消去 $a_{(k+1)k}, \dots, a_{nk}$, 不急于消去 $a_{1k}, \dots, a_{(k-1)k}$ 仅仅是为了减少计算量, 并非一定要留下一个回代过程。如果对 $k=1, 2, \dots, n$, 利用 a_{kk} 不为零同时消去 $a_{1k}, \dots, a_{(k-1)k}, a_{(k+1)k}, \dots, a_{nk}$, 显然, 相应的计算方法还简单一些。这就是约当消去法的基本思想。约当消去法通常也称为高斯-约当消去法或无回代过程的高斯消去法。

大概是由于以前人们使用的计算工具非常落后, 所以计算量较小的计算方法更受欢迎。在现代的计算机条件下, 为了节省不太多的计算量而使程序变复杂的做法并不可取。约当消去法虽然计算量比高斯消去法稍大一些, 但由于算法更简单, 编程的方式更灵活, 还可用来求矩阵的逆, 以后还可用来求解有无数组解的线性方程组, 所以约当消去法的潜在价值实际上已经超过了高斯消去法。

8.5.1 约当消去法的算法说明

假如求解一般的 n 个变元 n 个方程的线性方程组 $Ax=b$, 假如数组 A 存放方程组的增广矩阵的元素, 约当消去法的算法说明如下。

```
double A[M][N];
void Jordan()
{ int i, j, k;
  double temp;
  for(k = 0; k < M; k++)
  { temp = A[k][k];
    for(j = k; j < N; j++) A[k][j] /= temp;
    for(i = 0; i < M; i++)
    { if(i == k) continue;
      temp = A[i][k];
      for(j = k; j < N; j++) A[i][j] -= temp * A[k][j];
    }
  }
  return;
}
```

不难理解, 约当消去法主循环体内当循环参数为 k 时, 其计算量与增广矩阵第 k 列到最后一列的元素的个数大体相同, 由此不难得到总计算量为

$$\sum_{k=1}^n n(n+1-k) = n \sum_{k=1}^n k \approx n^3/2$$

所以高斯-约当消去法的计算量为 $O(n^3/2)$, 相对于高斯消去法来说, 大约多出 50%。

由于高斯-约当消去法实际上是高斯消去法加以简化而形成的方法, 所以可以直接由

选主元的高斯消去法程序得到选主元的高斯-约当消去法程序;也可以直接由高斯消去法的模块程序得到高斯-约当消去法的模块程序。

8.5.2 约当消去法解线性方程组的模块程序

由于约当消去法实际上比高斯消去法简单,所以只需把 8.4.3 节中得到的高斯消去法解线性方程组的模块程序文件“LEG_GAUS.C”用文件名“LEG_Jord.C”复制一份,然后再作些修改即可。主要改写的内容是把程序 8.07A 中的 Gauss()函数改写为 Jordan()函数,相应的源代码见程序 8.08。另外还应把程序 8.07B 中的 SolutionLEG()函数中的函数调用语句

```
ErrCode=Gauss();
```

改写为

```
ErrCode=Jordan();
```

其余一概不变,所得到的也是选主元的约当消去法的模块程序。由于“接口”函数 SolutionLEG()的参数约定完全相同,所以两个模块程序可以互相替代,真正体现了“模块”的含义。

程序 8.08 选主元约当消去法中的 Jordan()函数

```
int Jordan()
{ int i,j,k;
  double x,test=0;
  if(SetPrintMatrix) PrintMatrix();
  for(k=0;k<PM;k++)
  { test=GetMainElement(k);
    if(SetPrintMatrix) PrintMatrix();
    if(test<ZERO){return k+1;}
    x = PA[k][k];
    for(j=k;j<PN;j++)PA[k][j] /= x;
    if(SetPrintMatrix) PrintMatrix();
    for(i=0;i<PM;i++)
    { if(i==k) continue;
      x=PA[i][k];
      for(j=k;j<PN;j++)
        PA[i][j] -= x * PA[k][j];
    }
  }
  if(SetPrintMatrix)PrintMatrix();
  return 0;
}
```

读者可以简单地把程序 8.07C 中的第一个预编译行

```
#include "LEG_GAUS.C"
```

改写为

```
#include "LEG_Jord.C"
```

即可得到与程序 8.08 联编的应用例程。

作为模块的一个直接应用,读者可以把这里所得到的模块文件“LEG_Jord.C”复制一份放在程序 6.03D 所在的目录中(不必更改文件名),并把程序 6.03D 中相应的预处理行改写为

```
#include "LEG_Jord.C"
```

即可实现高斯-约当消去法在解非线性方程组中的应用。可以预期,仅从屏幕显示的信息看,并不能断言所采用的到底是高斯消去法还是高斯-约当消去法。

8.5.3 利用约当消去法求矩阵的逆

由线性代数中的结果可以得知,我们利用线性方程组 $Ax=b$ 的增广矩阵 (A,b) 来解线性方程组,实际上是对矩阵 (A,b) 进行一系列的行变换并使其成为 (I,x) ,其中 I 是单位矩阵。注意到对任何一个矩阵进行行变换相当于左乘了一个与 A 的行数相同的一个可逆的方阵,尽管我们目前还不知道这个矩阵中每个元素的具体值,但是可以首先把它记为 B 再说。

如果利用约当消去法解线性方程组 $Ax=b$,我们有

$$B(A,b)=(I,x)$$

利用矩阵分块相乘的运算规则立即可以得到 $BA=I$,从而有 $B=A^{-1}$ 。这意味着如果用约当消去法解线性方程组 $Ax=b$,实际效果相当于将原方程组的增广矩阵左乘了 A 的逆 A^{-1} ,从而直接得到 $x=A^{-1}b$,问题是如何将 $B=A^{-1}$ 保存下来。

对于 $M \times M$ 可逆矩阵 A ,可以构造 $M \times 2M$ 矩阵 $T=(A,I)$,那么用约当消去法相当于把 T 的左边 M 列化为单位矩阵,其效果相当于用 A^{-1} 左乘 T ,从而得到 $A^{-1}(A,I)=(I,A^{-1})$,所以 T 的右边的 M 列就是所要求的 A^{-1} 。

我们一开始编程就注意到了对矩阵(二维数组)的行数和列数用相互独立的变量名来表示,所以用约当消去法把一个线性方程组的增广矩阵的左边化为单位矩阵,与把任何一个列数大于行数的矩阵的左边化为单位矩阵的方法是一样的,包括选主元的方法也是一样的,所以可以把前面得到的模块程序文件“LEG_Jord.C”首先用文件名“MATRXINV.C”复制一份,接下来再进行一些简单的修改。

对于求矩阵 A 的逆 $B=A^{-1}$ 来说,可以把相应的 C 语言函数说明为

```
int MatrixInversion(double * A, double * B, int n)
```

是合理的,其中数组 A, B 分别存放矩阵 A, B 的元素, n 是矩阵的阶数。函数首先申请存放矩阵 T 的元素所需要的存储空间,并利用指针数组而得存放矩阵 T 所需要的二维数组数 PA ,相应的源代码见程序 8.09。如果希望跟踪程序的运行,可以修改 PrintMatrix() 函数,把数组 PA 按两个矩阵分别显示,读者可以自己完成。我们现在的目的只是展示程序的构架,读者可以直接通过阅读源代码来加深对计算方法的理解。

程序 8.09 矩阵求逆的模块程序

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <math.h>
#ifndef ZERO
    #define ZERO 1.0E-30
#endif
double *PBUF, *PA[25], *PW[25];
int PM, PN;
int SetPrintMatrix=0;
int MatrixInversion(double *A, double *B, int n)
{
    int i, j, k, ErrCode=0;
    if(n>25) return -1;
    PM=n; PN=PM*2;
    PBUF=malloc(PM*PN*sizeof(double));
    if(!PBUF) return -2;
    for(k=0; k<PN; k++)
    {
        PW[k]=A+k*PM;
        PA[k]=PBUF+k*PN;
    }
    for(i=0; i<PN; i++)
    {
        for(j=0; j<PM; j++)
        {
            PA[i][j]=PW[i][j];
            PA[i][j+PM]=0.0;
        }
        PA[i][i+PM]=1.0;
    }
    ErrCode=Jordan();
    for(i=0; i<PM; i++)
    {
        PW[i]=B+i*PM;
        for(j=0; j<PM; j++) PW[i][j]=PA[i][j+PM];
    }
    return ErrCode;
}

```

把程序 8.09 以文件名“MatrxINV. C”保存, 应用程序即可通过文件包含而利用语句行

```
ErrorCode=MatrixInversion(&A[0][0], &B[0][0], n);
```

完成数学计算 $B=A^{-1}$ 。其中 A, B 分别为存放矩阵 A, B 的二维数组。

8.6 追赶法解三对角线性方程组

如果线性方程组 $Ax=b$ 的系数矩阵除了主对角线和次主对角线上的元素外, 其余的元素均为零, 则称为是三对角线性方程组。三对角线性方程组的一般形式为

$$\left. \begin{array}{rcl} b_1 x_1 + c_1 x_2 & & = d_1 \\ a_2 x_1 + b_2 x_2 + c_2 x_3 & & = d_2 \\ & a_3 x_2 + b_3 x_3 + c_3 x_4 & = d_3 \\ & \ddots & \vdots \\ & a_{n-2} x_{n-3} + b_{n-2} x_{n-2} + c_{n-2} x_{n-1} & = d_{n-2} \\ & a_{n-1} x_{n-2} + b_{n-1} x_{n-1} + c_{n-1} x_n & = d_{n-1} \\ & a_n x_{n-1} + b_n x_n & = d_n \end{array} \right\}$$

由于方程组中的非零元素沿系数矩阵的主对角线呈带状分布,所以也称为是带形线性方程组。

许多大规模的线性方程组的系数矩阵正好是三对角矩阵的形式,所以利用它的特殊性而形成更有效的方法就很有意义。对于 n 个变元的三对角线性方程组,首先可以把它写成更紧凑的格式

$$\left. \begin{array}{rcl} & b_1 x_1 & + c_1 x_2 = d_1 \\ a_2 x_1 & + b_2 x_2 & + c_2 x_3 = d_2 \\ a_3 x_2 & + b_3 x_3 & + c_3 x_4 = d_3 \\ & \vdots & \\ a_{n-2} x_{n-3} & + b_{n-2} x_{n-2} & + c_{n-2} x_{n-1} = d_{n-2} \\ a_{n-1} x_{n-2} & + b_{n-1} x_{n-1} & + c_{n-1} x_n = d_{n-1} \\ a_n x_{n-1} & + b_n x_n & = d_n \end{array} \right\} \quad (6.1)$$

因此,如果利用计算机求解,可以说明 4 个长度为 n 的一维数组 A, B, C, D 来存储所有的非零元素。注意到方程组中的 a_1 和 c_n 没有定义,可以简单地把数组中相应的元素设为零。

8.6.1 解的递推形式

对于(6.1)式给出的三对角线性方程组,不难建立解的递推格式。为了讨论方便,假定推导过程中所需要的条件(分母不为零)都是满足的。首先由

$$b_1 x_1 + c_1 x_2 = d_1$$

不难得到

$$x_1 = \frac{d_1}{b_1} - \frac{c_1}{b_1} x_2 \quad (6.2)$$

再令

$$u_1 = \frac{d_1}{b_1}, \quad v_1 = \frac{c_1}{b_1} \quad (6.3)$$

从而有

$$x_1 = u_1 - v_1 x_2$$

对 $k=2, 3, \dots, n$, 令

$$\left. \begin{aligned} u_k &= \frac{d_k - a_k u_{k-1}}{b_k - a_k v_{k-1}} \\ v_k &= \frac{d_k - a_k v_{k-1}}{b_k - a_k v_{k-1}} \end{aligned} \right\} \quad (6.4)$$

可以证明

$$x_k = u_k - v_k x_{k+1} \quad (6.5)$$

从而可以形成一个算法。

下面将用数学归纳法证明, 对 $k=2, 3, \dots, n$, (6.4) 式和 (6.5) 式总是成立。

当 $k=2$ 时, 利用

$$\begin{cases} x_1 = u_1 - v_1 x_2 \\ a_2 x_1 + b_2 x_2 + c_2 x_3 = d_2 \end{cases} \quad (6.6)$$

不难得到

$$x_2 = \frac{d_2 - a_2 u_1}{b_2 - a_2 v_1} - \frac{c_2}{b_2 - a_2 v_1} x_3$$

在 (6.4) 式中取 $k=2$ 即得

$$u_2 = \frac{d_2 - a_2 u_1}{b_2 - a_2 v_1}, \quad v_2 = \frac{c_2}{b_2 - a_2 v_1} \quad (6.7)$$

由 (6.6) 式和 (6.7) 式即可得到

$$x_2 = u_2 - v_2 x_3$$

所以结论对于 $k=2$ 也是成立的。

假如当 $k=K (K < n)$ 时结论成立, 即 $x_K = u_K - v_K x_{K+1}$, 其中

$$\begin{cases} u_K = \frac{d_K - a_K \cdot u_{K-1}}{b_K - a_K \cdot v_{K-1}} \\ v_K = \frac{c_K}{b_K - a_K \cdot v_{K-1}} \end{cases}$$

那么当 $k=K+1$ 时由

$$\begin{cases} x_K = u_K + v_K x_{K+1} \\ a_{K+1} x_K + b_{K+1} x_{K+1} + c_{K+1} x_{K+2} = d_{K+1} \end{cases} \quad (6.8)$$

可推得

$$x_{K+1} = \frac{d_{K+1} - a_{K+1} u_K}{b_{K+1} - a_{K+1} v_K} + \frac{c_{K+1}}{b_{K+1} - a_{K+1} v_K} x_{K+2} \quad (6.9)$$

在 (6.4) 式中取 $k=K+1$ 即得

$$u_{K+1} = \frac{d_{K+1} - a_{K+1} u_K}{b_{K+1} - a_{K+1} v_K}, \quad v_{K+1} = \frac{c_{K+1}}{b_{K+1} - a_{K+1} v_K} \quad (6.10)$$

由 (6.9) 式和 (6.10) 式即可得到

$$x_{K+1} = u_{K+1} - v_{K+1} x_{K+2}$$

所以对 $k=2, 3, \dots, n-1$, (6.4) 式和 (6.5) 式都是成立的。

最后利用

$$\begin{cases} x_{n-1} = u_{n-1} + v_{n-1} x_n \\ a_n x_{n-1} + b_n x_n = d_n \end{cases}$$

不难推出

$$x_n = \frac{d_n - a_n u_{n-1}}{b_n - a_n v_{n-1}} = u_n \tag{6.11}$$

8.6.2 算法说明

对于求(6.1)式的数值解来说,首先利用(6.3)式和(6.4)式依次计算各 u_k, v_k ,亦即

$$\left. \begin{aligned} u_1 &= \frac{d_1}{b_1} & v_1 &= \frac{c_1}{b_1} \\ u_k &= \frac{d_k - a_k u_{k-1}}{b_k - a_k v_{k-1}} \\ v_k &= \frac{d_k - a_k v_{k-1}}{b_k - a_k v_{k-1}} \end{aligned} \right\} \forall k=2,3,\cdots,n$$

在程序中可以说两个长度为 n 的数组 U 和 V 分别将 u_k, v_k 保存下来。接下来再利用(6.6)式,(6.5)式以及(6.2)式按逆推的顺序依次计算 $x_n, x_{n-1}, \cdots, x_1$,亦即

$$\begin{cases} x_n = u_n \\ x_k = u_k - v_k x_{k+1}, \quad \forall k=n-1, n-2, \cdots, 1 \end{cases}$$

在程序中说明一个长度为 n 的数组 X 保存相应的结果,从而完成求(1.1)的数值解。

把上面的第一个过程称为追过程,把第二个过程称为赶过程,故把所形成的方法称为追赶法。下面通过一个具体例子说明相应的列表计算方法。

例 6.1 利用追赶法求解下面的三对角线性方程组

$$\begin{cases} 4x_1 & + & 2x_2 & & =16 \\ 2x_1 & + & 5x_2 & + & x_3 & =17 \\ & & 3x_2 & + & 7x_3 & =13 \end{cases}$$

解:采用列表的格式计算,主要源代码见程序 8.10,读者可以把省略的代码补充完整。程序显示的原始数据以及计算结果如图 6.1 所示,所显示的相关数组的定义可以直接阅读源代码来理解。

k	A	B	C	D
0	0.00000e+00	4.00000e+00	2.00000e+00	1.60000e+01
1	2.00000e+00	5.00000e+00	1.00000e+00	1.70000e+01
2	3.00000e+00	7.00000e+00	0.00000e+00	1.30000e+01

k	w	u	v	x
0	4.00000e+00	4.00000e+00	5.00000e-01	3.00000e+00
1	4.00000e+00	2.25000e+00	2.50000e-01	2.00000e+00
2	6.25000e+00	1.00000e+00	0.00000e+00	1.00000e+00

图 6.1 程序 8.10 显示的原始数据以及计算结果

程序 8.10 利用追赶法求解例 8.1 的源程序

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#define MN 51
```

```

int    RN=0;
double A[MN],B[MN],C[MN],D[MN];
double U[MN],V[MN],W[MN],X[MN+1];
char * fname="TB080601.TXT";
Operation()
{ int k;
  W[0]=B[0];
  U[0]=D[0]/B[0];
  V[0]=C[0]/B[0];
  for(k=1;k<RN;k++)
  { W[k]=B[k]-A[k]*V[k-1];
    U[k]=(D[k]-A[k]*U[k-1])/W[k];
    V[k]=C[k]/W[k];
  }
  X[RN]=0;
  for(k=RN-1;k>=0;k--)
  { X[k]= U[k]-V[k]*X[k+1];
  }
  return;
}
void ShowTable(){...}
void SaveTable(char * filename){...}
int FormProblem()
{ RN=3;
  A[0]=0; B[0]=4; C[0]=2; D[0]=16;
  A[1]=2; B[1]=5; C[1]=1; D[1]=17;
  A[2]=3; B[2]=7; C[2]=0; D[2]=13;
  return 0;
}
main()
{ FormProblem();
  Operation();
  ShowTable();
  SaveTable(fname);
  getchar();
}

```

把追赶法与高斯消去法对比分析可知,追赶法实际就是高斯消去法在特殊条件下的应用。不难看出,追赶法的计算量特别小,它仅与 n 成正比,而高斯消去法的计算量是与 n 三次方成正比。可以证明,如果所有的 $b_i \geq |a_i| + |c_i|$,则所给的方程组是主对角线占优的,此时算法的稳定性能好。

练 习 题

1. 修改程序 8.01 中的 FormProblem() 函数,使得所生成的增广矩阵的每个元素都在区间 $[0.5, 1.5]$ 内。

2. 随机生成一个4阶方阵 A , 然后调用函数 `MatrixInversion(A, B, 4)`, 求 $B=A^{-1}$, 并编程验算是否有 $AB=BA=I$ 。

3. 模仿高斯消去法把线性方程组化为下三角形线性方程组(读者自己给出定义), 然后求解下三角形线性方程组而得到不同形式的高斯消去法。

4. 编写应用于测试求解线性方程组的方法的案例构造程序, 方法是先随机生成线性方程组的系数矩阵 A , 再随机生成线性方程组的解 u , 计算出 $b=Au$, 从而得到有准确解 $x=u$ 的线性方程组 $Ax=b$, 假如用某个模块程序得到 $Ax=b$ 的解 x^0 , 则不难计算出 x^0-u 的模, 它可以表示 x^0 的误差。

5. 设 x^0 为采用某种方法得到的线性方程组 $Ax=b$ 的近似解, 令 $E=Ax^0-b$, E 的模可用来评估 x^0 的近似程度, 试在高斯消去法和约当消去法中增加一个误差分析模块。(学完第9章后再回过头来作这件事更容易一些)

6. 完成追赶法解三对角线性方程组的模块程序编写, 并构造案例验算(可以随机生成案例)。

最小二乘法是一个非常有用的数学方法,它的理论意义是用于求 $\mathbf{R}^n$ 中的某个向量在某个子空间上的投影,从而得到 $\mathbf{R}^n$ 中的点到子空间的距离。这种解释似乎有些玄乎,不过我们还是现实一些为好,直接利用它来求矛盾线性方程组的最小二乘解。

曲线拟合简单说来就是寻找一条在数学上满足一定条件的曲线来近似表示 xy 平面上的若干个离散点所满足的函数关系。曲线拟合有着广泛的用途,工程中,它可用来求经验公式;在科学试验中,它可以用来对实验数据进行统计分析,从而进一步得到物理学的定律;在生产过程控制领域,它可用来求某些指标比如产品的平均寿命,产品的合格率等的最小二乘估计;在统计学中,它可应用于多元回归分析等;另外,在数值计算领域,它可用来进行一般性的误差分析。

有意义的是,如果我们用某个函数类中的函数(比如线性函数,多项式函数等)进行曲线拟合,问题便转化为确定相应的参数,采用的方法恰好是最小二乘法。所以我们现在学习的这一章很能体现计算方法的魅力:借助计算机的支持,只用很简单的代码即可实现用复杂的数学方法解决复杂的工程计算问题。

9.1 线性函数拟合方法

假如需要确定一根弹簧的长度 y 与所受到的外界的拉力 x 之间的函数关系 $y=f(x)$,人们自然会联想到物理学中的定律:在弹性限度以内,弹簧的长度的改变量与所受到的外界的拉力成正比。由此不难得到弹簧的长度 y 与所受到的外界的拉力 x 满足一个线性关系 $y=a_0+a_1x$ 。所以如果要具体分析一根弹簧的性能,其实也就是确定两个重要的参数 a_0 和 a_1 。

9.1.1 获取原始数据

如果要具体分析一根弹簧的性能,首先必须通过物理试验获取相关的原

始数据,接下来再采用科学的方法对所获取的数据进行分析,从而得出科学的结论。这也是人类通过科学试验来认识客观世界的基本方法。

对于分析弹簧来说,通过实验来获取原始数据还是很容易的,比如把弹簧的上端固定在一个支架上,下端挂上一个砝码盘,在砝码盘中放上不同质量的砝码,测量出相应的弹簧的长度,即可得到若干对弹簧所受到的拉力与弹簧相应的长度的观测值。

一般地,我们总可以想办法通过物理试验或其他手段获取一对相关变量 x, y 的 $n+1$ 对观测值,从而得到如表 1.1 所示的原始数据列表。这样,我们就完成了通过物理试验获取原始数据的工作,接下来就是利用数学方法处理实验数据了。在一般情况下,可以假定表中的各 x_k 的值为准确值或精确度很高的值,而各 y_k 则带有随机观测误差。

表 1.1 弹簧性能测试数据列表

x	x_0	x_1	$\cdots$	x_n
y	y_0	y_1	$\cdots$	y_n

9.1.2 组织矛盾线性方程组

对于分析弹簧的性能而言,我们已经知道 x 与 y 之间满足线性关系 $y = a_0 + a_1 x$, 只是不知道这里的 a_0 和 a_1 的具体值。为了尽可能准确地确定出 a_0 和 a_1 , 可以把表 1.1 中列出的 $n+1$ 对观测值 $\{(x_k, y_k) | k=0, 1, \cdots, n\}$ 分别代入到线性函数 $y = a_0 + a_1 x$ 中, 从而得到两个变元的由 $n+1$ 个方程组成的线性方程组

$$\left. \begin{aligned} a_0 + a_1 x_0 &= y_0 \\ a_0 + a_1 x_1 &= y_1 \\ &\vdots \\ a_0 + a_1 x_n &= y_n \end{aligned} \right\} \quad (1.1)$$

对于数值计算来说,我们所需要的只是线性方程组的增广矩阵,它包括了关于线性方程组的所有原始数据以及相应的结构。为了讨论的方便,我们利用矩阵和向量的形式来表示上面的(1.1)式。记

$$C = \begin{pmatrix} 1 & x_0 \\ 1 & x_1 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, a = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, y = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}$$

从而得到方程组(1.1)的简写形式

$$Ca = y \quad (1.2)$$

一般地,线性方程组(1.1)中方程的个数大于变量的个数,也就是方程组的系数矩阵的行数大于列数。系数矩阵 C 通常是满列秩的,所以线性方程组(1.1)无通常意义下的解,故称为矛盾方程组。

9.1.3 转化为极小化二次函数问题

在一般情况下,前面得到的方程组(1.1)无解,这是很容易理解的,因为其中的 y_0 ,

$y_1, \dots, y_n$ 都是观测值, 它们不可避免地带有观测误差, 但我们有理由寻找它的尽可能准确的解。记

$$r_i = a_0 + a_1 x_i - y_i, i=0, 1, \dots, n \quad (1.3)$$

显然, r_i 就是 y_i 的误差。记

$$J(a_0, a_1) = \sum_{i=0}^n (r_i)^2 = \sum_{i=0}^n (a_0 + a_1 x_i - y_i)^2 \quad (1.4)$$

$J(a_0, a_1)$ 就是 $n+1$ 个误差项的平方和。

注意到(1.4)式中的 $x_0, x_1, \dots, x_n$ 和 $y_0, y_1, \dots, y_n$ 都是常数, 而 a_0, a_1 都是待确定的系数, 所以可以直接把 J 看作是 (a_0, a_1) 的函数。显然, 寻找 a_0, a_1 使得 $J(a_0, a_1)$ 达到它的极小值是合理的, 从而得到一个无约束的极小化问题

$$\min J(a_0, a_1) = \sum_{i=0}^n (a_0 + a_1 x_i - y_i)^2 \quad (1.5)$$

至少, 可以按求解第6章例2.1和例2.2的方式编写相应的数学问题模块, 从而可以利用最速下降法或三部曲算法求上面(1.5)式的解。不过对于这种特殊的二次函数来说, 可以寻找更好的办法。在一般情况下, $J(a_0, a_1)$ 是正定的二次函数, 它有全局的惟一极小点。所以可以通过求解

$$\left. \begin{aligned} \frac{\partial J(a_0, a_1)}{\partial a_0} &= 0 \\ \frac{\partial J(a_0, a_1)}{\partial a_1} &= 0 \end{aligned} \right\} \quad (1.6)$$

来寻找(1.5)式的解, 并把这样的解称为是(1.1)式的最小二乘解。

9.1.4 求解正规方程

不难利用(1.5)式给出的 $J(a_0, a_1)$ 求出它对两个变元的偏导数:

$$\left. \begin{aligned} \frac{\partial J(a_0, a_1)}{\partial a_0} &= 2 \sum_{i=0}^n (a_0 + a_1 x_i - y_i) \\ \frac{\partial J(a_0, a_1)}{\partial a_1} &= 2 \sum_{i=0}^n [(a_0 + a_1 x_i - y_i) x_i] \end{aligned} \right\} \quad (1.7)$$

再由上面的(1.7)式整理后即得

$$\left. \begin{aligned} \frac{\partial J(a_0, a_1)}{\partial a_0} &= 2 \left[n a_0 + \left(\sum_{i=0}^n x_i \right) a_1 - \sum_{i=0}^n y_i \right] \\ \frac{\partial J(a_0, a_1)}{\partial a_1} &= 2 \left[\left(\sum_{i=0}^n x_i \right) a_0 + \left(\sum_{i=0}^n (x_i)^2 \right) a_1 - \sum_{i=0}^n (x_i y_i) \right] \end{aligned} \right\} \quad (1.8)$$

利用上面的(1.6)式, 并略去(1.8)式中的常数因子2, 即可得到下面的以 a_0, a_1 为变元的线性方程组。

$$\left. \begin{aligned} n a_0 + \left(\sum_{i=1}^{i=n} x_i \right) a_1 &= \sum_{i=1}^{i=n} y_i \\ \left(\sum_{i=1}^{i=n} x_i \right) a_0 + \left(\sum_{i=1}^{i=n} (x_i)^2 \right) a_1 &= \sum_{i=1}^{i=n} x_i y_i \end{aligned} \right\} \quad (1.9)$$

上面的方程组(1.9)一般称为正规方程,在不同的应用领域内,也有人称之为法方程、回归方程。利用上一章得到的解线性方程组的模块程序,即可完成求上面的(1.9)式的数值解,从而得到上面的(1.5)式的数值解。

如果进一步记 Q, p 分别为方程组(1.9)的系数矩阵和常数向量,不难验证

$$Q = C^T C, p = C^T y \quad (1.10)$$

从而可以把方程组(1.9)用矩阵和向量的形式表示为

$$(C^T C)a = C^T y \quad (1.11)$$

它可以看成是由(1.2)式两边简单地左乘以 C^T 而来。

所以,如果我们的目的仅仅是求矛盾方程组的最小二乘解,相应的算法是非常简单的:将矛盾方程组的增广矩阵左乘以矛盾方程的系数矩阵的转置矩阵,即可得到正规方程的增广矩阵,接下来利用第8章给出的程序求解正规方程从而使问题得到解决。

9.1.5 分析解的有效性

假如通过求解正规方程(1.9)得到了(1.1)的最小二乘解 (a_0, a_1) ,那么也就找到了 y 与 x 的近似的函数关系 $y = a_0 + a_1 x$ 。在一般情况下这个结果是很有用的,但有时候仍然会产生较大的误差,为此,可以对所得到的结果的有效性作进一步检验。记

$$e = \sqrt{\frac{J(a_0, a_1)}{(n+1)-2}} \quad (1.12)$$

一般称 e 为标准误差。在实际工作中,可以规定出一个临界值 e_0 ,实际作用相当于控制误差限,当 e 小于这个值时就认为结果有效,否则作进一步的考虑。

9.1.6 一个完整的分析案例

例 1.1 假设已经得到弹簧所受到的拉力 x 与弹簧的长度 y 的观测值列表如下所示:

k	0	1	2	3	4	5	6	7
x_k	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
y_k	0.118	0.131	0.152	0.162	0.180	0.192	0.206	0.228

试确定 y 与 x 之间的函数关系。

解:把 x 与 y 的观测值依次代入到 $y = a_0 + a_1 x$ 中,得矛盾线性方程组

$$\left. \begin{aligned} a_0 + 0.1a_1 &= 0.118 \\ a_0 + 0.2a_1 &= 0.131 \\ a_0 + 0.3a_1 &= 0.152 \\ a_0 + 0.4a_1 &= 0.162 \\ a_0 + 0.5a_1 &= 0.180 \\ a_0 + 0.6a_1 &= 0.192 \\ a_0 + 0.7a_1 &= 0.206 \\ a_0 + 0.8a_1 &= 0.228 \end{aligned} \right\} \quad (1.13)$$

相应的系数矩阵 C 和常数向量 y 分别为

$$C = \begin{pmatrix} 1 & 0.1 \\ 1 & 0.2 \\ 1 & 0.3 \\ 1 & 0.4 \\ 1 & 0.5 \\ 1 & 0.6 \\ 1 & 0.7 \\ 1 & 0.8 \end{pmatrix}, a = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, y = \begin{pmatrix} 0.118 \\ 0.131 \\ 0.152 \\ 0.162 \\ 0.180 \\ 0.192 \\ 0.206 \\ 0.228 \end{pmatrix} \quad (1.14)$$

将方程组 $Ca=y$ 两边同时左乘以 C^T , 得

$$\left. \begin{aligned} 8a_0 + 3.6a_1 &= 1.369 \\ 3.6a_0 + 2.04a_1 &= 0.6802 \end{aligned} \right\} \quad (1.15)$$

解方程组(1.15)得: $a_0 = 0.102\ 39$, $a_1 = 0.152\ 47$ 。再由(1.12)式算得的标准差为 $e = 2.468 \times 10^{-3}$ 。

例 1.1 曲线拟合的结果如图 1.1 所示。从图中可以看出, 观测到的离散点的位置基本上是在拟合的直线附近徘徊, 所以也有人把这条直线称为回归直线, 故把线性拟合称为线性回归。

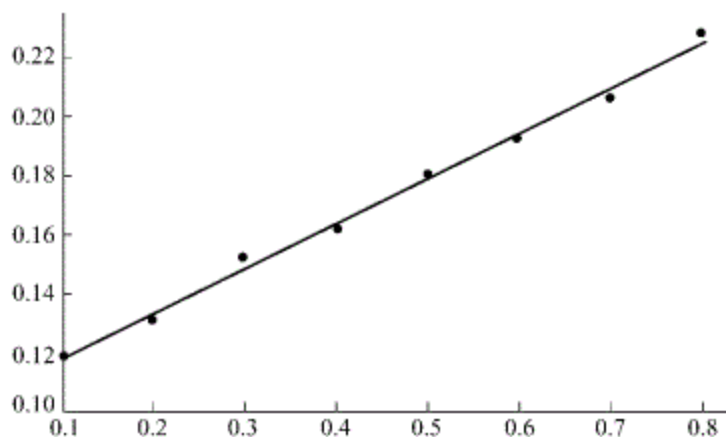


图 1.1 例 1.1 线性拟合结果的图形表示

9.2 最小二乘法

设有线性方程组

$$\left. \begin{aligned} c_{00}x_0 + c_{01}x_1 + \cdots + c_{0M}x_M &= y_0 \\ c_{10}x_0 + c_{11}x_1 + \cdots + c_{1M}x_M &= y_1 \\ \vdots &\vdots \\ c_{L0}x_0 + c_{L1}x_1 + \cdots + c_{LM}x_M &= y_L \end{aligned} \right\} \quad (2.1)$$

利用矩阵和向量的记号, 记

$$C = \begin{pmatrix} c_{00} & c_{01} & \cdots & c_{0M} \\ c_{10} & c_{11} & \cdots & c_{1M} \\ \vdots & \vdots & & \vdots \\ c_{L0} & c_{L1} & \cdots & c_{LM} \end{pmatrix}, x = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_M \end{pmatrix}, y = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_L \end{pmatrix}$$

上面的(2.1)式可简记为 $Cx=y$ 。

作为最小二乘法所求解的线性方程组,通常约定 C 是满列秩值的 $(L+1) \times (M+1)$ 矩阵,而且还有 $L>M$,所以方程组(2.1)无通常意义下的解,故称之为矛盾方程组。

虽然从理论上讲求一个矛盾方程组的解是不可思议的事情,但正如我们在上一节中所看到的那样,其原因是在模型中忽略了实际问题中的某些难以控制的因素所产生的误差项,而这种误差项在建立实际问题的数学模型时几乎普遍存在,所以专门研究这样的问题的求解方法就更有意义。

9.2.1 化为极小化二次函数问题

定义 $n+1$ 维向量 $r=(r_0, r_1, \cdots, r_n)^T$, 令

$$r=Cx-y \quad (2.2)$$

我们可以把 r 看作是用 y 近似表示 Cx 的误差向量。

现在,我们可以换一个思路来研究(2.1)式的解。如果能找到 x ,使得 r 为零向量,那么 x 就是(2.1)式的准确解;如果 x 使得 r 非常接近零向量,那么 x 就是(2.2)式的很好的近似解。为了度量这种接近的程度,我们将(2.2)式两边左乘以各自的转置向量得

$$r^T r = (Cx-y)^T (Cx-y) \quad (2.3)$$

利用

$$(Cx-y)^T = x^T C^T - y^T$$

以及

$$x^T C^T y = y^T Cx = (C^T y)^T x$$

所以由(2.3)式不难得到

$$r^T r = x^T (C^T C) x - 2(C^T y)^T x + y^T y \quad (2.4)$$

注意到 $r^T r$ 就是 r 的所有分量的平方和,所以用它来评价 r 接近零(向量)的程度是合理的。再注意到(2.4)式的右边是关于 x 的二次函数,从而得到相应的无约束极值问题

$$\min J(x) = x^T (C^T C) x - 2(C^T y)^T x + y^T y \quad (2.5)$$

假如 x^* 为相应的最优解,那么由(2.2)式所得到的 $r^* = Cx^* - y$ 的所有分量的平方和也随之达到最小,所以我们把上面的(2.5)式的解作为(2.1)式近似解,并称为是最小二乘解。

9.2.2 正规方程的矩阵形式与计算方法

对于无约束极值问题(2.5)式来说,可以证明,如果 C 是满列秩的矩阵,那么 $J(x)$ 便是正定二次函数,此时, $\nabla J(x)=0$ 是(2.5)式取得(惟一的)极值的充分必要条件。

不难验证,对于(2.5)式定义的 $J(x)$ 来说, $\nabla J(x)$ 可以简单地表示为

$$\nabla J(x) = (C^T C)x - C^T y$$

所以(2.5)式的解也就是线性方程组

$$(C^T C)x = C^T y \quad (2.6)$$

的解。(2.6)式通常称为正规方程。

如果我们的注意力只是利用计算机求矛盾方程组(2.1)的最小二乘解,相应的算法倒是特别简单:将矛盾方程 $Cx=y$ 两边同时左乘以系数矩阵 C 的转置矩阵 C^T 即可得到相应的正规方程。得到了正规方程的增广矩阵,接下来再利用上一章得到的解线性方程组的模块程序,从而问题得到解决。

9.2.3 应用模块的编写

由于最小二乘法的用途非常广泛,把生成正规方程和求解正规方程以及与主程序的通讯放在同一个模块中,显然可以得到一个很有用的工具。为了节省编程的时间,可以把第8章解线性方程组的模块程序,比如 LEG_Jord.c,复制一份,用文件名“CLEG_LSM.C”保存,在此基础上再进行修改。接口函数可以说明为

```
double SolutionCLEG(double * C, double * Y, double * X, int L, int M)
```

其中数组 C, Y 分别存放矛盾方程组的系数矩阵 C 和常数向量 y ; 数组 X 存放问题的解向量 x ; 变量 L, M 分别为系数矩阵 C 的行数和列数; 函数的返回值可约定为(2.5)式定义的 $J(x)$ 的值。所以接口函数的效率特别高。相应的源代码见程序 9.01A。

在程序 9.01A 中,函数 SolutionCLEG() 首先把主程序传过来的参数复制到本模块的公共变量中; 接下来申请存放正规方程增广矩阵的存储空间; 然后依次调用函数 FormNormalEquation() 生成正规方程的增广矩阵; 调用 Jordan() 函数求解正规方程; 把正规方程的解写入数组 X 中; 释放前面申请的存储空间, 最后利用函数 Objective() 计算目标函数值作为程序的返回值。程序中的 Jordan() 函数以及它所调用的函数都保持不变, 所以没有列出它们的源代码。

程序 9.01A 求矛盾方程最小二乘解应用模块

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double * buf;
double * PC[50], * PY, * PA[50];
int PL, PM, PN;
int SetPrintMatrix=0;
void FormNormalEquation()
{ int i, j, k;
  double temp;
  for(i=0; i<PM; i++)
    for(j=i; j<PM; j++)
    { temp=0.0;
      for(k=0; k<PL; k++) temp+=PC[k][i]*PC[k][j];
      PA[i][j]=temp;
      PA[j][i]=temp;
    }
  for(i=0; i<PM; i++)
```



```

    { temp=0.0;
      for(k=0;k<PL;k++)temp+=PC[k][i]*PY[k];
      PA[i][PM]=temp;
    }
    return;
}

double Objective(double *x)
{ int l,m;
  double OBJ=0.0,temp=0.0;
  for(l=0;l<PL;l++)
  { temp=0.0;
    for(m=0;m<PM;m++)temp+=x[m]*PC[l][m];
    temp-=PY[l];
    OBJ+=temp*temp;
  }
  return OBJ;
}

double SolutionCLEG(double *C,double *Y,double *X,int L,int M)
{ double StandError=0;
  int k,ErrCode=0;
  if(L>50||M>50) return -1;
  PL=L;PM=M;PN=M+1;PY=Y;
  for(k=0;k<PL;k++)PC[k]=C+k*PM;
  buf=malloc(PM*PN*sizeof(double));
  if(!buf) return -2;
  for(k=0;k<PM;k++)PA[k]=buf+k*PN;
  FormNormalEquation();
  ErrCode=Jordan();
  if(ErrCode>0)return -3;
  for(k=0;k<PM;k++)X[k]=PA[k][PN-1];
  free(buf);
  return Objective(X);
}

```

9.2.4 一个完整的计算案例

随机生成4个变量,8个方程组成的矛盾线性方程组,利用程序9.01A求最小二乘解,并返回最优目标值的完整例程见程序9.01B。这个例程完全是虚构的,并没有“实际”背景,但可以来说明模块程序“CLEG_LSM.C”的用法。与利用模块程序解线性方程组类似,可以在主程序中通过设置SetPrintMatrix为1或0来选择是否显示详细计算过程。

程序 9.01B 求矛盾线性方程组的最小二乘解例程

```
#include "CLEG_LSM.C"
```

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define ML 8
#define MM 4
unsigned int SEED=8000;
double C[ML][MM],Y[ML],X[MM];
unsigned int RL=ML,RM=MM;
void FormProblem()
{ int i,j,k;
  double x;
  srand(SEED);
  for(i=0;i<ML;i++)
  for(j=0;j<MM;j++)
  { k=rand();
    x=(double)k/RAND_MAX;
    /* RAND_MAX=32767,by system */
    C[i][j]=0.1+x*9.8;
  }
  for(i=0;i<ML;i++)
  { k=rand();
    x=(double)k/RAND_MAX;
    Y[i]=x*20.0;
  }
  return;
}
main()
{ int k;
  double OBJ;
  FormProblem();
  SetPrintMatrix=1;
  OBJ=SolutionCLEG(&C[0][0],Y,X,ML,MM);
  printf("\n OBJ = %13.5e",OBJ);
  for(k=0;k<MM;k++)
  printf("\n X[%d] = %13.5f",k,X[k]);
  getchar();
}

```

运行程序 9.01B 最后显示的结果如图 2.1 所示,我们现在的目的是说明模块程序“CLEG_LSM.C”已经能正常通过编译运行以及如何利用,所得到的结果只要正确即可,

```

OBJ = 3.8556e+01
X[0] = 1.10034
X[1] = 1.15453
X[2] = -1.78603
X[3] = 0.25880

```

图 2.1 程序 9.01B 的运行结果

本身并没有什么实际意义。

9.3 多项式拟合方法

我们在第一节中已经给出了一个系统的方法进行试验数据处理,在那里我们只是简单地把实验数据设想为满足一个线性关系。如果我们更一般地把实验数据设想为满足一个多项式函数关系,那么相应的问题就是多项式拟合。

9.3.1 问题的提法

假如得到一对相关变量 (x, y) 或某个未知函数 $y=f(x)$ 的 $n+1$ 对观测值的列表(见表3.1)

表 3.1 两个相关变量的观测值列表

x	x_0	x_1	$\cdots$	x_n
y	y_0	y_1	$\cdots$	y_n

现在要找一个 m 次多项式

$$y = a_0 + a_1 x + \cdots + a_m x^m \quad (3.1)$$

来近似地表示这个未知函数。在这里,我们通常约定 $n > m$,而且 $x_0, x_1, \cdots, x_n$ 中至少有 $m+1$ 个不同的值。

多项式拟合问题的核心就是如何尽可能准确地确定出这个多项式的系数 $a_0, a_1, \cdots, a_m$,对此,我们可以首先把它表示为一个线性方程组的解。

把表3.1中的 $n+1$ 对观测值 $\{(x_k, y_k) | k=0, 1, \cdots, n\}$ 分别代入到(3.1)式中,我们可以得到下面的线性方程组

$$\left. \begin{aligned} a_0 + x_0 a_1 + \cdots + (x_0)^m a_m &= y_0 \\ a_0 + x_1 a_1 + \cdots + (x_1)^m a_m &= y_1 \\ &\vdots \\ a_0 + x_n a_1 + \cdots + (x_n)^m a_m &= y_n \end{aligned} \right\} \quad (3.2)$$

方程组(3.2)其实也是矛盾方程组,相应的系数矩阵 C 和常数向量 y 分别为

$$C = \begin{bmatrix} 1 & x_0 & \cdots & (x_0)^m \\ 1 & x_1 & \cdots & (x_1)^m \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & \cdots & (x_n)^m \end{bmatrix}, \quad y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix} \quad (3.3)$$

所以不难用前面学过的最小二乘法来求解。

作为一个理论问题,我们强调系数矩阵 C 是满列秩的。在下一章介绍了范德蒙行列式的性质后,我们不难得到这样的结论:如果 $x_0, x_1, \cdots, x_n$ 中至少有 $m+1$ 个不同的值,那么 C 就是满列秩的矩阵。所以在实际应用中保证 C 满列秩并不困难。

9.3.2 直接利用“CLEG_LSM.C”求解

首先可以在程序中说明两个数组 X, Y , 简单地编写一个 `FormProblem()` 函数用表 3.1 中的数据给 X, Y 赋初值; 接下来可以再编写一个函数生成矛盾线性方程组 (的增广矩阵); 最后调用函数 `SolutionCLEG()` 求解即可。下面我们通过一个例子说明如何简单地利用上一节所编写的模块程序 `CLEG_LSM.C` 完成多项式拟合。

例 3.1 已知相关变量 x, y 的 8 对观测值列表如下:

k	0	1	2	3	4	5	6	7
x_k	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
y_k	1.377	1.480	1.693	1.909	2.255	2.393	2.771	3.415

试用最小二乘法求相应的 3 次拟合多项式

$$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

的系数 a_0, a_1, a_2, a_3 。

解: 首先用下面几行 C 语言代码定义维数, 说明相应的全局变量 A, X, Y, C :

```
#define ML 8
#define MM 4
double A[MM];
double X[ML], Y[ML], C[ML][MM];
```

接下来编写 C 语言函数 `FormProblem()` 给数组 X, Y 赋初值:

```
void FormProblem()
{ int k;
  X[0]=0.100; Y[0]=1.377;
  X[1]=0.200; Y[1]=1.480;
  X[2]=0.300; Y[2]=1.693;
  X[3]=0.400; Y[3]=1.909;
  X[4]=0.500; Y[4]=2.255;
  X[5]=0.600; Y[5]=2.393;
  X[6]=0.700; Y[6]=2.771;
  X[7]=0.800; Y[7]=3.415;
  return;
}
```

再接下来编写 C 语言函数 `FormCLEG()` 生成矛盾方程组的系数矩阵 C , 方法也很简单, 直接读源代码即可理解。

```
void FormCLEG()
{ int i, j, k;
  double x, temp;
  for(i=0; i<ML; i++)
```

```

{ x=X[i];
  temp=1.0;
  for(j=0;j<MM;j++)
  { C[i][j]=temp;
    temp*=x;
  }
}
return;
}

```

完整的源代码见程序 9.02, 程序中的数组 A 用于存放拟合多项式的系数。

程序 9.02 求解例 3.1 的 C 语言程序

```

#include "CLEG_LSM.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define ML 8
#define MM 4
double A[MM];
double X[ML], Y[ML], C[ML][MM];
void FormProblem() { ... }
void FormCLEG() { ... }
main()
{ int k;
  double ERR, SERR;
  FormProblem();
  FormCLEG();
  SetPrintMatrix=1;
  ERR=SolutionCLEG(&C[0][0], Y, A, ML, MM);
  SERR=sqrt(ERR/(ML-MM));
  printf("\n SERR = %14.6e", SERR);
  for(k=0; k<MM; k++) printf("\n A[%1d] = %10.5f", k, A[k]);
  getchar();
}

```

程序 9.02 显示的矛盾线性方程组为

$c[i]$	$c[i][0]$	$c[i][1]$	$c[i][2]$	$c[i][3]$	$Y[i]$
[0]	1.000 00	0.100 00	0.010 00	0.001 00	1.377 00
[1]	1.000 00	0.200 00	0.040 00	0.008 00	1.480 00
[2]	1.000 00	0.300 00	0.090 00	0.027 00	1.693 00
[3]	1.000 00	0.400 00	0.160 00	0.064 00	1.909 00

续表

$c[i]$	$c[i][0]$	$c[i][1]$	$c[i][2]$	$c[i][3]$	$Y[i]$
[4]	1.000 00	0.500 00	0.250 00	0.125 00	2.255 00
[5]	1.000 00	0.600 00	0.360 00	0.216 00	2.393 00
[6]	1.000 00	0.700 00	0.490 00	0.343 00	2.771 00
[7]	1.000 00	0.800 00	0.640 00	0.512 00	3.415 00

所显示的正规方程的增广矩阵为

$PA[i][j]$	[0]	[1]	[2]	[3]	[4]
[0]	8.000 00	3.600 00	2.040 00	1.296 00	17.293 00
[1]	3.600 00	2.040 00	1.296 00	0.877 20	8.940 20
[2]	2.040 00	1.296 00	0.877 20	0.617 76	5.499 40
[3]	1.296 00	0.877 20	0.617 76	0.446 96	3.678 80

最后所显示的结果是

```

SERR=      8.334 68E-02
A[0] =      1.107 57
A[1] =      2.666 86
A[2] =     -3.577 49
A[3] =      4.729 80

```

曲线拟合的情况如图 3.1 所示。

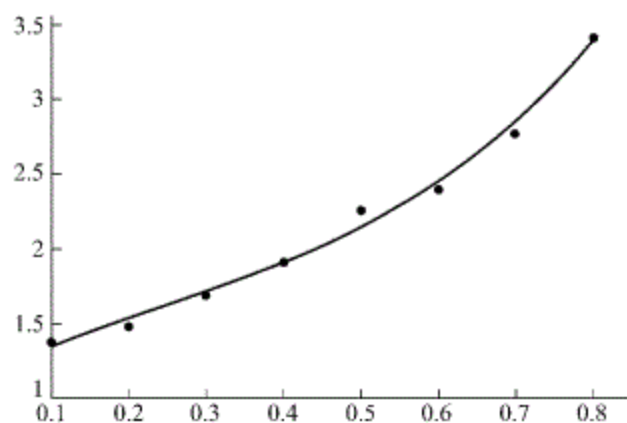


图 3.1 例 3.1 曲线拟合结果

注意到函数 SolutionCLEG() 的返回值是(2.5)式的最优解,记为 ERR,称

$$SERR = \sqrt{\frac{ERR}{N-M}}$$

为标准误差^①。

我们很容易应用程序 9.02 求解其他的多项式拟合问题,所作的修改只是问题的维数定义 ML 和 MM 的值,并重写 FormProblem()函数。

9.3.3 直接编写多项式拟合模块

利用 9.2 节中的最小二乘法进行曲线拟合可以概括为三个主要步骤:根据问题的要求写出矛盾方程;利用矛盾方程产生正规方程;求解正规方程。如果能根据问题的要求直接产生正规方程,可以进一步简化我们的计算。

利用线性方程组(3.2)的系数矩阵 C 和常数向量 y 的表示形式(3.3),我们不难得到

$$C^T C = \begin{pmatrix} n & \sum_{k=0}^n x_i & \cdots & \sum_{k=0}^n (x_i)^m \\ \sum_{k=0}^n x_i & \sum_{k=0}^n (x_i)^2 & \cdots & \sum_{k=0}^n (x_i)^{m+1} \\ & & \ddots & \\ \sum_{k=0}^n (x_i)^m & \sum_{k=0}^n (x_i)^{m+1} & \cdots & \sum_{k=0}^n (x_i)^{m+m} \end{pmatrix}$$

$$C^T y = \left(\sum_{k=0}^n y_i \quad \sum_{k=0}^n (x_i y_i) \quad \cdots \quad \sum_{k=0}^n [(x_i)^m y_i] \right)^T$$

对于 m 次多项式拟合来说,注意到 $C^T C$ 的元素中只有 $2m+1$ 个不同的元素

$$\left\{ \sum_{k=0}^n (x_k)^j \mid j = 0, 1, \cdots, 2m \right\}$$

为了编程方便,可以专门说明一个长度为 $2m+1$ 数组 PXX 来存放它们。记

$$PXX[j] = \sum_{k=0}^n (x_k)^j, j = 0, 1, \cdots, 2m$$

则对 $i, j=0, 1, \cdots, m, C^T C$ 的第 i 行第 j 列的元素即为 $PXX[i+j]$,所以只要完成了数组 PXX 中的元素的计算,就很容易写出正规方程的系数矩阵。至于正规方程的常数向量,其计算方法与各 $PXX[j], j=0, 1, \cdots, 2m$ 的计算方法基本相同。

多项式拟合模块的接口程序可以说明为

```
double GetFittingPoly(double * X, double * Y, double * A, int N, int M)
```

其中数组 X, Y 存放相关变量 x, y 的观测值列表, A 用来存放拟合多项式的系数, N 为数组 X, Y 的长度, M 为拟合多项式的次数。GetFittingPoly()首先申请内存空间,完成全局变量的初始设置,接下来调用函数 FormNormalEquation()生成正规方程的增广矩阵;调用函数 Jordan()解正规方程;调用函数 Objective()计算所有误差项的平方和。完整的源代码见程序 9.03。

程序 9.03 多项式拟合模块程序

^① 标准误差在不同的场合有不同的表示形式,这里指的是用 $M-1$ 次多项式拟合 N 对观测值的多项式拟合的标准误差,详细情况可参看文献[14]第 455 页。

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double * buf;
double * PX, * PY, * PXX, * PZ, * PA[50];
int PL, PM, PN;
int SetPrintMatrix=0;
int Jordan() { ... }
double Objective()
{ int k, m;
  double OBJ=0.0, temp, x;
  for(k=0; k<PL; k++)
  { x=PX[k];
    temp=PZ[PM-1];
    for(m=PM-1; m>0; m--) temp=temp * x + PZ[m-1];
    temp=PY[k] - temp;
    OBJ += temp * temp;
  }
  return OBJ;
}
void FormNormalEquation()
{ int i, j, k;
  double temp;
  for(k=0; k<PL; k++) PZ[k]=1.0;
  PXX[0]=PL;
  for(j=1; j<=(PM-1)*2; j++)
  { temp=0.0;
    for(k=0; k<PL; k++) PZ[k] *= PX[k];
    for(k=0; k<PL; k++) temp += PZ[k];
    PXX[j]=temp;
  }
  for(i=0; i<PM; i++)
    for(j=0; j<PM; j++)
      PA[i][j]=PXX[i+j];
  temp=0.0;
  for(k=0; k<PL; k++)
  { PZ[k]=PY[k];
    temp += PZ[k];
  }
  PA[0][PN-1]=temp;
  for(i=1; i<PM; i++)
  { temp=0.0;
    for(k=0; k<PL; k++) PZ[k] *= PX[k];
    for(k=0; k<PL; k++) temp += PZ[k];
  }
}

```

```

    PA[i][PN-1]=temp;
}
return;
}
double GetFittingPoly(double * X,double * Y,double * A,int n,int m)
{ int k,ErrCode=0;
  if(N>50||M>50) return -1;
  PL=n;PM=m;PN=m+1;PX=X;PY=Y;
  buf=malloc((PM * PN+PL * 3) * sizeof(double));
  if(!buf) return -2;
  for(k=0;k<PM;k++)PA[k]=buf+k * PN;
  PZ=buf+PM * PN;
  PXX=PZ+PL;
  FormNormalEquation();
  ErrCode=Jordan();
  if(ErrCode>0)return -3;
  for(k=0;k<PM;k++)A[k]=PA[k][PN-1];
  free(buf);
  PZ=A;
  return Objective();
}

```

9.4 一般基函数的曲线拟合方法

对于一般的工程问题来说,我们用多项式进行曲线拟合通常能够满足要求,但情形并不总是这样。在一些具体领域里面(比如经济预测)采用更为特殊的曲线来进行拟合可能效果更好,为此我们简要介绍一般的曲线拟合的原理和方法。

所谓多项式拟合,实际就是利用基函数 $\{1, x, x^2, \dots, x^m\}$ 的一个线性组合来近似表示某个未知的函数。如果需要的话,我们也可以采用其他的基函数,比如 $\{\sin(nx), \cos(nx), n=0, 1, \dots\}$ 的线性组合,从而形成更一般的曲线拟合方法。

9.4.1 问题的提法

假设我们收集到了两个相关变量 x, y 或某个函数 $y=f(x)$ 的 n 对观测值列表

x	x_0	x_1	$\dots$	x_n
y	y_0	y_1	$\dots$	y_n

我们希望用 $m+1$ 个基函数 $\omega_0(x), \omega_1(x), \dots, \omega_m(x)$ 的一个线性组合

$$y=a_0\omega_0(x)+a_1\omega_1(x)+\dots+a_m\omega_m(x) \quad (4.1)$$

来近似地表达 x, y 间的函数关系,我们的问题就是如何根据给出的观测值列表最佳的确定出组合系数 $a_0, a_1, \dots, a_m$ 。

如果我们取 $\omega_k(x) = x^k, k=0, 1, \dots, m$, 所得到的就是上一节的多项式拟合问题。

9.4.2 利用矛盾方程求最小二乘解

与多项式拟合类似, 把 $n+1$ 对观测值 $\{(x_k, y_k), k=0, 1, \dots, n\}$ 分别代入到 (4.1) 式中, 即可以得到下面的线性方程组

$$\left. \begin{aligned} \omega_0(x_0)a_0 + \omega_1(x_0)a_1 + \dots + \omega_m(x_0)a_m &= y_0 \\ \omega_0(x_1)a_0 + \omega_1(x_1)a_1 + \dots + \omega_m(x_1)a_m &= y_1 \\ &\vdots \\ \omega_0(x_n)a_0 + \omega_1(x_n)a_1 + \dots + \omega_m(x_n)a_m &= y_n \end{aligned} \right\} \quad (4.2)$$

一般说来, 当 $n > m$ 时, 上面的问题无解, 但我们同样可以求它的最小二乘解。方程组 (4.2) 的系数矩阵 C , 常数向量 y 分别为

$$C = \begin{pmatrix} \omega_0(x_0) & \omega_1(x_0) & \dots & \omega_m(x_0) \\ \omega_0(x_1) & \omega_1(x_1) & \dots & \omega_m(x_1) \\ \vdots & \vdots & \ddots & \vdots \\ \omega_0(x_n) & \omega_1(x_n) & \dots & \omega_m(x_n) \end{pmatrix}, y = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix} \quad (4.3)$$

对于给定的基函数来说, 直接编程计算 C 的元素也是很容易的。

得到矛盾线性方程组 (4.2) 的系数矩阵 C 和常数向量 y , 就可以直接利用程序 9.01A 求相应的最小二乘解而得到 (4.1) 式中的待定常数 $a_0, a_1, \dots, a_m$, 从而使问题得到解决。

9.4.3 直接利用正规方程求解

对于一般形式的曲线拟合问题来说, 直接利用矛盾方程求最小二乘解已经够用。作为计算方法来说, 也可以模仿多项式拟合的方法, 直接编写一个模块程序利用原始数据生成正规方程, 求解正规方程。但由于没有给出基函数的具体形式, 所以只能得到一个半成品, 在具体应用时, 需要用户自己编写计算基函数的代码。

对于由 (4.3) 式给出的 C, y , 记 $Q = C^T C, p = C^T y$, 记 $Q = (q_{ij}), p = (p_0, p_1, \dots, p_n)$, 则有

$$\left. \begin{aligned} q_{ij} &= \sum_{k=0}^n \omega_i(x_k) \omega_j(x_k) \\ p_i &= \sum_{k=0}^n y_k \omega_i(x_k) \end{aligned} \right\} \forall i, j \quad (4.4)$$

所以, 只要给出了基函数的具体形式, 利用上面的 (4.4) 式产生正规方程的增广矩阵并不困难。

练 习 题

1. 给出相关变量 x, y 的观测值列表如下:

x	0	1	2	3	4	5
y	2.08	7.68	13.8	27.1	40.8	61.2

试用二次多项式 $y=a_0+a_1x+a_2x^2$ 进行曲线拟合。

要求手工完成计算,捕捉误差分析,并严格按下面的步骤做:

- (1) 写出矛盾方程;
- (2) 写出矛盾方程的系数矩阵和常数向量;
- (3) 写出正规方程的增广矩阵;
- (4) 手工解正规方程;
- (5) 利用第8章的程序解正规方程。

2. 利用多项式拟合的两个模块程序求解第1题。

3. 从其他应用领域找一个曲线拟合的案例用几种不同次数的多项式拟合,并对比分析结果的有效性,写一篇简单的实验报告。

4. 任意构造一个多项式

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_mx^m$$

任取某个区间 $[a, b]$ 内的 $n+1$ 个等分点 $x_0, x_1, \cdots, x_n$ 计算出 $f_k = f(x_k)$, 记 y_k 等于 f_k 加上一个 $[-0.05, 0.05]$ 之间的随机数, 利用所得到的 $(x_k, y_k), k=0, 1, \cdots, m$ 分别作 $m-1, m, m+1$ 次多项式拟合, 对比分析多项式拟合的有效性。所有计算均由计算机完成, 并写一篇简单的实验报告。

10.1 引言

我们在第 1 章中曾经提到过,计算方法研究的基本问题是如何把复杂的数学问题求解有效地转化为对有限数位的数进行有限次的四则运算问题。从这个意义上讲,早期的插值方法的采用是计算方法成为一门独立的数学分支的重要标志。

假设 $y=f(x)$ 是某个表达式很复杂、甚至根本写不出来的实函数,且已知 $f(x)$ 在某个区间 $[a,b]$ 上的 $n+1$ 个互异的点 $x_0, x_1, \dots, x_n$ 处的函数值 $f(x_0), f(x_1), \dots, f(x_n)$, 希望找到一个简单的函数 $y=p(x)$, 使得

$$p(x_k) = f(x_k), k = 0, 1, \dots, n \quad (1.1)$$

这就是一般的 $n+1$ 个基点的插值问题,而且称:

$f(x)$ 为被插值函数;

$p(x)$ 为插值函数;

$x_0, x_1, \dots, x_n$ 为插值基点或插值节点;

$p(x_k) = f(x_k), k=0, 1, \dots, n$ 为插值条件;

$[a,b]$ 为插值区间。

对于一般的 $n+1$ 个基点的插值问题来说,有时还可以知道 $f(x)$ 在全部的或部分插值基点 $x_0, x_1, \dots, x_n$ 处的一阶或二阶导数值,从而进一步要求插值函数 $p(x)$ 在相应的点处的一阶或二阶导数值等于给定值。

对于一般的 $n+1$ 个基点的插值问题来说,如果找到了满足插值条件的函数 $y=p(x)$, 接下来就可以在一定范围内利用 $p(x)$ 近似表示 $f(x)$, 从而解决了相应的计算问题。有时候,还需要根据给定的插值点 x 的位置把相应的插值问题划分为内插或外推。对于给定的插值点 x 来说,如果总有插值基点 x_i, x_j , 使得 $x_i \leq x \leq x_j$, 则称相应的插值问题为“内插”;如果 x 大于所有的插值基点或小于所有的插值基点,则称相应的插值问题为“外推”。不管是“内插”还是“外推”,求插值函数的方法都是一样的,利用插值函数进行计算

的方法也是一样的,所以把它们统称为插值。

对于早期的插值问题来说, $f(x)$ 通常是已知函数,比如对数函数,指数函数,三角函数等,并且已经有了这些函数的函数值列表,插值法可以用来计算那些不在表中的点处的函数值。对于这一类问题来说,现在已经不需要用插值方法来计算。对于现在的许多实际问题来说,我们并不知道 $f(x)$ 的具体形式,所对应的函数值可能是由测量仪器或其他物理设备中直接读出来的, $f(x)$ 只是一个数学概念意义下的函数。

早期的插值方法主要应用于计算复杂的函数值,从而进一步制造各种数学用表。比如我们很容易制造 1 到 10 的具有 4 位数字的 3 次方表,反过来我们能否利用这个表制造出 1 到 1 000 以内的数的具有 4 位有效数字的 3 次方根表? 如果我们已经得到了具有 4 位数字的各种数学用表,能否利用它完成更精确的计算?

对于现代的计算工具来说,早期的插值方法所扮演的角色已经退出了历史舞台,但它的解决问题的原理、思想、方法对我们现代数值计算的仍然有重要的指导作用。

如果说早期的插值方法是利用有理函数的可计算性来解决非有理函数的计算问题,那么今天的插值方法给予同样的思路,利用初等函数的可计算性解决非初等函数的计算问题。由于非初等函数的计算问题大量出现在数值积分以及求微分方程数值解中,所以这一章的学习直接为后面进一步的学习打基础。另外,大量的科学计算问题往往需要通过一个迭代或递推过程来完成,如果我们在在这个过程中巧妙地利用已经得到的近似值进行插值,一般情况下可以极大地提高收敛的速度。总的说来,插值作为一种比较老的方法在现代计算机条件下不仅没有过时,恰恰相反,与现代计算工具的有机结合反而更能体现它强大的生命力。

10.2 一般的多项式插值问题

对于一个插值问题来说,如果已知条件就是 $n+1$ 个互异的插值基点 $x_0, x_1, \dots, x_n$ 点处的函数值 $y_0, y_1, \dots, y_n$, 而插值函数就是一般的次数不超过 n 的多项式,则称为是一般的 $n+1$ 个基点的多项式插值问题。此时,我们习惯上把插值基点以及相应的函数值采用下面的表 2.1 的形式列出,并简称为由表 2.1 给出的插值问题。

表 2.1 一般 $n+1$ 个基点的插值问题的函数值列表

k	0	1	...	n
x	x_0	x_1	...	x_n
y	y_0	y_1	...	y_n

对于一般的 $n+1$ 个基点的多项式插值问题,插值函数可以记为 $p_n(x)$, 并且把 $p_n(x)$ 称为是插值多项式。以后如无特别说明,一般的 $n+1$ 个基点的插值问题所指的都是由表 2.1 给出的多项式插值问题。由于次数不超过 n 的多项式的一般形式为

$$p_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (2.1)$$

所以只要确定了 $n+1$ 个系数 $a_0, a_1, \dots, a_n$, 我们便确定了一个插值多项式。

在这一节中,我们主要解决插值多项式的存在性和唯一性问题,为后面的理论分析打

基础,具体的方法留到下面两节讨论。

10.2.1 多项式插值的一般方法

对于由表 2.1 给出的一般的 $n+1$ 个基点的多项式插值问题,也可以采用上一章中介绍的曲线拟合的方法来求插值多项式 $p_n(x)$ 的系数 $a_0, a_1, \dots, a_n$ 。把表 2.1 中的 $n+1$ 对观测值 $(x_k, y_k), k=0, 1, \dots, n$ 分别代入到(2.1)式中,即可得到关于 $a_0, a_1, \dots, a_n$ 这 $n+1$ 个变元的、由 $n+1$ 个线性方程组成的方程组

$$\left. \begin{aligned} a_0 + x_0 a_1 + (x_0)^2 a_2 + \dots + (x_0)^n a_n &= y_0 \\ a_0 + x_1 a_1 + (x_1)^2 a_2 + \dots + (x_1)^n a_n &= y_1 \\ &\vdots \\ a_0 + x_n a_1 + (x_n)^2 a_2 + \dots + (x_n)^n a_n &= y_n \end{aligned} \right\} \quad (2.2)$$

把方程组(2.2)写成矩阵和向量相乘的形式即可得到下面向量形式的方程组

$$\begin{pmatrix} 1 & (x_0)^1 & \dots & (x_0)^n \\ 1 & (x_1)^1 & \dots & (x_1)^n \\ \vdots & \vdots & \vdots & \vdots \\ 1 & (x_n)^1 & \dots & (x_n)^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix} \quad (2.3)$$

由线性方程组的解的性质可知,(2.2)是否有解,是否有惟一解均由(2.3)式的系数矩阵决定。如果线性方程组(2.3)有惟一解,不难利用第 8 章介绍的高斯消去法或高斯-约当消去法以及相应的程序模块求出(2.3)式的解,从而使问题得到解决。

10.2.2 范德蒙(Vandermonde)行列式的性质与惟一性定理

设 $x = (x_0, x_1, \dots, x_n)^T$ 为任意的 $n+1$ 维向量,称下面的行列式

$$\begin{vmatrix} 1 & x_0 & (x_0)^2 & \dots & (x_0)^n \\ 1 & x_1 & (x_1)^2 & \dots & (x_1)^n \\ 1 & x_2 & (x_2)^2 & \dots & (x_2)^n \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & (x_n)^2 & \dots & (x_n)^n \end{vmatrix} \quad (2.4)$$

为 $n+1$ 级范德蒙行列式。

定理 2.1 $n+1$ 级范德蒙行列式(2.4)的值等于 $x_0, x_1, \dots, x_n$ 这 $n+1$ 个数的所有可能的差 $x_j - x_i (0 \leq i < j \leq n)$ 的乘积。

上面的定理 2.1 是代数学中很著名的一个定理,详细证明见参考文献[1]第 78~79 页,基本方法是数学归纳法。

推论 2.1 $n+1$ 级范德蒙行列式(2.4)不等于零的充要条件是诸 $x_0, x_1, \dots, x_n$ 两两互不相同。

由于线性方程组(2.3)的系数矩阵的行列式就是范德蒙行列式,再加上我们对插值基点互异的假设,所以对于给定的插值条件,存在惟一的插值多项式,它的系数就是线性方程组(2.2)的惟一解。

定理 2.2 如果两个次数都不超过 n 的多项式 $P(x)$ 和 $Q(x)$ 在 $n+1$ 个互不相同的点 $x_0, x_1, \dots, x_n$ 处的值相同, 则这两个多项式恒等。

这也是代数学中很著名的定理, 详细证明在文献[1]第 25~26 页中可找到, 基本思路是不恒为零的 n 次多项式不可能有多于 n 个的零点, 而 $P(x)-Q(x)$ 却有, 所以它要么是次数高于 n 的多项式, 要么恒等于零。显然 $P(x)-Q(x)$ 不可能是次数高于 n 的多项式, 所以一定恒等于零。

定理 2.2 进一步表明, 不管什么形式的多项式, 只要次数不超过 n , 而且满足相同的插值条件, 那么它们就是恒等的。

上面的两个定理已经表明, 对于多项式插值问题来说, 插值多项式由插值条件惟一决定, 所以我们在口语中所讲的 $n+1$ 个基点的插值问题求解指的就是求出由一组插值条件所惟一决定的多项式。但是允许这个多项式有不同的数学形式, 以便于采用不同的方法进行数值计算。

在后面的两节中, 我们将对相同的插值条件给出两种不同的插值多项式, 即拉格朗日插值多项式和牛顿插值多项式。所以它们应该是恒等的, 名称不同只是各自的形式有所不同。

10.2.3 与曲线拟合的差异

从形式上看, $n+1$ 个基点的插值问题与曲线拟合问题基本相同: 都是由 $n+1$ 个条件决定出一个多项式, 也考虑寻找基函数的线性组合, 但它们的原理、方法以及应用领域都不相同, 基本上是两个类别的问题。

曲线拟合主要应用于误差分析以及多元回归, 数学原理是投影理论, 方法是最小二乘法; 而插值则应用于精确计算, 或者在一个迭代过程中起加速作用, 应该说只是一种经验的方法, 理论基础比较脆弱, 不过可以通过验算来证实结果的有效性。

尽管插值问题与曲线拟合问题都可以看成是逼近理论的应用, 但由于背景知识和应用领域都相距甚远, 编程的风格也大相径庭, 过于看重数学形式上的共同点在这里并不是一件很有意义的事, 所以我们选择了分开讨论。

10.2.4 编程前的准备工作

对于表 2.1 给出的一般形式的 $n+1$ 个基点的多项式插值问题, 如果利用计算机求插值多项式的系数或直接利用插值多项式计算函数值, 可以定义两个长度为 $n+1$ 的数组 X 、 Y , 分别存放插值基点的值以及对应的函数值, 亦即

$$X[k] = x_k, \quad Y[k] = y_k, \quad k = 0, 1, \dots, n$$

在接下来的两节中, 我们将学习两种重要的多项式插值方法, 它们所用到的原始数据都是由表 2.1 形式的表格给出, 因此编写一个专门的程序用表中的数据给相关的数组赋初值是必要的。

例 2.1 给出某个函数 $y=f(x)$ 的一组对应值如表 2.2 所示, 试用插值方法求 $f(0.55)$ 的近似值。

表 2.2 函数 $y=f(x)$ 的对应值列表

k	0	1	2	3	4
x	0.5	0.6	0.7	0.8	0.9
y	1.648 7	1.822 1	2.013 8	2.225 5	2.459 6

解:作为利用计算机求解插值问题的第一步,可以按一般的约定说明两个数组 X, Y , 用一个 C 语言函数 FormProblem() 给数组 X, Y 赋初值,并返回插值基点的个数。相应的源代码见程序 10.01。为方便计,我们把数组 X, Y 放在调用程序中说明,通过指针传递参数。

程序 10.01 输入例 2.1 的原始数据例程

```
int FormProblem(double * X, double * Y, int MN)
{ int TableSize;
  TableSize=5;
  if(TableSize>=MN) return 0;
  X[0]=0.5; Y[0]=1.6487;
  X[1]=0.6; Y[1]=1.8221;
  X[2]=0.7; Y[2]=2.0138;
  X[3]=0.8; Y[3]=2.2255;
  X[4]=0.9; Y[4]=2.4596;
  return TableSize;
}
```

把程序 10.01 以文件名“EXAM21.C”保存为磁盘文件,在以后的案例计算中,简单地通过文件包含即可用相应的方法求解例 2.1。

10.3 拉格朗日(Lagrange)插值方法

拉格朗日插值多项式选用的基函数具有对称性,所以我们可以从中体验到数学的美,事实上它也是后面证明差商具有对称性的基础。正是因为拉格朗日插值多项式所具有的美学价值,所以它对后面的学习有很大的影响。

拉格朗日插值公式看上去形式较复杂,但实际上算法还是非常简单的。

10.3.1 拉格朗日插值公式的构造

对于表 2.1 所给定的 $n+1$ 个基点的多项式插值问题,相应的拉格朗日插值公式,我们记为 $\text{LAGRG}(x, X, Y, n)$,实际就是把插值多项式 $p_n(x)$ 表为基函数

$$LB_0(x), LB_1(x), \dots, LB_n(x)$$

的线性组合,其中 X, Y 是长度为 $n+1$ 的数组,分别存放插值基点以及相应的函数值; x 为插值点;

$$LB_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{(x-x_j)}{(x_k-x_j)}, k=0,1,\dots,n \quad (3.1)$$

而组合系数实际就是各 $y_0, y_1, \dots, y_n$, 因此我们有

$$\text{LAGRG}(x, X, Y, n) = \sum_{k=0}^n y_k L B_k(x) \quad (3.2)$$

在进一步证明由上面的(3.2)式给出的 $\text{LAGRG}(x, X, Y, n)$ 的确是一个插值多项式之前, 读者可以从这样一个事实来体验一下拉格朗日插值公式的对称性: 如果我们对 $n+1$ 个插值基点重新编号, 也就是把表 2.1 中的列的位置任意互换, 由于(3.1)式中的乘法具有交换律, (3.2)式中的加法也具有交换律, 所以相应的拉格朗日插值公式并没有发生变化, 从而验证了定理 2.2 的正确性。

显然, 所有的基函数 $L B_k(x)$ 都是 n 次多项式, 所以 $\text{LAGRG}(x, X, Y, n)$ 是次数不超过 n 的多项式。为了证明 $\text{LAGRG}(x, X, Y, n)$ 的确是插值多项式, 还需证明它一定满足插值条件。

定理 3.1 对于由表 2.1 给出的一般的 $n+1$ 个基点的多项式插值问题以及由(3.1)式和(3.2)式给出的拉格朗日插值多项式, 一定有

$$\text{LAGRG}(x_k, X, Y, n) = y_k, \quad k = 0, 1, \dots, n$$

证明: 注意到所有的基函数 $L B_k(x), k=0, 1, \dots, n$ 均具有性质

$$L B_k(x_j) = \delta_{kj} = \begin{cases} 1, & j = k \\ 0, & j \neq k \end{cases}, \quad j = 0, 1, \dots, n$$

所以对任意 $j=0, 1, \dots, n$, 用 x_j 替代 $\text{LAGRG}(x, X, Y, n)$ 中的 x , 我们有

$$\begin{aligned} \text{LAGRG}(x_j, X, Y, n) &= \sum_{k=0}^n y_k L B_k(x_j) \\ &= y_j L B_j(x_j) + \sum_{\substack{k=0 \\ k \neq j}}^n y_k L B_k(x_j) \\ &= y_j \end{aligned}$$

所以 $\text{LAGRG}(x, X, Y, n)$ 满足插值条件。证毕。

10.3.2 基函数的性质

定理 3.2 对于由表 2.1 给出的一般的函数值列表以及由(3.1)式定义的基函数 $\{L B_k(x), k=0, 1, \dots, n\}$, 我们有

$$\sum_{k=0}^n L B_k(x) \equiv 1$$

证明: 考虑被插函数为 $f(x) \equiv 1$ 的特殊多项式插值问题, 对于表 2.1 给出的插值基点 $x_0, x_1, \dots, x_n$, 对应的函数值 $y_0, y_1, \dots, y_n$ 均为 1, 从而相应的拉格朗日插值多项式为

$$\text{LAGRG}(x, X, Y, n) = \sum_{k=0}^n [1 \times L B_k(x)]$$

它满足插值条件

$$\text{LAGRG}(x_j, X, Y, n) = \sum_{k=0}^n [1 \times L B_k(x_j)] = y_j = 1, \quad \forall j = 0, 1, \dots, n$$

而 $f(x) \equiv 1$ 也是次数不超过 n 的多项式, 也满足插值条件

$$f(x_k) = y_k, k = 0, 1, \dots, n$$

由插值多项式的惟一性定理立即推出我们的结论。证毕。

10.3.3 拉格朗日插值的列表计算方法

如果采用列表的方式计算,可以按下面的表 3.1 的格式设计算法。

表 3.1 拉格朗日插值列表计算格式

k	$X[k]$	$Y[k]$	$LB[k]$	$LT[k]$
0	x_0	y_0	$LB_0(x)$	$y_0 LB[0]$
1	x_1	y_1	$LB_1(x)$	$y_1 LB[1]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	x_n	y_n	$LB_n(x)$	$y_n LB[n]$

其中:数组 X, Y 存放给定的函数值列表;

数组 LB 存放基函数值, $LB[k] = LB_k(x), k = 0, 1, \dots, n$;

数组 LT 存放插值多项式各项的值, $LT[k] = y_k LB[k], k = 0, 1, \dots, n$ 。

采用列表计算的方法求解例 2.1 的源代码见程序 10.02, 函数 $GetLB(\text{int } k, \text{double } x)$ 的作用是计算基函数值 $LB_k(x)$ 。无论是手工计算还是机器计算,把各 $LB[k]$ 的值累加起来,结果应该为 1,所以可以据此检验计算是否正确;各 $LT[k]$ 的值累加起来即为插值多项式的值,所以在程序中专门说明两个变量 $SUMLB$ 和 $SUMLT$ 来保存这两个累加值。

程序 10.02 列表计算拉格朗日插值多项式的值

```
#include "EXAM21.C"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#define MN 21
int RN=0;
double X[MN], Y[MN], LB[MN], LT[MN];
double X0, Y0, SUMLB=0, SUMLT=0;
char fname[32];
double GetLB(int k, double x)
{ double y=1.0;
  int j;
  for(j=0; j<RN; j++)
  { if(j==k) continue;
    y=y*(x-X[j])/(X[k]-X[j]);
  }
  return y;
}
double Operation(double x)
```

```

{ int k;
  X0=x;
  SUMLB=0;
  SUMLT=0;
  for(k=0;k<RN;k++)
  { LB[k]=GetLB(k,x);
    LT[k]=LB[k]*Y[k];
    SUMLB+=LB[k];
    SUMLT+=LT[k];
  }
  Y0=SUMLT;
  return Y0;
}
int ShowTable(){...}
int SaveTable(char * fname){...}
main()
{ double x,y;
  x=0.55;
  RN=FormProblem(X,Y,MN);
  Operation(x);
  ShowTable();
  SaveTable("TB0302.TXT");
  getchar();
  return;
}

```

程序 10.02 运行的结果见下面的表 3.2,从所显示的 SUMLB=1 印证了拉格朗日插值基函数的性质,也表明程序运行正常。

表 3.2 程序的运行结果

k	$X[k]$	$Y[k]$	$LB[k]$	$LT[k]$
0	0.500 000 0	1.648 700 0	0.273 437 5	0.450 816 4
1	0.600 000 0	1.822 100 0	1.093 750 0	1.992 921 9
2	0.700 000 0	2.013 800 0	-0.546 875 0	-1.101 296 9
3	0.800 000 0	2.225 500 0	0.218 750 0	0.486 828 1
4	0.900 000 0	2.459 600 0	-0.039 062 5	-0.096 078 1
X0=0.550 000 000 000 000 0				
SUMLB=1.000 000 000 000 000 0				
SUMLT=1.733 191 406 250 000 5				

10.3.4 应用程序设计

一方面利用计算机求拉格朗日插值多项式的值编程比较容易,另一方面许多应用程序中可以利用插值的方法“试探”一下,看能否得到更精确的结果,所以通常的应用只是对

已经得到的一些点处的函数值再利用插值方法计算 1 个或少量几个点处的插值的结果。在这种情况下,有一个便于应用的程序比起节省计算量来说显得更为重要。所以在实际应用中,应用拉格朗日插值多项式进行计算的机会也相当多。

拉格朗日插值的应用程序的源代码见程序 10.03,由于程序很简单,需要用到时,直接写在其他应用程序文件中也行。在程序 10.03 中,函数

`double LAGRGR(double x, double * X, double * Y, int n)`

的形式参数 n 为实际插值基点的个数,其余的形式参数以及函数的返回值与函数 `LAGRGR(x, X, Y, n)` 中的定义相同。

程序 10.03 拉格朗日插值应用程序

```
double LAGRGR(double x, double * X, double * Y, int n)
{ int k, j;
  double sum=0.0, base=1.0;
  for(k=0; k<n; k++)
  { base=1.0;
    for(j=0; j<n; j++)
    { if(j==k) continue;
      base *= (x-X[j])/(X[k]-X[j]);
    }
    sum += Y[k] * base;
  }
  return sum;
}
```

可以把程序 10.02B 以文件名“Lagrange.c”作为磁盘文件保存,应用程序可以通过文件包含来调用 `LAGRGR()` 函数。

10.3.5 基函数的等价形式

拉格朗日插值公式由于形式简单,在许多情况下还可用来解决与插值有关的一些理论问题,为此还需要对基函数作一些更深入的分析。

记
$$\lambda_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n (x - x_j)$$

则有
$$LB_k(x) = \frac{\lambda_k(x)}{\lambda_k(x_k)}$$

再记
$$\pi_{n+1}(x) = \prod_{j=0}^n (x - x_j)$$

则有
$$\lambda_k(x) = \frac{\pi_{n+1}(x)}{(x - x_k)}$$

以及
$$\lambda_k(x_k) = \pi'_{n+1}(x_k)$$

所以有
$$LB_k(x) = \frac{1}{(x - x_k) \cdot \pi'_{n+1}(x_k)} \pi_{n+1}(x)$$

利用各 $LB_k(x)$ 的等价形式,可以进一步得到 $LAGRG(x, X, Y, n)$ 的等价形式

$$LAGRG(x, X, Y, n) = \sum_{k=0}^n \frac{y_k \pi_{n+1}(x)}{(x - x_k) \pi'_{n+1}(x_k)} \quad (3.3)$$

亦即

$$LAGRG(x, X, Y, n) = \left(\sum_{k=0}^n \frac{y_k}{(x - x_k) \pi'_{n+1}(x_k)} \right) \pi_{n+1}(x) \quad (3.4)$$

作为数值计算来说,上面的(3.3)式和(3.4)式都没有太大的实际意义。尽管看起来由于提公因式而使得计算量有所减少,但程序明显地会变得复杂。如果单纯从计算量的角度考虑,拉格朗日插值法不如下一节将要介绍的牛顿插值法。(3.3)式的意义在于,它可以方便地导出拉格朗日插值多项式的最高次项的系数的数学形式,这正是我们所需要的结果。

定理 3.3 对于由表 2.1 给出的一般的 $n+1$ 个基点的多项式插值问题来说,相应的拉格朗日插值多项式的最高次项的系数为

$$\sum_{k=0}^n \frac{y_k}{\pi'_{n+1}(x_k)} \quad (3.5)$$

上面的(3.5)式也是拉格朗日插值公式具有对称性的表现形式。如果我们在表 2.1 中只是改变一下插值基点的排列顺序,相应的多项式插值问题本身并没有改变,再由插值多项式的惟一性即可得到相应的拉格朗日插值多项式的最高次项的系数也不会改变,利用加法的交换律,这个结果在上面的(3.5)式中同样得到充分体现。

10.4 差商与牛顿插值公式

对于表 2.1 给出的一般的 $n+1$ 个基点的多项式插值问题来说,相应的拉格朗日插值公式由于具有对称性,因而数学形式优美,求数值解的算法简单,但也有它自身的缺点,就是灵活性不够。假如利用 $n+1$ 个基点的插值多项式进行近似计算达不到所需要的精度,通常的办法是增加一个插值基点而得到更高次数的插值多项式。如果利用拉格朗日插值公式计算,那么前面所得到的 $n+1$ 个基点的拉格朗日插值多项式在新的计算中难以发挥作用。牛顿插值公式正好可以弥补拉格朗日插值公式这方面的不足,假如我们已经得到了 $n+1$ 个基点的牛顿插值多项式,如果再增加一个插值基点,也只是在原来的基础上增加一个修正项即可,我们把牛顿插值公式的这种特性称为承袭性。

10.4.1 具有承袭性插值多项式的基函数形式

假设对于具有 $k+1(k \geq 0)$ 个插值基点的多项式插值问题已经得到了相应的插值多项式 $p_k(x)$,如果它不能满足精度要求,则希望通过增加一个插值基点 (x_{k+1}, y_{k+1}) ,并利用已得到的 $p_k(x)$ 来构造新的、具有 $k+2$ 个插值基点的插值多项式 $p_{k+1}(x)$ 。为此,把 $p_{k+1}(x)$ 表示为 $p_k(x)$ 与一个修正项(记为 $r_{k+1}(x)$)的和的形式是合理的,即

$$p_{k+1}(x) = p_k(x) + r_{k+1}(x) \quad (4.1)$$

这样一来,只要确定了 $r_{k+1}(x)$,即可得到 $p_{k+1}(x)$ 。此时称上面的(4.1)式为具有承袭性

的插值多项式。

注意到(4.1)式中的 $p_k(x)$ 和 $p_{k+1}(x)$ 均应满足前面的 $k+1$ 个插值条件

$$p_k(x_j) = p_{k+1}(x_j) = y_j, j = 0, 1, \dots, k \quad (4.2)$$

所以我们可以立即得到

$$r_{k+1}(x_j) = 0, j = 0, 1, \dots, k \quad (4.3)$$

再注意到 $r_{k+1}(x)$ 在一般情况下应当是 $k+1$ 次多项式, 因此 $r_{k+1}(x)$ 可以表示为

$$r_{k+1}(x) = c_{k+1}(x-x_0)(x-x_1)\cdots(x-x_k) \quad (4.4)$$

其中 c_{k+1} 为待定常数。事实上它可以由最后新增的一个插值条件 $p_{k+1}(x_{k+1}) = y_{k+1}$ 来确定, 但我们现在不必去考虑它。记

$$\pi_{k+1}(x) = (x-x_0)(x-x_1)\cdots(x-x_k) \quad (4.5)$$

则有

$$r_{k+1}(x) = c_{k+1}\pi_{k+1}(x)$$

而且还有

$$\pi_{k+1}(x_j) = 0, j = 0, 1, \dots, k \quad (4.6)$$

为方便计, 我们特别约定 $\pi_0(x) \equiv 1$, 从而还有

$$\pi_{k+1}(x) = (x-x_k)\pi_k(x)$$

对于由表 2.1 给出的一般的具有 $n+1$ 个插值基点的多项式插值问题, 令 k 分别取 $0, 1, \dots, n-1$, 上面的(4.1)式到(4.6)式仍然成立。现在我们令 k 依次取 $n-1, n-2, \dots, 0$, 反复利用(4.1)式和(4.4)式即可得到

$$p_n(x) = c_0 + c_1\pi_1(x) + \cdots + c_n\pi_n(x)$$

或

$$p_n(x) = c_0\pi_0(x) + c_1\pi_1(x) + \cdots + c_n\pi_n(x) \quad (4.7)$$

上面的(4.7)式是具有承袭性的插值多项式的一般形式, 它可以看作是一组基函数

$$\{\pi_k(x) \mid k = 0, 1, \dots, n\}$$

的线性组合。

为了使(4.7)式真正成为插值多项式, 还需要根据插值条件确定组合系数 $c_0, c_1, \dots, c_n$ 。为此, 可以在(4.7)式中令 x 分别取 $x_0, x_1, \dots, x_n$, 并利用插值条件

$$p_n(x_j) = f(x_j), j = 0, 1, \dots, n$$

和(4.6)式, 从而得到

$$\left. \begin{aligned} f(x_0) &= c_0 \\ f(x_1) &= c_0 + c_1\pi_1(x_1) \\ &\vdots \\ f(x_n) &= c_0 + c_1\pi_1(x_n) + \cdots + c_n\pi_n(x_n) \end{aligned} \right\} \quad (4.8)$$

上面的线性方程组(4.8)是一个下三角形的线性方程组, 直接求它的数值解也很容易, 至少比前面给出的线性方程组(2.2)容易得多。

10.4.2 差商的定义与计算方法

为了深入研究线性方程组(4.8)的解的特性,并进一步引出牛顿插值公式,我们首先介绍差商的定义以及一些相关的知识。

定义 4.1 设 $f(x)$ 是定义在某区间内取连续变量值或离散变量值的实值函数,且已知 $f(x)$ 在 $n+1$ 个互异的离散点 $x_0, x_1, \dots, x_n$ 处的函数值 $f(x_0), f(x_1), \dots, f(x_n)$, 称

$$f[x_k] = f(x_k), \quad k = 0, 1, \dots, n \quad (4.9)$$

为 $f(x)$ 在 x_k 处的零阶差商;

$$f[x_i, x_j] = \frac{f[x_j] - f[x_i]}{x_j - x_i}, \quad i \neq j, 0 \leq i, j \leq n \quad (4.10)$$

为 $f(x)$ 在 x_i, x_j 处的一阶差商;

$$f[x_i, x_j, x_k] = \frac{f[x_i, x_k] - f[x_i, x_j]}{x_k - x_j}, \quad k \neq j, 0 \leq i, k, j \leq n \quad (4.11)$$

为 $f(x)$ 在 x_i, x_j, x_k 处的二阶差商。

一般地,如果定义了 $f(x)$ 的 $k-1$ 阶差商,则可以定义 $f(x)$ 的 k 阶差商为:

$$f[x_0, \dots, x_{k-1}, x_k] = \frac{f[x_0, \dots, x_{k-2}, x_k] - f[x_0, \dots, x_{k-2}, x_{k-1}]}{x_k - x_{k-1}} \quad (4.12)$$

这里, k 可取值 $2, 3, \dots, n$ 。

差商的数值计算可以按照与定义平行的方式完成。由于在后面的应用程序中所用到的差商只是 $f[x_0], f[x_0, x_1], \dots, f[x_0, x_1, \dots, x_k], \dots, f[x_0, x_1, \dots, x_n]$, 所以差商的计算可以按下面的表 4.1 的格式进行。

表 4.1 差商的数值计算表格

k	0	1	2	3	4	...
	x_0	x_1	x_2	x_3	x_4	...
0	$f[x_0]$	$f[x_1]$	$f[x_2]$	$f[x_3]$	$f[x_4]$	...
1		$f[x_0, x_1]$	$f[x_0, x_2]$	$f[x_0, x_3]$	$f[x_0, x_4]$	...
2			$f[x_0, x_1, x_2]$	$f[x_0, x_1, x_3]$	$f[x_0, x_1, x_4]$	...
3				$f[x_0, x_1, x_2, x_3]$	$f[x_0, x_1, x_2, x_4]$	...
4					$f[x_0, x_1, x_2, x_3, x_4]$	...
...						...

利用差商表进行手工计算还是比较简单的(由此说明相应的计算方法简单),开始的三行其实也就是表 2.1 的内容。由 k 值为 1 的行开始,既可以从上往下一行一行按定义计算,也可以从左向右一列一列按定义计算。每个单元格中的差商值总等于它上面一行中同列的差商值与主对角线上的差商值之差与相应的插值基点值之差的商。例如计算

$$f[x_0, x_1, x_2, x_4] = \frac{f[x_0, x_1, x_4] - f[x_0, x_1, x_2]}{x_4 - x_2}$$

就可以利用表 4.1 中带阴影的单元格中的元素计算。

对于例 2.1 给出的函数值列表,模仿列表格式计算差商的 C 语言源代码见程序 10.04。程序中的二维数组 DQT 存放表 4.1 给出的差商表, $DQT[i][j]$ 表示表 4.1 中 k 值为 i 的行 j 的列所对应的单元格中的数值。程序中的另一个变量 RN 表示插值基点的个数。程序运行所显示的结果如表 4.2 所示。

表 4.2 利用表 2.1 中的数据所得到的差商表

k:	0	1	2	3	4
X:	0.50000	0.60000	0.70000	0.80000	0.90000
Q[0]:	1.64870	1.82210	2.01380	2.22550	2.45960
Q[1]:		1.73400	1.82550	1.92267	2.02725
Q[2]:			0.91500	0.94333	0.97750
Q[3]:				0.28333	0.31250
Q[4]:					0.29167

程序 10.04 列表格式计算差商

```
#include "EXAM21.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 21
int RN=0;
double X[MN], Y[MN];
double DQT[MN][MN];
int Operation()
{ int i,j;
  double temp;
  for(i=0;i<MN;i++)
    for(j=0;j<RN;j++)
      DQT[i][j]=0.0;
  for(j=0;j<RN;j++) DQT[0][j]=Y[j];
  for(j=1;j<RN;j++)
  { for(i=0;i<j;i++)
    { temp= DQT[i][j]-DQT[i][i];
      DQT[i+1][j]=temp/(X[j]-X[i]);
    }
  }
  return 0;
}

int ShowTable(){...}

main()
{ RN=FormProblem(X, Y, MN);
  Operation();
  ShowTable();
  getchar();
  return;
}
```

由于在后面的实际应用中,我们所用到的差商也只是

$$f[x_0], f[x_0, x_1], \dots, f[x_0, x_1, \dots, x_n]$$

所以,专门用一个长度为 $n+1$ 的数组 F 来存放表 4.1 主对角线上的插商值是合理的,从而有

$$F[k] = f[x_0, x_1, \dots, x_k], k = 0, 1, \dots, n$$

直接编写一个 C 语言函数对表 2.1 给出的函数值列表计算相应的差商 $F[k], k=0, 1, \dots, n$ 是合理的,也是方便的,相应的 C 语言函数可以说明为

```
int GetDQTF(double * X, double * Y, double * F, int n)
```

其中 n 为插值基点的个数。以计算例 2.1 表 2 完整的源代码见程序 10.05, 程序运行的结果如表 4.3 所示。

表 4.3 利用表 2.1 中的数据所得到的实用差商表

k	X[k]	Y[k]	F[k]
0	0.5000	1.6487	1.64870000
1	0.6000	1.8221	1.73400000
2	0.7000	2.0138	0.91500000
3	0.8000	2.2255	0.28333333
4	0.9000	2.4596	0.29166667

程序 10.05 中的函数 GetDQTF() 实际上特别简单, 关键代码也就是一个二重循环体中的一条语句。程序特别简单反而不太好理解, 也不太好作详细解释, 相对于程序 10.04 来说也只是没有保留不必要的中间结果, 换了数组名而已。读者可以对照着阅读源代码来加深对算法的理解。

程序 10.05 利用例 2.1 给出的函数值列表计算差商表

```
#include "EXAM21.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 21
int RN=0;
double X[MN], Y[MN], F[MN];
int GetDQT(double * X, double * Y, double * F, int n)
{ int k, j;
  for(k=0; k<n; k++) F[k]=Y[k];
  for(k=1; k<n; k++)
    for(j=0; j<k; j++)
      F[k]=(F[k]-F[j])/(X[k]-X[j]);
  return 0;
}
ShowTable(){...}
main()
{ RN=FormProblem(X, Y, MN);
  if(RN==0) return;
  GetDQT(X, Y, F, RN);
  ShowTable();
}
```



```

    getch();
    return;
}

```

10.4.3 差商的重要性质

定理 4.1 设 $f(x)$ 是定义在某区间内取连续变量值或离散变量值的实值函数, 对该区间内任意 $k+1$ 个 ($k \geq 0$) 互异的离散点 $x_0, x_1, \dots, x_k$ 以及任意异于 $x_0, x_1, \dots, x_k$ 的点 x , 我们有

$$f[x] = f[x_0] + f[x_0, x_1]\pi_1(x) + \dots + f[x_0, \dots, x_k]\pi_k(x) + f[x_0, \dots, x_k, x]\pi_{k+1}(x) \quad (4.13)$$

证明: 显然, 当 $k=0$ 时结论平凡成立;

当 $k=1$ 时, 由

$$f[x_0, x] = \frac{f[x] - f[x_0]}{x - x_0}$$

立即推得

$$f[x] - f[x_0] = f[x_0, x](x - x_0)$$

亦即

$$f[x] = f[x_0] + f[x_0, x_1]\pi_1(x)$$

所以当 $k=1$ 时结论成立。

假设当 k 取某个 $0 \leq j < n$ 时结论也成立, 即

$$f[x] = f[x_0] + \dots + f[x_0, \dots, x_j]\pi_j(x) + f[x_0, \dots, x_j, x]\pi_{j+1}(x) \quad (4.14)$$

则当 $k=j+1$ 时, 由差商的定义得

$$f[x_0, \dots, x_j, x_{j+1}, x] = \frac{f[x_0, \dots, x_j, x] - f[x_0, \dots, x_j, x_{j+1}]}{x - x_{j+1}}$$

整理后得

$$f[x_0, \dots, x_j, x] = f[x_0, \dots, x_j, x_{j+1}] + f[x_0, \dots, x_j, x_{j+1}, x](x - x_{j+1})$$

再代入到 (4.14) 式中, 并利用

$$(x - x_{j+1})\pi_{j+1}(x) = \pi_{j+2}(x)$$

即可推得

$$f[x] = f[x_0] + \dots + f[x_0, \dots, x_j, x_{j+1}]\pi_{j+1}(x) + f[x_0, \dots, x_j, x_{j+1}, x]\pi_{j+2}(x)$$

所以结论对 $k=j+1$ 也成立。

所以, (4.13) 式对所有的 $k \geq 0$ 都成立, 证毕。

10.4.4 牛顿插值公式

现在, 我们只要把差商的定义以及上面的定理 4.1 与表 2.1 所确定的 $n+1$ 个基点的插值问题联系起来, 即可导出具有承袭性的牛顿插值多项式。

定理 4.2 对于表 2.1 所确定的 $n+1$ 个基点的多项式插值问题, 对任意 $k=0, 1, \dots, n-1$, 我们有

$$f[x_{k+1}] = f[x_0] + \dots + f[x_0, \dots, x_k]\pi_k(x_{k+1}) + f[x_0, \dots, x_k, x_{k+1}]\pi_{k+1}(x_{k+1}) \quad (4.15)$$

证明:在(4.13)式中令 $x=x_{k+1}$ 立即得证。

定理 4.3 对于表 2.1 所确定的 $n+1$ 个基点的多项式插值问题,我们有

$$c_k = f[x_0, x_1, \dots, x_k], k = 0, 1, \dots, n$$

为线性方程组(4.8)的解。

这个定理实际上是上面的定理 2 的推论。

事实上,只要按 $k=0, 1, \dots, n-1$ 的顺序把上面的(4.15)式重写一遍,在最前面补上 $f[x_0]=f[x_0]$,再与(4.8)式对比,利用解的惟一性以及 $f[x_k]=f(x_k), k=0, 1, \dots, n$ 即可完成证明。

作为练习,我们鼓励读者直接根据差商的定义采用数学归纳法完成证明。

定理 4.1 和定理 4.2 表明,对于 $k=1, 2, \dots, n$,记

$$N_k(x) = f[x_0] + f[x_0, x_1]\pi_1(x) + \dots + f[x_0, \dots, x_k]\pi_k(x) \quad (4.16)$$

是相应于插值条件

$$N_k(x_j) = f(x_j), j = 0, 1, \dots, k$$

的、具有承袭性的插值多项式。

特别地在(4.12)式和上面的(4.16)式中取 $k=n$,并令

$$R_n(x) = f[x_0, x_1, \dots, x_n, x]\pi_{n+1}(x) \quad (4.17)$$

我们有

$$\left. \begin{aligned} N_n(x) &= f[x_0] + f[x_0, x_1]\pi_1(x) + \dots + f[x_0, x_1, \dots, x_n]\pi_n(x) \\ R_n(x) &= f[x_0, x_1, \dots, x_n, x]\pi_{n+1}(x) \\ f(x) &= N_n(x) + R_n(x) \end{aligned} \right\} \quad (4.18)$$

它们分别称为牛顿插值多项式、插值余项和牛顿插值公式。至此,我们已经得到了具有承袭性的插值多项式。

10.4.5 计算表格与计算方法说明

对于给定的 $n+1$ 个基点的插值问题以及自变量的任意取值 x ,我们建议按如下的表格计算牛顿插值多项式的系数以及多项式的值:

k	$X[k]$	$Y[k]$	$F[k]$	$NB[k]$	$NT[k]$
0	x_0	x_0	$F[0]$	$NB[0]$	$NT[0]$
1	x_1	x_1	$F[1]$	$NB[1]$	$NT[1]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	x_n	x_n	$F[n]$	$NB[n]$	$NT[n]$

其中:数组 X, Y 和 F 已经在前面说明过;数组 NB 和 NT 分别存放基函数值以及插值多项式各项的值。从而对 $k=0, 1, \dots, n$,我们有

$$NB[k] = \pi_k(x);$$

$$NT[k] = F[k] \times NB[k].$$

数组 NT 的所有元素之和即为牛顿插值的结果。填表计算的方法还是很简单的,前面的

程序 10.05 已经解决了 F 列的计算问题,各 $NB[k]$ 和 $NT[k]$ 也可利用递推方式计算。

采用列表计算的方法求解例 2.1 的源代码见程序 10.06,运行程序所显示的结果如表 4.4 所示,与程序 10.02 所得到的结果相同又一次证实了插值多项式的惟一性。

表 4.4 牛顿插值法求解例 2.1 的计算结果

k	x[k]	y[k]	F[k]	NB[k]	NT[k]
0	0.5000	1.6487	1.64870000	1.00000000	1.64870000
1	0.6000	1.8221	1.73400000	0.05000000	0.08670000
2	0.7000	2.0138	0.91500000	-0.00250000	-0.00228750
3	0.8000	2.2255	0.28333333	0.00037500	0.00010625
4	0.9000	2.4598	0.29166667	-0.00009375	-0.00002734
X0 =	0.5500000000000000				
Y0 =	1.7331914062500000				

程序 10.06 牛顿插值法的列表计算程序

```
#include "EXAM21.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 21
int RN=0;
double X[MN],Y[MN],F[MN],NB[MN],NT[MN];
double X0,Y0;
int GetDQT(double *X,double *Y,double *F,int N){...}
double Operation(double x)
{ int k;
  double state=1.0,y=0;
  X0=x;
  for(k=0;k<RN;k++)
  { NB[k] = state;
    NT[k] = F[k] * NB[k];
    y += NT[k];
    state *= x-X[k];
  }
  Y0=y;
  return Y0;
}
ShowTable(){...}
main()
{ double x,y;
  x=0.55;
  RN=FormProblem(X,Y,MN);
  if(RN>MN) return;
  GetDQT(X,Y,F,RN);
  y=Operation(x);
  ShowTable();
  getchar();
  return;
}
```

10.5 插值余项与差商的性质讨论

10.5.1 关于插值余项的讨论

定义 5.1 对于一般的 $n+1$ 个基点的多项式插值问题, 设 $f(x)$ 为被插值函数, $P_n(x)$ 为相应的插值多项式, 记 $R_n(x)$ 为 $f(x)$ 与 $P_n(x)$ 的差, 即

$$f(x) = P_n(x) + R_n(x) \quad (5.1)$$

则 $R_n(x)$ 就是用 $P_n(x)$ 近似替代 $f(x)$ 的误差, 我们称它为插值余项。

显然, 由插值多项式的惟一性可以导出插值余项的惟一性。对于一般的 $n+1$ 个基点的多项式插值问题, 我们在前面的三节中分别给出了三种不同的方法: 一般方法、拉格朗日插值法和牛顿插值法。由惟一性定理可知, 不同的方法得到的不同形式的不超过 n 次的多项式由于在 $n+1$ 个不同的点处的值相同, 所以它们实际上是恒等的, 所以相应的插值余项也是恒等的。

定理 5.1 设 $f(x)$ 是定义在 $[a, b]$ 上的具有直至 $n+1$ 阶连续导数的被插值函数, 又假设 $n+1$ 个插值基点满足 $a \leq x_0 < x_1 < \cdots < x_n \leq b$, $P_n(x)$ 为相应的插值多项式, $R_n(x)$ 为插值余项, 则对任意固定的 $x \in (a, b)$, 我们有

$$R_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(\xi) \pi_{n+1}(x) \quad (5.2)$$

证明: 显然, 如果 x 等于 $x_0, x_1, \cdots, x_n$ 中的某一个值, 则有 $\pi_{n+1}(x) = 0$ 。由于 $p_n(x)$ 一定满足插值条件, 所以又有 $p_n(x) = f(x)$, 从而 $R_n(x) = 0$, 所以定理的结论平凡成立。

对任意给定的异于 $x_0, x_1, \cdots, x_n$ 的 $x \in (a, b)$, 作辅助函数

$$g(t) = f(t) - P_n(t) - \frac{R_n(x)}{\pi_{n+1}(x)} \pi_{n+1}(t)$$

显然当 t 取 $x_0, x_1, \cdots, x_n$ 以及给定的 x 时有 $g(t) = 0$, 从而 $g(t)$ 在 (a, b) 内有 $n+2$ 个零点, 所以由罗尔定理可知值 $g'(t)$ 在 (a, b) 内有 $n+1$ 个零点, $g''(t)$ 在 (a, b) 内有 n 个零点……依此类推, $n+1$ 次应用罗尔定理, 则存在 $\xi \in (a, b)$ 使得 $g^{(n+1)}(\xi) = 0$ 。注意到

$$[P_n(t)]^{(n+1)} = 0, \quad [\pi_{n+1}(t)]^{(n+1)} = (n+1)!$$

以及 x 是常数, 所以有

$$f^{(n+1)}(\xi) - \frac{R_n(x)}{\pi_{n+1}(x)} (n+1)! = 0$$

由此可以立即推出(5.2)式, 证毕。

10.5.2 差商与导数的关系

在前面的 10.4 节中, 我们实际上利用牛顿插值公式给出了多项式插值余项的另一种数学形式。对 (a, b) 内任意异于 $x_0, x_1, \cdots, x_n$ 的 x , 我们有

$$R_n(x) = f[x_0, x_1, \cdots, x_n, x] \pi_{n+1}(x) \quad (5.3)$$

对比上面的(5.2)式和(5.3)式, 利用插值余项的惟一性立即得出

$$f[x_0, \cdots, x_n, x] \pi_{n+1}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \pi_{n+1}(x)$$

从而进一步得到

$$f[x_0, \dots, x_n, x] = \frac{f^{(n+1)}(\xi)}{(n+1)!} \quad (5.4)$$

我们可以采用定理的形式说明上面的(5.4)式。

定理 5.2 设 $f(x)$ 是某区间 (a, b) 内的 n 阶连续可微函数, 则对 (a, b) 内任意 $n+1$ 个互异的点 $x_0, x_1, \dots, x_n$, 总存在 $\xi \in (a, b)$, 使得

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!} \quad (5.5)$$

10.5.3 多项式的差商

上面的定理 5.2 给出了一般的连续可微函数的差商与导数的关系。如果 $f(x)$ 为 n 次多项式, 那么我们有

$$f^{(n+1)}(x) = 0, f^{(n)}(x) = (n!)a_n$$

其中 a_n 为 $f_n(x)$ 的 n 次项的系数。这个结论可以表示为下面的定理。

定理 5.3 任何 n 次多项式函数的 $n+1$ 阶差商均为零, n 阶差商均为常数。

证明: $f_n(x)$ 是所讨论的 n 次多项式, 记 $p_n(x)$ 是 $f_n(x)$ 的 $n+1$ 个互异的基点 $x_0, x_1, \dots, x_n$ 的牛顿插值多项式, 由定理 2.2 可知, $p_n(x)$ 与 $f_n(x)$ 恒等, 从而插值余项 $R_n(x) = 0$ 。对任意异于 $x_0, x_1, \dots, x_n$ 的项 x , 显然 (4.17) 式中的 $\pi_{n+1}(x) \neq 0$, 从而有 $f[x_0, x_1, \dots, x_n, x] = 0$ 。

由于 $p_n(x)$ 与 $f_n(x)$ 恒等, 所以它们的最高次项的系数也相同。设 $f_n(x)$ 的 n 次项的系数为 a_n , 而 $p_n(x)$ 的 n 次项的系数为 $f[x_0, x_1, \dots, x_n]$, 所以 $f[x_0, x_1, \dots, x_n]$ 为常数。证毕。

10.5.4 差商的重要性质

利用前面已经得到的一系列结果, 我们可以推出关于差商的两个最重要的性质。

定理 5.4 设 $f(x)$ 是定义在某区间 (a, b) 内的实函数, $x_0, x_1, \dots, x_n$ 是 (a, b) 内 $n+1$ 个互异的点, 则 $f[x_0, x_1, \dots, x_n]$ 可表示为 $f(x_0), f(x_1), \dots, f(x_n)$ 的线性组合。

证明: 考察 $f(x)$ 的 $n+1$ 个基点 $x_0, x_1, \dots, x_n$ 的牛顿插值多项式 $N_n(x)$ 和拉格朗日插值多项式 $L_n(x)$, 由于它们在 $n+1$ 个不同点处的函数值相同, 所以是恒等多项式, 从而最高次项的系数相同。

利用 (3.5) 式给出的拉格朗日插值多项式 $L_n(x)$ 的最高次项的系数表示形式, 再把其中的各 y_k 替换为 $f(x_k)$, $k=0, 1, \dots, n$, 立即得到

$$f[x_0, x_1, \dots, x_n] = \sum_{k=0}^n [\pi'_{n+1}(x_k)]^{-1} f(x_k) \quad (5.6)$$

证毕。

定理 5.4 的意义在于, 如果我们需要某个 $f(x)$ 在某一点 x^* 处的 n 阶导数值, 可以利用 (5.5) 式, 找到 x_0 附近的 $n+1$ 个互异的点 $x_0, x_1, \dots, x_n$, 从而有

$$f^{(n)}(x^*) = n! f[x_0, x_1, \dots, x_n]$$

再利用(5.6)式,从而有

$$f^{(n)}(x^*) = n! \sum_{k=0}^n [\pi'_{n+1}(x_k)]^{-1} f(x_k) = \sum_{k=0}^n [\lambda_k f(x_k)] \quad (5.7)$$

这为我们后面求数值微分,求微分方程数值解提供了理论依据。而且,如果按某种规则确定 $x_0, x_1, \dots, x_n$, 那么相应的 $\lambda_0, \lambda_1, \dots, \lambda_n$ 都是固定常数,从而相应的计算特别方便。

定理 5.5 设 $f(x)$ 是定义在某区间 (a, b) 内的实函数, $x_0, x_1, \dots, x_n$ 是 (a, b) 内 $n+1$ 个互异的点,又假设 $K(0), K(1), \dots, K(n)$ 是 $0, 1, \dots, n$ 的任意一个排列,则有

$$f[x_0, x_1, \dots, x_n] = f[x_{K(0)}, x_{K(1)}, \dots, x_{K(n)}]$$

证明:仅改变插值基点的排列顺序所得到的两个牛顿插值多项式是恒等的,所以他们的最高次项的系数也相等,证毕。

定理 5.5 称为差商的对称性,直接从(5.6)式以及加法的交换律也可以得出这个结论。

大概是过去为了手工列表计算的方便,差商的递推形式的定义是:

若定义了 $f(x)$ 在 $x_0, x_1, \dots, x_k$ 处和 $x_1, \dots, x_k, x_{k+1}$ 处的 k 阶差商 $f[x_0, x_1, \dots, x_k]$ 和 $f[x_1, \dots, x_k, x_{k+1}]$, 则定义 $f(x)$ 在 $x_0, x_1, \dots, x_k, x_{k+1}$ 处的 $k+1$ 阶差商为

$$f[x_0, x_1, \dots, x_k, x_{k+1}] = (f[x_1, \dots, x_k, x_{k+1}] - f[x_0, \dots, x_k]) / (x_{k+1} - x_0)$$

相应的计算表格如下面的表 5.1 所示。

表 5.1 差商计算表格

x_k	零阶差商	一阶差商	二阶差商	三阶差商	四阶差商	...
x_0	$f[x_0]$					$\vdots$
x_1	$f[x_1]$	$f[x_0, x_1]$				$\vdots$
x_2	$f[x_2]$	$f[x_1, x_2]$	$f[x_0, x_1, x_2]$			$\vdots$
x_3	$f[x_3]$	$f[x_2, x_3]$	$f[x_1, x_2, x_3]$	$f[x_0, x_1, x_2, x_3]$		$\vdots$
x_4	$f[x_4]$	$f[x_3, x_4]$	$f[x_2, x_3, x_4]$	$f[x_1, x_2, x_3, x_4]$	$f[x_0, x_1, x_2, x_3, x_4]$	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

现在利用差商的对称性,不难得到差商的两不同形式的定义是等价的。如果手工计算,按表 4.1 计算与按表 5.1 计算的工作量大体相同。如果模仿上面的表 5.1 编程计算,即使只是为了计算 $f[x_0], f[x_0, x_1], \dots, f[x_0, \dots, x_n]$, 也需要说明一个二维数组,很难编出类似于程序 10.03B 那样简明的程序,不能充分体现牛顿插值法所具有的承袭性。

10.6 灵活应用插值方法举例

10.6.1 反(函数)插值

设 $y=f(x)$ 在 x^* 附近连续可微,如果 $f'(x^*) \neq 0$, 则 $f(x)$ 在 x^* 附近是单调函数,因此存在反函数 $x=g(y)$, 使得对 $y^*=f(x^*)$ 附近的任意 y , 总可找到 x^* 附近的 x , 满足 $y=f(x)$, 亦即 $g[f(x)]=x$ 。

在高等数学中我们已经注意到这样一个事实:有时候求直接函数值比较容易,但是求反函数值却比较麻烦。现在我们可以巧妙地利用插值方法来解决这个问题。

对于多项式插值来说,求反函数插值与求直接函数插值是一样容易的:只要简单地把表 2.1 给出的直接函数的插值基点与对应的函数值列表的两行互换一下位置,即可得到相应的反函数的插值基点与对应的函数值列表,再利用反函数的函数值列表构造相应的插值多项式即可简单地计算反函数值,我们把这样的插值称之为反插值。

注意到反函数当自变量取零时的函数值就是直接函数的零点,所以我们可以利用反插值求反函数当自变量取零时的函数值来求直接函数的零点。事实上,求 $y=f(x)$ 的反函数当 $y=y_0$ 时的函数值也就是求非线性方程 $f(x)-y_0=0$ 的解。所以求反函数值和解非线性方程这两个问题实际上密不可分。下面通过一个例子说明具体操作方法。

例 6.1 利用插值方法求 $y=x^2$ 当 $y=5$ 时的反函数值。

解:显然,所给出的问题相当于求非线性方程 $x^2=5$ 的解,也相当于求函数 $f(x)=x^2-5$ 的零点。显然, $2^2<5, 3^2>5$, 所以相应的反函数值 $x\in[2,3]$ 。把区间 $[2,3]$ 划分成 5 等份(也可以适当多一些,也可以适当少一些,这里只是介绍一种方法),得到 6 个等分点,计算这些点处的函数值,从而得到函数值列表。最后用反插值求 $y=0$ 时的反函数值即可。所以,只要得到了相应的反函数值列表,即可利用前面得到的程序完成计算。相应的源代码见程序 10.07A。其中 FormProblem() 根据调用程序的要求,把求解区间 $[a,b]$ 划分为 n 等分,把 $n+1$ 个等分点(包括两个端点)的值以及对应的函数值分别写入数组 Y 和 X 。

程序 10.07A 构造求解例 6.1 的反插值问题

```
double f(double x){return x*x-5;}
int FormProblem(double * X,double * Y,double a,double b,int n)
{ int K ;
  double step;
  step=(b-a)/n;
  Y[0]=a;
  for(K=1;K<=n;K++)Y[K]=Y[K-1]+step;
  for(K=0;K<=n;K++)X[K]=f(Y[K]);
  return 0;
}
```

把程序 10.07A 以文件名“Exam61.c”保存,再把程序 10.02 复制一份,把其中第一行改为

```
#include "EXAM61.C"
```

把 main() 函数改写为下面的程序 10.07B 即可。运行后所显示的结果如表 6.1 所示,不难验证,所得到的结果有 6 位有效数字。读者可以把程序中的 $a=2, b=3$ 改写为 $a=2.2, b=2.3$, 不难发现所得到的结果会更加精确。

表 6.1 反插值法求解例 6.1 计算结果

k	x[k]	y[k]	LB[k]	LT[k]
0	-1.0000000	2.0000000	-0.0310329	-0.0620658
1	-0.1600000	2.2000000	0.8205814	1.8052791
2	0.7600000	2.4000000	0.2931583	0.7035798
3	1.7600000	2.6000000	-0.1077531	-0.2801581
4	2.8400000	2.8000000	0.0285173	0.0798485
5	4.0000000	3.0000000	-0.0034710	-0.0104129
X0 = 0.0000000000000000				
SUMLB = 0.9999999999999998				
SUMLT = 2.2360706059939774				

程序 10.07B 求解例 6.1 的主函数

```
main()
{ double a=2,b=3;
  RN=6;
  RN=FormProblem(X,Y,a,b,RN-1);
  Operation(0);
  ShowTable();
  getchar();
  return;
}
```

10.6.2 反插值与二分法集成

在第 5 章 5.2 节中,我们介绍了用区间对分法(也称二分法)解非线性方程,并且说明了对分法算法简单,适用范围广,数字性能稳定等优点,缺点是如果求解区间较小,则算法的效率不高。如果利用对分法的每一步迭代所产生的区间的两个端点和中点以及相应的函数值作反插值,一般说来反插值所得到的解比所在区间的中点更为精确。

尽管第 5 章 5.6 节中介绍的 Brent 方法也基于类似的思想,但我们推荐的方法是保持对分法的算法框架不变,只是每一步利用对分法中的三个点再作一次反插值而得到一个独立的点列,从而可以再构成一个停机规则。这样一来,所形成的综合性的方法既保持了对分法的所有优点,又可以根据反插值的结果而提前停机,是一种在稳的基础上求快的方法。

结合求解第 5 章例 2.1,亦即求 $x^3-2x-2=0$ 在 $[1,2]$ 内的解,集成反插值与对分法求解非线性方程的例程见程序 10.08。程序中定义两个长度为 3 的数组 x 、 y 同时应用于二分法和插值法, $x[0]$ 、 $x[1]$ 、 $x[2]$ 分别存放当前的区间的左端点,中点及右端点,而 $y[0]$ 、 $y[1]$ 、 $y[2]$ 分别存放 $x[0]$ 、 $x[1]$ 、 $x[2]$ 处的函数值。数组 A 、 B 、 $X0$ 、 $Y0$ 、 $X1$ 、 $Y1$ 分别保存迭代过程中的区间的端点,中点及相应的函数值,插值的结果及相应的函数值。运行程序所显示的结果如表 6.2 所示。

从程序所显示的结果看,用二分法迭代 16 次可以得到具有 5 位有效数字的结果,与此同时,利用插值方法得到的结果已经有 15 位有效数字,足见利用插值方法加速收敛的效果。

表 6.2 运行程序 10.06 所显示的结果

K	A[K]	B[K]	X0[K]	Y0[K]	X1[K]	Y1[K]
0	1.00000	22.00000	1.50000	-1.6E+00	1.870846394984326	8.1E-01
1	1.50000	2.00000	1.75000	-1.4E-01	1.770429345450210	8.4E-03
2	1.75000	1.00000	1.87500	8.4E-01	1.768960540372932	-2.5E-03
3	1.75000	1.87500	1.81250	3.3E-01	1.769221010473420	-5.3E-04
4	1.75000	1.81250	1.78125	8.9E-02	1.769285673543704	-6.4E-05
5	1.75000	1.78125	1.76562	-2.7E-02	1.769293119194687	5.7E-06
6	1.76562	1.78125	1.77344	3.1E-02	1.769292192980396	-1.2E-06
7	1.76562	1.77344	1.76953	1.8E-03	1.769292350986050	-2.4E-08
8	1.76562	1.76953	1.76758	-1.3E-02	1.769292355590112	1.0E-08
9	1.76758	1.76953	1.76855	-5.4E-03	1.769292354509831	2.0E-09
10	1.76855	1.76953	1.76904	-1.8E-03	1.769292354278039	2.9E-10
11	1.76904	1.76953	1.76929	-3.9E-05	1.769292354238912	2.1E-12
12	1.76929	1.76953	1.76941	8.6E-04	1.769292354238500	-9.7E-13
13	1.76929	1.76941	1.76935	4.1E-04	1.769292354238901	-2.2E-13
14	1.76929	1.76935	1.76932	1.9E-04	1.769292354238625	-4.9E-14
15	1.76929	1.76932	1.76930	7.4E-05	1.769292354238631	-6.6E-15
16	1.76929	1.76930	1.76929	1.8E-05	1.769292354238631	-1.7E-15
ANS = 1.769292354238631						
ERR = 6.661E-16						

程序 10.08 集成对分法与反插值法解非线性方程

```

#include "LAGRANGE.C"
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 21
int RN=0;
double ANS,ERR;
double A[MN],B[MN],X0[MN],Y0[MN],X1[MN],Y1[MN];
double x[5],y[5];
double A0=1.0,B0=2.0,EPS=0.4E-14;
double f(double x){return(x*x*x-2.0*x-2);}
Operation()
{ int K;
  double y0,err0,err1;
  err0=(B0-A0)/2.0;
  y0=f(A0);
  x[0]=A0;x[2]=B0;x[1]=(x[0]+x[2])/2;
  y[0]=f(x[0]);y[2]=f(x[2]);y[1]=f(x[1]);
  A[0]=x[0];B[0]=x[2];
  X0[0]=x[1];Y0[0]=y[1];
  X1[0] = LAGRGR(0,y,x,3);Y1[0]=f(X1[0]);
  for(K=1;K<MN;K++)
  { if(y[1]*y0>0){x[0]=x[1];y[0]=y[1];}
    else {x[2]=x[1];y[2]=y[1];}
    x[1]=(x[0]+x[2])/2.0;y[1]=f(x[1]);
    A[K]=x[0];B[K]=x[2];
    X0[K]=x[1];Y0[K]=y[1];
    X1[K] = LAGRGR(0,y,x,3);Y1[K]=f(X1[K]);
    ERR=fabs(X1[K]-X1[K-1]);
    if(ERR<EPS)break;
    err0/=2.0;
    if(err0<ERR)ERR=err0;
  }
}

```



```

        if(ERR<EPS)break;
    }
    if(K>=MN)K--;
    RN=K+1;
    if(fabs(Y1[K])<fabs(Y0[K]))ANS=X1[K];
    else
        ANS=X0[K];
    return;
}
ShowTable(){...}
main()
{ Operation();
  ShowTable();
  getchar();
}

```

对程序 10.08 还可作进一步的改进,比如把 x, y 说明为长度为 4 的数组,程序的循环体内 $x[0]$ 或 $x[2]$ 被 $x[1]$ 替代之前可以保存为 $x[3]$,相应的函数值保存为 $y[3]$,并把程序中所有的 LAGR_G() 函数调用改写为“LAGRG(0, $y, x, 4$)”,改动不大,但效果十分显著,迭代 12 次即可达到相同精度的结果。

程序 10.08 的另一个改进方法是,在循环体内调用 LAGR_G() 函数前,把 $X1[k-1]$ 与 $Y1[k-1]$ 分别赋值给 $x[3], y[3]$,采用 3 次多项式反插值,效果更好,甚至令人觉得有些神奇。不过如果仔细回味一下也在情理之中。这种改进方式的遗留问题是 $y[3]$ 可能与 $y[0], y[1], y[2]$ 中的某一个非常接近,从而导致调用 LAGR_G() 函数失败,在具体应用中,这个潜在的问题倒也不是很难解决。

对于集成二分法和反插值方法求 $f(x)$ 在 (a, b) 内的零点来说,如果 $f(x)$ 不是单调函数,显然存在一些问题。在这种情况下,如果还希望利用插值方法来加速,集成二分法和抛物线插值方法也是合理的,只是还要求一个二次函数的零点。

10.6.3 二次插值与黄金分割法集成

在第 5 章 5.3 节中,我们介绍了求单变量函数极小值的黄金分割法,利用黄金分割法求函数的极小值与二分法求函数的零点算法的思路相同这一点,我们也可以模仿着利用插值法来加速黄金分割法的收敛速度并且同时保持黄金分割法数字性能稳定的优点。在第 5 章 5.6 节中,我们介绍了求解非线性方程的 Muller 方法,并提到改造 Muller 方法用于求函数的极值点更合乎情理一些。现在我们可以通过编程把这些设想转变为现实。

假设已知平面上三个点 $(x_0, y_0), (x_1, y_1), (x_2, y_2)$,我们希望有一个 C 语言函数,例如命名为 MinQuadritic(),其原型为

```
double MinQuadritic(double * x, double * y)
```

通过输入 $x=(x_0, x_1, x_2), y=(y_0, y_1, y_2)$ 而返回经过 $(x_0, y_0), (x_1, y_1), (x_2, y_2)$ 的抛物线的极值点。利用第 5 章(6.11)~(6.13)式以及(6.19)式,编写 MinQuadritic() 函数还是很容易的,相应的源代码见程序 10.09A。整个程序几乎就是照抄相应的公式,读者可以

对照着读。

程序 10.09A

```
double MinQuadratic(double * x, double * y)
{ double h0, h1, q0, q1, a, b;
  h0 = x[1] - x[0];
  h1 = x[2] - x[1];
  q0 = (y[1] - y[0]) / h0;
  q1 = (y[2] - y[1]) / h1;
  a = (q1 - q0) / (h1 + h0);
  b = q1 + a * h1;
  return x[2] - b / a / 2.0;
}
```

结合求解第 5 章例 3.1, 集成黄金分割法与插值法求函数极小值的源程序见程序 10.09B, 程序中的数组 $x[4]$ 用来依次存放区间的端点和两个黄金分割点, 数组 $y[4]$ 用来存放相应的函数值。数组 A, B 存放代过程中每一次的区间的端点; $X0$ 存放用黄金分割法得到的解, $Y0$ 存放相应的函数值; $X1$ 存放用插值法得到的解, $Y1$ 存放相应的函数值。

利用程序 10.09B 求解第 3 章例 3.1 所显示的计算结果见表 6.3。从程序运行的结果看, 如果不采用插值法加速而单纯看黄金分割法所得到的结果, 比较 $X0[14]$, $A[14]$ 和 $B[14]$ 可知, $X0[14]$ 只有 3 位有效数字; 比较 $X1[13]$ 和 $X1[14]$ 可以断定采用插值法加速所得到的结果 $X1[14]$ 具有 7 位有效数字, 所以插值法加速的效果也很明显。

表 6.3 利用程序 10.09B 求解第 5 章例 3.1 所显示的计算结果

k	A[k]	B[k]	X0[k]	Y0[k]	X1[k]	Y1[k]
0	1.500000	1.500000	2.263982	-0.62993214	2.04175488	-0.63341068832824
1	1.500000	2.736068	2.263982	-0.62993214	2.13817902	-0.63364403845863
2	1.972136	2.736068	2.263982	-0.62993214	2.14209133	-0.63387918457185
3	1.972136	2.444272	2.152476	-0.66016502	2.14691649	-0.66007315883348
4	1.972136	2.263982	2.152476	-0.66016502	2.15091922	-0.66015358210172
5	2.083592	2.263982	2.152476	-0.66016502	2.15238418	-0.66016465283798
6	2.083592	2.195048	2.152476	-0.66016502	2.15293795	-0.66016638457281
7	2.126165	2.195048	2.152476	-0.66016502	2.15316435	-0.66016651642859
8	2.126165	2.168737	2.152476	-0.66016502	2.15323904	-0.66016654767210
9	2.142426	2.168737	2.152476	-0.66016502	2.15327647	-0.66016655357210
10	2.142426	2.158687	2.152476	-0.66016502	2.15328436	-0.66016655408269
11	2.148637	2.158687	2.152476	-0.66016502	2.15329089	-0.66016655422550
12	2.148637	2.154848	2.152476	-0.66016502	2.15329244	-0.66016655423812
13	2.151010	2.154848	2.153382	-0.66016654	2.15329199	-0.66016655425026
14	2.152476	2.154848	2.153382	-0.66016654	2.15329219	-0.66016655425041
ANEX = 2.153292194581995, ERRX = 2.048E-07						
ANSY = -0.660166554230412, ENRY = 2.571E-13						

程序 10.09B 集成黄金分割法与插值法求函数极小值

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 51
int RN=0;
double ANSX, ANSY, ERRX, ERRY;
double GOLDEN=0.61803398874989481;
double AX1, AX2;
double A[MN], B[MN], X0[MN], X1[MN], Y0[MN], Y1[MN];
double x[4], y[4];
```

```

double A0=1.5,B0=3.5;
double f(double x){return exp(x)-2*x*x;}
double EPSX=0.4E-8,EPSY=0.4E-12;
    { X0[K]=x[2];Y0[K]=y[2];
int GoldenSection()
{ int K,flag;
    double ERR0,ERR1;
    AX2=(B0-A0)*GOLDEN;
    AX1=AX2*GOLDEN;
    x[0]=A0; x[1]=x[0]+AX1;
    x[2]=x[0]+AX2; x[3]=B0;
    y[0]=f(x[0]);y[1]=f(x[1]);
    y[2]=f(x[2]);y[3]=f(x[3]);
    A[0]=x[0]; B[0]=x[3];
    if(y[1]<=y[2])flag=1;
        else flag=2;
    if(flag==1)
    { X0[0]=x[1];Y0[0]=y[1];
        X1[0]=MinQuadratic(x,y);
    }
    else
    { X0[0]=x[2];Y0[0]=y[2];
        X1[0]=MinQuadratic(&x[1],&y[1]);
    }
    Y1[0]=f(X1[0]);
    for(K=1;K<MN;K++)
    { AX2=AX1;
        AX1=AX2*GOLDEN;
        if(flag==1)
        { x[3]=x[2];y[3]=y[2];
            x[2]=x[1];y[2]=y[1];
            x[1]=x[0]+AX1;
            y[1]=f(x[1]);
        }
        else
        { x[0]=x[1];y[0]=y[1];
            x[1]=x[2];y[1]=y[2];
            x[2]=x[0]+AX2;
            y[2]=f(x[2]);
        }
        A[K]=x[0];B[K]=x[3];
        if(y[1]<=y[2])flag=1;
            else flag=2;
        if(flag==1)

```

```

    { X0[K]=x[1]; Y0[K]=y[1];
      X1[K]=MinQuadratic(x,y);
    }
    else
    { X0[K]=x[2]; Y0[K]=y[2];
      X1[K]=MinQuadratic(&x[1], &y[1]);
    }
    Y1[K]=f(X1[K]);
    ERRX=fabs(X1[K]-X1[K-1]);
    ERRY=fabs(Y1[K]-Y1[K-1]);
    if(ERRX<EPSX||ERRY<EPSY)break;
    ERRX=AX1;
    ERRY=fabs(Y0[K]-Y0[K-1]);
    if(ERRX<EPSX)break;
  }
  if(K>=MN)K--;
  RN=K+1;
  if(Y1[K]<Y0[K]){ ANSX=X1[K]; ANSY=Y1[K];}
  else           { ANSX=X0[K]; ANSY=Y0[K];}
  return 0;
}
ShowTable(){...}
SaveTable(char * fname)
main()
{ GoldenSection();
  ShowTable();
  getchar();
}

```

10.7 分段埃尔米特(Hermit)插值

利用插值的方法进行有关计算,基本要求就是误差应尽可能小,通常的想法是,通过增加插值基点从而提高插值多项式的次数来提高插值的精度,但实际上这并不是最有效的方法。当插值多项式的次数较高时,可能会产生意想不到的龙格(Runge)现象,此时,次数的增高反而产生不利影响。

设被插函数为 $f(x)=1/(1+x^2)$, 插值区间为 $[-5, 5]$, 把插值区间划分为 n 等份, 插值基点为区间的等分点。我们会发现,随着 n 的增大,插值多项式在区间端点附近发生激烈的震荡,这就是龙格现象。取 $n=10$ 时插值函数的图形如图 7.1 所示,在 $x=-4.5$ 和 $x=4.5$ 附近,插值函数值与被插函数值相差太大而完全失去意义。

埃尔米特插值的基本思想就是使插值多项式与被插函数在插值基点处的导数值也相同。为了简化计算,还可以把插值区间划分成若干子区间,在各个子区间上独立地进行的埃尔米特插值,这就是分段埃尔米特插值。分段埃尔米特插值的最大优点是能保证插值

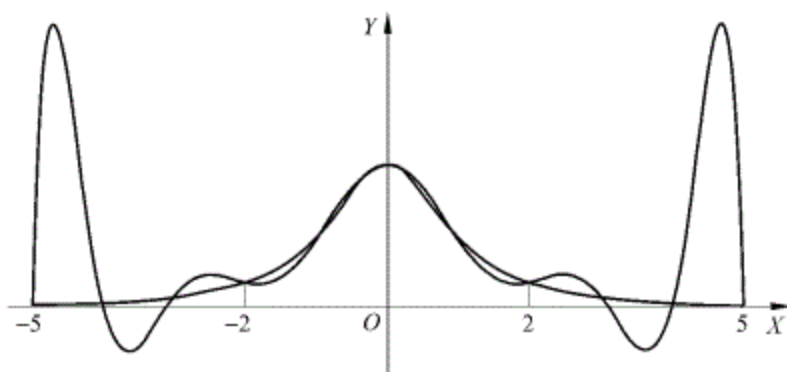


图 7.1 多项式插值的龙格现象示意图

函数在区间的交界点处是光滑的。一般说来,采用分段埃尔米特插值是消除龙格现象的有效方法^①,而且相应的计算也简单得多。

10.7.1 一般的 $n+1$ 个基点的埃尔米特插值简介

已知 $f(x)$ 在 $x_0, x_1, \dots, x_n$ 处的函数值 $y_0, y_1, \dots, y_n$ 和导数值 $z_0, z_1, \dots, z_n$, 可以把它们用表 7.1 的形式列出, 并称为是埃尔米特插值的原始数据列表。

表 7.1 埃尔米特插值的原始数据列表

k	0	1	...	n
x_k	x_0	x_1	...	x_n
y_k	y_0	y_1	...	y_n
z_k	z_0	z_1	...	z_n

利用表 7.1 给出的函数值和导数值, 我们可以构造一个 $2n+1$ 次多项式 $H(x)$, 满足所给出的 $2n+2$ 个条件, 即

$$\left. \begin{aligned} H(x_j) &= y_j \\ H'(x_j) &= z_j \end{aligned} \right\}, j = 0, 1, \dots, n$$

这就是埃尔米特插值问题。

对于一般的 $n+1$ 个基点的埃尔米特插值问题, 仿照拉格朗日插值公式的构造方法, 我们定义两组基函数 $\{\alpha_k(x), k=0, 1, \dots, n\}$ 和 $\{\beta_k(x), k=0, 1, \dots, n\}$ 如下:

$$\alpha_k(x) = \left[1 - 2(x - x_k) \sum_{\substack{j=0 \\ j \neq k}}^n \frac{1}{x_k - x_j} \right] [LB_k(x)]^2 \quad (7.1)$$

$$\beta_k(x) = (x - x_k) [LB_k(x)]^2 \quad (7.2)$$

^① 读者切不可由此产生误解, 认为应该用埃尔米特插值来替代拉格朗日插值或牛顿插值。实际上龙格现象一般不会出现, 即使出现也不是很难解决的问题。笔者本人的观点是, 在某些特定情况下应用埃尔米特插值来计算一些复杂的函数值还是很不错的, 比如后面求微分方程的数值解很容易得到理论解在离散点处的函数值和导数值列表, 接下来再用分段埃尔米特插值进行计算也可以说是恰到好处。仅仅为了避免龙格现象而特意采用埃尔米特插值的做法并不可取。

相应的埃尔米特插值多项式 $H(x)$ 也可以表示为两组基函数的线性组合, 且组合系数相应地分别为 $y_0, y_1, \dots, y_n$ 和 $z_0, z_1, \dots, z_n$, 亦即

$$H(x) = \sum_{k=0}^n y_k \alpha_k(x) + \sum_{k=0}^n z_k \beta_k(x) \quad (7.3)$$

显然, $H(x)$ 是 $2n+1$ 次多项式, 它由 $2n+2$ 个常数确定, 而 $2n+2$ 个常数正好由 $2n+2$ 个插值条件确定。不难看出, 对任意给定的 $k=0, 1, \dots, n$, 上面(7.1)式和(7.2)式定义的 $\alpha_k(x)$ 和 $\beta_k(x)$ 具有下面一些性质:

- (1) 对任意 $j=0, 1, \dots, n$, 当 $j \neq k$ 时, 有 $\alpha_k(x_j) = \beta_k(x_j) = 0$;
- (2) 对任意 $j=0, 1, \dots, n$, $[\alpha_k(x_j)]' = [\beta_k(x_j)]' = 0$;
- (3) $\alpha_k(x_k) = 1, \beta_k(x_k) = 0$;
- (4) $[\alpha_k(x_k)]' = 0, [\beta_k(x_k)]' = 1$ 。

再利用各 $\alpha_k(x)$ 和 $\beta_k(x)$ 所具有的这些性质立即可以得出, 对任意给定的 $j=0, 1, \dots, n$, 由上面的(7.3)式定义的 $H(x)$ 满足条件

$$H(x_j) = y_j, H'(x_j) = z_j$$

所以 $H(x)$ 就是所要寻找的埃尔米特插值多项式。

10.7.2 三次埃尔米特插值

最简单的埃尔米特插值问题可以设想为已知 $f(x)$ 在 x_0, x_1 处的函数值 y_0, y_1 和导数值 z_0, z_1 , 求 3 次多项式 $H(x)$, 满足相应的插值条件:

$$H(x_0) = y_0, H'(x_0) = z_0$$

$$H(x_1) = y_1, H'(x_1) = z_1$$

如果只是单纯解决这个问题, 可以有很多方法得到满足条件的 $H(x)$, 比如把 $H(x)$ 表为 $a_0 + a_1x + a_2x^2 + a_3x^3$, 接下来直接利用插值条件确定 a_0, a_1, a_2, a_3 。另一个容易想到的方法是利用(7.3)式取 $n=1$ 而得到

$$H_3(x) = y_0\alpha_0(x) + y_1\alpha_1(x) + z_0\beta_0(x) + z_1\beta_1(x) \quad (7.4)$$

其中 $\alpha_0(x), \alpha_1(x), \beta_0(x)$ 和 $\beta_1(x)$ 可由(7.1)式和(7.2)式取 $n=1$ 得到。

为了把问题解决得更彻底一些, 可以利用上面的(7.4)式构造 $[0, 1]$ 区间上的三次埃尔米特插值公式。其实方法也很简单, 把基函数 $\alpha_0(x), \alpha_1(x), \beta_0(x)$ 和 $\beta_1(x)$ 中的 x_0, x_1 分别改为 0 和 1, 并把相应的基函数名 $\alpha_0(x)$ 和 $\alpha_1(x)$ 改名为 $u_0(t)$ 和 $u_1(t)$, 把 $\beta_0(x)$ 和 $\beta_1(x)$ 改名为 $v_0(t)$ 和 $v_1(t)$, 从而有

$$\left. \begin{aligned} u_0(t) &= (t-1)^2(2t+1) \\ u_1(t) &= t^2(-2t+3) \\ v_0(t) &= t(t-1)^2 \\ v_1(t) &= t^2(t-1) \end{aligned} \right\} \quad (7.5)$$

所以相应的插值公式为

$$Y(t) = y_0u_0(t) + y_1u_1(t) + z_0v_0(t) + z_1v_1(t) \quad (7.6)$$

对于一般的区间 $[x_0, x_1]$ 上的三次埃尔米特插值问题来说, 令

$$h = x_1 - x_0, t = (x - x_0)/h$$

同样地有当 $x=x_0$ 时得 $t=0$, 而 $x=x_1$ 时得 $t=1$ 。另外, 由于 $t'(x)=1/h$, 从而有 $x'(t)=h$, 所以相应的三次埃尔米特插值多项式可以表示为

$$H[t(x)] = y_0 u_0 \left(\frac{x-x_0}{h} \right) + y_1 u_1 \left(\frac{x-x_0}{h} \right) + h z_0 v_0 \left(\frac{x-x_0}{h} \right) + h z_1 v_1 \left(\frac{x-x_0}{h} \right) \quad (7.7)$$

也可以直接验证上面的(7.7)式满足一般形式的三次埃尔米特插值条件。

利用上面的(7.6)式和(7.7)式编程计算是很容易的, 对于给定的插值基点 x_0, x_1 , 插值基点处的函数值 y_0, y_1 和导数值 z_0, z_1 , 以及插值点 x , 可按下面的步骤计算:

首先计算 $h=x_1-x_0, t=(x-x_0)/h$;

接下来计算数 $u_0(t), u_1(t), v_0(t)$ 和 $v_1(t)$;

最后再利用(7.7)式计算 $H_3(t)$ 即可完成。

例 7.1 给出函数 $f(x)$ 在 $x_0=1.96, x_1=2.25$ 处的函数值 y_0, y_1 和导数值 z_0, z_1 列表如下, 试用三次埃尔米特插值法求 $f(2)$ 的近似值。

k	x	y	z
0	1.96	1.4	0.5/1.4
1	2.25	1.5	0.5/1.5

解: 求解本例题的源代码见程序 10.10, 程序运行的结果如表 7.2 所示, 直接阅读源代码即可理解其中每个变量的定义。 H 为插值的结果, 简单分析后可知 H 有 7 位有效数字。

表 7.2 程序 10.10 的运行结果

step	=	0.290000000000
t0	=	0.137931034483
t1	=	-0.862068965517
u0	=	0.948175356841
u1	=	0.051824643159
v0	=	0.102505227767
v1	=	-0.016400836443
H0	=	1.338059312454
H1	=	0.076154550549
H	=	1.414213863002

程序 10.10 求解例 7.1 的源程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
double x0=1.96, x1=2.25;
double y0=1.4, y1=1.5;
double z0=0.5/1.4, z1=0.5/1.5;
double x=2.0;

char *str[] =
{ "step =",
  " t0 =", " t1 =",
  " u0 =", " u1 =",
  " v0 =", " v1 =",
```

```

    " H0 =", " H1 =",
    " H ="
};
double step,t0,t1,u0,u1,v0,v1,H0,H1,H;
void Operation()
{ double t;
  step=x1-x0;
  t=(x-x0)/step;
  t0=t;
  t1=t-1;
  u0=t1*t1*(3+2*t1);
  u1=t0*t0*(3-2*t0);
  v0=t1*t1*(t1+1);
  v1=t0*t0*(t0-1);
  H0=y0*u0+step*z0*v0;
  H1=y1*u1+step*z1*v1;
  H=H0+H1;
  return;
}
void ShowResult()
{ printf("\n %s %16.12f",str[0],step);
  printf("\n %s %16.12f",str[1],t0);
  printf("\n %s %16.12f",str[2],t1);
  printf("\n %s %16.12f",str[3],u0);
  printf("\n %s %16.12f",str[4],u1);
  printf("\n %s %16.12f",str[5],v0);
  printf("\n %s %16.12f",str[6],v1);
  printf("\n %s %16.12f",str[7],H0);
  printf("\n %s %16.12f",str[8],H1);
  printf("\n %s %16.12f",str[9],H);
  getchar();
  return;
}
main()
{ Operation();
  ShowResult();
}

```

关于三次埃尔米特插值余项 $R(x)$ 的推导,可以采用类似于多项式插值余项的推导方法(见定理 5.1),对区间 (x_0, x_1) 内的任意点 x ,构造辅助函数

$$g(\lambda) = f(\lambda) - H_3[t(\lambda)] - \frac{R(x)}{(x-x_0)^2(x-x_1)^2}(\lambda-x_0)^2(\lambda-x_1)^2$$

则有 $g(x_0)=g(x)=g(x_1)=0$, 所以 $g'(\lambda)$ 在 $[x_0, x_1]$ 内有 4 个零点(包括两个边界点 x_0 和 x_1), 如果 $f(x)$ 至少 4 次可微, 则存在 $\xi \in (x_0, x_1)$, 使得 $g^{(4)}(\xi)=0$, 由此不难推出

$$R(x) = \frac{f^{(4)}(\xi)}{4!}(x-x_0)^2(x-x_1)^2 \quad (7.8)$$

10.7.3 分段三次埃尔米特插值方法简介

对于表 7.1 给出的一般形式的 $n+1$ 个基点的埃尔米特插值问题来说,我们可以假定插值基点按由小到大的顺序排列,即 $a=x_0 < x_1 < \cdots < x_n=b$,从而把插值区间 $[a,b]$ 划分为 n 个小区间。

记第 i 个子区间为 $[x_i, x_{i+1}]$, $i=0, 1, \cdots, n-1$, 利用每个小区间的端点的函数值和导数值,我们可以得到每个小区间上的埃尔米特插值函数,从而得到区间 $[a,b]$ 上的一段形式给出的插值函数。

分段三次埃尔米特插值方法编程计算并没有特别的难处。对任意插值点 x , 首先确定出 x 所在的区间,接下来计算所在区间内的三次埃尔米特插值函数值即可。

分段三次埃尔米特插值方法主要应用于一些特殊函数值计算,一般说来,插值区间的长度愈短,插值结果的误差就愈小,为了兼顾计算量和结果的精度,在具体情况下可以利用(7.8)式取 x 为区间的中点而得到 $R(x)$ 的最大值,利用它来确定子区间的长即可保证分段三次埃尔米特插值的精度。

10.8 三次样条插值方法简介

样条(spline)本来是一种弹性很好而且容易自然弯曲的金属条之类的东西,技术工人们利用它在工件上划出自然弯曲的曲线把一些离散点连接起来,以便加工。从数学的意义上讲,这样画出来的曲线通常是二次连续可微或二次光滑。借助样条的启发,我们也可以用一定程度的光滑曲线把 xy 平面上的一些离散点连接起来,从而又得到一种插值方法,并称为样条插值。

假如我们得到一个如表 8.1 形式的函数值列表,最下面一行的含义可以灵活规定。不失一般性,可以假定表中的 $x_0, x_1, \cdots, x_n$ 满足关系 $x_0 < x_1 < \cdots < x_n$,从而得到 n 个子区间 $[x_i, x_{i+1}]$, $i=0, 1, \cdots, n-1$ 。

表 8.1 埃尔米特插值的原始数据列表

k	0	1	...	$n-1$	n
x	x_0	x_1	...	x_{n-1}	x_n
y	y_0	y_1	...	y_{n-1}	y_n
z	z_0	z_1	...	z_{n-1}	z_n

如果在每个子区间上作线性插值(一次多项式),则称为是一次样条插值,此时整个区间 $[x_0, x_n]$ 插值函数是连续的,但在插值基点处不可导。一般地,如果在每个子区间上用 k 次多项式插值,则称为是 k 次样条插值,适当选择插值多项式的系数通常可以使得插值函数在插值基点处 $k-1$ 阶可导。

对于实际应用来说,一次样条插值通常称为分段线性插值;二次样条插值由于计算比

较复杂,而所得到的插值函数在插值基点处只有一阶光滑,所以一般不用;三次样条插值可以借助分段三次埃尔米特插值方法而使得有关的计算大大简化,所得到的曲线在插值基点处二次光滑,所以样条插值一般指的都是三次样条插值。

对于表(8.1)给出的一般的 $n+1$ 个基点的三次样条插值问题,为叙述方便,对 $i=0, 1, \dots, n-1$, 记 $S_i[t(x)]$ 为区间 $[x_i, x_{i+1}]$ 上的三次样条函数,利用(7.7)式的表示形式, $S_i[t(x)]$ 可表示为

$$S_i[t(x)] = y_i u_0\left(\frac{x-x_i}{h_i}\right) + y_{i+1} u_1\left(\frac{x-x_i}{h_i}\right) + h_i z_i v_0\left(\frac{x-x_i}{h_i}\right) + h_i z_{i+1} v_1\left(\frac{x-x_i}{h_i}\right) \quad (8.1)$$

其中各 z_i 都是待定常数。

注意到对任意给定的一组 $z_0, z_1, \dots, z_n$ 的值,上面的(8.1)式所得到的分段插值函数在 $n+1$ 个插值基点处都是一阶光滑的,我们当然希望适当选择 $z_0, z_1, \dots, z_n$, 使得所得到的分段插值函数在插值基点处都是二阶光滑的。

假如 z_0 和 z_n 是已知的,注意到对 $i=1, 2, \dots, n-1$, x_i 既是第 $i-1$ 个子区间的右端点,也是第 i 个子区间的左端点,为了保证 x_i 处的二阶导数相同,可以令

$$(S_{i-1}[t(x_i)])'' = (S_i[t(x_i)])'' \quad (8.2)$$

从而得到 $n-1$ 个等式,由此可以确定出 $z_1, z_2, \dots, z_{n-1}$, 从而形成一个完整的方法。

利用(7.5)式,我们有

$$\begin{aligned} [u_0(t)]'' &= 6(2t-1), [u_1(t)]'' = -6(2t-1) \\ [v_0(t)]'' &= 6t-4, [v_1(t)]'' = 6t-2 \end{aligned}$$

由此可以进一步得到

$$\begin{aligned} [u_0(0)]'' &= -6, [u_1(0)]'' = +6, [v_0(0)]'' = -4, [v_1(0)]'' = -2 \\ [u_0(1)]'' &= +6, [u_1(1)]'' = -6, [v_0(1)]'' = +2, [v_1(1)]'' = +4 \end{aligned}$$

从而有

$$\begin{aligned} (S_{i-1}[t(x_i)])'' &= \frac{1}{(h_{i-1})^2} [+6y_{i-1} - 6y_i + 2h_{i-1}z_{i-1} + 4h_{i-1}z_i] \\ (S_i[t(x_i)])'' &= \frac{1}{(h_i)^2} [-6y_i + 6y_{i+1} - 4h_i z_i - 2h_i z_{i+1}] \end{aligned}$$

再由(8.2)式即得

$$-6 \frac{y_i - y_{i-1}}{(h_{i-1})^2} + 2 \frac{z_{i-1} + 2z_i}{h_{i-1}} = +6 \frac{y_{i+1} - y_i}{(h_i)^2} - 2 \frac{2z_i + z_{i+1}}{h_i}$$

从而有

$$\frac{z_{i-1} + 2z_i}{h_{i-1}} + \frac{2z_i + z_{i+1}}{h_i} = 3 \left[\frac{y_{i+1} - y_i}{(h_i)^2} + \frac{y_i - y_{i-1}}{(h_{i-1})^2} \right] \quad (8.3)$$

为了简化计算,将上面的(8.3)式两边同时乘以 $h_{i-1}h_i/(h_{i-1}+h_i)$, 整理后即得

$$\frac{h_i}{h_{i-1}+h_i} z_{i-1} + 2z_i + \frac{h_{i-1}}{h_{i-1}+h_i} z_{i+1} = 3 \left[\frac{h_{i-1}}{h_{i-1}+h_i} \frac{y_{i+1} - y_i}{h_i} + \frac{h_i}{h_{i-1}+h_i} \frac{y_i - y_{i-1}}{h_{i-1}} \right] \quad (8.4)$$

在上面的(8.4)式中,令

$$\alpha_i = \frac{h_{i-1}}{h_{i-1} + h_i} \tag{8.5}$$

则有

$$1 - \alpha_i = \frac{h_i}{h_{i-1} + h_i} \tag{8.6}$$

再令

$$\beta_i = 3 \left[\alpha_i \frac{y_{i+1} - y_i}{h_i} + (1 - \alpha_i) \frac{y_i - y_{i-1}}{h_{i-1}} \right] \tag{8.7}$$

则(8.4)式可简写为

$$(1 - \alpha_i)z_{i-1} + 2z_i + \alpha_i z_{i+1} = \beta_i \tag{8.8}$$

对 $i=1,2,\cdots,n-1$, 利用 z_0 和 z_{n+1} 已知, 由(8.5)~(8.8)式不难得到紧凑格式的三对角线性方程组, 从而可以方便地利用第 8 章 8.6 节介绍的追赶法求解。求出了各 $z_1, z_2, \cdots, z_{n-1}$ 后, 接下来的应用与前面 7.3 小节中介绍的分段埃尔米特插值完全相同, 读者完全可以自己编写这部分程序。

练 习 题

1. 给定 $y=f(x)$ 的离散函数值列表如下:

k	0	1	2	3
x_k	2	3	4	5
y_k	4	7	1	0

试利用拉格朗日差值方法求 $f(3.5)$ 的近似值。(按列表格式计算)

2. 给定 $y=f(x)$ 的离散函数值列表如下:

k	0	1	2	3	4
x_k	1	2	3	5	6
y_k	5.75	5	6.25	20.75	37

按 x_k 与 4 的差的绝对值由小到大的顺序对表中的各列重新排列, 分别用 2 次、3 次和 4 次牛顿差值多项式计算 $f(4)$ 的近似值。

3. 直接用数学归纳法证明 $c_k = f[x_0, x_1, \cdots, x_k], k=0, 1, \cdots, n$ 为线性方程组(4.8)的解。(提示: 对 $k=1, 2, \cdots, K, K+1$, 依次证明 $c_0 = f[x_0], c_1 = f[x_0, x_1], \cdots, c_k = f[x_0, x_1, \cdots, x_k]$ 是(4.8)式中的前 $k+1$ 个方程所组成的方程组的解。)

4. 构造 $y=x^2$ 在 x 为 2.6, 2.63, 2.65, 2.67, 2.7 处的函数值列表, 并利用(反)拉格朗日插值方法求 $7^{1/2}$ 的近似值。

5. 利用上题构造的函数值列表, 试用(反)牛顿插值方法求 $7.1^{1/2}$ 的近似值。

6. 构造 $y=x^2$ 在 x 为 2.6, 2.7 处的函数值和导数值列表, 并利用(反)埃尔米特插值方法求 $7^{1/2}$ 的近似值和 $7.1^{1/2}$ 的近似值(提示: 利用反函数的导数等于直接函数的导数的

倒数来简化计算)。

7. 模仿程序 10.08, 完成集成二分法与 Muller 方法求解非线性方程的程序设计。

8. 试完成分段三次埃尔米特插值的程序设计(可以与后面求常微分方程数值解的程序结合起来做)。

9. 试完成完整的三次样条插值的程序设计(可以与分段三次埃尔米特插值的程序结合起来做)。

数值微分与外推加速方法

第 11 章

设 $f(x)$ 是 $[x_0 - h, x_0 + h]$ 上连续可微的实函数, 我们希望得到 $f(x)$ 在 x_0 处的导数值, 这就是数值微分问题。如果 $f(x)$ 是一个初等函数, 对于大学生来说, 则没有数值计算方面的问题, 因为任何初等函数的导数都还是初等函数, 所以直接利用导数的表达式编程计算就行。另外, 利用第 4 章给出的求函数的增量比的程序计算当增量取零时的增量比也可得到相应的导数值。

然而, 许多工程计算中用到的函数虽然理论上可导, 但是如果给定的函数不是初等函数, 或者函数的表示形式就是一个函数值列表或者就是一段程序代码^①, 在这种情况下, 我们只能绕开导函数而直接计算导数值。

相对说来, 绕开导函数而直接求导数值的计算量将会增大, 但现在计算工具性能的大幅度提升使得这个问题并不像以前想象的那么严重。数值求导的另一个常见问题就是数值的稳定性, 我们的计算实践表明, 利用差商近似替代微商, 并辅之以外推法加速, 其数值性能也是很不错的。由于这些原因, 数值微分的实用价值将会不断提升。

11.1 利用一阶差商外推加速

11.1.1 利用差商替代微商

求数值微分最简单的方法, 也是最基本的方法, 就是利用差商替代微商, 余下的问题是如何估算误差, 如何进一步提高精度。

假设 $f(x)$ 是 $[x_0 - h, x_0 + h]$ 上连续可微的实函数, 当 h 的值充分小时, 可以利用 $f(x)$ 在 $(x_0, x_0 + h)$ 处的一阶差商 $f[x_0, x_0 + h]$ 作为 $f'(x_0)$ 的近

^① 按照数学中的实函数的定义, 如果一个可正常运行的 C 语言函数有一个或多个浮点型形式参数, 而返回值也是浮点型的数, 那么它也应该具有数学意义的函数。在近似计算的意义下也可以定义这样的函数的连续性、可微性等, 并进行相应的数值计算。

似值,即

$$f'(x_0) \approx f[x_0, x_0+h] = [f(x_0+h) - f(x_0)]/h \quad (1.1)$$

把 $f(x_0+h)$ 在 x_0 处一阶泰勒展开得

$$f(x_0+h) = f(x_0) + f'(x_0)h + O(h^2)$$

由此立即得出

$$|f'(x_0) - f[x_0, x_0+h]| = O(h)$$

这表明,如果利用 $f[x_0, x_0+h]$ 近似表示 $f'(x_0)$,其截断误差大体上与 h 成正比。此时我们称公式(1.1)具有 1 阶精度。

虽然利用 $f[x_0, x_0+h]$ 近似表示 $f'(x_0)$ 的精度很低,但还是值得编写一个专门的 C 语言函数来计算 $f[x_0, x_0+h]$,我们可以把这个函数原型说明为

double GetDiffQuat(double x0, double h)

按(1.1)式右边的格式计算,相应的源代码当然再简单不过了。

11.1.2 利用变步长方法提高精度

对于利用 $f[x_0, x_0+h]$ 近似表示 $f'(x_0)$,我们习惯称 h 为步长。一般说来,可以适当减小步长而使得 $f[x_0, x_0+h]$ 更接近 $f'(x_0)$ 。不过如果 h 取得太小,那么 $f(x_0+h) - f(x_0)$ 会失去太多的有效数字位数反而使精度下降。

实际上我们已经在第 1 章 1.7 节中给出了一个简单方法,首先设定适当的 h_0 ,接下对 $k=0,1,\dots$,依次令

$$\left. \begin{aligned} h_k &= h_0/2^k \\ z_k &= f[x_0, h_k] \end{aligned} \right\} \quad (1.2)$$

从而得到一个理论上收敛于 $f'(x_0)$ 的序列 $\{z_k\}$ 。利用上面的(1.2)式编程计算时,可以预先设定一个充分小的正数 EPS 来控制精度,当某个 $|z_k - z_{k-1}| < \text{EPS}$ 时,即可停止进一步的计算。由于计算各 z_k 时存在不利的减法运算,所以在没有想到理想的方法之前,误差限 EPS 的值不宜选得太小。

第 1 章 1.7 中的程序 1.01 实际上已经以求 $\ln x$ 在 $x_0=2$ 处的导数值,取初始 $h_0=0.32$ 为例给出了完整的源代码及计算结果,作为求数值微分的应用程序来说,可以在此基础上作些适当的简化,不必保留其中一些没有进一步利用价值的中间结果。

11.1.3 利用外推方法来加速收敛

对于利用变步长求 $f(x)$ 在每个给定的点 x_0 处的数值微分来说,利用(1.2)式得到的各 $(h_k, z_k), k=0,1,\dots$, 可以看成是自变量为 h 的函数 $F(x_0, h) = f[x_0, h]$ 在一些离散点处的函数值列表,所以可以从中挑出一些作插值多项式 $p(h)$,此时 $p(h)$ 可作为 $F(x_0, h)$ 的近似函数,从而可以利用 $p(0)$ 作为 $F(x_0, h)$ 当 h 趋近于零时的极限的近似值。在这样一个思路指引下,可以构造许多不同的加速收敛的迭代格式,而且基本方法都是一样的。下面以抛物线插值说明加速的技巧。记

$$p(h) = a_0 + a_1h + a_2h^2 \quad (1.3)$$

对给定的 $k=0,1,\dots$,取 $(h_k, z_k), (h_{k+1}, z_{k+1}), (h_{k+2}, z_{k+2})$ 作为插值基点,分别代入到

(1.3)式中,并利用 $h_{k+1}=2h_k, h_{k+2}=4h_k$, 从而有

$$\left. \begin{aligned} a_0 + a_1 h_k + a_2 (h_k)^2 &= z_k \\ a_0 + 2a_1 h_k + 4a_2 (h_k)^2 &= z_{k+1} \\ a_0 + 4a_1 h_k + 16a_2 (h_k)^2 &= z_{k+2} \end{aligned} \right\} \quad (1.4)$$

利用(1.4)式解出 a_0 , 并记为 A_{k+2} 得

$$A_{k+2} = (8z_{k+2} - 6z_{k+1} + z_k)/3 \quad (1.5)$$

上面的(1.5)式是一个非常优美的公式:它与步长 h_k 无关,仅仅是序列 $\{z_k\}$ 中相邻的三个元素的线性组合,组合系数也是固定的,计算量可以忽略不计。从而又得到一个加速收敛序列 $\{A_k, k=2, 3, \dots\}$ 。不难理解,序列 $\{A_k, k=2, 3, \dots\}$ 将比序列 $\{z_k\}$ 更快速地收敛于 $f'(x_0)$ 。下面通过一个例子说明具体的操作方法以及实际效果。

例 1.1 设 $f(x)=(x)^{1/2}$, 试求 $f'(4)$ 的数值解,取 $h_0=0.32$ 。

解:利用(1.2)式产生序列 $\{h_k\}$ 、 $\{z_k\}$, 同时利用(1.3)式产生加速收敛序列 $\{A_k\}$, 并利用数组 H, Z, A 分别存放相应的序列,完整的源代码见程序 11.01,运行程序所显示的结果如表 1.1 所示。

表 1.1 程序 11.01 的运行结果

k	H[k]	Z[k]	A[k]
0	3.20000e-01	0.245190528383290	0.245190528383290
1	1.60000e-01	0.247548783981962	0.247548783981962
2	8.00000e-02	0.248762345905195	0.249998863911026
3	4.00000e-02	0.249378105604452	0.249999851128801
4	2.00000e-02	0.249688278817101	0.24999980938432
5	1.00000e-02	0.249843945007852	0.24999997588220
6	5.00000e-03	0.249921923790006	0.2499999966679
ANS = 0.24999999666679			
ERR = 2.108E-09			

不难看出,如果不采用加速方法,所得到的 $Z[6]$ 只有 3 位有效数字,利用插值方法加速,相应的 $A[6]$ 有 9 位有效数字,所以利用外推方法加速收敛的效果十分明显。

程序 11.01 利用一阶差商结合外推加速方法求数值微分

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 32
double ANS=0, ERR=0, EPS = 0.4E-8;
static int RN=1;
static double H[MN], Z[MN], A[MN];
double x0=4.0, h0=0.32;
double f(double x){return sqrt(x);}
double GetDiffQuat(double x, double h)
{ return (f(x+h)-f(x))/h;
}
void Operation()
{ int K;
  H[0]=h0;
  Z[0]=GetDiffQuat(x0, H[0]);
```

```

A[0]=Z[0];
for(K=1;K<MN;K++)
{ H[K]=H[K-1]/2;
  Z[K]=GetDiffQuat(x0,H[K]);
  if(K<2) A[K]=Z[K];
  else    A[K]=(8 * Z[K]-6 * Z[K-1]+Z[K-2])/3;
  if(K<4) continue;
  ERR=fabs(A[K]-A[K-1]);
  if(ERR<EPS)break
}
if(K<MN)K++;
RN=K;
ANS=A[RN-1];
return;
}

```

11.2 利用中心差商外推加速

11.2.1 中心差商的优势

假如 $f(x)$ 在 x_0 处适当光滑,也就是说我们所要用到的各阶导数都存在,那么可以把 $f(x_0+h)$ 和 $f(x_0-h)$ 在 x_0 处泰勒展开得

$$f(x_0+h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{3!}f'''(x_0)h^3 + \dots$$

$$f(x_0-h) = f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{3!}f'''(x_0)h^3 + \dots$$

由此可得

$$\frac{f(x_0+h) - f(x_0-h)}{2h} = f'(x_0) + \frac{1}{3!}f'''(x_0) \cdot h^2 + O(h^4)$$

从而可以取

$$f'(x_0) \approx \frac{f(x_0+h) - f(x_0-h)}{2h} \quad (2.1)$$

上面的(2.1)式称为中点公式,也可以认为是利用中心差商替代微商。显然它具有二阶精度,相对于(1.1)式来说,精度提高了一阶。

对于求数值微分来说,如果问题允许利用中心差商来进行近似计算,作为一个不成文的约定,我们都应该自动地采用中心差商来计算。

11.2.2 变步长方法

我们也可以按变步长办法利用(2.1)式来求 $f(x)$ 在 x_0 处的数值微分。假如对于给定的 $h_0 > 0$, 对 $k=0, 1, \dots$, 记

$$\left. \begin{aligned} h_k &= h_0/2^k \\ z_k &= \frac{f(x_0 + h_k) - f(x_0 - h_k)}{2h_k} \end{aligned} \right\} \quad (2.2)$$

那么当 $k \rightarrow \infty$ 时, 我们也有 $z_k \rightarrow f'(x_0)$ 。类似地也可以预先设定一个充分小的正数 EPS 来控制精度, 当某个 $|z_k - z_{k-1}| < \text{EPS}$ 时, 即可停止进一步的计算。由于计算诸 z_k 时还是存在不利的减法运算, 所以如果不采用加速的技巧, 误差限 EPS 的值不宜选得太小。

11.2.3 采用外推方法加速

利用(2.2)式得到的各 (h_k, z_k) , $k=0, 1, \dots$, 同样可以看成是某个自变量为 h 的函数

$$F(x_0, h) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} \quad (2.3)$$

在一些离散点处的函数值列表, 所以也可以从中挑出一些作插值多项式 $p(h)$ 作为 $F(x_0, h)$ 的近似函数, 从而可以利用 $p(0)$ 作为 $F(x_0, h)$ 当 h 趋近于零时的极限的近似值。注意到 $F(x_0, h)$ 作为 h 的函数来说是偶函数, 所以可以把插值多项式的形式设想为

$$p(h) = a_0 + a_1 h^2 + a_2 h^4 + \dots + a_m h^{2m}$$

其中 m 的值可以灵活掌握。下面以 $m=2$ 为例说明加速的技巧, 此时相应的插值多项式为

$$p(h) = a_0 + a_1 h^2 + a_2 h^4 \quad (2.4)$$

对给定的 $k=0, 1, \dots$, 取 (h_k, z_k) , (h_{k+1}, z_{k+1}) , (h_{k+2}, z_{k+2}) 作为插值基点, 分别代入到(2.4)式中, 并利用 $h_{k+1}=2h_k$, $h_{k+2}=4h_k$, 从而有

$$\left. \begin{aligned} a_0 + a_1(h_k)^2 + a_2(h_k)^4 &= z_k \\ a_0 + 4a_1(h_k)^2 + 16a_2(h_k)^4 &= z_{k+1} \\ a_0 + 16a_1(h_k)^2 + 256a_2(h_k)^4 &= z_{k+2} \end{aligned} \right\} \quad (2.5)$$

同样利用(2.5)式解出 a_0 , 并记为 A_{k+2} 得

$$A_{k+2} = (64z_{k+2} - 20z_{k+1} + z_k)/45 \quad (2.6)$$

从而又得到一个加速收敛序列 $\{A_k, k=2, 3, \dots\}$ 。

上面的(2.6)式与前面的(1.3)式的作用和性质相同, 所以前面对(1.3)式的赞美之词对(2.6)式同样成立。

以求解例 1.1 为例, 模仿程序 11.01 并利用上面的(2.2)式和(2.6)编程计算也是很容易的, 我们把完整的源程序编号为程序 11.02, 它可以由程序 11.01 简单地改写而来, 而且改动的地方不多, 也容易理解, 所以下面只列出改动的部分, 读者可以自己完成具体的文字编辑工作。

程序 11.02 利用中心插商结合外推加速方法求数值微分

```
/* 把程序 11.01 中控制精度的 EPS 修改的更小一些 */
double EPS=0.4E-12;
/* 把程序 11.01 中计算差商的函数修改如下 */
double GetCDQ(double x, double h)
{ return (f(x+h)-f(x-h))/h/2;
```

```

}
/* 把程序 11.01 的 Operation() 函数中计算差商的两条语句修改如下 */
Z[0]=GetCDQ(x0,H[0]);
:
Z[K]=GetCDQ(x0,H[K]);
/* 把程序 11.01 的 Operation() 函数中计算 A[K] 的两条语句修改如下 */
if(K<2) A[K]=Z[K];
else A[K]=(64 * Z[K]-20 * Z[K-1]+Z[K-2])/45;
/* OK! 修改完毕 */

```

运行程序 11.02 所显示的结果如表 2.1 所示。与不用外推的结果对比,采用加速方法得到 $A[5]$ 有 13 位有效数字,而不用外推加速所得到的 $Z[5]$ 只有 6 位有效数字,两者相差十分显著。与程序 11.01 所得到的结果相比,按相同的计算量比较,这里的 $A[3]$ 有 11 位有效数字,而程序 11.01 所得到的 $A[6]$ 只有 9 位有效数字,所以利用中心差商结合外推加速的效果也更好一些。

表 2.1 程序 11.02 的运行结果

k	H[k]	Z[k]	A[k]
0	3.20000e-01	0.250200562121195	0.250200562121195
1	1.60000e-01	0.250050035033036	0.250050035033036
2	8.00000e-02	0.250012502188016	0.250000000033189
3	4.00000e-02	0.250003125136727	0.250000000000517
4	2.00000e-02	0.250000781258542	0.250000000000004
5	1.00000e-02	0.250000195313029	0.249999999999994
ANS = 0.249999999999994			
ERR = 1.069E-14			

11.3 理查逊(Richardson)外推加速方法

11.3.1 问题的提出

前面两节介绍的数值微分方法的基本思想是相同的:假如某个真值 $T(x_0)$ 是某个连续函数 $F(x_0, h)$ 当 h 趋近于零时的极限值,我们可以采用变步长方法构造一个单调收敛于零的序列 $\{h_k\}$,从而序列 $\{F(x_0, h_k)\}$ 当 k 趋近于无穷大时收敛于 $T(x_0)$;为了加快收敛速度,可以用递推的形式利用两个序列中若干个相邻的 h_k 以及相应的 $F(x_0, h_k)$ 构造插值多项式 $p(h)$,并利用 $p(0)$ 作为 $T(x_0)$ 的近似值,从而加快了收敛速度。理查逊外推加速方法就是直接利用 $F(x_0, h)$ 的泰勒展式来提高收敛速度的一种具有普遍意义的方法。

假设 $F(x_0, h)$ 在 $h=0$ 附近充分光滑,而且在 $h=0$ 处可展为泰勒级数

$$F(x_0, h) = T(x_0) + A_1 h + A_2 h^2 + \cdots + A_p h^p + \cdots \quad (3.1)$$

显然,如果利用 $F(x_0, h)$ 作为 $T(x_0)$ 的近似值,则相应的误差为 $A_1 h + A_2 h^2 + \cdots + A_p h^p + \cdots$ 。

一般地,在上面的(3.1)式中,如果 $A_1 \neq 0$,则称 $F(x_0, h)$ 具有 1 阶精度;如果 $A_1 = 0$, $A_2 = 0, \dots, A_{p-1} = 0, A_p \neq 0$,则称 $F(x_0, h)$ 具有 p 阶精度。显然,如果一个近似计算公式 $F(x_0, h)$ 的精度比较高,那么当 h 趋近于零时 $F(x_0, h)$ 也就会快速地趋近于 $T(x_0)$ 。问题是能否找到一种通用的方法来得到具有较高精度的近似公式。

理查逊外推加速方法实际上是给出了一种基本的思路,使得我们能够利用低阶精度

的公式构造高阶精度的公式。

11.3.2 中点公式与变步长方法

为了叙述的方便,假设 $F(x_0, h)$ 是计算 $T(x_0)$ 已知的具有一阶精度的公式,而且允许 h 取负值,把 $F(x_0, h)$ 和 $F(x_0, -h)$ 在 $h=0$ 处适当地泰勒展开,我们有

$$F(x_0, h) = T(x_0) + A_1 h + A_2 h^2 + \cdots + A_p h^p + A_{p+1} h^{p+1} + \cdots \quad (3.2)$$

$$F(x_0, -h) = T(x_0) - A_1 h + A_2 h^2 + \cdots + (-1)^p A_p h^p + A_{p+1} h^{p+1} + \cdots \quad (3.3)$$

从而进一步有

$$\frac{F(x_0, h) + F(x_0, -h)}{2} = T(x_0) + A_2 h^2 + A_4 h^4 + \cdots + A_{2m} h^{2m} + O(h^{2(m+1)}) \quad (3.4)$$

记

$$FT_0(x_0, h) = \frac{F(x_0, h) + F(x_0, -h)}{2} \quad (3.5)$$

由上面的(3.4)式和(3.5)式可知, $FT_1(x_0, h)$ 是计算 $T(x_0)$ 的具有二阶精度的公式。而且利用 $FT_0(x_0, h)$ 计算 $T(x_0)$ 时,可以约定 $h > 0$ 。由于在许多具体应用中,上面的(3.5)式相当于“中点公式”,所以我们这里也把它称为“中点公式”。

利用上面的(3.5)式求 $T(x_0)$ 的近似值还是很容易的,对于给定的 h 的初值 h_0 ,定义两个适当长度的数组 H, T_0 , 对 $k=0, 1, \dots$, 记

$$\left. \begin{aligned} H[k] &= h_0/2^k \\ T_0[k] &= FT_0(x_0, H[k]) \end{aligned} \right\} \quad (3.6)$$

当 k 趋近无穷大时, $T_0[k]$ 至少以二阶收敛的速率趋近于 $T(x_0)$ 。

11.3.3 高精度公式的构造方法

由上面的(3.5)式和(3.4)式不难得到

$$FT_0(x_0, h) = T(x_0) + A_2 h^2 + A_4 h^4 + \cdots + A_{2m} h^{2m} + O(h^{2(m+1)}) \quad (3.7)$$

在接下来的讨论中,我们总是假定 $FT_0(x, h)$ 是已知的连续函数,对任意固定的 x_0 , $FT_0(x_0, h)$ 作为 h 的函数在 $h=0$ 附近是适当光滑的偶函数,而且在 $h=0$ 处的泰勒展式具有上面的(3.7)式的形式。

为了利用上面的(3.7)式构造更高精度的计算公式,可以把 h 折半,从而有

$$FT_0(x_0, h/2) = T(x_0) + \frac{1}{4} A_2 h^2 + \frac{1}{16} A_4 h^4 + \cdots + \frac{1}{2^{2m}} A_{2m} h^{2m} + O(h^{2(m+1)}) \quad (3.8)$$

把上面的(3.8)式两边同时乘以 4,再减去(3.7)式两边,再把所得到的结果两边同时除以 3 得

$$\frac{4 \times FT_0(x_0, h/2) - FT_0(x_0, h)}{3} = T(x_0) + B_4 h^4 + B_6 h^6 + \cdots + B_{2m} h^{2m} + O(h^{2(m+1)}) \quad (3.9)$$

上面的(3.9)式中的 $B_4, B_6, \dots$, 虽然可以进一步用 $A_4, A_6, \dots$, 表示出来,由于我们一般并

不关心它们的具体值是多少,所以采用不同的符号来表示它们即可。记

$$FT_1(x_0, h) = \frac{4 \times FT_0(x_0, h/2) - FT_0(x_0, h)}{3} \quad (3.10)$$

利用(3.9)式和(3.10)式, $FT_1(x_0, h)$ 可以表示为

$$FT_1(x_0, h) = T(x_0) + B_4 h^4 + B_6 h^6 + \cdots + B_{2m} h^{2m} + O(h^{2(m+1)}) \quad (3.11)$$

显然 $FT_1(x_0, h)$ 是具有 4 阶精度的公式。

对于前面的(3.6)式给出的数组 H , 我们同样可以得到一个序列 $\{FT_1(x_0, H[k]), k=0, 1, \cdots\}$, 类似地, 利用数组 $T1$ 来存放序列中的元素。为了编程方便, 我们约定

$$T1[k+1] = FT_1(x_0, H[k]), k = 0, 1, \cdots \quad (3.12)$$

所以数组 $T1$ 中的第零个元素 $T1[0]$ 没有定义, 作为编程的技巧, 可以令 $T1[0] = T0[0]$ 。

实际上, 只要 $FT_0(x_0, h)$ 足够光滑, 我们还可以类似地利用(3.11)式对 h 折半而消去 h^4 项而得到具有 6 阶精度的近似公式

$$FT_2(x_0, h) = \frac{16 \times FT_1(x_0, h/2) - FT_1(x_0, h)}{15} \quad (3.13)$$

类似地, 对于前面的(3.6)式给出的数组 H , 我们同样可以得到一个序列 $\{FT_2(x_0, H[k]), k=0, 1, \cdots\}$, 同样可以利用数组 $T2$ 来存放序列中的元素。为了编程方便, 也同样约定

$$T2[k+2] = FT_2(x_0, H[k]), k = 0, 1, \cdots$$

所以数组 $T2$ 中的前两个元素 $T2[0]$ 和 $T2[1]$ 没有定义, 作为编程的技巧, 可以令 $T2[0] = T0[0]$, 形式上利用(3.13)式计算可得 $T2[1] = (16 \times T1[1] - T1[0])/15$ 。

我们已经完成了利用 $FT_0(x_0, h)$ 推出 $FT_1(x_0, h)$, 利用 $FT_1(x_0, h)$ 推出 $FT_2(x_0, h)$, 基于同样的思路, 我们可以更一般地利用 $FT_{k-1}(x_0, h)$ 推出 $FT_k(x_0, h)$, 相应的递推格式为

$$FT_k(x_0, h) = \frac{4^k \times FT_{k-1}(x_0, h/2) - FT_{k-1}(x_0, h)}{4^k - 1} \quad (3.14)$$

而且 $FT_k(x_0, h)$ 是具有 $2(k+1)$ 阶精度的公式。

一般地, 我们把上面的(3.14)式称为 $FT_0(x_0, h)$ 第 k 次外推所得到的结果, 并且把这一系列的外推统称为理查逊外推方法。其所以称为外推, 是由于实际效果相当于利用 $FT_0(x_0, h)$ 取 $h > 0$ 的一些离散点作插值多项式 $p(h)$, 并把 $p(0)$ 作为 $T(x_0)$ 的近似值, 具体的, 更灵活的操作方法我们在前面两节中已经尝试过, 而且效果也很好。

11.3.4 理查逊外推法求数值微分

如果一个问题可以利用外推法来加速收敛或提高精度, 我们当然会建议采用外推法来计算。如果利用机器计算, 经验表明, 一般情况下外推 3 次即可, 再多了, 要保存的中间结果也随之增多, 而收效并不显著。采用理查逊外推法求例 1.1 的源程序见程序 11.03, 源程序实际上特别简单, 读者可以对照上面的(3.5)式、(3.6)式、(3.12)式、(3.13)式和(3.14)式来读源代码, 以进一步加深对理查逊外推加速方法的理解。

程序 11.03 理查逊外推法求数值微分

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 32
int RN=0;
double EPS=0.4E-8,ERR;
double H[MN],T0[MN],T1[MN],T2[MN],T3[MN];
double x0=4.0;
double h0=0.32;
double FT0(double x,double h)
{ return (sqrt(x+h)-sqrt(x-h))/h/2;}
int Operation()
{ int k;
  H[0]=h0;
  T0[0]=FT0(x0,h0);
  T1[0]=T0[0];
  T2[0]=T0[0];
  T3[0]=T0[0];
  for(k=1;k<MN;k++)
  { H[k]=H[k-1]/2.0;
    T0[k]=FT0(x0,H[k]);
    T1[k]=(4.0*T0[k]-T0[k-1])/3.0;
    T2[k]=(16.0*T1[k]-T1[k-1])/15.0;
    T3[k]=(64.0*T2[k]-T2[k-1])/63.0;
    if(k<4) continue;
    ERR = fabs(T3[k]-T3[k-1]);
    if(ERR<EPS)break;
  }
  if(k<MN)k++;
  RN=k;
  ANS=T3[RN-1];
  return RN;
}
ShowTable(){...}
main()
{ Operation();
  ShowTable();
}

```

读者可以看到程序 11.03 中的 Operation()函数的代码几乎与具体问题无关,所以对于求解具体问题来说,也只需改写程序中的函数 FT0(),即可利用理查逊外推加速方法求解。运行程序 11.03 所显示的结果如表 3.1 所示,从给出的误差估计值 ERR 的值来看,最后结果应该有 15 位有效数字,所显示的结果也表明所得到的结果至少有 14 位有效数字。注意到不用外推加速法所得到的相应结果只有 5 位有效数字,两者根本不能相比。

表 3.1 程序 11.03 的运行结果

k	H[k]	T0[k]	T1[k]	T2[k]	T3[k]
0	3.200e-01	0.2502005621	0.2502005621	0.25020056212120	0.25020056212120
1	1.600e-01	0.2500000310	0.2499999893	0.2499999847915137	0.249999981000898
2	8.000e-02	0.2500125022	0.2499999912	0.25000000003319	0.250000021461038
3	4.000e-02	0.2500031251	0.2499999995	0.25000000000052	0.250000000000000
4	2.000e-02	0.2500007813	0.2500000000	0.25000000000000	0.250000000000000
ANS =		0.249999999999996			
ERR =		1.776E-15			

11.4 涅维尔(Neville)插值方法与通用外推格式^①

对于需要严格实行误差控制的多项式插值来说,拉格朗日插值和牛顿插值(两者本质上是相同的)的先天不足就是误差估计比较麻烦。虽然也有相应的余项估计公式,但用起来却不太方便。由于一般的多项式插值不太容易估计误差,所以确定插值多项式的次数通常也带有凭经验估计的成分。克服这个缺点的有效途径就是采用涅维尔插值方法或埃特金插值方法。

涅维尔方法和埃特金方法本质上都是基于递推的方法,也就是利用两个较低次,比如 m 次,多项式插值结果再作一次线性插值而得到 $m+1$ 次多项式插值结果,所以这两种方法的原理完全相同,甚至还容易混淆。如果仅用于一般的内插,这两种方法难分伯仲;如果应用于外推加速,涅维尔方法更为实用。在现代计算机条件下,由于插值方法的使命已由过去的计算函数值转向提高序列的收敛速率,以得到更精确的计算结果,所以人们现在只看重涅维尔插值方法。

11.4.1 涅维尔插值的递推格式

对于一般的利用 $n+1$ 个基点 $x_0, x_1, \dots, x_n$ 处的函数值 $y_0 = f(x_0), y_1 = f(x_1), \dots, y_n = f(x_n)$ 进行不同的 m 次多项式插值问题,对 $k=0, 1, \dots, n$ 以及 $m=0, 1, \dots, k$, 记 $p_{k,m}(x)$ ^② 为 $x_k, x_{k-1}, \dots, x_{k-m}$ 这 $m+1$ 个基点所确定的 m 次插值多项式,显然有 $p_{k,0}(x) \equiv y_k$, 而 $p_{k,k}(x)$ 就是经过 $(x_0, y_0), (x_1, y_1), \dots, (x_k, y_k)$ 的 k 次插值多项式。

对于适当的 k 值以及 $m \leq k$, 为了利用 $p_{k,m}(x)$ 和 $p_{k+1,m}(x)$ 得到 $p_{k+1,m+1}(x)$, 可以把 $p_{k+1,m+1}(x)$ 表示为

$$p_{k+1,m+1}(x) = a(x_{k-m} - x)p_{k+1,m}(x) + b(x_{k+1} - x)p_{k,m}(x) \quad (4.1)$$

其中 a, b 都是待定常数^③。为了确定这两个常数 a 和 b , 可以在上面的(4.1)式中令 x 分别取 x_{k+1} 和 x_{k-m} , 由于

$$\left. \begin{aligned} p_{k+1,m+1}(x_{k+1}) &= p_{k+1,m}(x_{k+1}) = y_{k+1} \\ p_{k+1,m+1}(x_{k-m}) &= p_{k,m}(x_{k-m}) = y_{k-m} \end{aligned} \right\} \quad (4.2)$$

所以由上面的(4.1)式和(4.2)式可以立即得到

① 通用外推格式是作者使用的提法,目的是为了推广外推方法的研究和应用,应能得到广泛的认同。

② 与参考文献[14]第112页的论述相比,作者在这里采用了更为简明的记号。

③ 与国内外讨论同类问题的流行表达方式相比,作者在这里对公式(4.1)作了细微的技术处理,目的是更容易推出下面的公式(4.4),从而进一步得到通用外推格式(4.5)式和(4.6)式。

$$a(x_{k-m} - x_{k+1}) = 1, \quad b(x_{k+1} - x_{k-m}) = 1$$

从而有

$$a = \frac{1}{x_{k-m} - x_{k+1}}, \quad b = -\frac{1}{x_{k-m} - x_{k+1}} \quad (4.3)$$

再由(4.1)式和(4.3)式即可得到

$$p_{k+1,m+1}(x) = \frac{(x_{k-m} - x)p_{k+1,m}(x) - (x_{k+1} - x)p_{k,m}(x)}{x_{k-m} - x_{k+1}} \quad (4.4)$$

注意到在上面的(4.4)式中, $p_{k+1,m}(x)$ 和 $p_{k,m}(x)$ 的组合系数满足关系式

$$\frac{(x_{k-m} - x) - (x_{k+1} - x)}{x_{k-m} - x_{k+1}} \equiv 1$$

而当 x 分别取 $x_{k-m+1}, x_{k-m+2}, \dots, x_k$ 时 $p_{k+1,m}(x)$ 和 $p_{k,m}(x)$ 的值都依次为 $y_{k-m+1}, y_{k-m+2}, \dots, y_k$, 从而得到(4.4)式的确是由 $(x_j, y_j), j = k-m, k-m+1, \dots, k, k+1$ 这 $m+2$ 个基点所确定的次数不超过 $m+1$ 的插值多项式。

11.4.2 涅维尔插值的计算方法

利用(4.4)式以及对任意 k 总有 $p_{k,0}(x) \equiv y_k$, 所以对适当的 k 和 m , 我们不难采用递推的方式计算 $p_{k,m}(x)$ 值。对于固定的 x , 记 $p_{k,m}$ 为多项式 $p_{k,m}(x)$ 的值, 可以按表 4.1 所示的格式按行由上到下, 按列由左到右的顺序计算。

表 4.1 Neville 插值的计算表格

		$m=0$	$m=1$	$m=2$	$m=3$	$m=4$	...
k	x_k	y_k	$p_{k,1}$	$p_{k,2}$	$p_{k,3}$	$p_{k,4}$	...
0	x_0	y_0					...
1	x_1	y_1	$p_{1,1}$				...
2	x_2	y_2	$p_{2,1}$	$p_{2,2}$			...
3	x_3	y_3	$p_{3,1}$	$p_{3,2}$	$p_{3,3}$		...
4	x_4	y_4	$p_{4,1}$	$p_{4,2}$	$p_{4,3}$	$p_{4,4}$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

按表 4.1 给出的计算格式编写相应的计算机程序也是很容易的, 表中的各 x_k, y_k 以及各 $p_{k,m}$ 可以说明相应的全局变量 X, Y, P 存放, 相应的 C 语言函数可以说明为

```
void Neville(double x, int K)
```

其中 x 为插值点, k 为插值多项式的最高次数。

程序 11.04 按表 4.1 递推计算的涅维尔插值程序

```
#define MN 32
double X[MN+1], Y[MN+1], P[MN+1][MN+1];
void Neville(double x, int K)
{ int k, m;
  P[0][0] = Y[0];
  for(k=0; k<K; k++)
```

```

{ P[k+1][0]=Y[k+1];
  for(m=0;m<=k;m++)
  { P[k+1][m+1]=(X[k-m]-x)*P[k+1][m]-(X[k+1]-x)*P[k][m];
    P[k+1][m+1]/=X[k-m]-X[k+1];
  }
}
return;
}

```

例 4.1^① 给出 $y=f(x)$ 的函数值列表如表 4.2 所示, 试利用涅维尔插值方法求 $f(1.5)$ 的近似值。

表 4.2 $y=f(x)$ 的函数值列表

k	0	1	2	3	4	5
$X[k]$	1.0	1.3	1.6	1.9	2.2	2.5
$Y[k]$	0.765 197 7	0.620 086 8	0.455 440 22	0.281 818 6	0.110 362 3	-0.048 383 8

利用表 4.2 给出的原始数据, 取 $K=5, x=1.5$ 运行程序 11.04 所得到的结果如表 4.3 所示。

表 4.3 运行程序 11.04 所得到的结果

k	$X[k]$	$P[k][0]$	$P[k][1]$	$P[k][2]$	$P[k][3]$	$P[k][4]$	$P[k][5]$
0	1.00	0.7651977					
1	1.30	0.6200860	0.5233449				
2	1.60	0.4554022	0.5102968	0.5124715			
3	1.90	0.2818186	0.5132634	0.5112857	0.5118127		
4	2.20	0.1103623	0.5104270	0.5137381	0.5118302	0.5118200	
5	2.50	-0.0483838	0.4807699	0.5301984	0.5119070	0.5118430	0.5118277

一般说来, 利用涅维尔方法插值的基本目的是能够自动进行误差控制, 这对原始数据列表有一定的要求, 否则涅维尔方法插值的优势很难体现出来。从表 4.3 所示的结果看, $P[5][3]$ 的精度反不如 $P[4][3]$ 和 $P[3][3]$ 的精度高, $P[5][4]$ 的精度也不如 $P[4][4]$ 的精度高, 原因应该是“最后”增加的插值基点 $X[5]$ 离插值点 $x=1.5$ 太远所致。把表 4.2 中的各 $X[k]$ 简单地按 $|X[k]-1.5|$ 的值由大到小的顺序重排而得到表 4.4, 再运行程序 11.04 所得到的结果如表 4.5 所示。由表 4.5 所显示的计算结果可以看出, $P[5][5]$, $P[4][3]$, $P[5][3]$, $P[4][4]$, $P[5][4]$, $P[5][5]$ 都可以得到近似值 0.511 8, 所以我们可以断言 $f(1.5)=0.511 8$ 。

表 4.4 表 4.2 按 $|X[k]-1.5|$ 重排序后的函数值列表

k	0	1	2	3	4	5
$X[k]$	2.5	2.2	1.0	1.9	1.3	1.6
$Y[k]$	-0.048 383 8	0.110 362 3	0.765 197 7	0.281 818 6	0.620 086 8	0.455 440 22

① 本例选自文献[3]第 94 页, 利用程序 11.04 所得到的计算结果与文献[3]中的完全相同。

表 4.5 重运行程序 11.04 所得到的结果

k	x[k]	P[k][0]	P[k][1]	P[k][2]	P[k][3]	P[k][4]	P[k][5]
0	2.50	-0.0483838					
1	2.20	0.1103623	0.4807699				
2	1.00	0.7651977	0.4823496	0.4884897			
3	1.90	0.7818186	0.4966538	0.5023926	0.5116612		
4	1.30	0.6200860	0.5073302	0.5244478	0.5117689	0.5117509	
5	1.60	0.4554022	0.5102968	0.5112857	0.5118127	0.5118200	0.5118277

实际上,由表 4.5 还可以看出,对于给定的 m 值, $P[k][m]$ 随着 k 值增大精度愈来愈高;对于给定的 k 值, $P[k][m]$ 随着 m 值的增大精度愈来愈高^①。涅维尔插值的这种性质特别适用于外推加速。

11.4.3 利用涅维尔插值公式构造外推格式

对于利用外推方法计算连续函数 $y=f(x)$ 当 x 趋近于某个 x_0 时的极限值来说,我们总可以通过一个平移变换而得到相应的 $y=f(x_0+h)$ 当 h 趋近于 0 时的极限值。为叙述的方便,可以把 x_0 看成是固定常数,把 $f(x_0+h)$ 看成是某个关于 h 的函数 $F(h)=f(x_0+h)$ 。为了计算 $F(h)$ 当 h 趋近于 0 时的极限值,可以构造一个单调收敛于 0 的序列 $\{h_n\}$,令 $y_n=F(h_n)$,从而可以利用涅维尔插值方法计算各 $p_{k,m}(0)$,相当于利用一系列 $F(h_n)$ 值来推算出一系列 $F(0)$ ^② 的近似值。由于实际计算总是利用有限多个大于零的 $h_k, k=0,1,\dots,n$ 来计算 $F(0)$,所以相应的方法也就称为外推。当我们利用涅维尔插值方法进行这样一类外推计算时,可以把各 $p_{k,m}(0)$ 直接简记为 $p_{k,m}$,从而可由前面的(4.4)式得到下面的外推格式:

$$p_{k+1,m+1} = \frac{h_{k-m}p_{k+1,m} - h_{k+1}p_{k,m}}{h_{k-m} - h_{k+1}}, k=0,1,\dots; m=0,1,\dots,k \quad (4.5)$$

其中 $P_{k,0}=y_k, k=0,1,\dots$ 。

对于利用外推方法计算某个连续函数 $F(h)$ 当 h 趋近于 0 时的极限值这一类应用来说,很多情况下 $F(h)$ 是关于 h 的偶函数,此时 $F(h)$ 的泰勒展式可以表示为

$$F(h) = a_0 + a_1h^2 + a_2h^4 + \dots + a_mh^{2m} + O(h^{2(m+1)})$$

相应的插值多项式可以表示为

$$p(h) = a_0 + a_1(h^2)^1 + a_2(h^2)^2 + \dots + a_m(h^2)^m$$

它可以利用 $m+1$ 对 $((h_j)^2, F(h_j)), j=k, k-1, \dots, k-m$ 的值来确定,具体计算方法当然也可以按涅维尔插值方法的递推格式计算,从而得到相应的外推格式为

$$p_{k+1,m+1} = \frac{(h_{k-m})^2 p_{k+1,m} - (h_{k+1})^2 p_{k,m}}{(h_{k-m})^2 - (h_{k+1})^2}, k=0,1,\dots; m=0,1,\dots,k \quad (4.6)$$

(4.5)式以及(4.6)式称为通用的外推格式是合理的。在这两个公式的基础上,只要得到一个不同的构造步长序列 $\{h_k, k=0,1,\dots\}$ 的策略,即可得到一个不同格式的外推加速方法。

① 虽然从理论上讲应该有这样的结果,实际计算也能体验到这种结果,但是当 k 和 m ,特别是 m ,大到一定程度后,情况会发生改变。

② 从通常意义上讲, $F(h)$ 当 $h=0$ 并没有定义,从数学意义上看,由于 $F(h)$ 当 h 趋于零时的极限存在,所以可以利用 $F(h)$ 当 h 趋于零时的极限值来补充定义。

11.4.4 涅维尔外推格式蕴含理查逊外推格式

回顾理查逊外推方法所采用的步长序列,对于给定的初始步长 h_0 ,我们有 $h_n = h_0/2^n, n=0,1,\dots$ 。把它们代入到(4.6)式中即得

$$p_{k+1,m+1} = \frac{\left(\frac{h_0}{2^{k-m}}\right)^2 p_{k+1,m} - \left(\frac{h_0}{2^{k+1}}\right)^2 p_{k,m}}{\left(\frac{h_0}{2^{k-m}}\right)^2 - \left(\frac{h_0}{2^{k+1}}\right)^2}, k=0,1,\dots; m=0,1,\dots,k$$

化简后即得

$$p_{k+1,m+1} = \frac{4^{m+1} p_{k+1,m} - p_{k,m}}{4^{m+1} - 1}, k=0,1,\dots; m=0,1,\dots,k$$

与(3.14)式对比分析即可得出的,对于使用步长序列 $\{h_n = h_0/2^n, n=0,1,\dots\}$ 而言,如果利用(4.6)式进行外推,实际效果就是使用理查逊外推方法!

基于不同的算法原理而得到了相同的计算方法,这显然是一件很有意义的事情。一方面由于(4.6)式是利用插值多项式导出的外推公式,所以理查逊外推方法也仅仅是一种通常意义下的外推方法;反过来,不难利用理查逊外推方法直接得到利用(4.5)式和(4.6)式所得到的外推结果 $p_{k+1,m+1}$ 的精度估计公式。

11.5 实用外推加速方法

对于计算适当光滑的实函数 $F(h)$ 当 h 趋近于零时的极限值来说,利用(4.5)式以及(4.6)式,只要得到一个不同的构造步长序列 $\{h_k, k=0,1,\dots\}$ 的策略,即可得到一个具体的外推加速方法。这样一来,我们就可以脱离具体问题而专门研究如何构造这样的步长序列,使得在一般情况下能得到尽可能好的外推效果。在具体应用的场合,又可以在一般性的方法的基础上采用一些具体措施以得到更好的结果。

11.5.1 构造步长序列应注意的问题

显然,我们不大可能利用最优化技术寻找对所有问题而言都效果最好的步长序列,惟一能做的事情就是根据(4.5)式和(4.6)式的特点以及通常计算 $F(h)$ 的函数值可能面临的问题而给出一些构造步长序列的基本原则。

在很多情况下,当 h 的值充分小时,计算 $F(h)$ 的函数值可能会遇到数值性能不好的问题。比如利用中心差商求 $f(x)$ 在 x_0 处的数值微分,这时 $F(h)$ 的形式为

$$F(h) = \frac{f(x_0 + h) - f(x_0 - h)}{2h}$$

当 h 的值充分小时,计算 $F(h)$ 的函数值会产生较大的舍入误差。

一般地,如果计算 $F(h)$ 不存在数值性能的问题,则让序列 $\{h_n\}$ 快速收敛于零可以相应地提高外推加速的收敛性能,否则应该放慢序列 $\{h_n\}$ 收敛于零的速度以提高外推加速的数值性能。由于现代计算机的运算速度已经有显著提高,所以强调计算的精度是合理的。为了得到更好的数值性能,应该适当放慢 $\{h_n\}$ 收敛于零的速度。

过于放慢 $\{h_n\}$ 收敛于零的速度的负作用是导致相邻的各 h_k 之间的距离减小,由此也会影响外推的性能。对于适当的 k 值,以计算 $p_{k+1,1}$ 为例,仔细分析(4.5)式不难得到,由于 $p_{k+1,1}$ 和 $p_{k,1}$ 都是同一个真值的近似值,所以它们相差不大,如果还有 h_k 和 h_{k+1} 也都相差很小,那么(4.5)式的分子中就存在不利的运算。

11.5.2 利用等比系列外推加速

可以说,最容易想到的步长序列就是等比数列。对于给定的初始步长 h_0 ,只要确定一个大于1的 q 值,令 $h_k = h_0/q^k, k=0,1,2,\dots$,即可得到一个步长序列。显然, q 值愈大,序列收敛于零的速度愈快。由于对收敛速度存在着互相矛盾的制约,所以选择合理的 q 值的余地并不是很大。一般地,把 q 值确定在1.5~2.0的范围内是比较明智的。当 $q=2$ 时,所形成的方法即为人们通常采用的理查逊外推方法。当 q 取1.5时,由于 $(1.5/2)^{10} < 1/10$,所以 $\{h_n\}$ 收敛于零的速度已经有显著的降低。

在(4.5)式、(4.6)式中利用 $h_n = h_0/q^n$ 进行相应的替换,化简后即可得到

$$p_{k+1,m+1} = \frac{q^{m+1} p_{k+1,m} - p_{k,m}}{q^{m+1} - 1}, k = 0, 1, \dots; m = 0, 1, \dots, k \quad (5.1)$$

以及

$$p_{k+1,m+1} = \frac{(q^2)^{m+1} p_{k+1,m} - p_{k,m}}{(q^2)^{m+1} - 1}, k = 0, 1, \dots; m = 0, 1, \dots, k \quad (5.2)$$

利用上面的(5.1)式和(5.2)式实现外推加速的源程序几乎是相同的,相应的C语言函数可以分别说明为

```
int Extrapol1(double h0, double q0, double EPS)
int Extrapol2(double h0, double q0, double EPS)
```

其中 h_0 为步长的初值, q_0 为比例常数,EPS为允许误差;函数返回值为1说明外推加速成功,返回值为0说明计算失败或达不到精度要求。利用(5.2)式采用等比系列外推完整的源代码见程序11.05A。程序中:符号常数MN表示变步长序列的最大长度,一般取为20已经足够;符号常数MM表示插值多项式的最高次数,如果利用(5.2)式编程,一般取6已经足够,如果利用(5.1)式编程,可以相应地取在8~16之间;整型变量RK,RM用于表示最后的计算结果为 $P[RK][RM]$ 。

程序 11.05A 利用等比序列实现外推加速的源程序

```
/* FILE Name: Extrapol2.c */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 28
#define MM 8
double F(double h);
double Q0=1.0, Q[MM+1], P[MN+1][MM+1];
int RK=0, RM=0;
int Extrapol2(double h0, double q0, double EPS)
```

```

{ double err,h;
  int k,m;
  h=h0; Q0=q0*q0;Q[0]=1.0;
  for(k=0;k<MM;k++) Q[k+1]=Q[k]*Q0;
  P[0][0]=F(h);h/=q0;
  for(k=0;k<MN;k++)
  { P[k+1][0]=F(h);h/=q0;
    for(m=0;m<k;m++)
    { err=P[k+1][m]-P[k][m];
      if(fabs(err)<EPS){RK=k+1;RM=m;return 1;}
      if(m>=MM) break;
      P[k+1][m+1]=Q[m+1]*P[k+1][m]-P[k][m];
      P[k+1][m+1]/=Q[m+1]-1.0;
    }
  }
  RK=MN;RM=MM;
  return 0;
}

```

利用程序 11.05A 求解例 1.1 的主程序见程序 11.05B。令允许误差 EPS 分别取 $0.4\text{E}-12$ 和 $0.4\text{E}-15$ ；令比值 q 分别取 1.6 和 2 运行程序所显示的结果如表 5.1 所示。

表 5.1 程序 11.05B 的运行结果

```

q=2.0000 EPS= 4.0E-13
K= 5 M= 2
P[K-1][M]= 2.500000000000043e-01
P[K][M] = 2.499999999999936e-01

q=2.0000 EPS= 4.0E-15
K=28 M= 8
P[K-1][M]= 2.500000282639999e-01
P[K][M] = 2.499999831291806e-01
Bad Problem !

q=1.6000 EPS= 4.0E-13
K= 5 M= 3
P[K-1][M]= 2.499999999999994e-01
P[K][M] = 2.500000000000000e-01

q=1.6000 EPS= 4.0E-15
K= 6 M= 3
P[K-1][M]= 2.500000000000000e-01
P[K][M] = 2.500000000000002e-01

```

从运行程序所显示的结果看，当允许误差控制精度 EPS 取 $0.4\text{E}-12$ 时， q 取 2.0 和 1.6 都达到了控制的精度，而且两者的计算量也大体相同。当允许误差控制精度 EPS 取 $0.4\text{E}-15$ 时，取 $q=2.0$ 达不到精度要求，而取 $q=1.6$ 却能够得到具有 15 位有效数值的结果，这种差异印证了我们前面的分析。

程序 11.05B

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ExtrapA2.C"

double x0=4.0;
double h0=0.32;
double F(double h)

```

```

    { return (sqrt(x0+h)-sqrt(x0-h))/2.0/h;
    }
main()
{ int OK;
  double q=1.6, EPS = 0.4E-15;
  OK=ExtrapA2(h0,q, EPS);
  printf("\n q=%6.4f EPS=%8.2E",q, EPS);
  printf("\n K=%2d M=%2d",RK,RM);
  printf("\n P[K-1][M]=%24.16e",P[RK-1][RM]);
  printf("\n P[K][M] =%24.16e",P[RK][RM]);
  if(!OK) printf("\n Bad Problem !");
  getchar();
}

```

11.5.3 利用等差数列外推加速

在外推方法的许多应用领域,不同的步长是将某个区间划分为不同的等分而得到。为了适应这样一类应用的需要,可以采用一个单调递增的正整数序列来构造步长序列。

设 $\{a_n\}$ 是一个单调递增的正整数序列,对于给定的初始步长 h_0 ,令 $h_n=h_0/a_n$ 即可得到单调收敛于零的步长序列 $\{h_n\}$ 。在(4.5)式、(4.6)式中利用 $h_n=h_0/a_n$ 进行相应的替换,化简后即可得到

$$p_{k+1,m+1} = \frac{a_{k+1}p_{k+1,m} - a_{k-m}p_{k,m}}{a_{k+1} - a_{k-m}}, k=0,1,\dots; m=0,1,\dots,k \quad (5.3)$$

以及

$$p_{k+1,m+1} = \frac{(a_{k+1})^2 p_{k+1,m} - (a_{k-m})^2 p_{k,m}}{(a_{k+1})^2 - (a_{k-m})^2}, k=0,1,\dots; m=0,1,\dots,k \quad (5.4)$$

所以,只要给出一个正整数序列的生成方法,即可形成一种特定的外推加速方法。

比较简单的正整数序列就是所谓等差数列,取首项为某个 n_0 ,公差为某个 n_1 ,那么我们有 $a_0=n_0, a_k=a_{k-1}+n_1, k=1,2,\dots$ 。通常 n_0 可以取为2或4,公差 n_1 也可以取2或4。

利用(5.4)式进行外推加速的源代码见程序11.06。与程序11.05A相比,增加了一个整形数组A存放相应的整数序列;增加了一个生成等差数列的函数FormSequence();省去了存放比值序列的数组Q。其余的地方几乎完全相同。程序中还说明了两个整形变量N0、N1分别存放等差数列的首项和公差,给它们赋不同的初值可以得到不同的等差数列。

程序 11.06 利用等差数列作外推加速的源程序

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 28
#define MM 8

```

```

int    RK, RM, A[MN+1];
double P[MN+1][MM+1];
double F(double h);
void FormSequence()
{ int k;
  A[0]=1; A[1]=2;
  for(k=2; k<=MN; k++) A[k]=A[k-1]+A[k-2];
  return;
}
int ExtrapolB2(double H0, double EPS)
{ double err, a1, a2;
  int k, m, N1, N2;
  FormSequence();
  P[0][0]=F(H0/A[0]);
  for(k=0; k<MN; k++)
  { P[k+1][0]=F(H0/A[k+1]);
    for(m=0; m<=k; m++)
    { err=P[k+1][m]-P[k][m];
      if(fabs(err)<EPS) {RK=k+1; RM=m; return 1;}
      if(m>MM) break;
      a1=(double)A[k+1]; a1*=a1;
      a2=(double)A[k-m]; a2*=a2;
      P[k+1][m+1]=a1*P[k+1][m]-a2*P[k][m];
      P[k+1][m+1]/= a1-a2;
    }
  }
  RK=MN; RM=MM;
  return 0;
}

```

利用等差数列形成外推方法也有很多需要注意的问题。首先是序列的长度不宜太长,一般不超过 16 即可。否则,相应的步长序列收敛于零的速度会显著变慢,从而使外推的性能变坏。由于步长序列的长度的限制,从而对初始步长也有一定的限制,否则会由于初始步长太大加之相应的步数较少从而相应的外推难以达到精度要求。由于这些原因,利用等差数列形成外推方法只是应用于一些典型的问题才会有较好的效果。

11.5.4 利用斐波那奇(Fibonacci)数列外推加速

对于克服等差数列外推加速的一些弊端来说,斐波那奇(Fibonacci)数列倒是具有某种得天独厚的优势。斐波那奇数列的前几项为

$$1, 2, 3, 5, 8, 13, 21, \dots$$

其构造规则为第 1 项为 1,第 2 项为 2,以后每一项都等于前面两项之和,所以生成斐波那奇数列也是很容易的。

斐波那奇数列的一个重要性质是对任意 $n > 0$, 均有 $a_n/a_{n-1} \approx 1.6$, 所以利用斐波那

奇数列产生的变步长序列也近似于一个等比数列,而且公比正好是比较理想的比值 1.6。回顾利用等比数列所形成的外推格式(5.1)式和(5.2)式不难理解,利用等比数列所形成的外推方法对初始步长 h_0 的选择是无关紧要的事,因为在计算过程中,步长实际上是按比例减小的。由于利用斐波那奇数列产生的变步长序列也近似于一个等比数列,所以也无需过于在意初始步长的选择。

在(5.3)式和(5.4)式中把序列 $\{a_n\}$ 取为斐波那奇数列,即可形成一个特殊的外推加速方法。利用斐波那奇数列作外推加速的源程序见程序 11.07,与程序 11.06 相比,只需改写其中的 FormSequence() 函数即可,读者可以自己完成文字编辑工作。

程序 11.07 利用等差数列作外推加速的源程序

```
void FormSequence()
{ int k;
  A[0]=1; A[1]=2;
  for(k=2;k<=MN;k++) A[k]=A[k-1]+A[k-2];
  return;
}
int ExtrapolB2(double h0,double EPS){...}
```

读者可以通过简单地修改程序 11.05B 而得到利用程序 11.06、11.07 求解例 1.1 的主函数程序。当 EPS 取 $0.4E-14$ 时,如果步长初值 h_0 取 0.32,程序 11.06 和 11.07 都能得到正确的结果;如果取初始步长为 $h_0 = 0.64$,只有程序 11.07 能得到正确的结果。这又一次印证了我们的分析。

11.5.5 关于停机规则的改进

不管是基于理查逊外推方法的理论基础,还是涅维尔插值方法的误差分析,通常的结论是利用 $p_{k,k}$ 作为 $F(0)$ 的近似值所得到的误差的阶数最高,所以人们喜欢利用表 4.1 的对角线上的元素来逼近 $F(0)$ 。但是,无论是基于直观判断还是基于实际计算的结果分析,确定合适的 k 值,把 $p_{k,k}$ 作为 $F(0)$ 的近似值并不见得总是最好的。利用程序 11.05A 求解例 1.1 的计算结果也表明,确定适当的 k, m , 利用 $p_{k,m}$ 作为 $F(0)$ 的近似值更为合理。对于一般的应用来说,完全可以利用

$$|p_{k+1,m} - p_{k,m}| < \text{EPS}$$

作为停机规则是合适的。如果允许误差 EPS 定得太小,或者由于达不到这样的精度而不收敛,或者由于 $F(h_{k+1})$ 与 $F(h_k)$ 的误差小于 EPS 而得到错误的结果。所以制定一个具有某种程度的“自适应性”的停机规则是有意义的。

尽管从理论上讲,对任意给定的 m , $p_{k,m}$ 随 k 值增大而收敛于问题的解,但实际上只能是某个 $p_{k,m}$ 最接近所要求解的真值。不难理解,这个特殊的 $p_{k,m}$ 与附近的几个 $p_{i,j}$ 之差也应该随之达到最小。为此,可以把相应的误差估计值表示为

$$\text{ERR} = \max\{|p_{k,m} - p_{k-1,m}|, |p_{k,m} - p_{k,m+1}|\} \quad (5.5)$$

这样,对每个固定的 k 值,总可得到某个相应的最好的 $p_{k,m}$, 以及相应的误差估计值 ERR_k 。可以在计算过程中一直保存当前的误差估计值,如果能随着 k 值的增大单调减小

而达到小于 EPS,则称之为正常返回;在计算过程中如果某个 $k+1$ 及所有 $m=0,1,\dots,k$ 所得到的误差估计值不能使得误差有所减小,则停止进一步的计算,把当前的 $p_{k,m}$ 作为所能得到的最好结果。对等比序列的外推加速方法采用这种自适应精度的停机规则的 C 语言源代码见程序 11.08,它可以由程序 11.05A 改写而来,读者可以对照阅读源代码以领悟实现这类技巧的具体方法。在程序 11.08 中,函数 ExtrapolA2() 的返回值如果是 1,则表示得到相应精度的结果;若为 0,则表示没有得到相应精度的结果。在后一种情况下, $P[RK][RM]$ 实际上是所能得到的 $F(0)$ 的最好的近似值。由于利用上面的(5.6)式估计误差偏于保守,所以即使函数的返回值为零, $P[RK][RM]$ 也有可能达到相应的精度,为此,可以对各 $P[k][m]$ 作进一步的分析。程序 11.08 可以用文件名“ExtrapD2.C”保存为磁盘文件。

程序 11.08

```
/* FileName ExtrapolD2.C */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 28
#define MM 8
double F(double h);
double Q0=1.0,Q[MM+1],P[MN+1][MM+1];
int RK=0,RM=0;
double ERR=1.0e100;
int ExtrapolA2(double h0,double q0,double EPS)
{ double h,err0,err1,token=0;
  int k,m;
  h=h0; Q0=q0*q0;Q[0]=1.0;
  for(k=0;k<MM;k++) Q[k+1]=Q[k]*Q0;
  P[0][0]=F(h);h/=q0;
  for(k=0;k<MN;k++)
  { token=0;
    P[k+1][0]=F(h);h/=q0;
    for(m=0;m<=k;m++)
    { if(m>=MM) break;
      P[k+1][m+1]=Q[m+1]*P[k+1][m]-P[k][m];
      P[k+1][m+1]/=Q[m+1]-1.0;
      err0=fabs(P[k+1][m]-P[k][m]);
      err1=fabs(P[k+1][m+1]-P[k+1][m]);
      if(err1<err0) err0=err1;
      if(err0<ERR){ ERR=err0;RK=k+1;RM=m;token=1;}
    }
  }
  if(ERR<EPS) return 1;
  if(token==0) return 0;
  token=0;
```

```

    }
    RK=MN;RM=MM;
    return 0;
}

```

适当修改程序 11.05A 即可应用程序 11.08 求解例 1.1。取 $q=2.0$, EPS 分别为 $0.4E-12$ 和 $0.4E-15$ 运行程序 11.08 求解例 1.1 所显示的结果见表 5.2。

表 5.2 运行程序 11.08 求解例 1.1 所显示的结果

q=2.0000	EPS= 4.0E-13
K= 4	M= 5
P[K-1][M]=	2.49999999999980e-01
P[K][M]=	2.49999999999982e-01
P[K][M+1]=	2.49999999999982e-01
q=2.0000	EPS= 4.0E-15
K= 4	M= 5
P[K-1][M]=	2.49999999999980e-01
P[K][M]=	2.49999999999982e-01
P[K][M+1]=	2.49999999999982e-01
Bad Problem !	

从表 5.2 可以看出,取 $q=2.0$, $EPS=0.4E-15$ 时,程序得不到相应精度的结果,但是程序仍然能“理智”地返回最精确的结果,对于本例来说,也达到了 14 位有效数字。对于一般的实际应用问题来说,这样的结果已经相当精确了。

11.5.6 关于外推方法的进一步讨论

外推方法是一种低成本(仅对前面得到的近似值按给定的组合系数再作一个线性组合,从而算法简单、额外的计算量很小)、高效率(相对于不用外推方法来说,计算量可以节省到几分之一甚至几千分之一)、高精度(采用外推方法可以得到更为精确的近似值)的方法。对于求解实际问题来说,如果能够应用外推方法,都应该采用外推方法来加速收敛。在高等数学、高等工程数学以及物理学中,许多重要的概念都是利用极限来定义的,所以相关的数值计算问题都可以通过外推方法来完成从而提高计算效率,所以外推方法在科学计算领域有着十分重要的意义。

由于外推方法本质上还是插值方法,所以外推方法也可能继承插值方法的一些弱点,虽然从理论上讲在一定条件下外推法产生的点列的收敛性不成问题,但实际上却不能保证计算机输出的结果总是正确的。其中部分原因与具体的外推方法有关,另外也与所涉及的数学问题的条件数有关。对于应用外推法可能出现的问题,从理论上寻找彻底解决的办法似乎不太容易,不过在实际应用中可以采用一些更灵活的处理方法来提高算法的性能。

对于求数值微分这样一类应用来说,由于对步长的选取比较自由,计算 $F(h)$ 的计算量与 h 的值的大小没有关系,所以利用等比序列外推加速是合适的。由于随着 h 的减小,计算 $F(h)$ 所产生的误差会明显增大,所以选用较小的公比值更合适一些。大量的算例表明,取公比 $q=1.6$ 比起 $q=2$ 来说,数值性能明显好一些。

对于后面求数值积分和常微分方程数值解这一类应用来说,计算 $F(h)$ 的计算量通常与 h 的值的大小成反比,所以外推的步数(亦即计算各 $p_{k,m}$ 时 k 的取值)不宜太大,否则,或者由于步长序列迅速趋于零而显著增大计算量,否则由于步长序列趋近于零的速度太慢而影响外推加速本身的性能。在这种情况下,初始步长 h_0 (通常对应某个区间的长度)适当选择得小一些似乎是惟一的办法,这样一来就可以利用等差数列外推加速。

改进外推性能的另一个重要方面是尽可能利用偶函数外推。对比分析公式(4.5)和(4.6)可知,对于任意单调减收敛于零的序列 $\{h_n\}$ 来说,对任意 $k, m > 0$,由于

$$\frac{(h_k)^2 - (h_{k+m})^2}{(h_k)^2} > \frac{h_k - h_{k+m}}{h_k} \quad (5.6)$$

所以,对任意相同的步长序列,对于避免因减法运算而造成有效数字损失来说,利用(4.6)式外推总比利用(4.5)式外推的效果较好^①。

改进外推性能还有一个容易被忽视的问题是如何充分利用计算机进行浮点数运算的特殊性。对于一般的数值计算问题来说是无需考虑这个问题的。但是由于外推方法对于加速序列的收敛速度的效果非常显著,所以一些细节的考虑也能在适当的时候产生很好的效果。以利用等比序列外推为例,如果初值 h_0 和公比 q 都选用可化为 $10^b \times 2^n$ 形式的数,那么无论是计算各 $F(h_k)$ 还是计算各 $p_{m,k}$,精度会有显著提高,从而可以改进外推加速的性能。对于利用整数序列外推加速来说,序列中的数都可以选用 $K \times 2^n$ 形式的整数,其中 K 为小于8或16的整数,这样的数化为二进制数后最多只有最高3位或4位不为零,在进行相关的运算时也能得到额外的好处。

11.6 二阶导数的计算方法

对于求某个已知函数 $f(x)$ 在 $x=x_0$ 处的二阶导数 $f''(x_0)$ 的数值解来说,我们也可以采用求一阶导数的数值解方法那样,构造一个关于 h 的偶函数 $F(h)$,而且当 h 为充分小的正数时,利用 $F(h)$ 作为 $f''(x_0)$ 的近似值,其误差大体上与 h^2 成正比。再接下来即可利用外推加速方法求 $f''(x_0)$ 的具有高精度的数值解。所以,利用前面已经得到的方法以及相应的源程序,剩下的工作已经很少了。

11.6.1 近似公式的推导

假如 $f(x)$ 在 $x=x_0$ 处四阶连续可导,把 $f(x_0+h)$ 和 $f(x_0-h)$ 同时在 $h=0$ 处四阶泰勒展开,得

$$\begin{aligned} f(x_0+h) &= f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 \\ &\quad + \frac{1}{3!}f^{(3)}(x_0)h^3 + \frac{1}{4!}f^{(4)}(x_0)h^4 + o(h^4) \end{aligned} \quad (6.1)$$

$$\begin{aligned} f(x_0-h) &= f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{3!}f^{(3)}(x_0)h^3 \\ &\quad + \frac{1}{4!}f^{(4)}(x_0)h^4 + o(h^4) \end{aligned} \quad (6.2)$$

其中 $o(h^4)$ 表示余项是比 h^4 高级的无穷小量。

^① 假设 $a > b > 0$,我们可以用 $(a-b)/a$ 来刻画 a 与 b “接近”的程度。若 $(a-b)/a$ 的值较大,则计算 $a-b$ 的精度就会高一些;反之,则会低一些。所以由上面的(5.6)式可以得出这里的结论。

把上面的(6.1)式和(6.2)式两边相加,整理后即得

$$\frac{f(x_0+h) - 2f(x_0) + f(x_0-h))}{h^2} = f''(x_0) + \frac{2}{4!}f^{(4)}(x_0)h^2 + o(h^2) \quad (6.3)$$

从而得到具有二阶解精度的近似公式

$$f''(x_0) \approx \frac{f(x_0+h) - 2f(x_0) + f(x_0-h))}{h^2} \quad (6.4)$$

在上面的(6.4)式中,把 x_0 看成固定的常数,即可得到关于步长 h 的函数

$$F(h) = \frac{f(x_0+h) - 2f(x_0) + f(x_0-h))}{h^2} \quad (6.5)$$

对于具体给定的 $f(x)$ 来说,只要给出一个步长的初值 h_0 ,即可利用外推方法求 $F(h)$ 当 h 趋近于零时的极限值。

11.6.2 理查逊外推法求二阶导数举例

例 6.1 试用外推法求 $\ln x$ 在 $x_0=1$ 处的二阶导数值,给定初始步长为 $h_0=0.32$ 。

解:可以简单地把程序 11.03 复制一份,改写其中的变量 x_0 和 h_0 的设置,利用(6.4)式重写函数 $FT0()$,相应的源代码见程序 11.08,为节省篇幅,只列出改写部分,具体的文字编辑工作读者可以自己完成。

运行程序 11.09 所显示的结果如表 6.1 所示。从计算结果可以看出如果不采用外推法计算,迭代 6 次所得到的结果只有 4 位有效数字,外推 3 次所得到的结果却有 13 位有效数字,加速的效果非常显著。

程序 11.09 理查逊外推法求 $\ln x$ 在 $x_0=1$ 处的二阶导数值

```

:
double x0=1.0;
double h0=0.32;
double FT0(double x,double h)
{ return (log(x+h)+log(x-h)-2.0*log(x))/h/h;
}
:

```

表 6.1 运行程序 11.09 所得的结果

k	H[k]	T0[k]	T1[k]	T2[k]	T3[k]
0	3.200e-01	-1.0549877365	-1.0549877365	-1.05498773646197	-1.05498773646197
1	1.600e-01	-1.0130227354	-0.9990344017	-0.99530417941076	-0.99435682138233
2	8.000e-02	-1.0032137192	-0.9999440471	-1.00000469016626	-1.00007930144809
3	4.000e-02	-1.000008544	-0.9999965661	-1.00000006733846	-0.99999999396024
4	2.000e-02	-1.0002000533	-0.999997863	-1.00000000103092	-0.9999999997842
5	1.000e-02	-1.0000500033	-0.999999887	-1.00000000001603	-0.9999999999992
ANS =		-0.99999999999919			
ERR =		2.150E-11			

11.6.3 利用一般的等比数列外推加速

对于求解例 6.1 来说,可以把程序 11.05B 复制一份,改写其中 x_0 、 h_0 的设置,按

(6.5)式重写函数 $F()$, 相应的源代码见程序 11.10, 这里只列出改写部分。

令比值 q 取 2.0, 允许误差 EPS 分别取 $0.4\text{E}-11$ 、 $0.4\text{E}-13$ 和 $0.4\text{E}-15$, 运行程序 11.10, 所显示的结果如表 6.2 所示。

由表 6.2 可以看出, 对于适当的误差控制来说, 理查逊外推方法 (相当于 $q=2$) 也能够得到高精度的解, 但是, 如果误差控制得太小, 则由于递推的步数增多导致计算 $F(h)$ 产生较大的误差而出现“假收敛的现象”, 图中当 EPS 取 $0.4\text{E}-13$ 和 $0.4\text{E}-15$ 时所得到的结果都是不准确的。

表 6.2 运行程序 11.10 所显示的结果

q=2.0000	EPS= 4.0E-12
K= 8	M= 2
P[K-1][M]=	-9.99999999979785e-01
P[K][M]=	-1.000000000000057e+00
q=2.0000	EPS= 4.0E-14
K=15	M= 1
P[K-1][M]=	-9.99999999981753e-01
P[K][M]=	-9.99999999981753e-01
q=2.0000	EPS= 4.0E-15
K=24	M= 0
P[K-1][M]=	-1.000000001862844e+00
P[K][M]=	-1.000000001862844e+00

令比值 q 取 1.6, 允许误差 EPS 还是分别取 $0.4\text{E}-11$ 、 $0.4\text{E}-13$ 和 $0.4\text{E}-15$, 运行程序 11.10, 所显示的结果如表 6.3 所示。

表 6.3 再运行程序 11.10 所显示的结果

q=1.6000	EPS= 4.0E-12
K= 6	M= 4
P[K-1][M]=	-1.000000000001494e+00
P[K][M]=	-1.00000000000014e+00
q=1.6000	EPS= 4.0E-14
K= 7	M= 4
P[K-1][M]=	-1.00000000000014e+00
P[K][M]=	-1.00000000000001e+00
q=1.6000	EPS= 4.0E-15
K= 8	M= 4
P[K-1][M]=	-1.00000000000001e+00
P[K][M]=	-1.00000000000001e+00

由表 6.3 可以看出, 对于利用等比序列外推加速来说, 适当减小比例常数对于提高外推加速的数值性能还是非常明显的。

程序 11.10

```

:
double x0=4.0;
double h0=0.32;
double F(double h)
{ return (sqrt(x0+h)-sqrt(x0-h))/2.0/h;
}
:

```

为了测试 q 取不同的浮点数对外推加速性能的影响, 适当修改程序 11.05B 以利用程序 11.08 求解例 6.1, 取 $\text{EPS}=0.4\text{E}-15$, q 分别取 1.599 7 和 1.6, 运行程序所显示的结果如表 6.4 所示。由表 6.4 不难看出, 本来这两者不应该有太大的差别, 但 $q=1.6$ 所得到的结果却多出 2 位有效数字, 原因应该是取 $q=1.6$ 充分利用了计算机做浮点数运算的特殊性。

表 6.4 利用程序 11.08 求解例 6.1 的运行结果

```

q=1.5997 EPS= 4.0E-16
K= 7 M= 6
P[K-1][M]= -1.000000000001154e+00
P[K][M] = -1.00000000000104e+00
P[K][M+1]= -1.00000000000103e+00
Bad Problem !

q=1.6000 EPS= 4.0E-16
K= 9 M= 5
P[K-1][M]= -1.00000000000001e+00
P[K][M] = -1.00000000000001e+00
P[K][M+1]= -1.00000000000001e+00

```

练 习 题

1. 试利用计算器采用理查逊外推法(外推 3 次)列表计算 $\ln x$ 在 $x_0=2$ 处的导数,取 $h_0=0.32$ 。要求得到具有 8 位有效数字的近似解,并说明比起不采用外推法来说,增加了多少位有效数字。

2. 分别利用程序 11.01 和 11.02 求 $\sin x$ 在 $x_0=\pi/3$ 处的导数值,问各自所能得到的最精确的解是多少,各有几位有效数字?

3. 利用 $\sin(x)/x$ 是偶函数的特点,试利用不同比值的等比序列的外推加速方法求 $\sin(h)/h$ 当 h 趋近于 0 的极限值。初始步长 h_0 可适当选取。问最好的情况下可以得到多少位有效数字的近似值。

4. 完成利用等比序列按(5.1)式外推加速的程序的编写,并利用不同的比值构造等比序列外推 $[\ln(1+h)]/h$ 当 h 趋近于 0 的极限值,并指出最好的结果有几位有效数字。

5. 假设 `double UserDefine(double x){...}` 是计算某个连续可微函数的函数值的 C 语言函数,试利用 C 语言的函数指针作为形式参数编写计算 `UserDefine(x)` 在 x_0 的条件数的 C 语言函数

```
double GetCondValue(double * f(double), double x){...}
```

设 $f(x)$ 是 $[a, b]$ 上连续可积的实函数, 求 $f(x)$ 在 $[a, b]$ 上的数值积分也就是求 $f(x)$ 在 $[a, b]$ 上的定积分的数值解。对于这个问题来说, 即使能找到 $f(x)$ 的一个原函数 $F(x)$ 的解析形式, 并利用牛顿-莱布尼兹 (Newton-Leibniz) 公式进行计算, 在许多情况下这样做也是非常麻烦的。问题的关键在于, 初等函数的原函数不一定还是初等函数, 在这种情况下并不能利用牛顿-莱布尼兹公式进行计算。所以直接研究一般形式的可积函数在给定区间上的数值积分就显得尤为重要。

12.1 求积公式与代数精度的概念

由定积分的定义可知, 连续函数 $f(x)$ 在区间 $[a, b]$ 上的定积分近似值可以表示为 $[a, b]$ 上的一些点 $x_0, x_1, \dots, x_n$ 处的函数值 $f(x_0), f(x_1), \dots, f(x_n)$ 的线性组合或加权和, 即

$$\int_a^b f(x) dx \approx \sum_{k=0}^n w_k f(x_k) \quad (1.1)$$

其中 $w_0, w_1, \dots, w_n$ 仅与 $x_0, x_1, \dots, x_n$ 的选取有关而与被积函数 $f(x)$ 无关。我们把这样的公式称为求积公式, 也称为机械求积公式。其意义在于, 只要我们对一般形式的数值积分问题找到了确定各 x_k 和 w_k 方法, 即可按机械操作的方式利用 (1.1) 式求数值积分。

12.1.1 术语和记号

为了计算 $f(x)$ 在区间 $[a, b]$ 上的定积分近似值, 我们通常的做法是首先按照某种规则给出适当的 n 值, 把积分区间 $[a, b]$ 划分为 n 等份, 记

$$h = (b-a)/n, \quad x_0 = a, \quad x_n = b, \quad x_k = a + kh, \quad k = 0, 1, 2, \dots, n$$

称 h 为步长, $x_0, x_1, \dots, x_n$ 为 $[a, b]$ 的一个等份分划。

假如 $x_0, x_1, \dots, x_n$ 为 $[a, b]$ 的一个等份分划, 那么求积公式 (1) 中的 w_0 ,

$w_1, \dots, w_n$ 的选取仅仅只与 n 有关, 从而可以简化对求积公式的研究。

12.1.2 求积公式的性质

微积分学中给出过关于定积分的一条重要性质, 即定积分保持函数的线性关系不变。它的含义是, 若 $f_1(x), f_2(x)$ 都是 $[a, b]$ 上的可积函数, 则对任意实数 c_1, c_2 , 我们有 $c_1 f_1(x) + c_2 f_2(x)$ 也是 $[a, b]$ 上的可积函数, 而且还有

$$\int_a^b [c_1 f_1(x) + c_2 f_2(x)] dx = c_1 \int_a^b f_1(x) dx + c_2 \int_a^b f_2(x) dx$$

不难验证, 求积公式也保持函数的线性关系不变, 即

$$\sum_{k=0}^n w_k [c_1 f_1(x_k) + c_2 f_2(x_k)] = c_1 \sum_{k=0}^n w_k f_1(x_k) + c_2 \sum_{k=0}^n w_k f_2(x_k) \quad (1.2)$$

12.1.3 几种常见的求积公式

在后面的讨论中, 我们将经常用到下面一些非常简单的求积公式, 它们是中矩公式、梯形公式和辛普森(Simpson)公式:

$$(1) \text{ 中矩公式 } \int_a^b f(x) dx \approx (b-a) f\left(\frac{a+b}{2}\right)$$

$$(2) \text{ 梯形公式 } \int_a^b f(x) dx \approx (b-a) \left[\frac{1}{2} f(a) + \frac{1}{2} f(b) \right]$$

$$(3) \text{ 辛普森公式 } \int_a^b f(x) dx \approx (b-a) \left[\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right]$$

中矩公式是我们特别强调的一个重要公式, 与求积公式(1.1)对比分析后, 可以认为它是这样一种机制: 把积分区间分为 2 等分, 取 $w_0 = w_2 = 0, w_1 = 1$ 所形成的求积公式。从几何上看, 它实际上是取区间中点的函数值与区间长的积作为定积分值, 类似于用中位线乘以高来计算梯形的面积。

12.1.4 截断误差与代数精度

在求积公式中, 我们使用的是近似等号, 这是因为, 对于一般的被积函数来说, 利用这些公式计算所得的结果除了舍入误差外, 还有截断误差。

有时为了进行误差分析, 我们可以把上面的(1.1)式写成

$$\int_a^b f(x) dx = \sum_{k=0}^n w_k f(x_k) + R[f]$$

其中 $R[f]$ 表示的就是截断误差。

对于求数值积分来说, 利用截断误差的概念进行误差分析比起数值微分来说要麻烦一些, 所以通常采用另一个概念, 代数精度的概念, 来评价求积公式的误差性能。

考察前面给出的三个求积公式, 如果被积函数是线性函数, 那么利用中矩公式或梯形公式所得到的结果就是准确值, 否则一般不是。读者还可以进一步验证, 对于一般的次数不超过 2 的多项式函数来说, 辛普森公式所得到的结果也是准确值。所以, 我们可以把一个求积公式到底可以对几次多项式能得到准确值作为求积公式的一个评价指标。

定义 1.1 一个求积公式

$$\int_a^b f(x) dx \approx \sum_{k=0}^n w_k f(x_k)$$

如果对所有的次数不超过 m 的多项式严格相等, 而对某些 $m+1$ 次多项式不相等, 则称该公式具有代数精度 m , 或者说该公式的代数精度为 m 。

利用上面的(1.2)式给出的求积公式的线性性, 我们不难证明下面的结论。

定理 1.1 如果求积公式对 $1, x, \dots, x^m$ 严格相等, 而对 x^{m+1} 不相等, 则该公式的代数精度为 m 。

我们可以方便地利用上面的定理 1.1 来计算一个求积公式的代数精度。

例 1.1 试证明辛普森公式的代数精度是 3。

证明: 当 $f(x)=1$ 时, 显然辛普森公式严格相等。

当 $f(x)=x$ 时

$$\int_a^b x dx = \frac{1}{2} x^2 \Big|_{x=a}^{x=b} = \frac{1}{2} (b^2 - a^2)$$

而

$$\begin{aligned} & (b-a) \left[\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right] \\ &= (b-a) \left[\frac{1}{6} a + \frac{4}{6} \cdot \frac{a+b}{2} + \frac{1}{6} b \right] \\ &= \frac{1}{2} (b-a) \left(\frac{2}{6} a + \frac{4}{6} a + \frac{4}{6} b + \frac{2}{6} b \right) \\ &= \frac{1}{2} (b^2 - a^2) \end{aligned}$$

所以当 $f(x)=x$ 时, 辛普森公式也严格相等。

当 $f(x)=x^2$ 时

$$\int_a^b x^2 dx = \frac{1}{3} x^3 \Big|_{x=a}^{x=b} = \frac{1}{3} (b^3 - a^3)$$

而

$$\begin{aligned} & (b-a) \left[\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right] \\ &= (b-a) \left[\frac{1}{6} a^2 + \frac{4}{6} \left(\frac{a+b}{2}\right)^2 + \frac{1}{6} b^2 \right] \\ &= \frac{1}{3} (b-a) \left[\frac{1}{2} a^2 + \frac{1}{2} (a+b)^2 + \frac{1}{2} b^2 \right] \\ &= \frac{1}{3} (b^3 - a^3) \end{aligned}$$

所以当 $f(x)=x^2$ 时, 辛普森公式也严格相等。

当 $f(x)=x^3$ 时

$$\int_a^b x^3 dx = \frac{1}{4} x^4 \Big|_{x=a}^{x=b} = \frac{1}{4} (b^4 - a^4)$$

而

$$\begin{aligned}
 & (b-a)\left[\frac{1}{6}f(a)+\frac{4}{6}f\left(\frac{a+b}{2}\right)+\frac{1}{6}f(b)\right] \\
 &= (b-a)\left[\frac{1}{6}a^3+\frac{4}{6}\left(\frac{a+b}{2}\right)^3+\frac{1}{6}b^3\right] \\
 &= \frac{1}{4}(b-a)\left[\frac{2}{3}a^3+\frac{1}{3}(a+b)^3+\frac{2}{3}b^3\right] \\
 &= \frac{1}{4}(b^4-a^4)
 \end{aligned}$$

所以当 $f(x)=x^3$ 时, 辛普森公式也严格相等。

不难验证, 当 $f(x)=x^4$ 时, 辛普森公式并不严格相等。所以辛普森公式的代数精度是 3。证毕。

读者不难验证, 中矩公式和梯形公式的代数精度都是 1。

值得一提的是, 代数精度的概念对于研究求积公式来说虽然有很重要的意义, 但也不应该盲目地单纯追求具有较高代数精度的求积公式。以前面提到的中矩公式、梯形公式和辛普森公式为例, 中矩公式和梯形公式的代数精度虽然都是 1, 但中矩公式只计算一个点的函数值, 而梯形公式要计算两个点处的函数值, 所以中矩公式优于梯形公式。与梯形公式相比, 辛普森公式只多计算一个点的函数值, 但代数精度却增加到 3, 显然辛普森公式更为优越。

12.2 牛顿-柯特斯(Newton-Cotes)求积公式

12.2.1 利用插值多项式导出求积公式

设 $f(x)$ 为被积函数, $[a, b]$ 为积分区间, $x_0, x_1, \dots, x_n$ 为 $[a, b]$ 内的 $n+1$ 个互异的点, 记 $L_n(x)$ 为相应的拉格朗日插值多项式, $R_n(x)$ 为相应的插值余项, 那么我们有

$$f(x) = L_n(x) + R_n(x) \quad (2.1)$$

将上面的(2.1)式两边同时积分得

$$\int_a^b f(x) dx = \int_a^b L_n(x) dx + \int_a^b R_n(x) dx$$

如果我们取

$$\int_a^b f(x) dx \approx \int_a^b L_n(x) dx \quad (2.2)$$

那么截断误差为

$$\int_a^b R_n(x) dx$$

利用第 10 章(3.2)式给出的拉格朗日插值公式, 我们有

$$L_n(x) = \sum_{k=0}^n f(x_k) LB_k(x)$$

其中 $LB_k(x), k=0, 1, \dots, n$, 是第 10 章(3.1)式给出的基函数。从而上面的(2.2)式可改写为

$$\int_a^b f(x) dx \approx \int_a^b \sum_{k=0}^n f(x_k) LB_k(x) dx \quad (2.3)$$

由于各 $f(x_k)$ 都是常数、 $LB_k(x)$ 是多项式函数, 所以从理论上讲, 上面(2.3)式的右边是可以计算的。记

$$w_k = \int_a^b LB_k(x) dx \quad (2.4)$$

由上面的(2.3)式即得

$$\int_a^b f(x) dx \approx \sum_{k=0}^n w_k f(x_k) \quad (2.5)$$

从而得到一种求数值积分的方法。上面的(2.4)式和(2.5)式被称为插值公式导出的求积公式。

回顾第10章关于多项式插值的结论可知, 由于任意次数不超过 n 的多项式与它的任意 $n+1$ 个基点的插值多项式恒等, 再由求积公式的代数精度的定义, 我们立即得到: 由 $n+1$ 个基点的拉格朗日插值多项式所形成的求积公式的代数精度至少是 n 。

为了强调上面的(2.5)式是利用 $n+1$ 个基点的插值多项式导出的求积公式, 我们把公式中的各 w_k 改写为 $w(n, k)$, 从而有

$$w(n, k) = \int_a^b LB_k(x) dx, \quad k = 0, 1, \dots, n \quad (2.6)$$

$$\int_a^b f(x) dx \approx \sum_{k=0}^n w(n, k) f(x_k) \quad (2.7)$$

12.2.2 牛顿-柯特斯求积公式的推导

虽然从理论上讲, 可以利用上面的(2.6)式和(2.7)式完成 $f(x)$ 在区间 $[a, b]$ 上的数值积分的近似计算, 但是利用(2.6)式计算各 $w(n, k)$ 仍然是一件非常麻烦的事情。牛顿-柯特斯求积公式的目的就是利用 $x_0, x_1, \dots, x_n$ 是区间 $[a, b]$ 的一个等分分划的特点进一步简化(2.6)式中的各 $w(n, k)$ 的计算。

假如我们将积分区间 $[a, b]$ 划分为 n 等分, 记

$$\left. \begin{aligned} h &= (b-a)/n \\ x_0 &= a \\ x_k &= a + kh, k = 0, 1, \dots, n \\ x &= a + th \end{aligned} \right\} \quad (2.8)$$

那么我们有

$$\left. \begin{aligned} x - x_j &= (t-j)h, j = 0, 1, \dots, n \\ x_k - x_j &= (k-j)h, k, j = 0, 1, \dots, n \\ dx &= hdt \\ x = a &\Rightarrow t = 0 \\ x = b &\Rightarrow t = n \end{aligned} \right\} \quad (2.9)$$

再注意到对 $k=0, 1, \dots, n$, 拉格朗日插值的基函数 $LB_k(x)$ 可表示为

$$LB_k(x) = \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j}$$

所以利用上面(2.8)式和(2.9)式给出的记号约定以及相应的积分变换,我们有

$$w(n, k) = \int_a^b \prod_{\substack{j=0 \\ j \neq k}}^n \frac{x - x_j}{x_k - x_j} dx = \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n \frac{t - j}{k - j} h dt$$

从而可以得到

$$w(n, k) = \frac{h}{\prod_{\substack{j=0 \\ j \neq k}}^n (k - j)} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt \quad (2.10)$$

再注意到

$$\begin{aligned} \prod_{\substack{j=0 \\ j \neq k}}^n (k - j) &= [k(k-1)\cdots 1] \times [(-1)(-2)\cdots (-n+k)] \\ &= (-1)^{n-k} k! (n-k)! \end{aligned}$$

所以由上面的(2.10)式可以得到

$$w(n, k) = \frac{(-1)^{n-k} h}{k! (n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt$$

再利用

$$h = \frac{b-a}{n}$$

立即可以得到

$$w(n, k) = (b-a) \frac{(-1)^{n-k}}{nk! (n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt \quad (2.11)$$

在上面的(2.11)式中,记

$$C(n, k) = \frac{(-1)^{n-k}}{nk! (n-k)!} \int_0^n \prod_{\substack{j=0 \\ j \neq k}}^n (t - j) dt \quad (2.12)$$

则上面的(2.11)式可表示为

$$w(n, k) = (b-a) C(n, k)$$

从而前面的(2.7)式可以表示为

$$\int_a^b f(x) dx = (b-a) \sum_{k=0}^n C(n, k) f(x_k) \quad (2.13)$$

上面的(2.12)式和(2.13)称为牛顿-柯特斯求积公式,利用(2.12)式得到的各 $C(n, k)$ 称为牛顿-柯特斯系数。虽然看上去利用(2.12)式计算牛顿-柯特斯系数也还是有些麻烦,但是与前面的(2.4)式对比后可知,各 $C(n, k)$ 与任何具体问题无关,可以把它们“一次性地”计算出来,并制成常数表。这样,人们就可以通过查表得到这些值,从而可以免去这一部分的计算。这样一来,上面的(2.13)式就真的成为机械求积公式了。

虽然在现代计算机条件下牛顿-柯特斯求积公式以及牛顿-柯特斯系数表的作用已经非常有限,但是牛顿-柯特斯求积公式所体现出来的独具创造性的思维方式和大胆的探

索精神为科学计算领域中的研究树立了光辉的榜样。

12.2.3 牛顿-柯特斯求积公式的注释

在数值积分的实际应用中,通常只是利用低阶的牛顿-柯特斯求积公式进行计算。为了提高精度,可以把积分区间划分为若干小区间。利用第10章中的拉格朗日插值公式的余项估计公式也不难定性地看出,在一个充分短的区间上利用低阶插值多项式进行近似计算,照样能得到非常精确的结果,而且无需采用任何预防措施也可以避免龙贝格现象的发生。在低阶的牛顿-柯特斯求积公式中,常用的几个公式是:梯形公式、辛普森公式和柯特斯公式,下面再对它们作一些简单介绍。读者需要注意的是,利用(2.12)式进行具体计算时,0!的值约定为1。

(1) 梯形公式

在牛顿-柯特斯求积公式中,如果取 $n=1$,那么 k 可以取 0 和 1。由此所形成的求积公式就是梯形公式。由(2.10)式可以得到

$$\begin{aligned} C(1,0) &= \frac{(-1)^1}{1 \times 0! \times 1!} \int_0^1 (t-1) dt = - \int_0^1 (t-1) dt = \frac{1}{2} \\ C(1,1) &= \frac{(-1)^0}{1 \times 1! \times 0!} \int_0^1 (t-0) dt = \int_0^1 t dt = \frac{1}{2} \end{aligned}$$

从而有

$$\int_a^b f(x) dx = (b-a) \left[\frac{1}{2} f(a) + \frac{1}{2} f(b) \right] \quad (2.14)$$

由于实际效果相当于计算梯形面积,所以称梯形公式。

(2) 辛普森公式

在牛顿-柯特斯求积公式中,如果取 $n=2$,那么 k 可以取 0, 1, 2, 由此所形成的求积公式就是辛普森公式。利用(2.10)可以得到

$$\begin{aligned} C(2,0) &= \frac{(-1)^2}{2 \times 0! \times 2!} \int_0^2 (t-1)(t-2) dt = \frac{1}{6} \\ C(2,1) &= \frac{(-1)^1}{2 \times 1! \times 1!} \int_0^1 (t-0)(t-2) dt = \frac{4}{6} \\ C(2,2) &= \frac{(-1)^0}{2 \times 2! \times 0!} \int_0^1 (t-0)(t-1) dt = \frac{1}{6} \end{aligned}$$

所以有

$$\int_a^b f(x) dx = (b-a) \left[\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right] \quad (2.15)$$

(3) 柯特斯公式

在牛顿-柯特斯求积公式中,如果取 $n=4$,那么 k 可以取 0, 1, 2, 3, 4, 由此所形成的求积公式就是柯特斯公式。利用(2.10)式也不难得到

$$\begin{aligned} C(4,0) &= \frac{(-1)^4}{4 \times 0! \times 4!} \int_0^4 (t-1)(t-2)(t-3)(t-4) dt = \frac{7}{90} \\ C(4,1) &= \frac{(-1)^3}{4 \times 1! \times 3!} \int_0^4 t(t-2)(t-3)(t-4) dt = \frac{32}{90} \end{aligned}$$

$$C(4,2) = \frac{(-1)^2}{4 \times 2! \times 2!} \int_0^4 t(t-1)(t-3)(t-4)dt = \frac{12}{90}$$

$$C(4,3) = \frac{(-1)^1}{4 \times 3! \times 1!} \int_0^4 t(t-1)(t-2)(t-4)dt = \frac{32}{90}$$

$$C(4,4) = \frac{(-1)^0}{4 \times 4! \times 0!} \int_0^4 t(t-1)(t-2)(t-3)dt = \frac{7}{90}$$

把所得到的 $C(4,k), k=0,1,\dots,4$ 代入到(2.13)式中,所得到的即为柯特斯公式。

常用的牛顿-柯特斯系数如下面的表 2.1 所示。

表 2.1 常用的牛顿-柯特斯系数表

$C(n,k), k=0,1,\dots,n$							
$n \backslash k$	0	1	2	3	4	5	6
1	1/2	1/2					
2	1/6	4/6	1/6				
3	1/8	3/8	3/8	1/8			
4	7/90	32/90	12/90	32/90	7/90		
5	19/288	75/288	50/288	50/288	75/288	19/288	
6	41/840	216/840	27/280	272/840	27/840	216/840	41/840

一般地,当 n 为偶数时相应的牛顿-柯特斯公式的效果更好一些,另外如果牛顿-柯特斯系数出现负值,可能会导致公式的数值稳定性不好。由于当 $n=8$ 时的牛顿-柯特斯系数出现负值,所以上面列出的 n 取 1~6 的牛顿-柯特斯系数表对于绝大多数的应用来说已经足够了。

还需要说明的是,牛顿-柯特斯求积公式并不能覆盖所有的求积公式。比如中矩公式就不能算是牛顿-柯特斯求积公式,我们在前面已经分析过,它的性能实际上优于梯形公式。在后面将要介绍的龙贝格求积算法中,相应的龙贝格求积公式也不是牛顿-柯特斯求积公式,但它的实际效果也是不错的。

12.3 变步长复化梯形公式方法

在上一节中我们介绍了牛顿-柯特斯公式以及它的一些特点,并且得到了牛顿-柯特斯公式的代数精度,接下来就是如何利用这个公式。

一般说来,在一个较大的积分区间上利用较高阶的牛顿-柯特斯公式虽然可以得到较高的代数精度,但实际效果并不好。主要原因一方面是可能的龙贝格现象使得相应的插值多项式在积分区间内的某些位置产生较大的误差,另外较高阶的牛顿-柯特斯系数由于出现负值而导致相应的求积公式的数值性能不稳定。解决这个问题的思路是把积分区间划分为若干等分,并在每个子区间上利用较低阶的牛顿-柯特斯公式求较小区间上的数值积分,并把这些积分值相加。按照这样的思路所得到的求积公式我们统称为复化型求积公式。数值积分的实际计算表明,利用复化型公式求数值积分是一种理想的方法。

12.3.1 复化中矩公式

对于求可积函数 $f(x)$ 在某个区间 $[a, b]$ 上的数值积分来说, 可以把积分区间 $[a, b]$ 划分为 n 等份, 记 $x_0, x_1, \dots, x_n$ 为等分点, 其中 $x_0 = a, x_n = b$, 记 $[x_{j-1}, x_j]$ 为第 j 个子区间, z_j 为第 j 个子区间的中点, $j = 1, 2, \dots, n$, 记 $h = (b-a)/n$, 记 M_n 为所有子区间上利用中矩公式所求得积分值的和, 那么我们有

$$M_n = \frac{b-a}{n} \sum_{i=1}^n f(z_i) \quad (3.1)$$

为了后面进一步讨论的方便, 我们也可以把积分区间 $[a, b]$ 划分为 $2n$ 等份, 如图 3.1 所示, 记 $y_0, y_1, \dots, y_{2n-1}, y_{2n}$ 为等分点, 其中 $y_0 = a, y_{2n} = b$, 那么我们也可以把下标值为奇数的点 y_{2k-1} , 看成是第 k 个子区间 $[y_{2(k-1)}, y_{2k}]$ 的中点, $k = 1, 2, \dots, n$ 。这样一来, 也可以把复化中矩公式写成

$$M_n = \frac{b-a}{n} \sum_{k=1}^n f(y_{2k-1}) \quad (3.2)$$

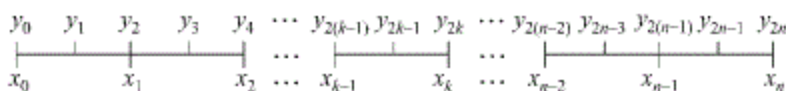


图 3.1 把积分区间划分为 n 等份后各子区间中点位置示意图

上面的(3.1)式和(3.2)式只是表示的形式有所不同, 如果编写数值计算的程序, 由于各 z_i 或各 y_j 在程序中并不需要区分开来, 甚至根本不需要保存, 所以无论利用哪个公式计算, 计算 M_n 的程序都是相同的。

在实际应用中, n 通常取为 2 的某个整数幂 2^k 。作为带有学习、研究性质的程序来说, 可以说明一个适当长度的数组 M 来存放计算结果, 约定 $M[k]$ 存放 $n = 2^k$ 时利用(3.1)式或(3.2)式计算所得到的 M_n 的值, 计算 $M[k]$ 的 C 语言函数可以说明为

```
double GetMK(double a, double b, int k)
```

其中 a, b 分别为积分区间的左右端点, k 表示将积分区间划分为 2^k 等份。

以计算 $f(x) = 4.0/(1+x^2)$ 在区间 $[0, 1]$ 上的数值积分为例, 相应的源代码见程序 12.01A。程序的调试工作将与程序 12.01B 合并在一起进行。

程序 12.01A 利用复化型中矩公式求数值积分

```
double f(double x) {return 4.0/(1.0+x*x);}
double GetMK(double a, double b, int k)
{ double x=0, y=0, h=0;
  unsigned long n=1, j=0;
  for(j=0; j<k; j++) n*=2;
  h = (b-a)/n;
  x=a+h/2;
  for(j=0; j<n; j++) { y+= f(x); x+=h; }
  return (h*y);
}
```


12.3.2 复化梯形公式

对于求 $f(x)$ 在 $[a, b]$ 上的数值积分来说, 把积分区间 $[a, b]$ 划分为 n 等份, 记 $x_0, x_1, \dots, x_n$ 为等分点, 记 $h = (b-a)/n$, 记 $I_j, j=1, 2, \dots, n$ 为 $f(x)$ 在第 j 个子区间 $[x_{j-1}, x_j]$ 上应用梯形公式所求得积分值, 则有

$$I_j = h \left[\frac{1}{2} f(x_{j-1}) + \frac{1}{2} f(x_j) \right]$$

记

$$T_n = \sum_{j=1}^n I_j$$

则有

$$\begin{aligned} T_n = h & \left[\frac{1}{2} f(x_0) + \frac{1}{2} f(x_1) + \frac{1}{2} f(x_1) + \frac{1}{2} f(x_2) + \dots \right. \\ & \left. + \frac{1}{2} f(x_{n-2}) + \frac{1}{2} f(x_{n-1}) + \frac{1}{2} f(x_{n-1}) + \frac{1}{2} f(x_n) \right] \end{aligned}$$

整理后得

$$T_n = \frac{b-a}{n} \left[\frac{1}{2} f(x_0) + \sum_{j=1}^{n-1} f(x_j) + \frac{1}{2} f(x_n) \right] \quad (3.3)$$

上面的(3.3)式就是我们所需要的复化梯形公式。虽然现在就可以直接编写求 T_n 的计算机程序, 但是我们目前并不急着解决这个问题, 而是希望把误差控制也合并在一起而形成自动调整精度的算法后再来编写相应的程序。

12.3.3 变步长复化梯形公式

假设对某个 n , 我们利用复化梯形公式, 也就是上面的(3.3)式, 得到了 T_n , 如果它不满足精度要求, 可以把每个子区间再对分一次, 这相当于把积分区间划分为 $2n$ 等份。记 $y_0, y_1, y_2, \dots, y_{2(n-1)}, y_{2n-1}, y_{2n}$ 为等分点, 利用(3.3)式计算 T_{2n} , 我们有

$$T_{2n} = \frac{b-a}{2n} \left[\frac{1}{2} f(y_0) + \sum_{j=1}^{2n-1} f(y_j) + \frac{1}{2} f(y_{2n}) \right] \quad (3.4)$$

在上面的(3.4)式中, 我们可以把等式右边方括号内对各 $f(y_j)$ 由 $j=1$ 到 $j=2n-1$ 求和改为按 j 为奇数和偶数分别求和, 并重新选用下标记号, 不难得到

$$\sum_{j=1}^{2n-1} f(y_j) = \sum_{i=1}^{n-1} f(y_{2i}) + \sum_{k=1}^n f(y_{2k-1})$$

再由图 3.1 可以看出, $y_{2i} = x_i, i=0, 1, \dots, n$, 所以上面的(3.4)式可以表示为

$$T_{2n} = \frac{b-a}{2n} \left[\frac{1}{2} f(x_0) + \sum_{i=1}^{n-1} f(x_i) + \frac{1}{2} f(x_n) \right] + \frac{b-a}{2n} \sum_{k=1}^n f(y_{2k-1}) \quad (3.5)$$

再利用上面的(3.1)式和(3.2)式, 即可由上面的(3.5)式得出

$$T_{2n} = \frac{1}{2} (T_n + M_n) \quad (3.6)$$

上面的(3.6)式显然是一个非常优美的公式, 我们干脆把它称为变步长复化梯形公

式,利用它不难形成一个自动调整精度的算法。

12.3.4 变步长复化梯形公式方法

为了方便地利用(3.6)式形成一个算法,我们总是取 n 等于 2 的某个指数幂 2^k ,并把 T_n 记为 $T[k]$,由 $n=2^k$ 可得 $2n=2^{k+1}$,从而可以由(3.6)式得到

$$T[k+1] = (T[k] + M[k])/2, \quad k = 0, 1, \dots \quad (3.7)$$

利用上面的(3.7)式形成一个完整的算法已经很容易了,加之可以利用程序 12.01A 完成各 $M[k]$ 的计算,所以只要得到了 $T[0]$ 的值,就可以反复利用(3.7)式计算 $T[1]$, $T[2]$, $\dots$ 直到某个 $|T[k+1] - T[k]|$ 小于预先给定的误差控制指标 EPS 即可。

例 3.1 利用变步长复化梯形公式法求 $4.0/(1+x^2)$ 在 $[0, 1]$ 上的数值积分,要求有 4 位有效数字。

解:求解例 3.1 的源代码见程序 12.01B,程序中多说明了一个数组是留待下一节进一步改进之用。程序运行所显示的结果如表 3.1 所示。由于正确的结果是科学常数 π ,所以, $M[7]$ 和 $T[7]$ 都有 5 位有效数字,不过进一步利用 $M[7]$ 和 $T[7]$ 还可以得到 $T[8]$ 。

表 3.1 程序 12.01B 的运行结果

k	M[k]	T[k]
0	3.200000000000000	3.000000000000000
1	3.162352941176470	3.100000000000000
2	3.146800518393943	3.131176470588235
3	3.142894729591689	3.138988494491089
4	3.141918174308560	3.140941612041389
5	3.141674033796337	3.141429893174974
6	3.141612998641847	3.141551963485656
7	3.141597739852815	3.141582481063752
ANS =	3.141582481063752	
ERR =	3.052E-05	

利用变步长复化梯形公式法所得到的两个序列 $M[k]$, $T[k]$, $k=0, 1, \dots$, 我们分别称为中点序列和梯形序列,从上面所显示的计算结果看,两个序列的收敛速度大体相同。精确评价结果是利用梯形序列大约可以节省一半的计算量。总的来讲两个序列的收敛速度都不能令人满意,不过由于算法简单,又能自动控制精度,这些都是难得的优点。如果能进一步提高收敛速度,就可以得到理想的方法,这个工作留到下一节完成。

程序 12.01B 变步长复化梯形公式法求解例 2.1

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define PI 3.141592653589793L
#define MN 32
int RN=1;
double M[MN], T[MN], S[MN];
double ANS, ERR, EPS=0.4E-4;
double a=0, b=1;
double f(double x)
{ return 4.0/(1+x*x); }
double GetMK(double a, double b, int K){...
```

```

ShowTable(){...}
int Operation(double a,double b,double EPS)
{ int K=0;
  M[0]=GetMK(a,b,0);
  T[0]=(b-a)*(f(a)+f(b))/2.0;
  RN=1;
  for(K=1;K<MN;K++)
  { M[K]=GetMK(a,b,K);
    T[K]=0.5*(T[K-1]+M[K-1]);
    ERR=fabs(T[K]-T[K-1]);
    if(ERR<EPS) break;
  }
  if(K<MN)K++;
  RN=K;
  ANS=T[KRN-1];
  return RN;
}
main()
{ Operation(a,b,EPS);
  ShowTable();
  getchar();
}

```

12.4 变步长复化辛普森(Simpson)公式方法

变步长复化梯形公式法的核心思想是:假设对于某个 $n=2^k$, 我们计算出相应的 $T[k]$, 如果 $T[k]$ 不满足精度要求, 则进一步把区间划分为 2^{k+1} 等份, 也就是在原来的每个子区间补上中点的函数值, 并利用 $T[k+1]=(T[k]+M[k])/2$ 来计算 $T[k+1]$ 。

现在我们可以这样考虑问题, 利用原来每个子区间的端点函数值 $f(y_{2(k-1)})$ 和 $f(x_{2k})$ 以及后来补算的中点的函数值 $f(y_{2k-1})$ 采用辛普森公式计算第 k 个子区间的积分值, 再把他们相加结果记为 $S[k+1]$, 从而又可以得到一个序列。无论是理论分析还是直观考虑, 这样做的效果应该更好一些, 问题是如何简化我们的计算。

12.4.1 复化辛普森公式

假设 $x_0, x_1, x_2, \dots, x_{2(k-1)}, x_{2k-1}, x_{2k}, \dots, x_{2(n-1)}, x_{2n-1}, x_{2n}$ 把积分区间 $[a, b]$ 划分为 $2n$ 等份, 我们也可以认为是其中的 $x_0, x_2, \dots, x_{2(k-1)}, x_{2k}, \dots, x_{2(n-1)}, x_{2n}$ 把区间 $[a, b]$ 划分为 n 等份, 并且 x_{2k-1} 就是第 k 个子区间 $[x_{2(k-1)}, x_{2k}]$ 的中点。

记 $I_k, k=1, 2, \dots, n$ 为 $f(x)$ 在第 k 个子区间 $[x_{2(k-1)}, x_{2k}]$ 应用辛普森公式所求得的积分值, 则有

$$I_k = \frac{b-a}{6n} [f(x_{2(k-1)}) + 4f(x_{2k-1}) + f(x_{2k})]$$

记

$$S_{2n} = \sum_{k=1}^{n-1} I_k$$

则有

$$S_{2n} = \frac{b-a}{6n} \left[f(x_0) + f(x_{2n}) + 4 \sum_{k=1}^n f(x_{2k-1}) + 2 \sum_{k=1}^{n-1} f(x_{2k}) \right] \quad (4.1)$$

上面的(4.1)式称为复化辛普森公式,虽然也可以直接编写求 S_{2n} 的计算机程序,但是首先不必急着这么做,而是等待着寻找类似于(3.6)式那样的变步长公式,以改进变步长复化梯形公式法的性能。

12.4.2 变步长复化辛普森公式

假设 $x_0, x_1, x_2, \dots, x_{2(k-1)}, x_{2k-1}, x_{2k}, \dots, x_{2(n-1)}, x_{2n-1}, x_{2n}$ 把积分区间 $[a, b]$ 划分为 $2n$ 等份,那么我们可以得到下面 3 个不同的积分值:

- (1) 利用 $x_{2k}, k=0, 1, \dots, n$ 这 $n+1$ 个点处的函数值和复化梯形公式计算出 T_n ;
- (2) 利用 $x_j, j=0, 1, \dots, 2n$ 这 $2n+1$ 个点处的函数值和复化梯形公式计算出 T_{2n} ;
- (3) 利用复化辛普森公式计算出 S_{2n} ;

显然, S_{2n}, T_n, T_{2n} 之间应该存在着一定的关系。

按照(3.3)式和(3.4)式方括号内的表示形式整理(4.1)式的方括号内的数学表达式,不难得到

$$S_{2n} = \frac{b-a}{6n} (\Sigma_1 - \Sigma_2) \quad (4.2)$$

其中

$$\begin{aligned} \Sigma_1 &= 2f(x_0) + 2f(x_{2n}) + 4 \sum_{k=1}^n f(x_{2k-1}) + 4 \sum_{k=1}^{n-1} f(x_{2k}) \\ \Sigma_2 &= f(x_0) + f(x_{2n}) + 2 \sum_{k=1}^{n-1} f(x_{2k}) \end{aligned}$$

上面的 Σ_1 和 Σ_2 还可以进一步整理为

$$\left. \begin{aligned} \Sigma_1 &= 4 \left[\frac{1}{2} f(x_0) + \sum_{j=1}^{2n-1} f(x_j) + \frac{1}{2} f(x_{2n}) \right] \\ \Sigma_2 &= 2 \left[\frac{1}{2} f(x_0) + \sum_{k=1}^{n-1} f(x_{2k}) + \frac{1}{2} f(x_{2n}) \right] \end{aligned} \right\} \quad (4.3)$$

利用前面的(3.3)式和(3.4)式以及上面的(4.3)式不难进一步得到

$$\frac{b-a}{6} \Sigma_1 = \frac{4}{3} T_{2n}, \quad \frac{b-a}{6} \Sigma_2 = \frac{1}{3} T_n \quad (4.4)$$

再由上面的(4.1)式和(4.4)式即可得到

$$S_{2n} = \frac{4T_{2n} - T_n}{3} \quad (4.5)$$

按照我们的习惯记法,取 $n=2^k$,则有 $2n=2^{k+1}$,利用 $S[k+1]$ 表示由(4.5)式所得到的 S_{2n} ,利用上一节得到的梯形序列 $T[0], T[1], \dots$,我们有

$$S[k+1] = (4T[k+1] - T[k])/3 \quad (4.6)$$

所以利用程序 12.01B 所产生的序列 $T[0], T[1], \dots$, 不难进一步利用(4.6)式再产生一个序列式 $S[1], S[2], \dots$, 并称之为辛普森序列。更有意义的是, 辛普森序列相当于利用梯形序列作一次理查逊外推所得到的序列, 所以我们有理由期待更好的结果。在辛普森序列中, $S[0]$ 没有定义, 为了让程序更简洁一些, 可以令 $S[0]=T[0]$ 。

12.4.3 变步长复化辛普森公式加速算法

利用前面得到的程序 12.01B, 接下来编写相应的变步长复化辛普森公式法的程序几乎是水到渠成, 由于程序 12.01B 中已经说明了数组 S , 所以最重要的改进是在 $\text{Operation}()$ 函数中对 $S[0]$ 赋初值; 在循环体内适当位置增加一条按(4.6)式计算的语句; 修改停止规则即可。改写后的 $\text{Operation}()$ 函数见程序 12.02。

程序 12.02 变步长复化辛普森公式加速算法

```
int Operation(double a, double b, double EPS)
{ int K=0;
  M[0]=GetMK(a, b, 0);
  T[0]=(b-a)*(f(a)+f(b))/2.0;
  S[0]=T[0];
  RN=1;
  for(K=1; K<MN; K++)
  { M[K]=GetMK(a, b, K);
    T[K]=0.5*(T[K-1]+M[K-1]);
    S[K]=(4.0*T[K]-T[K-1])/3.0;
    if(K<4)continue;
    ERR=fabs(S[K]-S[K-1]);
    if(ERR<EPS) break;
  }
  if(K<MN)K++;
  RN=K;
  ANS=S[RN-1];
  return RN;
}
```

利用程序 12.02 求解例 2.1 所显示的结果如表 4.1 所示。程序中, 控制精度的变量 EPS 所设定的值为 $0.4\text{E}-12$ 。不难看出, 最后得到的 $S[8]$ 已经有 15 位有效数字。

表 4.1 利用程序 12.02 求解例 2.1 所显示的结果

k	M[k]	T[k]	S[k]
0	3.200000000000000	3.000000000000000	3.000000000000000
1	3.162352941176470	3.100000000000000	3.133333333333333
2	3.146800518393943	3.131176470588235	3.141568627450980
3	3.142894729591689	3.138988494491089	3.141592502458707
4	3.141918174308560	3.140941612041389	3.141592651224822
5	3.141674033796337	3.141429893174974	3.141592653552836
6	3.141612998641847	3.141551963485656	3.141592653589216
7	3.141597739852815	3.141582481063752	3.141592653589783
8	3.141593925155547	3.141590110458283	3.141592653589794
ANS =	3.141592653589794		
ERR =	1.066E-14		

为了比较程序 12.02 与程序 12.01 的性能差别,我们专门在程序 12.01B 中把 EPS 修改为 $0.4\text{E}-12$ 重新运行,得到相应的循环次数为 21,比起程序 12.02 给定相同的精度设值来说,足足多出 13 次,有效数字还少了两位。注意到对于变步长数值积分来说,循环次数每增加 1 次,计算量就翻一番,所以利用复化辛普森公式对梯形序列加速,对于本例来说,计算量降到了八千分之一以下,而且有效数字还多出两位,这就是前面的(4.5)式和(4.6)式所体现的价值。

12.5 龙贝格(Romberg)求积算法

12.5.1 龙贝格求积算法简介

在前面两节中,我们介绍了复化梯形公式,复化中矩形公式,以及复化辛普森公式;利用变步长方法定义了三个序列 $M[k], T[k], S[k], k=0, 1, \dots$;并进一步利用关系式

$$T[k+1] = (T[k] + M[k])/2 \quad (5.1)$$

$$S[k+1] = (4T[k+1] - T[k])/3 \quad (5.2)$$

形成自动调整步长以适应精度要求的一种简单而高效的算法。

假设 $2n+1$ 个等分点 $x_0, x_1, x_2, \dots, x_{2(k-1)}, x_{2k-1}, x_{2k}, \dots, x_{2(n-1)}, x_{2n-1}, x_{2n}$ 把积分区间 $[a, b]$ 划分为 n 个子区间 $[x_{2(k-1)}, x_{2k}], k=1, \dots, n$, 并利用(4.1)式给出的复化辛普森公式计算 S_{2n} 。如果所得到的结果不满足精度要求,我们既可以利用 $4n+1$ 等分点把积分区间 $[a, b]$ 划分为 $2n$ 个子区间而计算 S_{4n} , 也可以利用这 $4n+1$ 等分点把积分区间 $[a, b]$ 划分 n 个子区间,利用每个子区间上已知 5 个点处的函数值和柯特斯公式求相应的数值积分,从而形成类似于(4.1)式的复化柯特斯公式。对于利用复化柯特斯公式来说,同样可以进一步利用变步长方法得到相应的柯特斯序列。我们记 $C[k], k=2, 3, \dots$, 表示利用 2^k 个等分点把积分区间划分为 2^{k-2} 个子区间,在每个子区间上利用柯特斯公式所得到的积分值的总和。有意义的是,我们同样可以得到类似如(5.2)式的结果

$$C[k+1] = (16S[k+1] - S[k])/15 \quad (5.3)$$

由(5.3)式不难看出,我们同样能采用类似于变步长复化辛普森公式的方法得到相应的柯特斯序列 $C[k], k=2, 3, \dots$, 实际效果相当于利用梯形序列作两次理查逊外推所得到的序列。

龙贝格求积算法相当于利用梯形序列采用理查逊外推法逐次外推而得到一系列的数值积分序列。一次外推的结果相当于辛普森序列,两次外推的结果相当于柯特斯序列,三次外推的计算格式为

$$R[k+1] = (64C[k+1] - C[k])/63 \quad (5.4)$$

我们称 $R[k], k=3, 4, \dots$ 为龙贝格序列。

如果我们采用统一记号,记 $T_0(k)$ 为梯形序列,记 $T_m(k)$ 为第 m 次外推所得到的序列,那么有

$$T_m(k) = \frac{4^m T_{m-1}(k) - T_{m-1}(k-1)}{4^m - 1}, \quad m = 1, 2, \dots, k \geq m \quad (5.5)$$

过去,人们用手工方式按上面的(5.5)式计算数值积分,通常按下面的表 5.1 给出的

表格计算。计算顺序也很简单,一行一行地往下计算即可,每一行按从左到右的顺序计算,直到 $|T_{n+1}(n+1)-T_n(n+1)|$ 小于某个预先给定的充分小的正数 EPS。

现在利用计算机求数值积分,通常只需计算到龙贝格序列即可。程序的编写方法与第 11 章的程序 11.03 大体相同,也可通过修改前面的程序 12.02 而直接得到。上面的公式(5.5)和表 5.1 只是作为资料来介绍。

表 5.1 龙贝格求积算法的计算表格

k	$T_0[k]$	$T_1[k]$	$T_2[k]$	...	$T_n[k]$	$T_{n+1}[k]$
0	$T_0(0)$					
1	$T_0(1)$	$T_1(1)$				
2	$T_0(2)$	$T_1(2)$	$T_2(2)$			
$\vdots$	$\vdots$	$\vdots$	$\vdots$			
n	$T_0(n)$	$T_1(n)$	$T_2(n)$	...	$T_n(n)$	
$n+1$	$T_0(n+1)$	$T_1(n+1)$	$T_2(n+1)$	...	$T_n(n+1)$	$T_{n+1}(n+1)$

12.5.2 龙贝格算法的实现方法

有了程序 11.03、11.04、12.01 以及 12.02,读者自己完全能够独立编写龙贝格算法求数值积分的 C 语言程序。作为带有总结性的方法,我们还是给出完整的源程序。程序 12.03 就是结合求解例 2.1 的完整源程序。

我们实际的做法是把梯形序列外推 3 次,即 m 值取 0,1,2,3,并把相应的序列称为梯形序列、辛普森序列、柯特斯序列和龙贝格序列,并相应地利用数组 T 、 S 、 C 、 R 来存放它们。相对于程序 12.01 和 12.02 而言,没有必要显示中点序列。尽管如此,在程序中还是保存这个序列为好,所占用的存储空间不多,但程序读起来显得格外有条理。另外,作为练习,读者也可以直接利用中点序列进行理查逊外推加速,也能得到很好的效果。

值得特别一提的是,数组 T 、 S 、 C 、 R 的长度说明为 32 即可。这是因为 C 语言程序中无符号长整型数的最大值为 $N=2^{32}-1$,其他程序设计语言的设置也大体相同。所以,如果把积分区间划分为 2^k 等份, k 的最大值也就是 31,超过这个值就会出现计算机整数溢出,反而惹出麻烦来了。把数组 T 、 S 、 C 、 R 的长度说明为 32 对于一般的应用来说也足够了,很难想象一个数值积分问题会病态到这种程度;要把积分区间划分为 $2^{31} \approx 20$ 亿等份,并计算这大约 20 亿个点处的函数值。

程序 12.03 适当修改后可以作为应用程序使用,在应用程序中即使保留 5 个序列的中间结果,也只占用 160×8 字节的存储空间,在现代计算机条件下是微不足道的。

程序 12.03 龙贝格算法求解例 2.1

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define PI 3.141592653589793L
#define MN 32
```

```

int      RN=1;
double M[MN],T[MN],S[MN],C[MN],R[MN];
double ANS,ERR,EPS=0.4E-12;

double a=0,b=1;
double f(double x)
{ return 4.0/(1+x*x); }

double GetMK(double a,double b,int K){...}
void      ShowTable(){...}

int Operation(double a,double b,double EPS)
{ double temp;
  int K=0;
  T[0]=(b-a)*(f(a)+f(b))/2.0;
  S[0]=T[0];
  C[0]=T[0];
  R[0]=T[0];
  RN=1;
  for(K=1;K<MN;K++)
  { M[K-1]=GetMK(a,b,K-1);
    T[K]=(T[K-1]+M[K-1])/2.0;
    S[K]=(4.0*T[K]-T[K-1])/3.0;
    C[K]=(16.0*S[K]-S[K-1])/15.0;
    R[K]=(64.0*C[K]-C[K-1])/63.0;
    if(K<4)continue;
    ERR=fabs(R[K]-R[K-1]);
    if(ERR<EPS) break;
  }
  if(K<MN)K++;
  RN=K;
  ANS=R[RN-1];
  return RN;
}

main()
{ Operation(a,b,EPS);
  ShowTable();
  getchar();
}

```

运行程序 12.03 所显示的结果如表 5.2 所示。与程序 12.02 相比,达到相同精度的结果却少循环两次,相当于节省了 3/4 的计算时间。

表 5.2 运行程序 12.03 所显示的结果

k	T[k]	S[k]	C[k]	R[k]
0	3.0000000000	3.000000000000000	3.000000000000000	3.000000000000000
1	3.1000000000	3.133333333333333	3.142222222222222	3.144479717813051
2	3.1311764706	3.141568627450980	3.142117647058823	3.142115987135595
3	3.1389884945	3.141592502458707	3.141594094125888	3.141585783761874
4	3.1409416120	3.141592651224822	3.141592661142563	3.141592638396796
5	3.1414298932	3.141592653552836	3.141592653708037	3.141592653590029
6	3.1415519635	3.141592653589216	3.141592653591641	3.141592653589794
ANS = 3.141592653589794				
ERR = 2.349E-13				

12.6 复化梯形公式的精度分析

我们已经看到,所谓龙贝格求积公式,实际上就是对梯形序列按理查逊外推加速方法外推若干次。

在第 11 章中我们得到,如果用某个近似公式 $FT_0(x_0, h)$ 表示真值 $T(x_0)$, 如果 $FT_0(x_0, h)$ 在 $h=0$ 处的泰勒展式为

$$FT_0(x_0, h) = T(x_0) + A_2 h^2 + A_4 h^4 + \cdots + A_{2m} h^{2m} + O(h^{2(m+1)}) \quad (6.1)$$

那么 $FT_0(x_0, h)$ 作为计算 $T(x_0)$ 的近似公式就具有二阶精度, 而利用理查逊外推法得到的 $FT_k(x_0, h)$ 则具有 $2(k+1)$ 阶精度。

对于数值积分来说, 如果对梯形序列也得到了类似于上面(6.1)式形式的误差计算公式, 那么我们也可以得到复化梯形公式具有二阶精度; 复化辛普森公式具有四阶精度; 复化柯特斯公式具有六阶精度; 而龙贝格序列则具有八阶精度。

假设 $f(x)$ 是适当光滑的函数, 对于求 $f(x)$ 在区间 $[a, b]$ 上的数值积分来说, 我们利用 $x_0, x_1, \cdots, x_n$ 把积分区间划分为 n 等份从而得到区间的长为 $h = (b-a)/n$ 。在第 k 个子区间 $[x_{k-1}, x_k]$ 上, 可以把 $f(x)$ 同时表示为 $f(x_{k-1}+t)$ 和 $f(x_k-t)$, 显然, 无论在何种情况下, 均有 $t \in [0, h]$, 所以有

$$\int_{x_{k-1}}^{x_k} f(x) dx = \int_0^h f(x_{k-1}+t) dt = \int_0^h f(x_k-t) dt \quad (6.2)$$

从而也有

$$\int_{x_{k-1}}^{x_k} f(x) dx = \frac{1}{2} \left[\int_0^h f(x_{k-1}+t) dt + \int_0^h f(x_k-t) dt \right] \quad (6.3)$$

由此可以进一步得到

$$\int_a^b f(x) dx = \frac{1}{2} \sum_{k=1}^n \left[\int_0^h f(x_{k-1}+t) dt + \int_0^h f(x_k-t) dt \right] \quad (6.4)$$

把 $f(x_{k-1}+t)$ 和 $f(x_k-t)$ 都在 $t=0$ 处适当地泰勒展开后得

$$\begin{aligned} f(x_{k-1}+t) &= f(x_{k-1}) + f'(x_{k-1})t + \frac{1}{2}f''(x_{k-1})t^2 + \cdots \\ &\quad + \frac{1}{n!}f^{(n)}(x_{k-1})t^n + \cdots \end{aligned} \quad (6.5)$$

$$\begin{aligned} f(x_k-t) &= f(x_k) - f'(x_k)t + \frac{1}{2}f''(x_k)t^2 + \cdots \\ &\quad + \frac{(-1)^n}{n!}f^{(n)}(x_k)t^n + \cdots \end{aligned} \quad (6.6)$$

把上面的(6.5)式和(6.6)式两边同时在区间 $[0, h]$ 上积分,得

$$\begin{aligned}\int_0^h f(x_{k-1} + t) dt &= f(x_{k-1})h + \frac{1}{2}f'(x_{k-1})h^2 + \frac{1}{3!}f''(x_{k-1})h^3 + \cdots \\ &\quad + \frac{1}{(n+1)!}f^{(n)}(x_{k-1})h^{n+1} + \cdots \\ \int_0^h f(x_k - t) dt &= f(x_k)h - \frac{1}{2}f'(x_k)h^2 + \frac{1}{3!}f''(x_k)h^3 + \cdots \\ &\quad + \frac{(-1)^n}{(n+1)!}f^{(n)}(x_k)h^{n+1} + \cdots\end{aligned}$$

再由(6.4)式即可得到

$$\int_a^b f(x) dx = \frac{b-a}{n} \left[\frac{1}{2}f(x_0) + \sum_{k=1}^{n-1} f(x_k) + \frac{1}{2}f(x_n) \right] + \Sigma_1 + \Sigma_2 \quad (6.7)$$

其中

$$\Sigma_1 = \sum_{k=1}^m \frac{f^{(2k-1)}(x_0) - f^{(2k-1)}(x_n)}{(2k)!} h^{2k} + O(h^m) \quad (6.8)$$

$$\Sigma_2 = \sum_{k=1}^m \left(\sum_{j=1}^n \frac{f^{(2k)}(x_{j-1}) + f^{(2k)}(x_j)}{(2k+1)!} \right) h^{2k+1} \quad (6.9)$$

显然,上面的 Σ_2 可以进一步表示为

$$\Sigma_2 = (b-a) \sum_{k=1}^m \left(\frac{2}{(2k+1)!} \sum_{j=1}^n \frac{f^{(2k)}(x_{j-1}) + f^{(2k)}(x_j)}{2n} \right) h^{2k} \quad (6.10)$$

把上面的(6.6)式和前面的(3.3)式对比即可看出,复化梯形公式的误差为 $\Sigma_1 + \Sigma_2$,再由(6.8)式和(6.10)式即可得出,复化梯形公式具有二阶精度。

在(6.10)式中对于固定的 k 值,虽然 h^{2k} 的系数与 h 有关,但我们不难看出当 h 趋近于零,或者等价地当 n 趋近于无穷大时,对于适当给定的 k 值,(6.10)式中的和式部分

$$\sum_{j=1}^n \frac{f^{(2k)}(x_{j-1}) + f^{(2k)}(x_j)}{2n} \quad (6.11)$$

将收敛于常数,从而得到 h^{2k} 的系数将收敛于常数。再注意到(6.8)式中 h^{2k} 的系数是与 h 无关的常数,所以当 n 充分大时,我们还是近似地有

$$\Sigma_1 + \Sigma_2 \approx \sum_{k=1}^m A_k h^{2k}$$

从而由(6.7)式即可得到

$$\int_a^b f(x) dx \approx \frac{b-a}{n} \left[\frac{1}{2}f(x_0) + \sum_{k=1}^{n-1} f(x_k) + \frac{1}{2}f(x_n) \right] + \sum_{k=1}^m A_k h^{2k} \quad (6.12)$$

大概是由于这个原因,基于理查逊外推加速方法的龙贝格求积算法也有一定的效果。特别是当 $n=2^k$ 适当大时,采用理查逊外推加速方法还是很有效的。再加之理查逊外推加速方法是一种“稳赚而不亏”的方法,所以值得采用,但次数不宜太多。

12.7 关于数值积分方法的注释

利用上面的(6.12)式我们可以得出,如果被积函数 $f(x)$ 适当光滑,基于理查逊外推加速方法的龙贝格求积算法是比较理想的方法,特别是当 $n=2^k$ 适当大时,采用理查逊外

推加速方法的效果还会更好一些。注意到如果在应用程序中采用更为灵活的停机规则,那么理查逊外推加速方法实际上是一种“稳赚而不亏”的方法,所以应该作为首选方法。由于(6.12)式是近似推导出来的,而且近似程度还与具体的被积函数有关,所以外推的次数不宜太多,利用梯形序列外推到龙贝格序列即可。

作为龙贝格求积算法的具体应用,如果在小的区间上作数值积分,显然上面的(6.11)式将收敛得更快,这意味着采用龙贝格求积算法的实际效果会更好一些。所以,如果实际问题所给定的积分区间比较大,可以把区间划分为一些小的区间,用龙贝格求积方法求出每个小区间上的积分值,然后相加。由于龙贝格求积方法对步长的选择具有自适应性,所以总的计算量有可能显著减少。

求数值积分的另一个办法是将其处理为常微分方程的初值问题,从而可以利用求常微分方程的数值解的方法来求解。具体说来,如果求某个连续函数 $f(x)$ 在某区间 $[a, b]$ 上的数值积分,它的等价问题是求下述初值问题

$$\begin{cases} y'(x) = f(x) \\ y(a) = 0 \end{cases}$$

的解 $y(x)$ 在 $x=b$ 处的函数值 $y(b)$ 的近似值。

在下面的第 13 章中,我们将系统介绍求常微分方程的数值解的各种方法,它们都可以直接应用于求相应的数值积分,其中更具有实用价值的方法是自适应步长的龙格-库塔方法和大步长问题的外推加速方法。利用求常微分方程的数值解的方法求数值积分正好能够弥补龙贝格求积算法在灵活性方面的不足,不过由于这些方法一般不能利用(3.6)式来节省计算量,所以计算效率通常比龙贝格求积算法低。龙贝格求积算法的高效性和常微分方程的数值解方法的灵活性相辅相成,使我们求数值积分的方法也达到完善的地步。

数值积分的另一类方法为高斯(Gauss)型求积公式,其基本思路是首先把积分区间统一变换到区间 $[-1, 1]$ 上,从而有利于寻找统一的公式;接下来通过寻找插值基点连同相应的组合系数使得利用 n 个基点的插值多项式可以得到具有 $2n$ 阶精度的求积公式。虽然从代数精度的意义上讲,对于相同的插值基点而言,高斯型求积公式优于牛顿-柯特斯求积公式,但是高阶的高斯型求积公式同样不具实用意义。对于复化型高斯求积公式来说,实际上已经失去了(3.6)式、(4.5)式和(5.5)式这样一些美好的结果。如果我们也进一步利用低阶的复化型高斯求积公式产生一个序列,然后利用外推方法加速收敛,那么相应的方法已经退化得非常简单了:一个基点的(也就是二阶的)高斯求积公式实际上就是我们前面介绍的中距公式,所以在龙贝格求积算法中直接利用复化中距公式产生的序列替代梯形序列即可,我们把这项工作作为练习留给读者完成。

练 习 题

1. 对于下面的定积分问题,首先用解析方法找到具有 15 位有效数字的近似解(具体的数值计算还需由计算机完成),再用龙贝格求积算法计算到 $R[12]$,指出相应的 $T[12]$ 、 $S[12]$ 、 $C[12]$ 以及 $R[12]$ 各有几位有效数字。

$$(1) \int_1^2 \frac{1}{x} dx \quad (2) \int_0^1 e^{\sqrt{x}} dx \quad (3) \int_0^1 x \sqrt{1+x^2} dx$$

$$(4) \int_0^{\pi/4} \sin x dx \quad (5) \int_1^2 \ln x dx \quad (6) \int_0^{\pi/4} e^{\sin x} \cos x dx$$

2. 对于下面的定积分问题,用龙贝格求积算法计算到 $R[12]$,指出相应的 $T[12]$ 、 $S[12]$ 、 $C[12]$ 以及 $R[12]$ 各有几位有效数字。

$$(1) \int_0^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} dx \quad (2) \int_0^{\pi/2} \frac{\sin x}{x} dx$$

提示:对于第(2)小题,由于

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1$$

所以有

$$\int_0^{\pi/2} \frac{\sin x}{x} dx = \epsilon + \int_{\epsilon}^{\pi/2} \frac{\sin x}{x} dx$$

3. 用牛顿法解下面关于 x 的非线性方程,其中常数 a 可以在 $0.1 \sim 0.4$ 之间任意选取,要求至少得到具有 10 位有效数字的近似解。

$$\int_0^x \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt = a$$

4. 对于复化型数值积分方法来说,也可以应用理查逊外推加速方法直接利用中点序列 $M[k], k=0, 1, \dots, n$,外推三次而形成一个算法,试完成相应的程序编写,并通过求解例 2.1 与龙贝格求积算法的结果对比分析。

常微分方程的数值解法

第 13 章

在生产实践和科学研究过程中,我们经常会遇到求解常微分方程或常微分方程组的定解问题。在许多情况下,首先找到问题的解析解,然后再进行相关的计算往往非常困难,有时甚至是行不通的。在上一章中,我们已经成功地绕过牛顿-莱布尼兹公式而直接求一般的定积分的数值解。基于同样的理由,对于常微分方程或常微分方程组来说,我们也可以避免求解析解而直接求相应的数值解。

一般的高阶常微分方程都可以通过相应的变量代换而化为一阶常微分方程组来求解。一阶常微分方程组的初值问题求数值解与一阶常微分方程的初值问题求数值解的方法基本相同。所以在这一章中,我们重点讨论一阶常微分方程的初值问题求数值解的方法。

13.1 常微分方程初值问题数值解方法概述

13.1.1 问题的提出

一阶常微分方程初值问题可以表述为寻找定义在某个区间 $[a, b]$ 上的实函数 $y=y(x)$,使得

$$\left. \begin{aligned} y' &= f(x, y) \\ y(x_0) &= y_0 \end{aligned} \right\} \quad (1.1)$$

其中 $f(x, y)$ 为已知函数, x_0, y_0 都是已知常数。由于通常假定 $x_0=a$,所以 x_0 也称为初始点,相应的问题也就称为初值问题。

为了避免讨论解的存在性和惟一性之类的问题,我们约定 $f(x, y)$ 是适当光滑的函数,而且关于 y 满足李普希兹(Lipshitz)条件,从而保证了解的存在性和惟一性。

13.1.2 问题求解的三个基本步骤

假如 $y=y(x)$ 为(1.1)式的理论解,如果无法求得,或者所得到的结果不

是初等函数,我们可以直接转向求它的数值解,也就是 $y=y(x)$ 在一些离散点 $x_0, x_1, \dots, x_n$ 处的近似值 $y_0, y_1, \dots, y_n$ 。为了将问题适当简化,可以约定把求解区间划分为 n 等份,并记

$$x_0 = a, \quad x_n = b, \quad h = (b-a)/n, \quad x_k = x_0 + kh, \quad k = 0, 1, \dots, n$$

从而得到所需要的离散点。作为灵活的处置方法,也可以事先根据某种要求确定步长 h , 接下来再根据 $x_0 + nh \geq b$ 来确定步数 n 也是合理的,只是在这种情况下并不能保证仍然有 $x_n = b$, 对于一些特殊的应用来说可能需要单独计算 $y(b)$ 的近似值。

所谓求常微分方程的数值解的方法研究,关键问题是建立或寻找一个递推格式

$$y_k = y_{k-1} + hT(x_{k-r}, y_{k-r}; \dots; x_{k-1}, y_{k-1}) \quad (1.2)$$

利用已知的 y_0 和已经得到的 $y_1, y_2, \dots, y_{k-1}$ 进一步计算 y_k , 由此可以进一步计算 $y_{k+1}, y_{k+2}, \dots, y_n$ 。我们把这个计算过程称为按步进方式前进。

综上所述,求常微分方程数值解通常包括三个重要步骤:将问题离散化,建立或寻找一个递推格式,按步进方式计算。

13.1.3 问题求解的基本方法

对于形为(1.2)式的递推格式来说,如果有 $r=1$, 则相应的递推格式为

$$y_k = y_{k-1} + hT(x_{k-1}, y_{k-1}) \quad (1.3)$$

此时把(1.3)式确定的方法称为单步法,也就是每一步计算只需要用到前面一步的计算结果。单步法的优点是可以自行启动,亦即利用已知的 x_0, y_0 计算 y_1 , 接下来利用已知的 x_1, y_1 计算 y_2 , 其余 $y_3, y_4, \dots$, 可由此类推。单步法的计算效率通常比较低,或者精度不够,或者计算量偏大。

在(1.2)式中,如果 $r>1$, 则相应的方法称为多步法,也就是每一步的计算要用到前面好几步(r 步)的计算结果。多步法的优点是计算效率相对说来明显高一些,但多步法不能自行启动,必须首先利用单步法计算出 $y_1, y_2, \dots, y_{r-1}$, 才能启动一个 r 步(y_0 是已知的)的多步法。在口语中,为了启动一个 r 步的多步法,我们把求解 $y_1, y_2, \dots, y_{r-1}$ 称为计算表头。

对于求常微分方程数值解来说,为了使得求解的方法具有更好的数值稳定性,也经常采用下述形式的计算格式

$$y_k = y_{k-1} + hT(x_{k-r}, y_{k-r}; \dots; x_{k-1}, y_{k-1}; x_k, y_k) \quad (1.4)$$

这里,所需要计算的 y_k 相当于高等数学中的隐含数,此时需要利用第5章介绍的非线性方程求解方法利用(1.4)式求出 y_k 。

求常微分方程数值解的递推格式如果具有(1.2)式的形式,则相应的方法称为显式方法;如果具有(1.4)式的形式,则相应的方法称为隐式方法。隐式方法的优点是数值性能更好一些,但缺点是需要解一个非线性方程。

求常微分方程数值解的一种更有效的方法是同时建立形如(1.2)式和(1.4)式的计算格式,首先利用(1.2)式计算出一个初步的 y_k , 把它代入到(1.4)式的右边从而得到更精确的 y_k 。我们把利用(1.2)式得到的 y_k 称为预报值,代入到(1.4)式的右边所得到的结果称为校正值,称(1.2)式和(1.4)式构成一个预报-校正系统。在现代计算机条件下,在

一些特殊的应用场合,也可以把预报值作为不动点迭代法的初值,利用(1.4)式的特殊形式,直接采用带加速技巧的不动点算法也是非常方便的。

13.1.4 术语和记号的约定

为了求(1.1)式在某个区间 $[a, b]$ 上的数值解,我们通常根据精度要求把区间 $[a, b]$ 划分为 n 等份,记 $h = (b - a)/n$,称之为步长;我们总是约定 $x_0 = a, x_k = x_0 + kh, k = 0, 1, \dots, n$,可以认为它们都是精确值。

记 $y = y(x)$ 为问题的解,尽管有时候我们写不出它的表达式来,但 $y(x)$ 总是有定义的;记 $y(x_k)$ 表示当 $x = x_k$ 时 $y(x)$ 的精确值;记 y_k 为我们采用某个算法求得的 $y(x_k)$ 的近似值,记 $f_k = f(x_k, y_k)$ 。

对于上面的记号,我们一般地有 $y_k \neq y(x_k)$,从而还有 $f_k \neq f(x_k, y(x_k))$ 。

假如我们利用某个递推格式 $y_k = y_{k-1} + hT(x_{k-1}, y_{k-1})$ 或其他形式的递推关系来计算 y_k 的值,那么 y_k 与真值 $y(x_k)$ 之间的误差既与步长 h 有关,也与 y_{k-1} 的值有关。

无论是隐式方法还是显式方法,无论是单步法还是多步法,对于一个给定的递推格式来说,为了误差分析的方便,我们把仅与步长 h 有关的误差称为局部截断误差,而把 $|y_k - y(x_k)|$ 称为整体截断误差。简言之,记

$$\begin{aligned} y_k &= y_{k-1} + hT(x_{k-1}, y_{k-1}) \\ \bar{y}_k &= y(x_k) + hT[x_k, y(x_k)] \\ e_k &= |y(x_k) - \bar{y}_k| \\ \epsilon_k &= |y(x_k) - y_k| \end{aligned}$$

则 e_k 为局部截断误差,而 ϵ_k 即为整体截断误差。

13.1.5 精度评价方法

对于一个给定的递推格式而言,利用局部截断误差和整体截断误差的概念,可以进一步定义所给定的递推格式的精度。

记 $\epsilon_1(h), \epsilon_2(h), \dots, \epsilon_n(h)$ 和 $e_1(h), e_2(h), \dots, e_n(h)$ 分别为 $y_1, y_2, \dots, y_n$ 的整体截断误差和局部截断误差,如果有 $\epsilon_n(h) = O(h^p)$,则称相应的数值解方法具有 p 阶精度。如果 $e_n(h) = O(h^{p+1})$,则称相应的数值解法的局部截断误差是 $p+1$ 阶的。

在一般情况下,直接求 $\epsilon_n(h)$ 比较麻烦,而直接求局部截断误差 $e_k(h), k = 1, 2, \dots, n$ 的阶则较为容易,但我们的目的是希望尽量减少整体截断误差。所以,如果能通过局部截断误差来评估整体截断误差,就可以大大简化相应的误差评价方法。实际上,我们有下面一般的结果:

定理 1.1 如果一种方法的局部截断误差是 $p+1$ 阶的,那么这种方法就具有 p 阶精度。

13.1.6 编程的准备工作

前面已经提到过,求常微分方程数值解一般由三个基本步骤:将问题离散化;建立递推格式;按步进方式计算。所以求微分方程的数值解的算法框架都是相同的,不同的方法

的差别只在于所使用的递推形式不同,所以求常微分方程数值解的不同方法却有许多相同的代码,把这些相同的代码写在一个源程序文件中,几乎所有的不同的方法均可利用其中的代码,从而可以减少大量文本编辑的累赘,增加编程的乐趣。从学习常微分方程数值解方法的意义上讲,我们可以对同一个例题采用不同的方法求解,对不同方法的计算结果用统一的格式来显示,更有利于对比分析不同的方法的差异。

常微分方程数值解的主要挑战来自误差的积累,不过现代计算机条件为我们寻找高精度的数值解提供了很好的条件。从研究数值解方法的意义上讲,如果对所讨论的案例能找到理论上的准确解,我们更容易看清所讨论的方法的性能。下面我们通过一个具体例子说明编程前的准备工作。

例 1.1 求初值问题

$$\left. \begin{aligned} y' &= y - 2.0x/y \\ y(0) &= 1 \end{aligned} \right\}$$

在区间 $[0,1]$ 上的数值解,并与理论解

$$y(x) = \sqrt{1+2x}$$

对比指出相应的误差。

解:利用 C 语言程序求数值解的预编译行,相关的函数说明,相关的变量说明见程序 13.01A。程序中

符号常数 MN 表示允许取的离散点的个数 $n+1$ 的最大值;

变量 RN 表示实际取的离散点的个数 $n+1$;

数组 X,Y 分别存放各 $x_k, y_k, k=0,1,\dots,n$;

数组 F 用于存放各 $f(x_k, y_k), k=0,1,\dots,n$;

数组 YX 用于离散点处的精确解的值,亦即以 $y(x_k), k=0,1,\dots,n$;

数组 E 存放 y_k 的整体误差,亦即 $|y_k - y(x_k)|, k=0,1,\dots,n$;

变量 a,b 分别表示求解区间的左右端点;

变量 x_0, y_0 分别表示给定的初始点以及相应的初值;

函数 $y(x)$ 表示问题的精确解;

函数 CheckSolution() 用于计算各 $YX[k], E[k], k=0,\dots,n$;

函数 ShowTable() 和 SaveTable() 的作用是显示计算结果,把计算结果保存为磁盘文件。

可以把程序 13.01A 单独用文件名“ODE_COM.C”保存为磁盘文件,实现不同的方法的应用程序可以简单地通过文件包含而使用其中的代码。

值得特别说明的是,求常微分方程数值解的应用程序本身也需要按列表计算的格式编写,这是因为进一步的应用程序可以利用表中给出的离散点处的函数值作插值多项式从而得到连续的近似解。所以我们特别推荐保留 $y(x)$ 在给定的离散点处的一阶导数值,它们可以专门存放在数组 F 中,这样进一步的应用程序可以利用第 10 章介绍的分段埃尔米特插值方法求问题的连续解。

程序 13.01A 求常微分方程数值解的“公用”代码

```

/* FILE NAME: ODE_COM.C */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 201
int RN=1;
double X[MN],Y[MN],F[MN],YX[MN],E[MN];
double f(double x,double y)
{ return y-2.0*x/y;}
double x0=0.0,y0=1.0;
double a=0.0,b=1.0;
double g(double x)
{ return sqrt(2.0*x+1.0);}

void CheckSolution()
{ int n;
  for(n=0;n<=RN;n++)
  { YX[n]=g(X[n]);
    E[n]=fabs(YX[n]-Y[n]);
  }
  return;
}

void ShowTable()
{ int j0=0,j=0;
  while(1)
  { printf("\n=====");
    printf("=====");
    printf("\n k X[k] F[k] ");
    printf("Y[k] YX[K] E[K] ");
    printf("\n-----");
    printf("-----");
    for(j=j0;j<21+j0;j++)
    { if(j>RN) break;
      printf("\n%3d",j);
      printf("%11.4E%17.10E",X[j],F[j]);
      printf("%17.10E%17.10E",Y[j],YX[j]);
      printf("%11.4E",E[j]);
    }
    printf("\n=====");
    printf("=====");
    if(j>RN)break;
    j0+=20;
  }
}

```

```

        getchar();
    }
    return;
}
int SaveTable(char * fname)
{ FILE * fp;
  int j=0;
  fp=fopen(fname,"w");
  if(!fp){printf("Open File Err !");getchar();return 1;}
  fprintf(fp,"\\n=====\\n");
  fprintf(fp,"=====\\n");
  fprintf(fp,"\\n k X[k] F[k] ");
  fprintf(fp,"Y[k] YX[K] E[K] ");
  fprintf(fp,"\\n-----\\n");
  fprintf(fp,"-----\\n");
  for(j=0;j<=RN;j++)
  { if(j>RN) break;
    fprintf(fp,"\\n%3d",j);
    fprintf(fp,"%11.4E%17.10E",X[j],F[j]);
    fprintf(fp,"%17.10E%17.10E",Y[j],YX[j]);
    fprintf(fp,"%11.4E",E[j]);
  }
  fprintf(fp,"\\n=====\\n");
  fprintf(fp,"=====\\n");
  return 0;
}

```

13.2 欧拉(Euler)方法及其改进形式

欧拉方法是常微分方程初值问题数值解中最简单的方法,由于几何特征明显,有利于我们分析误差的原因,寻找改进的方法,所以总是研究的入门方法。

13.2.1 欧拉折线公式

用 $y(x)$ 在 x_{k-1}, x_k 处的差商近似替代 $y(x)$ 在 x_{k-1} 处的微商 $y'(x_{k-1})$, 得

$$\frac{y(x_k) - y(x_{k-1})}{x_k - x_{k-1}} \approx f(x_{k-1}, y(x_{k-1}))$$

再利用

$$y_k \approx y(x_k), \quad y_{k-1} \approx y(x_{k-1}), \quad x_k - x_{k-1} = h$$

即可得到欧拉折线公式

$$y_k = y_{k-1} + hf(x_{k-1}, y_{k-1}) \quad (2.1)$$

把 $y(x_k)$ 在 $x=x_{k-1}$ 处泰勒展开,得

$$y(x_k) = y(x_{k-1}) + f[x_{k-1}, y(x_{k-1})]h + \frac{1}{2}y''(x_{k-1})h^2 + O(h^3)$$

再由(2.1)式得

$$\tilde{y}_k = y(x_{k-1}) + f[x_{k-1}, y(x_{k-1})]h$$

所以有

$$e_k = |y(x_k) - \tilde{y}_k| = O(h^2)$$

从而得到欧拉折线公式具有一阶精度。

欧拉折线公式只有一阶精度是很容易理解的。以计算 y_1 为例,此时上面的(2.1)式演变成 $y_1 = y_0 + y'(x_0)h$,这表明 y_1 只是曲线 $y=y(x)$ 在 $x=x_0$ 处的切线上的点。

取步长 $h=0.1$,利用欧拉折线公式求解例 1.1 的源代码见程序 13.01B,程序运行所显示的结果见表 2.1。从表 2.1 不难看出,最后得到的 $Y[10]$ 只有 1 位有效数字。这样的精度当然无法满足实际应用的需要。

表 2.1 利用欧拉折线公式求解例 1.1 的计算结果

k	x[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.181818182E-01	1.100000000E+00	1.095445115E+00	4.555E-03
2	2.000E-01	8.561965190E-01	1.191818182E+00	1.183215817E+00	8.602E-03
3	3.000E-01	8.077475181E-01	1.277437834E+00	1.264911064E+00	1.253E-02
4	4.000E-01	7.692031910E-01	1.358212600E+00	1.341640788E+00	1.657E-02
5	5.000E-01	7.383333491E-01	1.435132919E+00	1.414213562E+00	2.092E-02
6	6.000E-01	7.137198409E-01	1.508966254E+00	1.483239697E+00	2.573E-02
7	7.000E-01	6.944519339E-01	1.580338238E+00	1.549193338E+00	3.114E-02
8	8.000E-01	6.799591681E-01	1.649783431E+00	1.612451550E+00	3.733E-02
9	9.000E-01	6.699148484E-01	1.717779348E+00	1.673320053E+00	4.446E-02
10	1.000E+00	6.64178758E-01	1.784770832E+00	1.732050808E+00	5.272E-02

程序 13.01B 利用欧拉折线公式求解例 1.1

```
#include "ODE_COM.C"

int Euler01(double x0,double y0,double h,int n)
{ int K;
  X[0]=x0;
  Y[0]=y0;
  F[0]=f(x0,y0);
  for(K=1;K<=n;K++)
  { X[K]=X[K-1]+h;
    Y[K]=Y[K-1]+h * F[K-1];
    F[K]=f(X[K],Y[K]);
  }
  return 0;
}

main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(a+b * h<b)n++;
  if(n>MN-1)n=MN-1;
  RN=n;
```

```

Euler01(x0,y0,h,n);
CheckSolution();
ShowTable();
SaveTable("TB130201.txt");
getchar();
return;
}

```

13.2.2 两步欧拉方法

利用 $y(x)$ 在 x_{k-1}, x_{k+1} 处的中心差商近似替代 $y(x)$ 在 x_k 处的微商 $y'(x_k)$, 得

$$\frac{y(x_{k+1}) - y(x_{k-1}))}{x_{k+1} - x_{k-1}} = y'(x_k)$$

由此不难得到

$$y_{k+1} = y_{k-1} + 2hf(x_k, y_k) \quad (2.2)$$

公式(2.2)称为两步欧拉公式,它需要利用前面两步计算得到的 y_{k-1} 和 y_k 来计算 y_{k+1} 。由于开始时只是已知 y_0 的值,对此,可以先利用欧拉折线公式计算 y_1 ,接下来即可利用(2.2)式按步进方式计算 $y_2, y_3, \dots, y_n$ 。

为了确定(2.2)式的精度,可以把 $y(x_{k+1})$ 和 $y(x_{k-1})$ 同时在 $x = x_k$ 处泰勒展开,得

$$y(x_{k+1}) = y(x_k + h) = y(x_k) + f[x_k, y(x_k)]h + \frac{1}{2}y''(x_k)h^2 + O(h^3) \quad (2.3)$$

$$y(x_{k-1}) = y(x_k - h) = y(x_k) - f[x_k, y(x_k)]h + \frac{1}{2}y''(x_k)h^2 + O(h^3) \quad (2.4)$$

由(2.3)式两边减去(2.4)式两边,整理后即得

$$y(x_{k+1}) - y(x_{k-1}) = 2f[x_k, y(x_k)]h + O(h^3) \quad (2.5)$$

从而立即得出两步欧拉方法具有二阶精度。

利用两步欧拉方法求解例 2.1 的源代码见程序 13.02,运行程序所显示的结果见下面的表 2.2,不难看出,两步欧拉方法所得到的结果比欧拉折线方法所得到的结果更精确一些,比如这里得到的 $Y[10]$ 有 2 位有效数字。总的来讲,所得到的结果的精度总不能令人满意。虽然可以通过减小步长来提高结果的精度,但由于本身还是低精度的方法,所以效果也不会很明显。另外,两步欧拉方法还有一个不能令人满意的地方是误差呈振荡的方式出现,这表明两步欧拉方法在数值稳定性方面存在一些问题。后面将会看到,改进的欧拉方法和二阶龙格库塔-库塔方法可以避免这个弊端。

表 2.2 利用两步欧拉公式求解例 1.1 的计算结果

k	x[k]	r[k]	y[k]	yx[k]	e[k]
0	0.0000e+00	1.0000000000e+00	1.0000000000e+00	1.0000000000e+00	0.0000e+00
1	1.0000e-01	9.181818182e-01	1.1000000000e+00	1.095445115e+00	4.555e-03
2	2.0000e-01	8.456947554e-01	1.183636364e+00	1.183215957e+00	4.204e-04
3	3.0000e-01	7.963774725e-01	1.269138947e+00	1.264911064e+00	4.228e-03
4	4.0000e-01	7.471914502e-01	1.342911858e+00	1.341640786e+00	1.271e-02
5	5.0000e-01	7.136455818e-01	1.418577237e+00	1.414213562e+00	4.264e-03
6	6.0000e-01	6.779088100e-01	1.485640974e+00	1.483239697e+00	2.401e-03
7	7.0000e-01	6.53502654e-01	1.554138999e+00	1.549193338e+00	4.966e-03
8	8.0000e-01	6.264025429e-01	1.616311028e+00	1.612451550e+00	3.859e-03
9	9.0000e-01	6.076533601e-01	1.679439508e+00	1.673320053e+00	6.119e-03
10	1.0000e+00	5.869688915e-01	1.737841700e+00	1.732050808e+00	5.791e-03

程序 13.02 利用两步欧拉方法求解例 1.1 的源程序

```
#include "ODE_COM.C"
int Euler02(double x0,double y0,double h,int n)
{ int K;
  X[0]=x0;
  Y[0]=y0;
  F[0]=f(x0,y0);
  X[1]=X[0]+h;
  Y[1]=Y[0]+h * F[0];
  F[1]=f(X[1],Y[1]);
  for(K=2;K<=n;K++)
  { X[K]=X[K-1]+h;
    Y[K]=Y[K-2]+2.0 * h * F[K-1];
    F[K]=f(X[K],Y[K]);
  }
  return 0;
}

main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(a+b * h<b)n++;
  if(n>MN-1)n=MN-1;
  RN=n;
  Euler02(x0,y0,h,n);
  CheckSolution();
  ShowTable();
  getchar();
  return;
}
```

13.2.3 改进的欧拉方法

将 $y' = f(x, y)$ 两边同时在区间 $[x_{k-1}, x_k]$ 上求定积分, 整理后即得

$$y_k = y_{k-1} + \int_{x_{k-1}}^{x_k} f(x, y(x)) dx$$

利用数值积分的梯形公式, 我们不难得到

$$\int_{x_{k-1}}^{x_k} f(x, y(x)) dx = \frac{1}{2} h [f(x_{k-1}, y_{k-1}) + f(x_k, y_k)]$$

从而有

$$y_k = y_{k-1} + \frac{1}{2} h [f(x_{k-1}, y_{k-1}) + f(x_k, y_k)] \quad (2.6)$$

上面的(2.6)式也可以称为梯形公式,注意到 y_k 在等号的两边同时出现,所以(2.6)式给出的方法是一种隐式方法。在现代计算机条件下,直接利用(2.6)式求关于 y_k 的非线性方程的解也是很容易的。

过去,人们利用手工方式解非线性方程组当然是一件非常困难的事,所以比较看好预估-校正法,其基本思想就是分两步来计算 y_k :第一步利用折线公式

$$z_k = y_{k-1} + hf(x_{k-1}, y_{k-1}) \quad (2.7)$$

计算 z_k 作为 y_k 预报值,第二步利用公式

$$y_k = y_{k-1} + \frac{1}{2}h[f(x_{k-1}, y_{k-1}) + f(x_k, z_k)] \quad (2.8)$$

计算 y_k 。我们把第一步称为预报,第二步称为校正,从而(2.7)式及(2.8)式构成一个预报-校正系统。由此所形成的方法称为改进的欧拉方法。

利用改进的欧拉方法求解例 1.1 的源代码见程序 13.03,运行程序所显示的结果见下面的表 2.3,不难看出,改进的欧拉方法所得到的结果与两步欧拉方法所得到的结果的精度大体相同,但是误差随 k 值的增大而增大,这样的结果表明改进的欧拉方法具有更好的数值性能。

表 2.3 利用改进的欧拉方法求解例 1.1 的计算结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.194122017E-01	1.095909091E+00	1.095445115E+00	4.640E-04
2	2.000E-01	8.462862838E-01	1.184096569E+00	1.183215957E+00	8.806E-04
3	3.000E-01	7.923430801E-01	1.266201361E+00	1.264911084E+00	1.290E-03
4	4.000E-01	7.478385416E-01	1.343360151E+00	1.341640786E+00	1.719E-03
5	5.000E-01	7.103876399E-01	1.416401929E+00	1.414213562E+00	2.188E-03
6	6.000E-01	6.783944626E-01	1.485955602E+00	1.483239697E+00	2.716E-03
7	7.000E-01	6.507509397E-01	1.552514091E+00	1.549193338E+00	3.321E-03
8	8.000E-01	6.266665797E-01	1.616474783E+00	1.612451550E+00	4.023E-03
9	9.000E-01	6.055671036E-01	1.678166364E+00	1.673320053E+00	4.846E-03
10	1.000E+00	5.870316130E-01	1.737867401E+00	1.732050808E+00	5.817E-03

程序 13.03 利用改进的欧拉方法求解例 1.1 的源程序

```
#include "ODE_COM.C"
int Euler11(double x0,double y0,double h,int n)
{ int K;
  double ZK
  X[0]=x0;
  Y[0]=y0;
  F[0]=f(x0,y0);
  for(K=1;K<=n;K++)
  { X[K]=X[K-1]+h;
    ZK = Y[K-1]+h * F[K-1];
    F[K] = f(X[K],ZK);
    Y[K] = Y[K-1]+0.5 * h * (F[K-1]+F[K]);
    F[K] = f(X[K],Y[K]);
  }
  return 0;
}
main()
{ double h=0.1;
```

```

int n;
n=(b-a)/h;
if(a+b*h<b)n++;
if(n>MN-1)n=MN-1;
RN=n;
Euler11(x0,y0,h,n);
CheckSolution();
ShowTable();
getchar();
return;
}

```

如果我们进一步分析改进的欧拉方法,它只不过是利用欧拉折线公式产生梯形公式(2.6)式的一个初始解,再利用不动点算法迭代一次,我们当然可以利用不动点算法多迭代几次,比如 2 次,4 次,从而更能体现现代机器计算的特点。

13.3 龙格-库塔(Runge-Kutta)方法

13.3.1 龙格-库塔方法的基本思想

对于由(1.1)式给出的一般形式的常微分方程初值问题来说,如果 $f(x, y)$ 关于它的两个变元均有连续的高阶偏导数,那么问题的理论解 $y(x)$ 也有连续的高阶导数。对于某个给定的 k ,利用 $x_{k+1}=x_k+h$,我们可以把 $y(x_{k+1})$ 在 x_k 处适当地泰勒展开,得

$$y(x_k+h) = y(x_k) + y'(x_k)h + \cdots + \frac{1}{(p+1)!}y^{(p+1)}(x_k)h^{p+1} + O(h^{p+2}) \quad (3.1)$$

注意到

$$\left. \begin{aligned} y'(x_k) &= f[x_k, y(x_k)] \\ y''(x_k) &= \left[\frac{\partial f(x, y)}{\partial x} + \frac{\partial f(x, y)}{\partial y} y'(x_k) \right] \Big|_{x=x_k, y=y(x_k)} \\ y'''(x_k) &= \left[\frac{\partial^2 f(x, y)}{\partial x^2} + 2 \frac{\partial^2 f(x, y)}{\partial x \partial y} y'(x_k) + \frac{\partial^2 f(x, y)}{\partial y^2} y''(x_k) \right] \Big|_{x=x_k, y=y(x_k)} \\ &\vdots \\ y^{(p+1)}(x_k) &= \left[\sum_{m=0}^p \frac{\partial^p f(x, y)}{\partial x^{p-m} \partial y^m} y^{(m)}(x_k) \right] \Big|_{x=x_k, y=y(x_k)} \end{aligned} \right\} \quad (3.2)$$

所以,从理论上讲,可以利用(3.1)式和(3.2)式得到具有任意 $p+1$ 阶精度的递推格式

$$y_{k+1} = y_k + y'(x_k)h + \cdots + \frac{1}{(p+1)!}y^{(p+1)}(x_k)h^{p+1} \quad (3.3)$$

显然如果直接利用上面的(3.1)式和(3.2)式形成递推格式,相应的数值计算会变得非常繁琐,相应的程序就会变得非常复杂。回顾牛顿插值多项式的余项公式,我们可以得到这样的结论:任意一个函数 $u(x)$ 在某一点 x_0 处的 n 阶导数可以用 x_0 附近的 $n+1$ 个点处的 n 阶差商来表示,而这个 n 阶差商又可表示为 $n+1$ 个点处的函数值的线性组合。

类似地,对于上面的(3.2)式中的各种偏导数项,也可以表示为 $f(x,y)$ 在一些不同的点处的函数值的线性组合。更有意义的是当我们在(3.3)式中利用差商近似替代微商时,还可以附带把(3.3)式化为关于 h 的线性函数,这样一来,可以对(3.3)式重新按不同点处的函数值进行合并,从而可以得到更为简单的递推格式。

13.3.2 龙格-库塔法的计算格式

为了利用一些不同点处的函数值的线性组合构造具有一般的 p 阶精度的递推格式,龙格-库塔法的具体计算格式为

$$\left. \begin{aligned} y_{k+1} &= y_k + \sum_{i=1}^p C_i K_i \\ K_1 &= hf(x_k, y_k) \\ K_{i+1} &= hf(x_k + \alpha_i h, y_k + \sum_{j=1}^i \beta_{ij} K_j), i = 1, \dots, p-1 \end{aligned} \right\} \quad (3.4)$$

为了得到具体的具有 p 阶精度的递推格式,把 y_{k+1} 在 x_k 处 p 阶展开,比较 h 的同次幂的系数即可求出各 α_i, β_{ij} 以及 C_i 。由于一般情况下是利用 p 个方程决定多于 p 个的参数,所以这些参数的取值并不惟一,我们当然可以选择更容易计算的参数值。另外一个做法是利用差商替代相应的微商把上面的(3.3)式化为上面的(3.4)式中的

$$y_{k+1} = y_k + \sum_{i=1}^p C_i K_i$$

的形式,也可以计算出(3.4)式中的各 α_i, β_{ij} 以及 C_i 。

13.3.3 二阶龙格-库塔法

在上面的(3.4)式中,如果取 $p=2$,所得到的方法即为二阶龙格-库塔法,此时相应的计算格式为

$$\left. \begin{aligned} y_{k+1} &= y_k + C_1 K_1 + C_2 K_2 \\ K_1 &= hf(x_k, y_k) \\ K_2 &= hf(x_k + \alpha h, y_k + \beta K_1) \end{aligned} \right\} \quad (3.5)$$

把上面的 K_2 在 x_k 处泰勒展开,得

$$K_2 = hf(x_k, y_k) + h[f_x(x_k, y_k)\alpha h + f_y(x_k, y_k)\beta K_1] + O(h^3)$$

所以由 $y_{k+1} = y_k + C_1 K_1 + C_2 K_2$ 即可得到

$$\begin{aligned} y_{k+1} &= y_k + (C_1 + C_2)f(x_k, y_k)h \\ &\quad + C_2 \alpha \left[f_x(x_k, y_k) + \frac{\beta}{\alpha} f_y(x_k, y_k)f(x_k, y_k) \right] h^2 + O(h^3) \end{aligned} \quad (3.6)$$

注意到在(3.3)式中我们有

$$\begin{aligned} y'(x_k) &= f(x_k, y_k) \\ y''(x_k) &= f_x(x_k, y_k) + f_y(x_k, y_k)f(x_k, y_k) \end{aligned}$$

所以比较(3.6)式和(3.3)式比较系数后即可得到

$$C_1 + C_2 = 1, \quad C_2 \alpha = \frac{1}{2}, \quad \frac{\beta}{\alpha} = 1 \quad (3.7)$$

在上面的(3.7)式中,如果取 $C_1 = C_2 = 1/2, \alpha = \beta = 1$, 相应的计算格式为

$$\left. \begin{aligned} K_1 &= hf(x_k, y_k) \\ K_2 &= hf(x_k + h, y_k + K_1) \\ y_{k+1} &= y_k + (K_1 + K_2)/2 \end{aligned} \right\} \quad (3.8)$$

不难理解,由上面的(3.8)式所形成的方法实际上就是改进的欧拉方法,由此可以得到改进的欧拉方法具有二阶精度。

在上面的(3.7)式中,如果取 $\alpha = \beta = 1/2$, 则有 $C_2 = 1, C_1 = 0$, 即可得到另一种不同形式的二阶龙格-库塔法,相应的计算格式为

$$\left. \begin{aligned} K_1 &= hf(x_k, y_k) \\ K_2 &= hf(x_k + h/2, y_k + K_1/2) \\ y_{k+1} &= y_k + K_2/2 \end{aligned} \right\} \quad (3.9)$$

上面的(3.9)式相当于两步欧拉方法,只不过在这里是把一步分为两个半步,所以实际效果应该比前面的两步欧拉方法好。以后如无特别说明,我们指的二阶龙格-库塔都是按上面的(3.9)式计算。

利用上面的(3.9)式求解例 1.1 的源代码见程序 13.04, 相应的计算结果如下面的表 3.1 所示。

表 3.1 利用二阶龙格-库塔方法求解例 1.1 的计算结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.129071837E-01	1.095476190E+00	1.095445115E+00	3.108E-05
2	2.000E-01	8.452802780E-01	1.183298420E+00	1.183215957E+00	8.246E-05
3	3.000E-01	7.907899818E-01	1.265056955E+00	1.264911064E+00	1.459E-04
4	4.000E-01	7.456726134E-01	1.341859988E+00	1.341640786E+00	2.192E-04
5	5.000E-01	7.075611150E-01	1.414516473E+00	1.414213562E+00	3.029E-04
6	6.000E-01	6.748158859E-01	1.483638339E+00	1.483239697E+00	3.986E-04
7	7.000E-01	6.463028437E-01	1.549702212E+00	1.549193338E+00	5.089E-04
8	8.000E-01	6.212021151E-01	1.613088300E+00	1.612451550E+00	6.368E-04
9	9.000E-01	5.989055097E-01	1.674106148E+00	1.673320053E+00	7.861E-04
10	1.000E+00	5.789524146E-01	1.733012308E+00	1.732050808E+00	9.615E-04

从表 3.1 所显示的计算结果看,二阶龙格-库塔虽然和改进的欧拉方法(也是二阶龙格-库塔法)具有相同的精度和相同的计算量,但这里的二阶龙格-库塔所得到的结果明显优于改进的欧拉方法。这个事实说明,对于寻找好的递推格式来说,精度固然重要,但具体的递推格式形式也是很重要的。

程序 13.04 二阶龙格库塔法求解例 1.1 的源代码

```
#include "ODE_COM.C"
void RungeKutta2(double x0, double y0, double h, int N)
{ int K;
  double k1, k2;
  X[0]=x0;
  Y[0]=y0;
  F[0]=f(x0, y0);
  for(K=0; K<N; K++)
  { X[K+1]=X[K]+h;
```



```

    k1=h * F[K];
    k2=h * f(X[K]+h/2.0,Y[K]+k1/2.0);
    Y[K+1] = Y[K]+k2;
    F[K+1] = f(X[K+1],Y[K+1]);
}
return;
}

main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(a+b * h<b)n++;
  if(n>MN-1)n=MN-1;
  RN=n;
  RungeKutta2(x0,y0,h,n);
  CheckSolution();
  ShowTable();
  getchar();
  return;
}

```

13.3.4 四阶龙格-库塔法

在工程应用中,人们常用的是标准四阶龙格-库塔法,其推导方式与二阶龙格-库塔法相同,它的各项参数值为:

$$\begin{cases} C_1 = 1/6, C_2 = 1/3, C_3 = 1/3, C_4 = 1/6 \\ \alpha_1 = \alpha_2 = 1/2, \alpha_3 = 1 \\ b_{11} = 1/2; b_{21} = 0, b_{22} = 1/2; b_{31} = b_{32} = 0, b_{33} = 1 \end{cases}$$

所以相应的计算公式是

$$\left. \begin{aligned} K_1 &= hf(x_k, y_k) \\ K_2 &= hf(x_k + \frac{1}{2}h, y_k + \frac{1}{2}K_1) \\ K_3 &= hf(x_k + \frac{1}{2}h, y_k + \frac{1}{2}K_2) \\ K_4 &= hf(x_k + h, y_k + K_3) \\ y_{k+1} &= y_k + \frac{1}{6}K_1 + \frac{1}{3}K_2 + \frac{1}{3}K_3 + \frac{1}{6}K_4 \end{aligned} \right\} \quad (3.10)$$

利用标准的四阶龙格-库塔方法求解例 1.1 的 C 语言源代码见程序 13.05,运行程序所显示的计算结果如下面的表 3.2 所示。从所显示的结果看,在求解例 1.1 的所有方法中,对于相同的步长,标准的四阶龙格-库塔法得到了到目前为止的最好的结果,所有离散点处的数值解的近似值至少有 5 位有效数字。在程序中,如果取步长 $h=0.05$,所得到

的结果都可以达到 7 位有效数字;如果取步长 $h=0.01$,所得到的结果都可以达到 9 位有效数字。所以适当减小步长对于提高精度也有显著的作用。

表 3.2 标准四阶龙格-库塔法求解例 1.1 的计算结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.128714153E-01	1.095445532E+00	1.095445115E+00	4.167E-07
2	2.000E-01	8.451552690E-01	1.183216740E+00	1.183215957E+00	7.889E-07
3	3.000E-01	7.905710159E-01	1.264912228E+00	1.264911084E+00	1.164E-06
4	4.000E-01	7.453582563E-01	1.341842354E+00	1.341840788E+00	1.567E-06
5	5.000E-01	7.071098045E-01	1.414235578E+00	1.414233562E+00	2.016E-06
6	6.000E-01	6.742037653E-01	1.483242223E+00	1.483239697E+00	2.525E-06
7	7.000E-01	6.455021546E-01	1.549196452E+00	1.549193338E+00	3.114E-06
8	8.000E-01	6.201798134E-01	1.612455350E+00	1.612451550E+00	3.800E-06
9	9.000E-01	5.976218716E-01	1.673324659E+00	1.673320053E+00	4.606E-06
10	1.000E+00	5.773595318E-01	1.732056365E+00	1.732050808E+00	5.554E-06

在过去计算工具非常落后的时代,高阶的龙格-库塔法由于计算量相对说来比较大,所以平时总是在多步法中专门用来计算表头。在现代计算机条件下,专门用高阶的龙格-库塔法求解实际问题也是可行的。

程序 13.05 采用标准的四阶龙格-库塔法求阶例 1.1 的源程序

```
#include "ODE_COM.C"
void RungeKutta4(double x0,double y0,double h,int N)
{ int k;
  double K1,K2,K3,K4;
  X[0]=x0;
  Y[0]=y0;
  F[0]=f(x0,y0);
  for(k=0;k<N;k++)
  { X[k+1]=X[k]+h;
    K1=h * F[k];
    K2=h * f(X[k]+0.5 * h,Y[k]+0.5 * K1);
    K3=h * f(X[k]+0.5 * h,Y[k]+0.5 * K2);
    K4=h * f(X[k]+h,Y[k]+K3);
    Y[k+1] = Y[k]+(K1+2.0 * K2+2.0 * K3+K4)/6.0;
    F[k+1] = f(X[k+1],Y[k+1]);
  }
  return;
}

main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(n>MN-1) n=MN-1;
  RN=n;
  RungeKutta4(x0,y0,h,n);
  CheckSolution();
  ShowTable();
  getch();
}
```

```

return;
}

```

13.3.5 利用龙格-库塔法求常微分方程组的初值问题的数值解

与常微分方程初值问题类似,一阶常微分方程组的初值问题可以表示为

$$\left. \begin{aligned} \frac{dy_1(x)}{dx} &= f_1(x, y_1, y_2, \dots, y_m) \\ \frac{dy_2(x)}{dx} &= f_2(x, y_1, y_2, \dots, y_m) \\ &\vdots \\ \frac{dy_m(x)}{dx} &= f_m(x, y_1, y_2, \dots, y_m) \\ y_1(x_0) &= y_1^0 \\ y_2(x_0) &= y_2^0 \\ &\vdots \\ y_m(x_0) &= y_m^0 \end{aligned} \right\} \quad (3.11)$$

我们的目的是寻找定义在某区间 $[a, b]$ 上的 m 个实函数 $y_1(x), y_2(x), \dots, y_m(x)$,使之满足上面的(3.11)式。一般地,我们也可以假定 x_0 即为区间的左端点 a ,所以相应的问题也称为初值问题。

在上面的(3.11)式中,记

$$\begin{cases} \mathbf{y} = (y_1(x), y_2(x), \dots, y_m(x))^T \\ \mathbf{y}' = \left(\frac{dy_1(x)}{dx}, \frac{dy_2(x)}{dx}, \dots, \frac{dy_m(x)}{dx} \right)^T \\ \mathbf{f}(x, \mathbf{y}) = (f_1(x, \mathbf{y}), f_2(x, \mathbf{y}), \dots, f_m(x, \mathbf{y}))^T \\ \mathbf{y}^0 = (y_1^0, y_2^0, \dots, y_m^0)^T \end{cases}$$

其中 $f_k(x, \mathbf{y})$ 为 $f_k(x, y_1, y_2, \dots, y_m)$ 的简写形式, $k=1, 2, \dots, m$ 。

利用上面的记号,常微分方程组的初值问题可以简单地表示为

$$\left. \begin{aligned} \mathbf{y}' &= \mathbf{f}(x, \mathbf{y}) \\ \mathbf{y}(x^0) &= \mathbf{y}^0 \end{aligned} \right\} \quad (3.12)$$

如果不记 $\mathbf{f}, \mathbf{y}$ 都是向量,那么(3.12)式与前面的(1.1)式的形式完全相同。所以,所有求解(1.1)式的方法只要把其中的 f, y 改写为向量 $\mathbf{f}, \mathbf{y}$,即可形成求解(3.12)式的递推格式。

只需简单地把前面的(3.1)式中的 f, y, K_1, K_2, K_3, K_4 都写成向量的形式,并按我们的习惯,用不同的上标来区分不同的向量,即可得到求解线性常微分方程组(3.12)的四阶标准龙格-库塔法的递推格式。

$$\left. \begin{aligned} K^1 &= hf(x_k, y^k) \\ K^2 &= hf(x_k + \frac{1}{2}h, y^k + \frac{1}{2}K^1) \\ K^3 &= hf(x_k + \frac{1}{2}h, y^k + \frac{1}{2}K^2) \\ K^4 &= hf(x_k + h, y^k + K^3) \\ y^{k+1} &= y^k + \frac{1}{6}K^1 + \frac{1}{3}K^2 + \frac{1}{3}K^3 + \frac{1}{6}K^4 \end{aligned} \right\} \quad (3.13)$$

读者只要理解了程序 13.05 上面的(3.13)式中的向量的各个分量的具体形式,结合具体问题求解编写相应的程序不应该有太大的困难。掌握了标准四阶龙格-库塔法求常微分方程组的初值问题,那么利用二阶龙格-库塔法或改进的欧拉方法求解常微分方程组的初值问题也不应该有太多的困难。

13.4 线性多步法

上一节介绍的龙格-库塔法不仅给出了一种有效的方法,还给我们这样的启示:对于计算某个 y_k 来说,利用 $y(x)$ 和 $f[x, y(x)]$ 在一些离散点处的函数值的线性组合也可以得到 $y(x_k)$ 的高精度的近似值。龙格-库塔方法相当于按照一定的格式首先找到相应的离散点,接下来继续寻找相应的组合系数。如果我们直接利用前面的计算已经得到的各 y_j 和各 $f(x_j, y_j)$ 而直接寻找相应的组合系数,显然可以得到一系列效率更高的方法。

对于某个给定的整数 $r > 1$, 当 $k > r$ 时,我们希望利用 $y_{k-1}, y_{k-2}, \dots, y_{k-r}$ 以及 $f(x_{k-1}, y_{k-1}), f(x_{k-2}, y_{k-2}), \dots, f(x_{k-r}, y_{k-r})$ 的线性组合来确定 y_k , 那么 y_k 可以表示为

$$y_k = \sum_{j=1}^r \alpha_j y_{k-j} + \sum_{j=1}^r \beta_j f_{k-j} \quad (4.1)$$

其中 $f_{k-j} = f(x_{k-j}, y_{k-j}), j=1, 2, \dots, r$ 。

线性多步法其实就是寻找形如上面的(4.1)式的递推格式。不难理解,如果在(4.1)式中允许 j 取零值,所得到的方法就是隐式形式的线性多步法。

13.4.1 利用辛普森公式建立递推格式

对于某个 $k > 1$, 将 $y'(x) = f(x, y(x))$ 两边同时在区间 $[x_{k-2}, x_k]$ 上求定积分,我们有

$$y(x_k) = y(x_{k-2}) + \int_{x_{k-2}}^{x_k} f(x, y(x)) dx \quad (4.2)$$

利用辛普森公式求上面(4.2)中的积分值,利用 y_{k-2}, y_{k-1}, y_k 近似替代 $y(x_{k-2}), y(x_{k-1}), y(x_k)$, 利用 $x_k - x_{k-2} = 2h$, 可以立即得到

$$y_k = y_{k-2} + h(f_{k-2} + 4f_{k-1} + f_k)/3 \quad (4.3)$$

根据我们前面第 12 章对辛普森求积公式的认识,上面的(4.3)式具有四阶精度。

上面的(4.3)式是一种隐式格式,所以寻找 y_k 的初始估计值非常重要,当然我们也有许多办法来处理这个问题。利用经过 $(x_{k-3}, y_{k-3}), (x_{k-2}, y_{k-2}), (x_{k-1}, y_{k-1})$ 这三点的 2

次插值多项式来计算 y_k 的近似值也是一种很好的方法,而且利用 $x_k, x_{k-1}, x_{k-2}, x_{k-3}$ 等距离的条件可以把表示 y_k 的数学形式化简为

$$y_k = y_{k-3} + 3(y_{k-1} - y_{k-2}) \quad (4.4)$$

利用(4.4)式和(4.3)式也可以形成一个预报-校正系统。对于几乎所有的预报-校正系统来说,在现代计算机条件下,我们可以利用预报公式得到迭代的初值,然后再利用校正公式采用不动点算法少量迭代几次(一般 2~4 次为宜),这样得到的结果一般说来更为准确,另外一些理论上的遗留问题(如果存在的话)也可以通过迭代计算而暴露出来。

利用(4.4)式和(4.3)式求解例 1.1 的源代码见程序 13.06,程序利用四阶龙格-库塔法来计算表头,这是所有线性多步法都要用到的操作。程序中利用迭代法计算 y_k 所设置的迭代次数 ITERN=2,运行程序所显示的计算结果如下面的表 4.1 所示,不难看出,实际计算的效果与四阶龙格-库塔法大体相同,可以得到有 5~6 位有效数字的结果。如果设置的迭代次数 ITERN=1,所得到的方法即成为预报-校正方法,程序运行的结果也优于二阶龙格-库塔法,不过只有 3~4 位有效数字,所以多迭代一次的效果非常好。利用(4.4)式所得到的初值本身就是很好的近似解,在本例中,它们已经具有 3~4 位有效数字,所以一般少量迭代几次即可。如果把迭代次数 ITERN 设置得太大,反不如把步长减小更为划算。把程序中的步长 h 改为 0.05,仍然设置迭代次数 ITERN=2,所得到的结果可以达到 7 位有效数字。由于辛普森公式的精度比二阶龙格-库塔法和改进的欧拉方法高,所以减小步长对于提高结果的精度的效果更好一些。

表 4.1 利用辛普森方法求解例 1.1 的计算结果

k	x[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.128714153E-01	1.095445532E+00	1.095445511E+00	4.167E-07
2	2.000E-01	8.451552690E-01	1.183218746E+00	1.183218957E+00	7.689E-07
3	3.000E-01	7.908693979E-01	1.264911052E+00	1.264921084E+00	1.250E-06
4	4.000E-01	7.453556795E-01	1.341640570E+00	1.341640786E+00	2.167E-07
5	5.000E-01	7.071049872E-01	1.414212366E+00	1.414213562E+00	1.196E-06
6	6.000E-01	6.741974368E-01	1.483238128E+00	1.483239697E+00	1.569E-06
7	7.000E-01	6.454930598E-01	1.549190708E+00	1.549193338E+00	2.630E-06
8	8.000E-01	6.201685211E-01	1.612448360E+00	1.612451550E+00	3.189E-06
9	9.000E-01	5.976070701E-01	1.673315649E+00	1.673320053E+00	4.404E-06
10	1.000E+00	5.773415317E-01	1.732045565E+00	1.732050808E+00	5.243E-06

程序 13.06 利用辛普森公式求解例 1.1 的源程序

```
#include "ODE_COM.C"
int ITERN=2;
void Simpson(double x0,double y0,double h,int n)
{ int i,K;
  double temp;
  RungeKutta(x0,y0,h,2);
  for(K=3;K<=n;K++)
  { X[K]=X[K-1]+h;
    Y[K] = Y[K-3]+3.0*(Y[K-1]-Y[K-2]);
    for(i=0;i<ITERN;i++)
    { F[K]=f(X[K],Y[K]);
      Y[K]=Y[K-2]+h*(F[K]+4*F[K-1]+F[K-2])/3.0;
    }
    F[K] = f(X[K],Y[K]);
```



```

    YX[K] = Y[K-3]+3.0*(Y[K-1]-Y[K-2]);
}
return;
}
main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(a+b*h<b)n++;
  if(n>MN-1)n=MN-1;
  RN=n;
  Simpson(x0,y0,h,n);
  CheckSolution();
  ShowTable();
  getchar();
  return;
}

```

13.4.2 四阶阿达姆斯(Adams)预报-校正方法

在线性多步法的应用中,目前最常用的方法为四阶阿达姆斯外推与内插法所形成的预报-校正系统。

假如对于 $k>3$, 我们已经得到了 $y(x)$ 在 $x_{k-4}, x_{k-3}, x_{k-2}, x_{k-1}$ 处的近似值 $y_{k-4}, y_{k-3}, y_{k-2}, y_{k-1}$, 为了计算 $y(x)$ 在 x_k 处的近似值 y_k , 可以对 $y'(x)=f(x, y(x))$ 两边同时在区间 $[x_{k-1}, x_k]$ 求定积分得

$$y(x_k) = y(x_{k-1}) + \int_{x_{k-1}}^{x_k} f(x, y(x)) dx \quad (4.5)$$

利用 $f(x, y(x))$ 在 $x_{k-4}, x_{k-3}, x_{k-2}, x_{k-1}$ 处的函数值列表(表 4.2)构造相应的 3 次拉格朗日插值多项式替代(4.5)中的被积函数 $f(x, y(x))$ 。

表 4.2 阿达姆斯外推法的函数值列表

j	0	1	2	3
x_{k-4+j}	x_{k-4}	x_{k-3}	x_{k-2}	x_{k-1}
f_{k-4+j}	f_{k-4}	f_{k-3}	f_{k-2}	f_{k-1}

立即可以得到

$$y_k = y_{k-1} + \sum_{j=0}^3 \omega_j f_{k-4+j} \quad (4.6)$$

其中

$$f_j = f(x_j, y_j), j = 0, 1, 2, 3$$

$$\omega_j = \int_{x_{k-1}}^{x_k} LB_j(x) dx, j = 0, 1, 2, 3 \quad (4.7)$$

所以,只要求出各个 w_j , 上面的(4.6)式便成为一个迭代格式。

为了简化各 w_j 的计算,可以在(4.7)式中作积分变换。令 $t = x - x_{k-1}$, 则有 $x = t + x_{k-1}$, 从而有

$$w_j = \int_0^h LB_j(t + x_{k-1}) dt, j = 0, 1, 2, 3 \quad (4.8)$$

所以有

$$w_0 = \int_0^h \frac{(t + x_{k-1} - x_{k-3})(t + x_{k-1} - x_{k-2})(t + x_{k-1} - x_{k-1})}{(x_{k-4} - x_{k-3})(x_{k-4} - x_{k-2})(x_{k-4} - x_{k-1})} dt$$

利用等距基点的条件,不难得到

$$w_0 = \int_0^h \frac{(t + 2h)(t + h)t}{(-h)(-2h)(-3h)} dt = -\frac{1}{6h^3} \int_0^h (t^3 + 3ht^2 + 2h^2t) dt = -\frac{9h}{24}$$

类似地,我们不难得到

$$w_1 = \int_0^h \frac{(t + 3h)(t + h)t}{(h)(-h)(-2h)} dt = \frac{1}{2h^3} \int_0^h (t^3 + 4ht^2 + 3h^2t) dt = \frac{37h}{24}$$

$$w_2 = \int_0^h \frac{(t + 3h)(t + 2h)t}{(2h)(h)(-h)} dt = -\frac{1}{2h^3} \int_0^h (t^3 + 5ht^2 + 6h^2t) dt = -\frac{59h}{24}$$

$$w_3 = \int_0^h \frac{(t + 3h)(t + 2h)(t + h)}{(3h)(2h)(h)} dt = \frac{1}{6h^3} \int_0^h (t^3 + 6ht^2 + 11h^2t + 6h^3) dt = \frac{55h}{24}$$

从而得到四阶阿达姆斯外推公式

$$y_k = y_{k-1} + \frac{h}{24}(55f_{k-1} - 59f_{k-2} + 37f_{k-3} - 9f_{k-4}) \quad (4.9)$$

上面的(4.9)式是一个显式格式,所以容易计算。使用外推法外推一步或少量几步也许没有太大的问题,如果一直用外推法来计算显然容易导致误差的积累。

阿达姆斯内插公式就是用下面的表 4.3 给出的插值多项式替代(4.5)式中的被积函数 $f(x, y(x))$ 所形成的递推格式。从而由(4.5)式可以得到

表 4.3 阿达姆斯内插法的函数值列表

j	0	1	2	3
x_{k-3+j}	x_{k-3}	x_{k-2}	x_{k-1}	x_k
f_{k-3+j}	f_{k-3}	f_{k-2}	f_{k-1}	f_k

$$y_k = y_{k-1} + \sum_{j=0}^3 \lambda_j f_{k-3+j} \quad (4.10)$$

其中各个 f_{k-3+j} 的意义与(4.6)式中的完全相同,各 λ_j 也和(4.6)式、(4.7)式以及(4.8)式给出的各 w_j 意义完全相同,只是在这里时利用表 4.3 给出的插值条件进行计算。用类似的方法,我们同样可以方便地得到

$$\lambda_0 = \int_0^h \frac{(t + h)(t)(t - h)}{(-h)(-2h)(-3h)} dt = -\frac{1}{6h^3} \int_0^h (t^3 - h^2t) dt = \frac{h}{24}$$

$$\lambda_1 = \int_0^h \frac{(t + 2h)(t)(t - h)}{(h)(-h)(-2h)} dt = \frac{1}{2h^3} \int_0^h (t^3 + ht^2 - 2h^2t) dt = -\frac{5h}{24}$$

$$\lambda_2 = \int_0^h \frac{(t+2h)(t+h)(t-h)}{(2h)(h)(-h)} dt = -\frac{1}{2h^3} \int_0^h (t^3 + 2ht^2 - h^2t - 2h^3) dt = \frac{19h}{24}$$

$$\lambda_3 = \int_0^h \frac{(t+2h)(t+h)(t)}{(3h)(2h)(h)} dt = \frac{1}{6h^3} \int_0^h (t^3 + 3ht^2 + 2h^2t) dt = \frac{9h}{24}$$

所以四阶阿达姆斯内插公式为

$$y_k = y_{k-1} + \frac{h}{24}(9f_k + 19f_{k-1} - 5f_{k-2} + f_{k-3}) \quad (4.11)$$

利用上面的(4.9)式和(4.11)式即可形成一个预报-校正系统:首先利用外推公式求出 y_k 作为预报值,接下来再利用内插公式计算校正后的 y_k 。所以相应的方法也可以称为阿达姆斯外推内插法。结合例 1.1 求解,完整的源代码见程序 13.07,运行程序所显示的结果见下面的表 4.4A。

表 4.4A 阿达姆斯外推内插法求解例 1.1 的计算结果($h=0.1$)

k	X[k]	F[k]	Y[k]	YK[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	1.000E-01	9.128714153E-01	1.09544532E+00	1.09544515E+00	4.167E-07
2	2.000E-01	8.451552690E-01	1.233216746E+00	1.233215957E+00	7.889E-07
3	3.000E-01	7.905710159E-01	1.264912228E+00	1.264911064E+00	1.164E-06
4	4.000E-01	7.453568168E-01	1.341641357E+00	1.341640786E+00	5.707E-07
5	5.000E-01	7.071071878E-01	1.414213833E+00	1.414213562E+00	2.711E-07
6	6.000E-01	6.742000585E-01	1.483239824E+00	1.483239697E+00	1.268E-07
7	7.000E-01	6.454972909E-01	1.549193380E+00	1.549193338E+00	4.200E-06
8	8.000E-01	6.201736516E-01	1.612451536E+00	1.612451550E+00	1.319E-08
9	9.000E-01	5.976142164E-01	1.673319999E+00	1.673320053E+00	5.371E-08
10	1.000E+00	5.773501230E-01	1.732050720E+00	1.732050808E+00	8.769E-08

从表 4.4A 所显示的结果看,取 $h=0.1$ 导致利用阿达姆斯外推内插法求解例 1.1 的数值性能不太稳定。一般地按步进方式计算时,随着步数的增大,误差也会随之增大,但图中所显示的结果显然不具有这条规律。即使所得到的结果精度比较高,但这样的结果并不是我们所希望的。简单地在程序中把 h 的值改为 0.09,重新运行程序所显示的结果如表 4.4B 所示。按道理讲,步长减小了,精度通常会随之提高,但是实际的精度与表 4.4A 相比反而降低了,不过根据误差的变化规律看,这里所得到的结果更能说明问题。与表 3.2 和表 4.1 对比分析后,不难看出四阶阿达姆斯外推内插法所得到的结果与四阶辛普森公式法和四阶龙格-库塔法所得到的结果的精度大体相同,这也是我们所期待的结果。

表 4.4B 阿达姆斯外推内插法求解例 1.1 的计算结果($h=0.9$)

k	X[k]	F[k]	Y[k]	YK[k]	E[k]
0	0.000E+00	1.000000000E+00	1.000000000E+00	1.000000000E+00	0.000E+00
1	9.000E-02	9.205749015E-01	1.086278295E+00	1.086278049E+00	2.460E-07
2	1.800E-01	8.574935148E-01	1.166190845E+00	1.166190379E+00	4.658E-07
3	2.700E-01	8.058238881E-01	1.240968049E+00	1.240967365E+00	6.842E-07
4	3.600E-01	7.624939157E-01	1.311488455E+00	1.311487705E+00	7.500E-07
5	4.500E-01	7.254775390E-01	1.378408743E+00	1.378404875E+00	8.679E-07
6	5.400E-01	6.93767922E-01	1.442231528E+00	1.442220510E+00	1.018E-06
7	6.300E-01	6.651919636E-01	1.503530831E+00	1.503529638E+00	1.193E-06
8	7.200E-01	6.401866176E-01	1.562051330E+00	1.562049935E+00	1.395E-06
9	8.100E-01	6.178046975E-01	1.618643033E+00	1.618641406E+00	1.628E-06
10	9.000E-01	5.976174231E-01	1.673321951E+00	1.673320053E+00	1.898E-06
11	9.900E-01	5.792881302E-01	1.726269863E+00	1.726267650E+00	2.213E-06
12	1.080E+00	5.625482966E-01	1.777841465E+00	1.777838883E+00	2.582E-06

程序 13.07 利用阿达姆斯外推内插法求解例 1.1

```
#include "ODE_COM.C"
int ITERN=1;
void Adams14(double x0,double y0,double h,int n)
{ int i, K;
  double temp;
```

```

RungeKutta(x0,y0,h,3);
for(K=4;K<=n;K++)
{ X[K]=X[K-1]+h;
  temp = 55.0 * F[K-1]-59 * F[K-2]+37.0 * F[K-3]-9.0 * F[K-4];
  Y[K] = Y[K-1]+h * temp/24.0;
  F[K] = f(X[K],Y[K]);
  for(i=0;i<ITERN;i++)
  { temp = 9.0 * F[K]+19.0 * F[K-1]-5.0 * F[K-2]+F[K-3];
    Y[K] = Y[K-1]+h * temp/24.0;
    F[K] = f(X[K],Y[K]);
  }
}
return;
}

main()
{ double h=0.1;
  int n;
  n=(b-a)/h;
  if(a+n*h<b)n++;
  if(n>MN-1)n=MN-1;
  RN=n;
  Adams14(x0,y0,h,n);
  CheckSolution();
  ShowTable();
  getchar();
  return;
}

```

13.5 求常微分方程数值解的模块化程序设计方法

我们在第6章中已经感受到,对于利用多种方法求解同一个有些复杂的数学问题来说,采用模块化程序设计方法编程可以带来很多方便:减少对源程序进行文字编辑的工作量,使程序设计变得简单,有利于对不同的方法进行对比分析。在前面几节中,我们已经这样做了,只是对模块没有明确定义。

对于求常微分方程数值解来说,目前我们只需4个模块即可:数学问题模块,格式输出模块,求解方法模块以及主控程序模块。相对说来,这些模块的本身以及完整的模块程序的结构都比较简单,所以我们直接通过一个例子说明模块程序的编写以及使用方法。

例 5.1 求初值问题

$$\left. \begin{aligned} y' &= 4e^{0.8x} - 0.5y \\ y(0) &= 2 \end{aligned} \right\}$$

在区间 $[0,2]$ 上的数值解,并与理论解

$$y(x) = \frac{4}{1.3}(e^{0.8x} - e^{-0.5x}) + 2e^{-0.5x}$$

对比指出相应的误差。

13.5.1 数学问题模块

描述例 5.1 的数学问题模块的源代码见程序 13.08A, 可以把源代码用文件名“ODE_NO2.C”保存为磁盘文件。这样, 采用不同的方法求解例 5.1 时, 即可简单地通过文件包含来利用其中的代码。

作为研究方法的案例, 有时候可以给出问题的理论解, 以便对比分析。在这种情况下, 可以在程序中定义一个标识符“ForErrTest”, 以便其他模块利用。

程序 13.08A

```
#include <stdlib.h>
#include <math.h>
#define ForErrTest 1
double x0=0.0, y0=3.0;
double a=0.0, b=2.0;
double eps=0.4e-6;
double f(double x, double y)
{ return 4.0 * exp(0.8 * x) - 0.5 * y;
}
double g(double x)
{ double y=4.0/1.3;
  y *= exp(0.8 * x) - exp(-0.5 * x);
  y += 2 * exp(-0.5 * x);
  return y;
}
```

对于求解一般的实际问题来说, 我们通常不知道问题的理论解 $y=g(x)$ 的数学表达式, 这时程序中也可以不利用函数 $g()$ 。相应地, 在其他模块中也不应有函数 $g()$ 的调用。

13.5.2 经典四阶龙格-库塔法模块

由于在后面的应用中, 我们主要考虑自适应步长的龙格-库塔法, 而实际中用得较多的又是经典四阶龙格-库塔法, 所以我们把相应的 C 语言函数说明为

```
double RungeKuttaC4(double x, double y, double H, int n)
```

其中 x, y 为给定的初始点以及初始点处的函数值, H 为求解区间的长度, n 为步数, 函数的返回值为 $y(x+H)$ 的近似值。完整的源代码见程序 13.08B, 它可以由程序 13.05 改写而来, 步长 h 可利用区间长度 H 除以步数 n 而得到。约定保存源程序的磁盘文件名为“ODE_RKC4.C”。程序中的 WriteTable() 函数为格式输出模块中的函数, 其作用是把重要的中间结果保存在相应的数组中, 以便按格式输出。

程序 13.08B

```

double f(double x,double y);
double K1,K2,K3,K4;
double RungeKuttaC4(double x0,double y0,double H,int n)
{ int k;
  double x,y,h;
  x=x0;y=y0; h=H/n;
  for(k=0;k<n;k++)
  { K1=f(x,y);
    WriteTable(x,y,K1);
    K2=f(x+0.5*h,y+0.5*K1*h);
    K3=f(x+0.5*h,y+0.5*K2*h);
    K4=f(x+h,y+K3*h);
    x+=h;
    y+=h*(K1+2.0*(K2+K3)+K4)/6.0;
  }
  WriteTable(x,y,f(x,y));
  return y;
}

```

13.5.3 格式输出模块

格式输出模块可由程序 13.01B 改写而来,增加 WriteTable()函数。为了既便于研究方法的性能之用,又具有应用程序的基本功能,可以设计两种表格形式输出计算结果。如果问题给出了理论解,所输出的表格应包括精确解以及近似解的误差这两栏;如果没有给出,所输出的表格可以包括精确的步长值。完整的源代码见程序 13.08C,其中 ShowTable()函数按不同的问题计算所需要的结果,调用相应的表格显示函数。函数 ShowTableA()显示精确解以及近似解的误差,可由程序 13.01B 中 ShowTable()函数改写函数名而来;函数 ShowTableB()按便于分段埃尔米特插值所需要的格式显示结果,读者可以自己模仿着完成源代码的编写。约定保存源程序的磁盘文件名为“ODE_TAB.C”。

程序 13.08C

```

/* FileName:ODE_TAB.C */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define TMN 1200
int RN=0;
double X[TMN+1],Y[TMN+1],F[TMN+1],YX[TMN+1],E[TMN+1];
double f(double x,double y);
double g(double x);
int WriteTable(double x,double y,double z)
{ int ans=1;

```

```

    if(RN==TMN){RN--;ans=0;}
    X[RN]=x;Y[RN]=y;F[RN]=z;
    RN++;
    return ans;
}
int ShowTable()
{ int n;
  # ifdef ForErrTest
  for(n=0;n<RN;n++){YX[n]=g(X[n]);E[n]=fabs(YX[n]-Y[n]);}
  ShowTableA();
  # else
  YX[0]=0.0;
  for(n=1;n<RN;n++) YX[n]=X[n]-X[n-1];
  ShowTableB();
  # endif
  return;
}
int ShowTableA(){...}
int ShowTableB(){...}

```

13.5.4 主控程序模块

最后的主控程序模块已经非常简单,只需按函数调用的逻辑顺序定义相应的文件包含程序,依次调用相应的 C 语言函数即可,完整的源代码见程序 13.08D,main()函数中的整型变量 n 给出所用的步数。利用 C 语言中同名的全局变量和局部变量的使用规则,可以把数学问题模块中说明过的一些变量在 main()函数中重新说明或重新赋值,对于通过案例研究算法来说,这样处理可以带来很多方便。

程序 13.08D

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO2.C" /* prog 13.08A */
#include "ODE_TAB.C" /* prog 13.08C */
#include "ODE_RK4.C" /* prog 13.08B */
main()
{ int n=5;
  double x0=0.0,y0=2.0;
  double a=0.0,b=3.0;
  RungeKutta4(x0,y0,b-a,n);
  CheckSolution();
  ShowTable();
  getchar();
  return;
}

```

13.5.5 计算结果分析

在程序 13.08D 中,对步数 n 分别取 5、10 和 15,所得到的计算结果分别如表 5.1~5.3 所示。

表 5.1 n 取 5 时程序 13.08D 的运行结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	3.000000000E+00	2.000000000E+00	2.000000000E+00	0.000E+00
1	6.000E-01	4.376704564E+00	4.175186090E+00	4.174732384E+00	4.537E-04
2	1.200E+00	6.723761519E+00	7.446048750E+00	7.444961233E+00	1.088E-03
3	1.800E+00	1.060731049E+01	1.255094556E+01	1.254891211E+01	2.033E-03
4	2.400E+00	1.695048282E+01	2.066670212E+01	2.066320152E+01	3.301E-03
5	3.000E+00	2.725120792E+01	3.368299521E+01	3.367717177E+01	5.823E-03

表 5.2 n 取 10 时程序 13.08D 的运行结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	3.000000000E+00	2.000000000E+00	2.000000000E+00	0.000E+00
1	3.000E-01	3.592680198E+00	2.984632806E+00	2.984619565E+00	1.324E-05
2	6.000E-01	4.376917166E+00	4.174760886E+00	4.174732384E+00	2.850E-05
3	9.000E-01	5.400381344E+00	5.634702997E+00	5.634656485E+00	4.651E-05
4	1.200E+00	6.724271188E+00	7.445029411E+00	7.444961233E+00	6.818E-05
5	1.500E+00	8.426899406E+00	9.707136569E+00	9.707041936E+00	9.463E-05
6	1.800E+00	1.060826356E+01	1.254903942E+01	1.254891211E+01	1.273E-04
7	2.100E+00	1.339586641E+01	1.613271494E+01	1.613254695E+01	1.680E-04
8	2.400E+00	1.695212363E+01	2.066342050E+01	2.066320152E+01	2.190E-04
9	2.700E+00	2.148578820E+01	2.640152487E+01	2.640124175E+01	2.831E-04
10	3.000E+00	2.725395759E+01	3.367755586E+01	3.367717177E+01	3.643E-04

表 5.3 n 取 15 时程序 13.08D 的运行结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	3.000000000E+00	2.000000000E+00	2.000000000E+00	0.000E+00
1	2.000E-01	3.375861440E+00	2.636364088E+00	2.636362384E+00	1.705E-06
2	4.000E-01	3.830706196E+00	3.355609723E+00	3.355606196E+00	3.567E-06
3	6.000E-01	4.376928604E+00	4.174738009E+00	4.174732384E+00	5.625E-06
4	8.000E-01	5.029198228E+00	5.113450578E+00	5.113442656E+00	7.922E-06
5	1.000E+00	5.804842771E+00	6.194641887E+00	6.194631377E+00	1.051E-05
6	1.200E+00	6.724298554E+00	7.444974680E+00	7.444961233E+00	1.345E-05
7	1.400E+00	7.811640186E+00	8.895532548E+00	8.895536452E+00	1.680E-05
8	1.600E+00	9.095203055E+00	1.058271169E+01	1.058269104E+01	2.065E-05
9	1.800E+00	1.060831466E+01	1.254893721E+01	1.254891211E+01	2.510E-05
10	2.000E+00	1.239025362E+01	1.484395215E+01	1.484392191E+01	3.024E-05
11	2.200E+00	1.448675837E+01	1.752598241E+01	1.752594620E+01	3.621E-05
12	2.400E+00	1.695221154E+01	2.066324468E+01	2.066320152E+01	4.316E-05
13	2.600E+00	1.985003036E+01	2.433569060E+01	2.433563934E+01	5.126E-05
14	2.800E+00	2.325479887E+01	2.863705255E+01	2.863699185E+01	6.071E-05
15	3.000E+00	2.725408376E+01	3.367724352E+01	3.367717177E+01	7.175E-05

从表 5.1~5.3 不难看出,对于经典四阶龙格-库塔方法来说,适当缩小步长,可以明显改进计算的精度。在下面一节中,我们将依据这个简单的事实推导自适应步长的龙格-库塔方法,使我们关于常微分方程数值解方法的研究从练兵阶段过渡到实战阶段。

13.6 自适应步长的龙格-库塔方法

在前面的 13.2 节、13.3 节和 13.4 节中,我们介绍了求常微分方程数值解的一些基本方法:包括单步法和多步法,显式方法和隐式方法。这些方法都是采用固定步长按步进方式计算的,所以还有一些遗留问题需要进一步解决。即使对于求解比较“友好”的问题,我们也希望找到一个合理的步长,一方面能保证计算精度,另外也希望相应的步数尽可能少,以减少计算量,减少舍入误差的积累。更重要的是,如果问题的解在某个局部区间上发生急剧变化(突变),而在其余的地方变化比较均匀(渐变),那么采用固定步长的方法求解显然存在弊端:或者由于步长太小而显著增大计算量,或者由于局部区间上没有采用小步长而产生较大的误差。所以寻找能够自动调整步长的方法就显得很有意义。

注意到利用等距基点插值多项式求数值积分所导出的线性多步法并不适宜于改造为自适应步长的方法,所以我们主要讨论自适应步长的龙格-库塔方法。目前已经有两种比较成熟的方法:步长减半的龙格-库塔方法和嵌入式龙格-库塔方法。

13.6.1 步长减半的龙格-库塔方法

对比分析图 5.1 和图 5.2 所显示的对同一个问题采用不同的步长所得到的计算的结果,我们不难看出,对于经典的四阶龙格-库塔方法来说,如果步长缩小一半,那么所得到的结果的精度会有明显提高。如果能根据两者的差别而导出更精确的误差估计公式,我们还能进一步改进龙格-库塔法的性能。

假如已知初值问题在某个 x_0 处的解为 $y_0 = y(x_0)$, 对于适当给定的步长 $h > 0$, 为了求 $y(x_0 + h)$ 的近似值, 我们可以利用经典的四阶龙格-库塔法分别采用步长为 h 计算一步和步长为 $h/2$ 计算两步, 相应的结果分别记为 z_1 和 z_2 , 这样我们就有

$$y(x_0 + h) = z_1 + A(h^5) + O(h^6) \quad (6.1)$$

$$y(x_0 + h) = z_2 + 2A(h/2)^5 + O(h^6) \quad (6.2)$$

其中 A 是某个非零常数, 其具体值是多少并不重要。记

$$\Delta = \frac{z_2 - z_1}{15} \quad (6.3)$$

利用上面的(6.1)式和(6.2)式不难得到

$$y(x_0 + h) = z_2 + \Delta + O(h^6) \quad (6.4)$$

上面的(6.3)式给出的 Δ 可以看成是 z_2 作为 $y(x_0 + h)$ 的近似值的误差的主要部分, 从而得到 $z_2 + \Delta$ 作为 $y(x_0 + h)$ 的近似值比 z_2 更为精确。下面我们利用表 5.1 和表 5.2 所显示的实际计算结果来验证上面的(6.3)式和(6.4)式的意义。

利用例 5.1 给出的数学问题以及运行程序 13.08D 所得到的计算结果, 取 $x_0 = 0, h = 0.6$, 在表 5.1 中可以找到相应的 $z_1 = 4.175\ 186\ 090$ (对应着图中的 $Y[1]$), 在表 5.2 中可以找到相应的 $z_2 = 4.174\ 760\ 886$ (对应着图中的 $Y[2]$), 从而有

$$\Delta = \frac{z_2 - z_1}{15} = \frac{4.174\ 760\ 886 - 4.175\ 186\ 090}{15} = -\frac{0.000\ 425\ 204}{15} = -0.000\ 028\ 347$$

所以有

$$z_2 + \Delta = 4.174\ 760\ 886 - 0.000\ 028\ 347 = 4.174\ 732\ 539$$

与 $y(0.6)$ 的真值 $4.174\ 732\ 384$ 之差为

$$y(0.6) - (z_2 + \Delta) = 4.174\ 732\ 384 - 4.174\ 732\ 539 = -0.000\ 000\ 159$$

所以 $z_2 + \Delta$ 作为 $y(0.6)$ 的近似值, 已经精确到小数点后 6 位, 比 z_1 和 z_2 的精度都高一些。

利用上面的(6.3)式和(6.4)式, 我们实际上有

$$y(x_0 + h) = \frac{16z_2 - z_1}{15} + O(h^6) \quad (6.5)$$

对于上面的(6.5)式, 我们真的能感受到“似曾相识燕归来”。与第 11 章(3.13)式对比分析即可看出, 它相当于利用理查逊外推加速方法外推一次的结果。由于只是在局部

范围内应用,而且也仅仅使用一次,所以也称为是局部外推公式。注意到 z_1 和 z_2 可直接由程序 13.08B 中的函数 RungeKuttaC4() 的返回值得到,所以我们不难利用(6.5)得到一个五阶的龙格-库塔法,我们称之为半步长五阶的龙格-库塔法。在后面的应用中,我们主要利用(6.3)式和(6.5)式形成自适应步长的方法,所以相应的 C 语言函数说明为

```
double RungeKutta5(double x0,double y0,double h,double * delt)
```

其中 delt 用于保存(6.3)式得到的误差估计值 Δ ,函数的返回值 $y(x_0+h)$ 也是具有五阶精度的近似值,完整的源代码见程序 13.09A。程序中的函数 Runge_KuttaC4() 直接由程序 13.08B 中的函数 RungeKuttaC4() 改名而来,不过需要删去其中两条 WriteTable() 函数调用语句,读者可以自己完成相应的文字编辑工作。约定保存源程序的磁盘文件名为“ODE_RKH5.C”。

程序 13.09A

```
/* File Name ODE_RKH5.C */
double f(double x,double y);
double K1,K2,K3,K4;
double Runge_KuttaC4(double x0,double y0,double H,int n){...}
double RungeKutta5(double x0,double y0,double h,double * delt)
{ double z1,z2,err;
  z1=Runge_KuttaC4(x0,y0,h,1);
  z2=Runge_KuttaC4(x,y,h,2);
  err=(z2-z1)/15;
  *delt=err;
  return y+err;
}
```

利用程序 13.09A 直接得到一个五阶的龙格-库塔法也是可以考虑的。取步数 $n=5$, 利用五阶的龙格-库塔法求解例 5.1 的主控模块的源代码见程序 13.09B, 程序运行后所显示的计算结果如表 6.1 所示。

表 6.1 程序 13.09B 的运行结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	3.000000000E+00	2.000000000E+00	2.000000000E+00	0.000E+00
1	6.000E-01	4.376931339E+00	4.174732539E+00	4.174732384E+00	1.549E-07
2	1.200E+00	6.724305166E+00	7.444961453E+00	7.444961233E+00	2.219E-07
3	1.800E+00	1.060832710E+01	1.254891234E+01	1.254891211E+01	2.317E-07
4	2.400E+00	1.695223301E+01	2.066320173E+01	2.066320152E+01	2.019E-07
5	3.000E+00	2.725411957E+01	3.367717191E+01	3.367717177E+01	1.403E-07

取步数 $n=5$ 运行程序 13.09B 与步数 $n=15$ 运行程序 13.08D 的计算量大体相同(如果重新精心地设计程序 13.09A,前者的计算量更少一些),利用表 6.1 和表 5.3 简单地对比两个程序所计算出来的 $y(0.6)$ 、 $y(1.2)$ 、 $y(1.8)$ 、 $y(2.4)$ 和 $y(3)$ 近似值可知,对于相同的计算量来说,采用局部外推法所得到的结果更精确一些。所以,如果初值问题给出的 $f(x,y)$ 足够光滑,又希望得到更精确的结果,那么利用程序 13.09A 形成一个五阶龙格-库塔法的程序以替代程序 13.08B 是合理的。

程序 13.09B

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO2.C"
#include "ODE_TAB.C"
#include "ODE_RKH5.C" /* SEE PROG 13.09A */
main()
{ int k,n=5;
  double x0=0.0,y0=2.0;
  double x,y,z,h,e;
  double a=0.0,b=3.0;
  h=(b-a)/n;
  x=x0;y=y0;z=f(x,y);
  WriteTable(x,y,z);
  for(k=1;k<=n;k++)
  { y= RungeKutta5(x,y,h,&e);
    x+=h;z=f(x,y);
    WriteTable(x,y,z);
  }
  ShowTable();
  getchar();
  return;
}

```

13.6.2 嵌入式龙格-库塔方法

在龙格-库塔法的实际应用中,由于四阶以上龙格-库塔法所需要计算的函数值的个数大于所约定的阶数,所以四阶龙格-库塔法就特别受到人们的重视,甚至形成了不成文的约定:如无特别声明,人们所指的龙格-库塔法就是经典的四阶龙格-库塔法。应该说,到目前为止,经典的四阶龙格-库塔法仍然处于这个领域中的霸主地位上。不过,由于 Fehlberg 的嵌入式五阶龙格-库塔法的采用,四阶龙格-库塔法的霸主地位似乎受到了挑战。

Fehlberg 的嵌入式龙格-库塔法的算法原理其实还是很容易理解的:按照一般的五阶龙格-库塔法的计算格式

$$\left. \begin{aligned}
 K_1 &= f(x_k, y_k) \\
 K_2 &= f(x_k + a_2 h, y_k + b_{21} K_1 h) \\
 K_3 &= f(x_k + a_3 h, y_k + b_{31} K_1 h + b_{32} K_2 h) \\
 K_4 &= f(x_k + a_4 h, y_k + b_{41} K_1 h + b_{42} K_2 h) \\
 K_5 &= f(x_k + a_5 h, y_k + b_{51} K_1 h + b_{52} K_2 h + b_{53} K_3 h + b_{54} K_4 h) \\
 K_6 &= f(x_k + a_6 h, y_k + b_{61} K_1 h + b_{62} K_2 h + b_{63} K_3 h + b_{64} K_4 h + b_{65} K_5 h)
 \end{aligned} \right\} \quad (6.6)$$

寻找 6 个不同点处的函数值 $K_1, K_2, \dots, K_6$, 同时进一步寻找两组组合系数 $\{c_1, \dots, c_6\}$ 和

$\{d_1, \dots, d_6\}$, 使得

$$y_{k+1} = y_k + h(c_1 K_1 + \dots + c_6 K_6) \quad (6.7)$$

与(6.6)式构成一个五阶龙格-库塔法, 而

$$y_{k+1} = y_k + h(d_1 K_1 + \dots + d_6 K_6) \quad (6.8)$$

与(6.6)式构成一个四阶龙格-库塔法, 从而可以进一步将

$$\Delta = h[(c_1 - d_1)K_1 + \dots + (c_6 - d_6)K_6] \quad (6.9)$$

看成是(6.8)式得到的 y_{k+1} 作为 $y(x_k + h)$ 的近似值的误差。

现在, 人们使用 Fehlberg 的由(6.7)~(6.9)式给出嵌入式龙格-库塔法并不采用 Fehlberg 原来给出的参数, 而是采用 J. R. Cash 和 A. H. Karp 给出的参数, 所形成的嵌入式龙格-库塔方法的效果更好些。完整的 Cash-Karp 参数见表 6.2^①。

表 6.2 嵌入式五阶龙格-库塔方法的 Cash-Karp 参数表

i	a_i	b_{i1}	b_{i2}	b_{i3}	b_{i4}	b_{i5}	c_i	d_i
1							$\frac{2\ 825}{27\ 648}$	$\frac{37}{378}$
2	$\frac{1}{5}$	$\frac{1}{5}$					0	0
3	$\frac{3}{10}$	$\frac{3}{40}$	$\frac{9}{40}$				$\frac{18\ 575}{48\ 384}$	$\frac{250}{621}$
4	$\frac{3}{5}$	$\frac{3}{10}$	$-\frac{9}{10}$	$\frac{6}{5}$			$\frac{13\ 525}{55\ 296}$	$\frac{125}{594}$
5	1	$-\frac{11}{54}$	$\frac{5}{2}$	$-\frac{70}{27}$	$\frac{35}{27}$		$\frac{277}{14\ 336}$	0
6	$\frac{7}{8}$	$\frac{1\ 631}{55\ 296}$	$\frac{175}{512}$	$\frac{575}{13\ 824}$	$\frac{44\ 275}{110\ 592}$	$\frac{253}{4\ 096}$	$\frac{1}{4}$	$\frac{512}{1\ 771}$

利用上面的表 6.2 以及(6.7)、(6.8)和(6.9)式不难编写具有误差估计的五阶龙格-库塔的程序。相应的源代码见程序 13.10, 相应的函数名及调用约定与程序 13.09A 相同, 这样便于对比分析两种不同的带有误差估计的五阶龙格-库塔法的性能差别。约定保存源程序的磁盘文件名为“ODE_RKF5.C”。

程序 13.10

```

/* File Name: ODE_RKF5.C */
double f(double x, double y);
/*****
*****/
const double a1=0.0, c1=37.0/378, d1=2825.0/27648;
const double a2=0.2, c2=0.0, d2=0.0;
const double a3=0.3, c3=250.0/621, d3=18575.0/48384;
const double a4=0.6, c4=125.0/594, d4=13525.0/55296;
const double a5=1.0, c5=0.0, d5=277.0/14336;

```

① 表 6.2 中的数据来自文献[13]第 713 页及文献[14]第 722 页。

```

const double a6=0.875, c6=512.0/1771, d6=1.0/4;
/*****
*****/
const double b21=0.2, b31=3.0/40, b32=9.0/40;
const double b41=0.3, b42=-0.9, b43=1.2;
const double b51=-11.0/54, b52=5.0/2, b53=-70.0/27;
const double b54=35.0/27, b61=1631.0/55296, b62=175.0/512;
const double b63=575.0/13824, b64=44275.0/110592, b65=253.0/4096;
/*****
*****/
double K1,K2,K3,K4,K5,K6;
double RungeKutta5(double x0,double y0,double h,double * err)
{ double dy,e;
  K1=f(x0,y0);
  dy=b21 * K1;
  K2=f(x0+a2 * h,y0+dy * h);
  dy=b31 * K1+b32 * K2;
  K3=f(x0+a3 * h,y0+dy * h);
  dy=b41 * K1+b42 * K2+b43 * K3;
  K4=f(x0+a4 * h,y0+dy * h);
  dy=b51 * K1+b52 * K2+b53 * K3+b54 * K4;
  K5=f(x0+a5 * h,y0+dy * h);
  dy=b61 * K1+b62 * K2+b63 * K3+b64 * K4+b65 * K5;
  K6=f(x0+a6 * h,y0+dy * h);
  dy=d1 * K1+d3 * K3+d4 * K4+d5 * K5+d6 * K6;
  * err=dy-(c1 * K1+c3 * K3+c4 * K4+c6 * K6);
  return y0+dy * h;
}

```

我们可以简单地把程序 13.09B 中的语句行

```
include "ODE_RHH5.C" /* SEE PROG 13.09A */
```

改写为

```
include "ODE_RHF5.C" /* SEE PROG 13.10 */
```

即可得到利用嵌入式五阶龙格-库塔法求解例 5.1 的程序。取步数 $n=10$ 运行程序所显示的结果如表 6.3 所示。

表 6.3 $n=10$ 时程序的运行结果

k	X[k]	F[k]	Y[k]	YX[k]	E[k]
0	0.000E+00	3.000000000E+00	2.000000000E+00	2.000000000E+00	0.000E+00
1	3.000E-01	3.59268800E+00	2.984619603E+00	2.984619565E+00	3.801E-08
2	6.000E-01	4.376931381E+00	4.174732457E+00	4.174732384E+00	7.206E-08
3	9.000E-01	5.400404548E+00	5.634656589E+00	5.634656485E+00	1.043E-07
4	1.200E+00	6.724305209E+00	7.444961370E+00	7.444961233E+00	1.369E-07
5	1.500E+00	8.426946637E+00	9.707042108E+00	9.707041936E+00	1.720E-07
6	1.800E+00	1.060832711E+01	1.254891232E+01	1.254891211E+01	2.120E-07
7	2.100E+00	1.339595028E+01	1.613254721E+01	1.613254695E+01	2.595E-07
8	2.400E+00	1.695223296E+01	2.066320184E+01	2.066320152E+01	3.177E-07
9	2.700E+00	2.148392957E+01	2.640124214E+01	2.640124175E+01	3.902E-07
10	3.000E+00	2.725411940E+01	3.367717225E+01	3.367717177E+01	4.816E-07

从表 6.1 和表 6.3 所显示的结果看,对于大体相同的计算量来说,半步长五阶龙格-库塔法与嵌入式五阶龙格-库塔法所得到的精度大体相同。不过从误差的变化规律可以看出,在开始几步,嵌入式五阶龙格-库塔法所得到的结果更精确一些;但是到后面几步反而是半步长五阶龙格-库塔法所得到的结果更精确一些。这个矛盾的结果表明,对于控制整体截断误差来说,半步长五阶龙格-库塔法仍保持着某种潜在的优势。当然,仅通过一个算例所显示出来的细微差别并不能说明哪个方法更好一些。

13.6.3 自适应步长方法

对于求解一般的初值问题来说,假如利用步进方法已经得到某个 x_k 处的函数值 $y(x_k)$ 的近似值 y_k ,接下来利用某个步长 h 计算 $x_{k+1}=x_k+h$ 处函数值 $y(x_{k+1})$ 的近似值 y_{k+1} ,如果利用前面所得到的两个五阶龙格-库塔法的计算格式计算 y_{k+1} ,那么同时还可以得到 y_{k+1} 的误差估计值 Δ 。由于 Δ 大体上与 h^5 成正比,从而可以把 Δ 近似表示为 $\Delta = Ah^5$,这里 A 是某个非零常数,具体数值是多少无关紧要。一方面,如果 y_{k+1} 的误差 Δ 不满足精度要求,那么需要“适当”减小 h ,重新计算 y_{k+1} ,直到满足精度要求为止;另一方面,如果 y_{k+1} 的误差 Δ 满足精度要求,那么就无需再计算 y_{k+1} ,不过下一步计算 y_{k+2} 时希望“适当”增大 h ,以减少不必要的步数,从而提高计算效率。

假如希望的精度为 Δ_0 ,希望下一次计算(不管是重新计算 y_{k+1} ,还是进一步计算 y_{k+2})所选用的步长为 h_0 ,利用

$$\frac{\Delta_0}{\Delta} = \frac{A(h_0)^5}{Ah^5} \quad (6.10)$$

即可求得

$$h_0 = \left(\frac{\Delta_0}{\Delta}\right)^{1/5} h \quad (6.11)$$

由于利用上面的(6.11)式所得到的 h_0 并不是精确计算的结果,而只是一种估计值,所以在具体应用时可采用一些“技术”处理的方法^①。比如在程序中设置两个正的常数因子 factor0, factor1,且 factor0 < factor1。若 $h_0 > h$,则将 h_0 乘以 factor0,以示加大惩罚的力度;否则将 h_0 乘以 factor1。实现自适应步长的龙格-库塔法的 C 语言函数可以说明为

```
int AdaptiveRungeKutta(double x0, double y0, double H, double EPS)
```

其中 H 为求解区间的长度, EPS 为允许误差。对于具有研究意义的程序来说,约定函数的返回值为实际计算的总“步数”是很有意义的。这里的总步数约定为有效的步数与失败的步数的总和,它与总计算量成正比。一般说来,如果失败的步数所占的比例较大,应该适当减小两个常数因子的值;如果太小,则应该适当增大两个常数因子的值。对于一个具体问题来说,在允许误差一定的条件下,通过试验调整两个常数因子的值,使得总步数达到最小并不太难。

实现自适应步长的龙格-库塔法的 C 语言源代码见程序 13.11A,程序的循环体外重

^① 文献[14]对(6.2)式的处置方式是, $h_0 = \theta(\Delta_0/\Delta)^\alpha$ 。若 $\Delta < \Delta_0$,取 $\alpha = 0.2$;否则取 $\alpha = 0.25$ 。 θ 为比例常数,一般取 0.9。由于取 $\alpha = 0.25$ 并没有带来明显的效果,又过于偏离理论分析的结果,所以我们并不提倡这样处理。

新计算最后一次的 h_0, x_1, y_1 , 目的是控制端点值。程序中的两个控制参数 factor0 和 factor1 的值分别取 0.8 和 0.9, 在一般情况下效果还是不错的。程序中调用的一步五阶龙格-库塔法函数 RungeKutta5() 既可以来自程序 13.09A, 也可以来自程序 13.10, 这样有利于比较半步长五阶龙格-库塔法和嵌入式五阶龙格-库塔法的性能。为了应用的方便, 约定保存源程序的磁盘文件名为“ODE_ADAP.C”。

程序 13.11A 自适应步长的龙格-库塔法程序

```
/* File Name :ODE_ADAP.C */
#include<math.h>
double factor0=0.8,factor1=0.9;
int AdaptiveRungeKutta(double x0,double y0,double H,double EPS)
{ double x1,y1,z0,z1,h0,h1;
  double err,abserr,scale,xend;
  int count=0;
  xend=x0+H;
  h1=H/4;
  x1=x0;y1=y0;
  while(x1<xend)
  { x0=x1,y0=y1,h0=h1;
    while(1)
    { y1=RungeKutta5(x0,y0,h0,&err);
      count++;
      abserr=fabs(err);
      scale=pow(EPS/abserr,1.0/5);
      if(abserr<EPS){h1=h0*scale*factor1;break;}
      h0=h0*scale*factor0;
    }
    WriteTable(x0,y0,f(x0,y0));
    x1=x0+h0;
  }
  h0=xend-x0;
  y1=RungeKutta5(x0,y0,h0,&err);
  x1=x0+h0;
  WriteTable(x1,y1,f(x1,y1));
  return count;
}
```

13.6.4 举例计算

现在, 我们已经完成了利用自适应龙格-库塔方法求解初值问题的所有准备工作, 接下来可以通过具体算例说明如何应用这些方法, 如何评价这些方法。

例 6.1^① 求解初值问题

$$\begin{cases} y'(x) + 0.6y = 10e^{-(x-2)^2/[2(0.075)^2]} \\ x \in [0, 4], y(0) = 0.5 \end{cases}$$

利用计算机求解上面的例 6.1 的数学问题模块见程序 13.11B, 约定保存源程序的磁盘文件名为“ODE_NO3.C”。

程序 13.11B 求解例 6.1 的数学问题模块程序

```
/* File Name: ODE_NO3.C */
#include <stdlib.h>
#include <math.h>
double x0=0.0, y0=0.5;
double a=0.0, b=4.0;
double eps=0.4e-6;
double f(double x, double y)
{ double temp = 40.0/3.0;
  temp *= x-2.0;
  return 10.0 * exp(-temp * temp/2.0);
}
```

利用自适应步长的龙格-库塔法求解例 6.1 的主控模块见程序 13.11C。程序虽然很简单, 也可把其中的文件包含语句行中的文件名分别写为“ODE_RKF5.C”或“ODE_RKH5.C”从而得到相应的嵌入式五阶龙格-库塔法和半步长五阶龙格-库塔法的自适应步长的程序。另外, 简单地改写 main() 函数中的 EPS 的值从而得不同精度设置的程序。接下来就可以检测所形成的方法了。

程序 13.11C 用不同方案求解例 6.1 的测试程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO3.C"
#include "ODE_TAB.C"
#include "ODE_RKF5.C"
/* OR Include "ODE_RKH5.C" */
#include "ODE_ADAP.C"

main()
{ int n;
  double H=4.0, EPS=0.4E-4;
  n = AdaptiveRungeKutta(x0, y0, H, eps);
}
```

^① 这是文献[12]中专门给出的测试自适应方法程序的例题, 所用的算法来自文献[14], 程序中采用的比例常数为 0.9。文献[12]中没有给出数值计算的结果, 而是通过图形显示计算结果。虽然文献[12]中给出的图形显示结果比较粗糙, 也足以说明方法的有效性。

```

ShowTable();
printf("\n eps = %10.4e count = %3d",eps,n);
getchar();
return;
}

```

首先取 $\text{EPS}=0.4\text{E}-8$ 运行程序 11.13C, 可以得到 $y(x_0+H)$ 的比较精确的近似值为

$$y(x_0+H) \approx 0.612\ 169\ 0$$

取 $\text{EPS}=0.4\text{E}-3$, 采用自适应嵌入式和半步长的五阶龙格-库塔法求解例 6.1 的计算结果如表 6.4 所示; 取 $\text{EPS}=0.4\text{E}-4$, 采用自适应半步长的五阶龙格-库塔法的计算结果如表 6.5 所示。

表 6.4 自适应嵌入式五阶龙格-库塔法求解例 6.1 的计算结果

k	x[k]	y[k]	F[k]	H[k]
0	0.0000000000E+00	5.0000000000E-01	-3.0000000000E-01	0.0000000000E+00
1	5.0000000000E-01	3.7040874511E-01	-2.224524707E-01	5.0000000000E-01
2	8.6710105386E-01	2.9718246090E-01	-1.7830947654E-01	3.6710105386E-01
3	1.3678212875E+00	2.2006284455E-01	-1.3203770673E-01	5.0072023368E-01
4	1.5512605068E+00	1.9712728468E-01	-1.1827620237E-01	1.8343921928E-01
5	1.6764285660E+00	1.8287982514E-01	-1.0881960416E-01	1.2516805916E-01
6	1.7802298984E+00	1.7497045442E-01	-9.1624296121E-02	1.0380133242E-01
7	1.8658050040E+00	1.6821802594E-01	-8.1878685720E-02	8.5575105597E-02
8	2.0108888564E+00	1.1578048778E+00	9.2004774509E+00	1.4508385241E-01
9	2.0862321664E+00	1.6882803757E+00	4.1505001160E+00	7.5343110015E-02
10	2.1552741530E+00	1.8138503557E+00	8.4577007542E-02	6.9041986544E-02
11	2.2253792532E+00	1.7718137479E+00	-9.1367221318E-01	7.0105100282E-02
12	2.3319400499E+00	1.6644605448E+00	-9.9811855792E-01	1.0656079611E-01
13	2.4641858578E+00	1.5375029344E+00	-9.2250171255E-01	1.3224580794E-01
14	2.8490343377E+00	1.2204874228E+00	-7.3229245367E-01	3.8484847989E-01
15	4.0000000000E+00	6.1174540530E-01	-3.6704724318E-01	1.1509656623E+00
eps = 4.000e-04 count = 25				

表 6.5 自适应半步长五阶龙格-库塔法求解例 6.1 的计算结果

k	x[k]	y[k]	F[k]	H[k]
0	0.0000000000E+00	5.0000000000E-01	-3.0000000000E-01	0.0000000000E+00
1	5.0000000000E-01	3.7040903446E-01	-2.224542067E-01	5.0000000000E-01
2	1.5975799888E+00	1.9171950855E-01	-1.1502610235E-01	1.0975799888E+00
3	1.7102052894E+00	1.7929650066E-01	-1.0184990927E-01	1.1262530063E-01
4	1.8062991680E+00	1.7823891957E-01	-2.4916255096E-01	9.6093878603E-02
5	1.8828822296E+00	2.7072419340E-01	2.7920796788E+00	7.6583061600E-02
6	1.9617467572E+00	7.1141588775E-01	8.3534721162E+00	7.8864527629E-02
7	2.0240375877E+00	1.2772276701E+00	8.7330249674E+00	6.2290830489E-02
8	2.0638538142E+00	1.7186894884E+00	3.5392007908E+00	6.9816226454E-02
9	2.1746937739E+00	1.8112797640E+00	-4.2316589637E-01	8.0839459750E-02
10	2.2575956776E+00	1.7408468006E+00	-1.0170661881E+00	8.2902403647E-02
11	2.3677350892E+00	1.6300526227E+00	-9.7797137003E-01	1.1013941162E-01
12	2.5337292835E+00	1.4755287315E+00	-8.8531723879E-01	1.6599419431E-01
13	3.1404091914E+00	1.0253230783E+00	-6.1519384699E-01	6.0667990791E-01
14	4.0000000000E+00	6.1216527459E-01	-3.6729922476E-01	8.5959080858E-01
eps = 4.000e-05 count = 25				

由表 6.4 和表 6.5 所显示的结果可以看出, 对于所给出的精度控制而言, 两种方法所得到的 $y(4)$ 的近似值都达到了相应的精度要求, 半步长方法所得到的结果还多出一位有效数字。由表 6.4 可以看出, $y(x)$ 从 $x=0$ 开始下降, 在 $x \approx 1.806$ 处达到最小值约为 0.178, 紧接着急剧上升, 在 $x \approx 2.175$ 处达到最大值约为 1.811。在长度不到 0.4 的区间 $[1.806, 2.175]$ 内, 函数值大约上升了 10 倍。当 $x > 2.715$ 时, 函数 $y(x)$ 单调下降, 而且下降的速度也愈来愈缓慢。另外, 由表 6.5 还可以看出, 在函数值发生急剧变化的区间内, 用自适应步长方法所形成的步长特别小, 在 $x \approx 2.024$ 处, $y'(x)$ 达到最大值, 约为 8.73, 此时相应的步长达到最小, 约为 0.062 3, 大约是平均步长 4.0/14 的 1/5。而在函数值变化缓慢的地方, 用自适应步长方法所形成的步长则适当增大。所有这些正是我们所企盼的结果。

13.7 大步长问题与外推加速方法

13.7.1 引言

对于(1.1)给出的初值问题,如果只是计算理论解 $y(x)$ 在某个点 $x_0 + H$ 处的函数值,那么相应的问题可以称为固定步长问题或单步问题。在通常情况下,步长 H 的值相对而言比较大,用前面几节介绍的方法计算一步通常难以达到精度要求。所以把这样的问题称为大步长问题是合适的。

研究大步长问题的数值解方法是很有意义的,首先它本身有着直接的应用;其次可以利用外推加速方法显著提高精度,并减少相应的计算量;最后,所形成的方法也可直接应用于多步问题,只是把其中的每一步都看成是一个独立的大步长问题即可。

由于步长 H 通常比较大,所以具体的求解方法还是把 H 划分为某个 n 等份,利用 n 小步的计算来完成一大步的计算问题。记 $G(x_0, y_0, H, n)$ 表示利用某个 n 步方法所得到的 $y(x_0 + H)$ 的近似值,我们当然可以预期,当 n 趋于无穷大时,相应的小步长 H/n 趋近于零,而 $G(x_0, y_0, H, n)$ 则趋近于 $y(x_0 + H)$,从而可以利用外推方法来加速收敛。

13.7.2 修正的中点公式方法

利用外推方法求解大步长问题的关键在于,对于利用某个 n 步方法所得到的 $y(x_0 + H)$ 的近似值 $G(x_0, y_0, H, n)$ 的误差可以表示为只有偶数次幂的幂级数,亦即

$$y(x_0 + H) = G(x_0, y_0, H, n) + A_1(H/n)^2 + A_2(H/n)^4 + \cdots + A_m(H/n)^{2m} + O(H/n)^{2(m+1)} \quad (7.1)$$

从而可以利用第 11 章 11.5 节的(5.4)式形成外推法。这个问题 Gragg 已经通过简单地修改两步欧拉方法而得以解决,所做的修改也仅仅是把最后的结果取为:

$$G(x_0, y_0, H, n) = [y_{n-1} + hf(x_n, y_n) + y_n]/2 \quad (7.2)$$

其中 $h = H/n$, $y_1, y_2, \dots, y_n$ 是利用两步欧拉方法得到的序列。

由于两步欧拉方法是利用中心差商替代微商而形成的方法,所以相应的方法也称为修正的中点公式法或 Gragg 方法。对于某个给定的大步长问题,利用 Gragg 方法计算 $G(x_0, y_0, H, n)$ 处的 C 语言函数可以说明为

```
double Gragg(double x0, double y0, double H, int n)
```

函数的返回值为(7.2)式给出的 $G(x_0, y_0, H, n)$ 。完整的源代码见程序 13.12A, 程序中计算 $G(x_0, y_0, H, n)$ 的函数值的 C 语言函数为 $T()$, 而不是 $Gragg()$, 目的是方便后面与不同形式的外推方法做对比分析。约定保存源程序的磁盘文件名为“ODE_GRAG.C”。

程序 13.12A

```
/* FileName: ODE_GRAG.C */
double f(double, double);
double T(double x0, double y0, double H, long n)
{ int K;
```

```

double x1,y1,y2,h;
h=H/n;
y1=y0+h*f(x0,y0);
x1=x0+h;
for(K=1;K<n;K++)
{ y2=y0+2.0*h*f(x1,y1);
  x1+=h;
  y0=y1;
  y1=y2;
}
y2=(y1+y0+h*f(x1,y1))/2;
return y2;
}

```

13.7.3 精心确定外推所利用的整数序列

利用修订的中点公式法以及第11章11.5节中介绍的利用整数序列外推的方法,接下来应用外推方法求解大步长问题并没有什么实质上的困难。为了得到更好的外推效果,有必要进一步研究采用什么样的整数序列以得到更好一些的效果。

一段时间以来,人们为了得到更好的整数序列做了大量的研究。由于没有相应的理论支撑,所以,相应的研究也只是采用数值试验的方法。除了通常采用的步长减半或步数倍增的外推方法外,Bulirsch 和 Stoer 建议的序列为

$$\{n_k\} = \{2, 4, 6, 8, 12, 16, 24, 32, \dots\}$$

序列的构造规则为

$$n_0 = 2, \quad n_1 = 4, \quad n_2 = 6; n_k = 2n_{k-2}, k = 3, 4, \dots$$

Deuffhard 建议的序列为

$$\{n_k\} = \{2, 4, 6, 8, 10, 12, 14, 16, \dots\}$$

也就是通常的偶数序列。不过他建议序列的长度不要超过8,如果在这个范围内所得到的外推序列不收敛,则适当减小步长。显然这为进一步的程序设计增添了麻烦。

对于外推方法来说,由于问题的多样性,只用某一个序列作外推显然是不够的。虽然不少人所采用的某个序列针对他们自己的特定问题可能是最好的,但这并不能证明这个序列对所有问题都是最好的或最合适的。我们可以按如下一些规则来构造有利的整数序列:首先序列中的数都可表示为某个 $2^L \sim 2^{L+1}$ 或 $2^L \sim 2^{L+2}$ 之间的数与某个 2^m 的积的形式,这样,转换为二进制整数或浮点数后,最多只有高位的 $L+1$ 位或 $L+2$ 位是1,其余低位均为零,这样的数有利于二进制计算;其次,序列中相邻两个数的比 n_{k+1}/n_k 最好比较接近某个常数,一方面吸取了几何序列与初始步长基本无关的优点,另外也可保持几何序列收敛比较均匀的特性;最后,序列中相邻两个数的比值的取值范围应该是

$$1.1 \leq n_{k+1}/n_k \leq 2$$

如果超出这个范围,则存在明显的问题。这样一来,我们就可以在 $[1.1, 2]$ 内确定少量几个合适的比值,并根据 n_{k+1}/n_k 的近似取值而得到不同的序列,序列的性能也随比值的不

同而有所差别,由此不难根据特定的问题确定出最合适的序列。

表 7.1 是我们精心设计的 5 个序列,连同公比为 2 的等比序列 $\{2, 4, 8, \dots\}$,一共有 6 个序列可以利用。有意义的是,对多个算例的实际检测表明,表中每一个序列都曾夺取过在某种意义下最佳的桂冠;在一般情况下,总有一个序列能取得非常好的结果。所以除此之外,再继续寻找新的序列并不是很有意义的事。

表 7.1 应用于外推方法的几种实用整数序列列表

K	N1[K]	N2[K]	N3[K]	N4[K]	N5[K]
0	2	2	2	2	2
1	4	4	4	4	4
2	8	8	8	8	6
3	13	12	11	12	8
4	21	16	16	16	10
5	32	24	22	20	12
6	52	32	32	24	14
7	84	48	44	32	16
8	128	64	64	40	20
9	208	96	88	48	24
10	336	128	128	64	28
11	512	192	176	80	32
12	832	256	256	96	40
13	1 344	384	352	128	48
14	2 048	512	512	160	56
15	3 328	768	704	192	64
16	5 376	1 024	1 024	256	80

对表 7.1 中的每个序列的具体说明见表 7.2。决定序列性能的一个重要指标是 n_{k+1}/n_k , 比值愈大, 序列增长的速度越快, 一方面可以加快外推的速度, 另一方面又可能显著增大计算量。决定序列性能的另一个重要指标是序列中的数转换成二进制数后从高位算起非零数字所占据的位数, 显然这个数字愈小愈好。

表 7.2 表 7.1 中每个序列的具体说明

序列	表头长	代表元素	计算 n_k 的递推格式	n_{k+1}/n_k	二进制数高位数字非零个数
N0	1	2, 4	$2n_{k-1}$	2	1
N1	5	8, 13, 21, 32	$4n_{k-3}$	≈ 1.6	5
N2	4	8, 12, 16	$2n_{k-2}$	1.33~1.5	2
N3	4	8, 11, 16	$2n_{k-2}$	≈ 1.4	4
N4	6	12, 16, 20, 24	$2n_{k-3}$	1.2~1.33	3
N5	7	8, 10, 12, 14, 16	$2n_{k-4}$	1.14~1.25	3

我们不难利用表 7.1 和表 7.2 中的数据自动生成所需要的整数序列,相应的 C 语言函数可以说明为

```
int FormSequence(int NO)
```

当形式参数 NO 分别取 0,1,⋯,5 时,所得到的序列分别为 $N_0, N_1, \dots, N_5$ 。

13.7.4 外推加速方法的模块程序

对于初始条件为 $y(x_0)=y_0$,步长为 H ,允许误差为 EPS 的大步长问题,利用外推加速方法求解的 C 语言函数可以说明为

```
int Extrap(double x0,double y0,double H,double EPS)
```

从而使外推加速方法成为一个非常便于使用的工具。模仿第 11 章程序 11.06,不难完成模块程序的编写,完整的源代码见程序 13.12B,约定保存源程序的文件名为“ODE_EXTR.C”。

在程序 13.12B 中,我们用 int 型全局变量 SN 表示所用到的整数序列编号,MNK 为序列的长度。不同的序列采用不同的长度是合理的。对序列 $N_0 \sim N_5$,我们采用的序列长度依次是 15, 20, 25, 25, 28, 30, 它们都在函数 FormSequence() 中设定。函数 FormSequence() 中当序列号取 6 时是 Bader 和 Deuflhard 用半隐式外推方法求解刚性问题所采用的序列,在下一节中我们将会用到它。

程序 13.12B 大步长问题的外推加速方法的源代码

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MN 32
#define MM 8
int RK, RM;
double P[MN+1][MM+1];
long NK[MN+1]={2,4};
int SN=-1, MNK=0;

int FormSequence(int NO)
{ int k;
  double N;
  if(NO<5)NK[2]=8; else NK[2]=6;
  switch(NO)
  { case 0:SN=0; MNK=15;
    for(k=3;k<=MNK;k++)NK[k]=2 * NK[k-1];
    break;
    case 1:SN=1; MNK=20;
    NK[3]=13; NK[4]=21;
    for(k=5;k<=MNK;k++)NK[k]=4 * NK[k-3];
    break;
    case 2:SN=2; MNK=25; NK[3]=12;
```

```

    for(k=4;k<=MN;k++)NK[k]=2 * NK[k-2];
    break;
case 3:SN=3;MNK=25;NK[3]=11;
    for(k=4;k<=MNK;k++)NK[k]=2 * NK[k-2];
    break;
case 4:SN=4;MNK=28;
    NK[3]=12;NK[4]=16;NK[5]=20;NK[6]=24;
    for(k=6;k<=MNK;k++)NK[k]=2 * NK[k-3];
    break;
case 5:SN=5;MNK=30;
    NK[3]=8;NK[4]=10;NK[5]=12;NK[6]=14;
    for(k=7;k<=MNK;k++)NK[k]=2 * NK[k-4];
    break;
case 6:SN=6;MNK=25;NK[0]=2;NK[1]=6;
    for(k=2;k<=MNK;k++)
    { N=(double)NK[k-1];
      while(N/NK[k-1]<1.4) N+=4.0;
      NK[k]=N;
    }
    break;
default:
    SN=4;MNK=28;
    NK[3]=12;NK[4]=16;NK[5]=20;NK[6]=24;
    for(k=7;k<=MN;k++)NK[k]=2 * NK[k-3];
}
return MNK;
}

```

```

int Extrap(double x0,double y0,double H,double EPS)

```

```

{ double err,a1,a2;
  int k,m;
  if(SN<0)FormSequence(4);
  P[0][0]=T(x0,y0,H,NK[0]);
  for(k=0;k<MNK;k++)
  { P[k+1][0]=T(x0,y0,H,NK[k+1]);
    for(m=0;m<=k;m++)
    { if(m>=MM) break;
      a1=(double)NK[k+1];a1*=a1;
      a2=(double)NK[k-m];a2*=a2;
      P[k+1][m+1]=a1 * P[k+1][m]-a2 * P[k][m];
      P[k+1][m+1]/= a1-a2;
      err=fabs(P[k+1][m]-P[k][m]);
      if(err<EPS){RK=k+1;RM=m;return 1;}
    }
  }
}

```

```

    }
    RK=MNK;RM=MM;
    return 0;
}

```

13.7.5 计算举例

对于例 6.1 给出的初值问题,取 $H=4.0$, $EPS=0.4E-12$,利用外推法求 $y(x_0+H)$ 的近似值的 C 语言源代码见程序 13.12C。令序号值 SN 依次取 0,1,...,5 运行程序所显示的结果如表 7.3 所示。由此进一步估算不同方法的计算量并不困难,但是意义似乎不大。

表 7.3 利用不同的外推方法求例 6.1 的 $y(x_0+H)$ 近似值的计算结果

SN= 0 EPS= 4.0E-13 K= 9 M= 2 P[K-1][M]= 6.121690271855234e-01 P[K][M] = 6.121690271852263e-01 P[K][M+1]= 6.121690271852216e-01	SN= 3 EPS= 4.0E-13 K=14 M= 2 P[K-1][M]= 6.121690271855571e-01 P[K][M] = 6.121690271852619e-01 P[K][M+1]= 6.12169027185224e-01
SN= 1 EPS= 4.0E-13 K=12 M= 2 P[K-1][M]= 6.121690271852827e-01 P[K][M] = 6.121690271852084e-01 P[K][M+1]= 6.121690271852035e-01	SN= 4 EPS= 4.0E-13 K=17 M= 3 P[K-1][M]= 6.121690271852299e-01 P[K][M] = 6.121690271852157e-01 P[K][M+1]= 6.121690271852129e-01
SN= 2 EPS= 4.0E-13 K=14 M= 2 P[K-1][M]= 6.121690271854635e-01 P[K][M] = 6.121690271852508e-01 P[K][M+1]= 6.121690271852159e-01	SN= 5 EPS= 4.0E-13 K=23 M= 3 P[K-1][M]= 6.121690271852582e-01 P[K][M] = 6.121690271851674e-01 P[K][M+1]= 6.121690271851371e-01

读者不难利用程序 13.12C 通过不同的文件包含,通过不同的精度 EPS 和不同的步长 H 的设置而实现,用不同的方法求解不同的问题。总的说来,如果步长比较大,则用 n_{k+1}/n_k 的比值较大的序列更好一些。在许多情况下,使用外推法甚至能得到具有 15 位有效数字的结果,比如令 $EPS=0.4E-14$, $H=1.5$ 求解例 1.1,利用序列 N1,N2,N3 都能得到这样的结果。

程序 13.12C

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO3.C"
#include "ODE_GRAG.C"
#include "ODE_EXTR.C"
main()
{ int k,OK,SN=4;
  double EPS = 0.4E-12,H=4.000;
  FormSequence(SN);
  OK=Extrap(x0,y0,H,EPS);
  printf("\n    SN=%2d EPS=%8.2E",SN,EPS);
  printf("\n    K= %2d M= %2d",RK,RM);
  printf("\n    P[K-1][M]= %24.16e",P[RK-1][RM]);
  printf("\n    P[K][M] = %24.16e",P[RK][RM]);
  if(RM<MM) printf("\n    P[K][M+1]= %24.16e",P[RK][RM+1]);
}

```

```

    else      printf("\n P[K][M-1]=%24.16e",P[RK][RM-1]);
    if(!OK) printf("\n Bad Problem !");
    getchar();
}

```

13.7.6 积分表的制作方法

对于第 12 章讨论的求某个连续函数 $y=f(x)$ 在区间 $[a,b]$ 上的数值积分问题,我们也可以简单地把它处理为下面的初值问题

$$\left. \begin{aligned} y'(x) &= f(x), x \in [a, b] \\ y(a) &= 0 \end{aligned} \right\} \quad (7.3)$$

从而可以利用步进的方式求解,所形成的列表显示的结果即为相应的积分表。

虽然可以利用上一节介绍的自适应步长方法求解上面的(7.3)式给出的初值问题,但所得到的结果使用起来不太方便,理想的方式还是按固定步长进行计算。为了使得按固定步长计算也能保证所需要的精度,可以按大步长的方法求解每一步的问题,从而也得到一个完整的方法。这也是我们在第 12 章最后一节中所建议的,把一个大区间上的定积分问题划分成若干小区间上的定积分问题,在小区间上利用外推加速方法求定积分还可以进一步减少计算量。下面以正态分布的概率积分来说明具体操作方法。

标准正态分布的概率密度函数为

$$\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, x \in (-\infty, +\infty)$$

相应的概率分布函数为

$$\Phi(x) = 0.5 + \int_0^x \varphi(t) dt, x \geq 0$$

由此不难完成相应的数学问题模块的编写。相应的源代码见程序 13.13A,约定保存源程序的磁盘文件名为“ODE_NO4.C”。

程序 13.13A

```

#include <math.h>
#include <stdlib.h>
const double PI=3.14159265358979321;
double x0=0.0,y0=0.5;
double a=0.0, b=1.0;
double eps=0.4e-8;
double f(double x,double y)
{ return exp(-x*x/2.0)/sqrt(2.0*PI);
}

```

为简便计,我们只计算 $\Phi(x)$ 在 $x=0,0.1,0.2,\dots,1.0$ 处的函数值列表,完整的源代码见程序 13.13B。由于程序本身很简单,所以无需解释。程序运行的结果如表 7.4 所示。应用外推方法求解这种小问题当然是轻而易举的事,由程序中所设置的 EPS 的值可知,图中所显示的计算结果实际上都有 11 位有效数字。

表 7.4 标准正态分布的分布函数 Y 和密度函数 F 的函数值列表

k	x[k]	Y[k]	F[k]	H[k]
0	0.0000000000E+00	5.0000000000E-01	3.9894228040E-01	0.0000000000E+00
1	1.0000000000E-01	5.3982783728E-01	3.9695254748E-01	1.0000000000E-01
2	2.0000000000E-01	5.7925970944E-01	3.9104269398E-01	1.0000000000E-01
3	3.0000000000E-01	6.1791142219E-01	3.8138781546E-01	1.0000000000E-01
4	4.0000000000E-01	6.5542174161E-01	3.6827014030E-01	1.0000000000E-01
5	5.0000000000E-01	6.9146246127E-01	3.5206532676E-01	1.0000000000E-01
6	6.0000000000E-01	7.2574688225E-01	3.3322460289E-01	1.0000000000E-01
7	7.0000000000E-01	7.5803634778E-01	3.1225393337E-01	1.0000000000E-01
8	8.0000000000E-01	7.8814460147E-01	2.8969155276E-01	1.0000000000E-01
9	9.0000000000E-01	8.1593987465E-01	2.6608524990E-01	1.0000000000E-01
10	1.0000000000E+00	8.4134474607E-01	2.4197072452E-01	1.0000000000E-01

程序 13.13B 利用外推方法制作概率积分表的源程序

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO4.C"
#include "ODE_TAB.C"
#include "ODE_EXTR.C"
main()
{ int k,n=10,OK=1;
  double EPS=0.4E-12,H=1.0;
  double x,y,z,h;
  h=H/n;
  x=x0;y=y0;z=f(x,y);
  WriteTable(x,y,z);
  FormSequence(4);
  for(k=1;k<=n;k++)
  { OK=Extrap(x,y,h,EPS);
    if(!OK){printf("Bad Problem !");break;}
    y=P[RK][RM];
    x+=h;z=f(x,y);
    WriteTable(x,y,z);
  }
  if(OK) ShowTable();
  getchar();
  return;
}

```

13.8 刚性方程与隐式外推方法

13.8.1 刚性方程与稳定性问题的提出

刚性方程是常微分方程初值问题中的一类特殊的问题,其基本特征是问题的解包含着性质完全不同的两个项:一个项迅速趋近于零,从而使积分曲线发生突变;另一个项则变化缓慢,使积分曲线趋于平滑。如果我们把自变量 x 理解为时间,那么使积分曲线产生突变的成分起作用的时间是很短暂的,但是对于整个求解过程来说,它却一直支配着步

长的选择,从而导致求数值解的困难。下面通过一个例子来说明这一点。

例 8.1 求解下述初值问题^①

$$\left. \begin{aligned} y' &= 3\,000 - 1\,000y - 2\,000e^{-x}, x \geq 0 \\ y(0) &= 0 \end{aligned} \right\} \quad (8.1)$$

上面(8.1)式的理论解为

$$y(x) = 3 - \frac{997}{999}e^{-1\,000x} - \frac{2\,000}{999}e^{-x} \quad (8.2)$$

函数在区间 $[0, 0.01]$ 上的图像如图 8.1 所示。

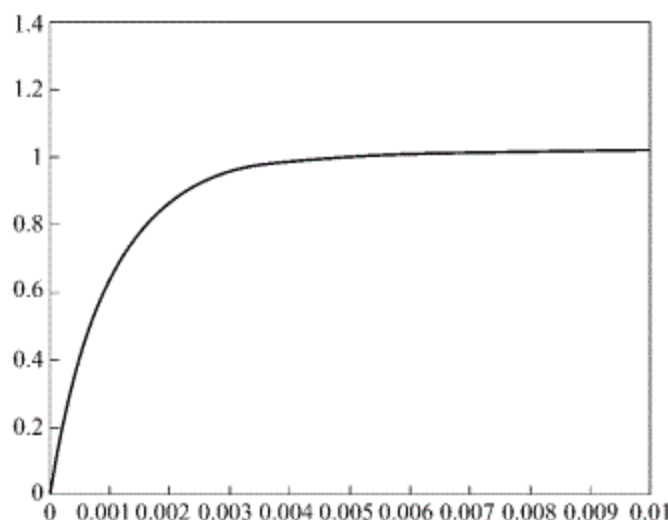


图 8.1 例 8.1 理论解的图像表示

由图 8.1 不难看出,在区间 $[0, 0.002]$ 上, $y(x)$ 的值随 x 增大而急剧上升;在区间 $[0.002, 0.004]$ 上, $y(x)$ 的上升速度相对说来明显地变得缓慢;当 $x > 0.004$ 时, $y(x)$ 的上升速度则非常缓慢。究其原因,主要是 $y(x)$ 中的含有 $e^{-1\,000x}$ 的项所致。

从曲线的形状看,似乎可以采用 13.6 节介绍的自适应步长方法求解,然而实际情况却完全不是这样的。我们可以把例 8.1 与例 6.1 给出的微分方程写成一般的线性微分方程 $y' + py = q(x)$ 的形式,在这里假定 p 为常数。不难看出,对于例 6.1 来说,使得 $y(x)$ 发生急剧变化的原因是由 $q(x)$ 项所致;而对于例 8.1 来说,使得 $y(x)$ 发生急剧变化的原因是由 py 项所致。如果进一步追根溯源,则是例 6.1 中的 $f(x, y)$ 所确定的李普希兹常数为 0.6,而例 8.1 的相应的李普希兹常数为 1 000。两者间的巨大差别引出了数值解方法的稳定性问题。

为简便计,设想求最简单的齐次线性常微分方程 $y' = -cy$ 的数值解,其中 c 为正的常数。假如初始条件为 $y(0) = 1$,则问题的精确解为 $y(x) = e^{-cx}$ 。显然,当 x 趋于无穷大时 $y(x)$ 趋近于零。假如采用单步欧拉方法求解,取固定步长 h 按步进方式计算,则有

$$y_k = y_{k-1} - chy_{k-1} \quad (8.3)$$

^① 本例题选自文献[12]第 719 页,但原文给出的理论解中的两个重要常数只精确到小数点后 3 位,难以应用于评价不同方法所得到的数值解的精度。

从而有

$$y_k = (1 - ch)y_{k-1} \quad (8.4)$$

由于 $1 - ch$ 是一个常数, 所以由 (8.4) 式所得到的序列 $\{y_k\}$ 是一个等比序列。由此立即得出, 如果 $|1 - ch| > 1$, 则序列 $\{y_k\}$ 无条件发散 (意指与初始条件无关)。即使 $|1 - ch| = 1$ 时序列 $\{y_k\}$ 收敛, 也不能收敛到零。所以, 把 $|1 - ch| < 1$ 称为稳定性条件是合理的, 由此可以解出 $h < 2/c$, 此时序列 $\{y_k\}$ 无条件收敛到零。

一般地, 如果初值问题的理论解 $y(x)$ 当 x 趋于无穷大时的极限 (记为 $y(\infty)$) 存在的话, 则称所讨论的问题具有稳态解, 也称相应的系统是稳定的。稳定性几乎是所有人工设计的系统的共同特性。比如人造地球卫星升空后再经过一个短暂的过渡阶段即可进入预定轨道 (其实总是带有一定的误差), 也就是达到稳定状态, 从此卫星将按确定的椭圆轨道运行, 椭圆的长轴和短轴从而都是固定的常数。对于一个可以趋近于稳定状态的系统来说, 如果按某个方法求数值解, 我们当然要求所得到的序列 $\{y_k\}$ 当 k 趋于无穷大时也能保证收敛到 $y(\infty)$, 此时称相应的方法是稳定的。

假如采用单步欧拉方法求例 8.1 稳态解的近似值, 我们有 $c = 1\,000$, 从而由稳定性条件得出 $h < 2/1\,000$ 。由图 8.1 不难看出, 当 $x < 0.004$ 时, 采用 $h < 2/1\,000$ 还可以接受; 但是当 $x > 0.004$ 时如果还采用步长 $h < 2/1\,000$ 进行计算, 似乎突破了人们的心理防线, 感觉难以接受, 然而稳定性的要求似乎只能这样做, 这就是刚性问题的由来。

现在进一步考虑一般形式的常系数线性微分方程组

$$y'(x) = Ay + q(x) \quad (8.5)$$

的刚性问题。这里 y, q 都是 m 维向量值函数, A 是 $m \times m$ 矩阵。记 $\lambda_1, \lambda_2, \dots, \lambda_m$ 为 A 的特征值, 记 $\operatorname{Re}(\lambda)$ 为复数 λ 的实部, 当所有的 $\operatorname{Re}(\lambda_j), j = 1, 2, \dots, m$ 都小于零时, 如果还是利用单步欧拉方法求解, 相应的稳定性条件为

$$h < \frac{2}{\max\{|\operatorname{Re}(\lambda_j)|, j = 1, 2, \dots, m\}} \quad (8.6)$$

记

$$s = \frac{\max\{|\lambda_j|, j = 1, \dots, m\}}{\min\{|\lambda_j|, j = 1, \dots, m\}} \quad (8.7)$$

如果 h 充分小而 $s \gg 1$, 则称上面的 (8.5) 式为刚性方程或刚性系统, 称 s 为刚性比。

对于刚性比充分大的刚性线性方程组来说, 如果采用单步欧拉方法求稳态解 (的近似值), 同样面临着用小步长进行长途跋涉的艰辛。

对于一般的非线性或非常系数线性常微分方程组 $y'(x) = f(x, y)$ 来说, 可以利用 $f(x, y)$ 在 (x_0, y^0) 处的雅可比矩阵替代 (8.5) 式中的 A 而得到所讨论的初值问题在 (x_0, y^0) 附近的刚性比。

13.8.2 隐式方法的重要意义

对于求解刚性方程或刚性方程组来说, 采用隐式方法或近似的隐式方法具有重要意义。仍以求解常系数齐次线性微分方程 $y' = -cy$ 为例, 如果采用隐式欧拉方法求解, 则相应的递推格式为

$$y_k = y_{k-1} - ch y_k \quad (8.8)$$

从而有

$$y_k = y_{k-1} / (1 + ch) \quad (8.9)$$

显然,由上面的(8.9)式所产生的序列 $\{y_k\}$ 还是等比序列,但是只要 $c > 0$,无论对怎样大的 $h > 0$,序列的公比 $(1 + ch)^{-1}$ 总是小于1的正数,从而得到序列 $\{y_k\}$ 无条件收敛到零,这正是我们所期待的结果。

对于求解常系数齐次线性常微分方程组 $y' = -cy$ 来说,记 $\lambda_1, \lambda_2, \dots, \lambda_m$ 为 c 的特征值,不难证明,对任意 $h > 0$, $(1 + h\lambda_j)^{-1}, j = 1, 2, \dots, m$ 为矩阵 $(I + hc)^{-1}$ 的特征值。如果 $y' = -cy$ 是一个刚性问题,则对所有 $j = 1, 2, \dots, m$,有 $\operatorname{Re}(\lambda_j) > 0$,从而得到

$$0 < \operatorname{Re}[(1 + h\lambda_j)^{-1}] < 1$$

如果还是利用隐式欧拉方法求解,类似地有

$$y_k = y_{k-1} - hc y_k$$

亦即

$$y_k = (I + hc)^{-1} y_{k-1} \quad (8.10)$$

再由 $(I + hc)^{-1}$ 的特征值的性质不难得出向量序列 $\{y_k\}$ 无条件收敛于零向量。

对于一般的非线性或非齐次线性常微分方程(组)所形成的刚性问题来说,很难得到像(8.9)式和(8.10)式那样的由隐式方法所形成的递推格式,但总可以通过求解相应的非线性方程(组)的数值解得到与(8.9)式和(8.10)式等效的递推格式。

正是由于隐式方法对于求解刚性方程来说其稳定性与步长无关,这相当于解除了我们的后顾之忧,所以求解刚性方程的方法都是由隐式方法派生而来,基本思路是寻找高阶的隐式方法、寻找求隐式递推格式的近似解的简便方法。目前这方面的研究可以说是兵分三路,它们是^①:

(1) Rosenbrock 给出的隐式龙格-库塔法,又由于是 Kaps 和 Rentrop 首先实现了 Rosenbrock 的方法,所以也称为是 Kaps-Rentrop 方法。

(2) Bader 和 Deuflhard 模仿 Bulirsh 和 Stoer 的外推方法而形成的所谓半隐式外推方法,基本操作方法是模仿 Gragg 的修正的中点公式法而得到一种隐式中点公式;采用线性近似的方法求相应的近似解;模仿 Bulirsh 和 Stoer 也构造一个更有效的整数序列外推加速,从而形成一种完整的方法。

(3) 预报-校正方法,它们大多由 Gear 的后退差分方法发展而来。

虽然这三种类型的方法都获得了成功,但是,结合刚性问题的特点以及现代计算机技术发展的趋势,更具鲁棒性(Robust,也称健壮性或适应性)的方法将更受欢迎。注意到外推方法一方面蕴含着对步长的自动化选择,另外也蕴含着对不同阶数的方法的自动化选择,所以我们更看好隐式外推方法。在接下来的两小节中,我们将结合求单个的刚性常微分方程的数值解具体介绍 Bader 和 Deuflhard 的工作以及我们在这个领域中的扩展研究。当然,在特殊情况下采用其他方法也是合理的选择。

^① 详细介绍请参阅文献[14]第741页以及第750页中列出的参考文献。

13.8.3 Bader 和 Deuflhard 的半隐式外推方法

Bader 和 Deuflhard 的半隐式外推方法采用的递推格式为

$$y_{k+1} = y_{k-1} + 2hf\left(x_k, \frac{y_{k-1} + y_{k+1}}{2}\right), k = 1, 2, \dots, n-1 \quad (8.11)$$

其中 n 是预先指定的步数。与中点公式类似,上面的(8.11)式也不能自行启动,需要另外计算 y_1 ,采用的方法自然是隐式欧拉方法,即

$$y_1 = y_0 + hf(x_1, y_1) \quad (8.12)$$

从而得到一个完整的递推格式。

为了能够利用上面的(8.12)式和(8.11)式如同显式方法那样计算 $y_1, y_2, \dots, y_n$,可以先把它们各自分别在 y_0 和 y_k 处线性化,得

$$\left. \begin{aligned} y_0 + \Delta_0 &= y_0 + hf(x_1, y_0) + h \frac{\partial f(x_1, y_0)}{\partial y_1} \Delta_0 \\ y_k + \Delta_k &= y_{k-1} + 2hf(x_k, y_k) + 2h \frac{\partial f(x_k, y_k)}{\partial y} \cdot \frac{\Delta_k - \Delta_{k-1}}{2} \\ k &= 1, 2, \dots, n-1 \end{aligned} \right\} \quad (8.13)$$

其中 $\Delta_k = y_{k+1} - y_k, k=0, 1, \dots, n-1$ 。

利用上面的(8.13)式不难进一步得到

$$\left. \begin{aligned} \Delta_0 &= \left[1 - h \frac{\partial f(x_1, y_0)}{\partial y}\right]^{-1} \cdot hf(x_1, y_0) \\ y_1 &= y_0 + \Delta_0 \end{aligned} \right\} \quad (8.14)$$

以及

$$\left. \begin{aligned} \Delta_k &= \Delta_{k-1} + 2\left[1 - h \frac{\partial f(x_k, y_k)}{\partial y}\right]^{-1} \cdot [hf(x_k, y_k) - \Delta_{k-1}] \\ y_{k+1} &= y_k + \Delta_k \\ k &= 1, 2, \dots, n-1 \end{aligned} \right\} \quad (8.15)$$

为了修匀最后一步的结果(记为 y),也是模仿 Gragg 的处理修正的中点公式的方法,把 y 表示为下述问题的解:

$$y = y_{n-1} + hf(x_n, y) \quad (8.16)$$

并且利用(8.15)式所得到的 y_n 作为初值利用牛顿法对上面的(8.16)式迭代一次,从而得到

$$y = y_n + \left[1 - h \frac{\partial f(x_n, y_n)}{\partial y}\right]^{-1} \cdot [hf(x_n, y_n) - \Delta_{n-1}] \quad (8.17)$$

Bader 和 Deuflhard 的半隐式外推方法采用的整数序列是

$$\{n_k\} = \{2, 6, 10, 14, 22, 34, 50, 70, 98, 138, \dots\} \quad (8.18)$$

其构造规则为:当 $k > 1$ 时, n_k 是由 n_{k-1} 逐次加 4,直到与 n_{k-1} 之比不小于 1.4 而得到。

由于已经有了框架形式的外推方法的模块程序,为了进一步得到具体的方法,可以利用(8.14)式、(8.15)式以及(8.17)式编写相应的方法函数 $T(x_0, y_0, H, n)$,其意义是采用某种 n 步方法计算初值问题的解 $y(x_0 + H)$ 的近似值。当 n 依次取自某个给定的整数序

列时,我们便得到相应的 $y(x_0+H)$ 的近似值序列。完整的源代码见程序 13.14A。程序中调用的函数 `double fy(double x,double y)` 专门用于计算 $f(x,y)$ 关于 y 的偏导数,它可以和计算 $f(x,y)$ 的同名 C 语言函数放在同一个数学问题模块中编写。程序最后一行定义的文件包含实现了与外推框架程序的有机连接,这仅仅是一种简单的编程技巧。为了便于和后面的程序对比分析,约定保持源程序的磁盘文件名为“STIFF_T11.C”。

程序 13.14A Bader 和 Deuflhard 的半隐式外推方法

```
/* File ame :STIF_T11.C */
double T(double x0,double y0,double H,long n)
{ long K;
  double h,x,y,delt0,delt1;
  h=H/n;
  x=x0+h;y=y0;
  delt0=h*f(x,y);
  delt0/=1-h*fy(x,y);
  y+=delt0;
  for(K=1;K<n;K++)
  { delt1=h*f(x,y)-delt0;
    delt1/=1-h*fy(x,y);
    delt1=delt0+2.0*delt1;
    y+=delt1; delt0=delt1;
    x+=h;
  }
  delt1=h*f(x,y)-delt0;
  delt1/=1-h*fy(x,y);
  y+=delt1;
  return y;
}
#include "ODE_EXTR.C"
```

Bader 和 Deuflhard 的半隐式外推方法的实际效果还是很不错的。但是,从递推格式到求相应的非线性方程的近似解方法都与现成的主流方法相距甚远从而限制了这种方法的进一步发展;太多的模仿也使得这个方法缺乏应有的理论基础;(8.18)式给出的序列的最大优点是大体上相当于一个公比为 1.4 的等比序列,除此之外其合理性的理由并不充分。所以在这个领域里面还存在着进一步拓展的空间。

13.8.4 直接利用隐式梯形序列外推

Bader 和 Deuflhard 的半隐式外推方法由于太多的具有探索意义的创新而使得我们应该考虑一下这样的问题:如果采用比较常规的方法求刚性问题的数值解,比如利用隐式梯形方法采用不同步长也可得到 $y(x_0+H)$ 的近似值序列,也可以利用牛顿法求相应的非线性方程的更为精确的数值解,这样一来情形又会是怎么样子呢? 如果情形更好一些或者大体相同,那么就没有必要让我们的方法过于偏离正常的轨道;如果情形非常糟糕,

那么相关的理论基础的坚实性似乎受到了挑战。所以有必要在这方面作进一步的探讨。

如果简单地利用隐式梯形方法求 y_k 的数值解,相应的递推格式为

$$y_k = y_{k-1} + h[f(x_{k-1}, y_{k-1}) + f(x_k, y_k)]/2 \quad (8.19)$$

由此所确定的 y_k 可以看成是下面关于 y 的(非线性)函数

$$F(y) = y_{k-1} + h[f(x_{k-1}, y_{k-1}) + f(x_k, y)]/2 - y \quad (8.20)$$

在 $y=y_{k-1}$ 附近的零点。虽然上面(8.20)式定义的 $F(y)$ 还与 k 以及 x_{k-1}, x_k, y_{k-1} 有关,但这个问题可以在求数值解时予以考虑,对于理论分析来说则无需顾及。

如果我们直接利用牛顿法求 $F(y)$ 在 $y=y_{k-1}$ 附近的零点 z ,当然可以利用 y_{k-1} 作为 z 的初值 z_0 ,利用

$$\left. \begin{aligned} F(z_i) &= h[f(x_{k-1}, y_{k-1}) + f(x_k, z_i)]/2 + y_{k-1} - z_i \\ F'(z_i) &= -\left[2 - h \frac{\partial f(x_k, z_i)}{\partial y}\right]/2 \end{aligned} \right\} \quad (8.21)$$

从而得到递推格式

$$\left. \begin{aligned} z_0 &= y_{k-1} \\ \Delta_{i-1} &= -F(z_{i-1})/F'(z_{i-1}) \\ z_i &= z_{i-1} + \Delta_{i-1} \\ i &= 1, 2, \dots \end{aligned} \right\} \quad (8.22)$$

对于刚性问题来说, $f(x, y)$ 关于 y 的偏导数的绝对值通常特别大,由此可以得出 $|F'(y)|$ 的值也特别大,再加之初始解 y_{k-1} 已经很靠近 $F(y)$ 的零点,所以采用牛顿法求 $F(y)$ 在 $y=y_{k-1}$ 附近的零点实现了问题与方法之间的完美结合。事实上,在一般情况下利用(8.22)式迭代一次即可得到满意的结果(只要保证算法的稳定性即可,利用外推方法来消除整体误差效果当然更好一些)。考虑到 $f(x, y)$ 的凸凹性可能会对迭代格式(8.22)产生一定的影响,所以保险起见,也可以利用(8.22)式迭代两次,至少可以增强算法的稳定性。

利用隐式梯形序列外推加速的 C 语言源代码见程序 13.14B 和程序 13.14C。它们的差别仅在于利用(8.22)式迭代一次和迭代两次。为应用的方便,约定保存源程序的磁盘文件名分别为“STIF_T21.C”和“STIF_T22.C”。与程序 13.14A 类似,程序中调用的函数 `double fy(double x, double y)` 专门用于计算 $f(x, y)$ 关于 y 的偏导数,程序最后一行定义的文件包含简化了与外推框架程序的有机连接。

程序 13.14B

```
double T(double x0, double y0, double H, long n)
{ long K;
  double h, x, y, z0, z1, delt;
  h = H/n;
  x = x0; y = y0;
  for(K=1; K<=n; K++)
  { z0 = f(x, y);
    x += h;
    z1 = f(x, y);
    delt = h * (z0 + z1) / 2.0;
```

```

        delt/=1.0-h*fy(x,y)/2.0;
        y+=delt;
    }
    return y;
}
#include "ODE_EXTR.C"

```

程序 13.14C

```

double T(double x0,double y0,double H,long n)
{ long K;
  double h,x1,y1,z0,z1,delt;
  h=H/n;
  for(K=1;K<=n;K++)
  { x1=x0+h;
    z0=f(x0,y0);
    z1=f(x1,y0);
    delt = h*(z0+z1)/2.0;
    delt /= 1.0-h*fy(x1,y0)/2.0;
    y1 = y0+delt;
    z1=f(x1,y1);
    delt = h*(z0+z1)/2.0+y0-y1;
    delt /= 1.0-h*fy(x1,y1)/2.0;
    y0=y1+delt;x0=x1;
  }
  return y0;
}
#include "ODE_EXTR.C"

```

程序 13.14B 的循环体内有 3 处函数 $f()$ 或 $fy()$ 的调用,程序 13.14C 的循环体内则是 5 次,所以对于相同的循环次数 n 来说,程序 13.14C 的计算量比程序 13.14B 大约多出 66%。注意到程序 13.14A 的循环体内只有两处函数 $f()$ 或 $fy()$ 的调用,所以对于相同的步数 n 来说,程序 13.14B 和程序 13.14C 的计算量分别是程序 13.14A 的 1.5 倍和 2.5 倍。这也许是 Bader 和 Deuflhard 的半隐式外推方法所独有的优势。

13.8.5 计算举例与程序性能的简单测试

为了利用已经编写好的程序求解例 8.1,可以模仿程序 13.08A、13.11B 完成相应的数学问题模块程序的编写,由于利用隐式外推方法求解,所以还需要编写计算 $f(x,y)$ 相对于 y 的偏导的 C 语言函数,函数原型可以说明为

```
double fy(double x,double y){...}
```

完整的源代码见程序 13.14D。约定保存源程序的文件名为“ODE_NO5.C”。

程序 13.14D

```
#define ForErrTest 1
```

```

#include <math.h>
#include <stdlib.h>
double x0=0.0,y0=0.0;
double a=0.0, b=1.0;
double eps=0.4e-8;
double f(double x,double y)
{ return 3000.0-1000.0*y-2000.0*exp(-x);
}
double fy(double x,double y){return -1000.0;}
double g(double x)
{ return 3.00-(997.0*exp(-1000.0*x)+2000.0*exp(-x))/999;
}

```

由于例 8.1 是一个线性常微分方程,不利于评价一些不同的方法之间的性能差别,我们再举一个非线性的刚性方程的例子。

例 8.2 求解下述初值问题在 x_0+H 处的解 $y(x_0+H)$ 的近似值。

$$\left. \begin{aligned} y' &= 3\,000 - 1\,000y - 0.5xy^2 - 2\,000e^{-x}, x \geq 0 \\ y(0) &= 0 \end{aligned} \right\}$$

读者可以模仿程序 13.14E 编写求解例 8.2 的数学问题模块程序,约定相应的源程序文件名为“ODE_NO6.C”。

用不同的隐式外推方法求解例 8.1 或例 8.2 的主控模块程序的源代码见程序 13.14E,它可以由程序 13.12 修改而来。为节省篇幅,我们略去了相关的显示语句,读者可以自己补充完整,也可以根据自己的喜爱来编写。

在程序 13.14E 中,注释为“Note 1”的行可以改写为

```
#include "ODE_NO6.C"
```

相当于求解例 8.1;注释为“Note 2”的行所包含的文件名可以改写为“STIF_T21.C”或“STIF_T22.C”则相当于利用程序 13.14B 或 13.14C 求解所选择的例题;注释为“Note 3”的行中所说明的变量 SN 的取值可以是 0、1、2、3、4、5、6,当 SN 取 6 时相当于利用 (8.18) 式给出的整数序列外推;在注释为“Note 4”的行中可以修改变量 EPS 和 H 的初值,以设定不同的精度和求解区间。

程序 13.14E

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "ODE_NO5.C" /* Note 1 */
#include "STIF_T21.C" /* Note 2 */
main()
{ int k,OK,SN=6; /* Note 3 */
  double EPS = 0.4E-12,H=0.05; /* Note 4 */
  FormSequence(SN);
  OK=Extrap(x0,y0,H,EPS);
  :
}

```

令序列号 SN 分别取 6 和 3, EPS 分别取 $0.4\text{E}-12$ 和 $0.4\text{E}-14$, 利用程序 13.14A 求解例 6.1 的计算结果如表 8.1 所示; 利用程序 13.14B 求解例 6.1 的计算结果如表 8.2 所示。

表 8.1 利用程序 13.14A 按不同序列不同精度求解例 8.1 的计算结果

SN= 6 H= 0.05 EPS= 4.0E-13 K = 8 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786357e+00 True Aans= 1.095636787786358e+00	SN= 3 H= 0.05 EPS= 4.0E-13 K = 9 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786358e+00 True Aans= 1.095636787786358e+00
SN= 6 H= 0.05 EPS= 4.0E-15 K = 8 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786357e+00 True Aans= 1.095636787786358e+00	SN= 3 H= 0.05 EPS= 4.0E-15 K = 9 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786358e+00 True Aans= 1.095636787786358e+00

表 8.2 利用程序 13.14B 按不同序列不同精度求解例 8.1 的计算结果

SN= 6 H= 0.05 EPS= 4.0E-13 K = 6 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786359e+00 P[K][M+1]= 1.095636787786359e+00 True Aans= 1.095636787786358e+00	SN= 3 H= 0.05 EPS= 4.0E-13 K = 7 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786358e+00 True Aans= 1.095636787786358e+00
SN= 6 H= 0.05 EPS= 4.0E-15 K = 6 M= 1 P[K-1][M]= 1.095636787786358e+00 P[K][M] = 1.095636787786359e+00 P[K][M+1]= 1.095636787786359e+00 True Aans= 1.095636787786358e+00	SN= 3 H= 0.05 EPS= 4.0E-15 K = 7 M= 1 P[K-1][M]= 1.095636787786359e+00 P[K][M] = 1.095636787786358e+00 P[K][M+1]= 1.095636787786358e+00 True Aans= 1.095636787786358e+00

表 8.1 和表 8.2 所显示的计算结果的确令人鼓舞: 在所有情况下, 都只用到比理论上保证稳定性所确定的最大步长稍小一些的步长而得到具有 15 位有效数字的精确结果

$$y(0.05) = 1.095\ 636\ 787\ 786\ 36$$

从所显示的结果还可以看出, 对于相同序列和相同精度来说, 程序 13.14B 明显优于程序 13.14A。如果我们再仔细分析一下不同的序列的差异, 当 SN 取 6 时, 有 $N6[6]=50$, $N6[8]=98$; 当 SN 取 3 时, 有 $N3[7]=44$, $N3[9]=88$ 。所以无论在何种情况下, 选用序列 N3 都比选用序列 N6 稍好。进一步检测的结果是, 程序 13.14A 与程序 13.14B 谁能更胜一筹, 似乎难分伯仲; 但在绝大多数情况下, 序列 N3 比序列 N6 的确胜出一筹, 当然也只能以微弱的优势胜出。

序列号 SN 分别取 0、3 和 5; 步长 H 分别取 0.1 和 0.5; 允许误差取 $0.4\text{E}-12$, 分别运行程序 13.14A、13.14B 和 13.14C 求解例 8.2, 程序所显示的结果如表 8.3~8.5 所示。

表 8.3 利用程序 13.14A 按不同序列不同步长求解例 8.2 的计算结果

SN= 0 H= 0.10 EPS= 4.0E-13 K = 7 M= 1 P[K-1][M]= 1.188443974614302e+00 P[K][M] = 1.188443974614303e+00 P[K][M+1]= 1.188443974614303e+00	SN= 0 H= 0.50 EPS= 4.0E-13 K = 8 M= 1 P[K-1][M]= 1.784930581013940e+00 P[K][M] = 1.784930581013937e+00 P[K][M+1]= 1.784930581013936e+00
SN= 3 H= 0.10 EPS= 4.0E-13 K =10 M= 1 P[K-1][M]= 1.188443974614302e+00 P[K][M] = 1.188443974614302e+00 P[K][M+1]= 1.188443974614302e+00	SN= 3 H= 0.50 EPS= 4.0E-13 K =12 M= 1 P[K-1][M]= 1.784930581013950e+00 P[K][M] = 1.784930581013936e+00 P[K][M+1]= 1.784930581013932e+00
SN= 5 H= 0.10 EPS= 4.0E-13 K =16 M= 1 P[K-1][M]= 1.188443974614304e+00 P[K][M] = 1.188443974614303e+00 P[K][M+1]= 1.188443974614303e+00	SN= 5 H= 0.50 EPS= 4.0E-13 K =21 M= 1 P[K-1][M]= 1.784930581013942e+00 P[K][M] = 1.784930581013950e+00 P[K][M+1]= 1.784930581013957e+00

仔细分析表 8.3~8.5 所显示的计算结果, 可以大体得出这样的结论: 对于给定的允许误差来说, 程序 13.14A、13.14B 以及 13.14C 都能得到具有相应的精度的结果, 计算两方面的差别虽然各有长短, 但总的来说差别不大。进一步的检验大体可以得出这样的结论: 在某种特定条件下, 如果三种方法出现了较大的差别, 那么程序 13.14B 和 13.14C 中总有一个比程序 13.14A 的性能有所超出或大体持平。

表 8.4 利用程序 13.14B 按不同序列不同步长求解例 8.2 的计算结果

SN= 0 H= 0.10 EPS= 4.0E-13 K = 7 M= 1 P[K-1][M]= 1.188443974614040e+00 P[K][M] = 1.188443974614270e+00 P[K][M+1]= 1.188443974614286e+00	SN= 0 H= 0.50 EPS= 4.0E-13 K = 9 M= 1 P[K-1][M]= 1.784930581013708e+00 P[K][M] = 1.784930581013908e+00 P[K][M+1]= 1.784930581013922e+00
SN= 3 H= 0.10 EPS= 4.0E-13 K = 9 M= 2 P[K-1][M]= 1.188443974613780e+00 P[K][M] = 1.188443974614112e+00 P[K][M+1]= 1.188443974614163e+00	SN= 3 H= 0.50 EPS= 4.0E-13 K =13 M= 2 P[K-1][M]= 1.784930581013477e+00 P[K][M] = 1.784930581013766e+00 P[K][M+1]= 1.784930581013810e+00
SN= 5 H= 0.10 EPS= 4.0E-13 K =14 M= 2 P[K-1][M]= 1.188443974613493e+00 P[K][M] = 1.188443974613831e+00 P[K][M+1]= 1.188443974613995e+00	SN= 5 H= 0.50 EPS= 4.0E-13 K =20 M= 4 P[K-1][M]= 1.784930581012948e+00 P[K][M] = 1.784930581013330e+00 P[K][M+1]= 1.784930581013402e+00

表 8.5 利用程序 13.14C 按不同序列不同步长求解例 8.2 的计算结果

SN= 0 H= 0.10 EPS= 4.0E-13 K = 6 M= 1 P[K-1][M]= 1.188443974614302e+00 P[K][M] = 1.188443974614302e+00 P[K][M+1]= 1.188443974614302e+00	SN= 0 H= 0.50 EPS= 4.0E-13 K = 7 M= 1 P[K-1][M]= 1.784930581013947e+00 P[K][M] = 1.784930581013937e+00 P[K][M+1]= 1.784930581013937e+00
SN= 3 H= 0.10 EPS= 4.0E-13 K = 8 M= 1 P[K-1][M]= 1.188443974614303e+00 P[K][M] = 1.188443974614302e+00 P[K][M+1]= 1.188443974614302e+00	SN= 3 H= 0.50 EPS= 4.0E-13 K =10 M= 1 P[K-1][M]= 1.784930581013959e+00 P[K][M] = 1.784930581013942e+00 P[K][M+1]= 1.784930581013936e+00
SN= 5 H= 0.10 EPS= 4.0E-13 K =12 M= 1 P[K-1][M]= 1.188443974614306e+00 P[K][M] = 1.188443974614303e+00 P[K][M+1]= 1.188443974614301e+00	SN= 5 H= 0.50 EPS= 4.0E-13 K =17 M= 1 P[K-1][M]= 1.784930581013962e+00 P[K][M] = 1.784930581013947e+00 P[K][M+1]= 1.784930581013934e+00

13.9 常微分方程数值解方法小结

对于求一般的常微分方程初值问题的数值解来说,在前面几节中,我们已经介绍了大量的方法。这些方法按不同的属性分类,可以划分为:高阶方法与低阶方法;单步方法与多步法;显式方法与隐式方法等。在特定条件下,这些方法几乎都可以与自适应步长方法或外推加速方法结合使用,从而构成求常微分方程数值解的方法体系。在实际应用中,我们当然希望能够结合具体问题的特点,充分利用不同方法的差异,选择一种更为合适的方法,力争得到尽可能好的结果。

一般说来,利用显式方法计算预报值,再利用隐式格式计算校正值无论是高阶方法还是低阶方法,总能获得比较好的数值稳定性能。所以对于求解实际问题来说,最好采用预报-校正系统来完成。即使需要更大的计算量,在现代计算机条件下,这个过去的瓶颈问题也正在逐步消失。

如果问题本身不适宜采用高阶方法求解,比如 $f(x, y)$ 的光滑性能不太好就属于这种情形,或者问题本身不需要太高精度的解,那么采用低阶的方法辅之以适当的步长是合适的。对于这样的问题,采用二阶龙格-库塔法或改进的欧拉方法已经足够。如果 $f(x, y)$ 的光滑性能比较好,那么采用二阶龙格-库塔法可以得到更精确的结果;否则应该采用改进的欧拉方法以获得更好的数值性能。

如果问题适宜于采用高阶方法求解,也希望得到高精度的结果,那么采用四阶龙格-库塔法计算表头,并利用阿达姆斯外推内插法求解应该是理想的方法。对于所有高阶方法来说,通过减小步长来提高结果的精度是比较务实的考虑。所以在一般情况下我们并不鼓励继续研究更高精度的方法,当然,对于一些特殊的问题还是可以例外的。

对于求解实际问题来说,我们通常并不能立即得出所得到的结果到底有几位有效数字。虽然可以通过理论分析来估计误差,但这样做一是劳神费力,二是所得到的结果也未

必靠得住,这中间不确定的因素太多。务实的方法是采用多种方法计算,采用不同阶的方法计算,采用不同的步长计算,这样通过多个结果对比分析,也可以确定结果的精度。在现代计算机条件下,采用这种基于试验的方法一般比理论分析的结果更为直观,更为具体。在这个基础上再辅之以理论分析,结论当然更可靠一些。

对于求解实际问题来说,如果有一种技巧能帮助我们分析一下到底采用哪种方法更好一些,那当然再好不过了。现在提出有这种“万能”的方法为时过早,不过随着认识的深入,完全可以在这个方面有所作为。如果拿到一个问题却不知道一下子该怎么办时,可以设计几个较大的步长,首先用显式的外推方法计算,如果成功,而且实际的步长也不是太小,说明不是刚性问题,可以转而用自适应步长方法得到大致的积分曲线,由此确定出几个关键的点,比如一阶导数为零的点,曲线与 x 轴的交点等。接下来再重新用外推方法求出这些点处的比较精确的函数值,从而把一个初值问题划分为多个初值问题,然后再用常规方法求解每个问题。如果用显式的外推方法计算失败,则相应的问题应该是刚性问题,此时可以利用隐式外推方法计算。虽然显式的和隐式的外推方法只适用于求解大步长问题,但我们可以模仿程序 13.13 制作积分表的方法把一个多步的问题看成多个一步问题,并用外推方法求解相应的一步问题,从而实现了用外推方法求解多步问题。在所有的方方法中外推方法的适应性最强,算法简单。所以两类外推方法犹如我们手中的两把利剑,几乎所有的问题都能迎刃而解。

我们不难把外推方法推广到求解微分方程组的问题,所以本章的方法不难应用于求解常微分方程组。如果方程个数比较多,恐怕还会遇到一些新问题,这也是正常的事,读者应该有勇气、有能力去解决它们。

练 习 题

1. 对下面给出的常微分方程初值问题,分别用改进的欧拉方法、二阶龙格-库塔法和阿达姆斯外推内插法(利用四阶龙格-库塔法计算表头),分别用步长 $h=0.08$ 、 $h=0.04$ 、 $h=0.02$ 和 $h=0.01$ 求数值解。假如要求结果有 3 位有效数字,试说明采用哪种方法以及何种步长较合适。假如要求结果有 6 位有效数字,又应该采用哪种方法以及何种步长较合适?

$$(1) \begin{cases} y' = yx^2 - 1.2y \\ y(0) = 1.0 \\ x \in [0, 0.8] \end{cases}$$

$$(2) \begin{cases} y' = (1+x)\sqrt{y} \\ y(0) = 1.0 \\ x \in [0, 0.8] \end{cases}$$

$$(3) \begin{cases} y' = e^{-x} - y \\ y(0) = 0 \\ x \in [0, 0.8] \end{cases}$$

$$(4) \begin{cases} y' = x - y + 1 \\ y(0) = 1 \\ x \in [0, 0.8] \end{cases}$$

2. 试利用经过 (x_{k-3}, y_{k-3}) , (x_{k-2}, y_{k-2}) , (x_{k-1}, y_{k-1}) 这三个点的拉格朗日插值多项式导出 $y(x_k)$ 的近似值 y_k 。

3. 模仿四阶阿达姆斯外推内插公式的推导方法,试推出三阶阿达姆斯外推内插公式,并利用所得到的方法求解例 1.1。

4. 试利用经过 $(x_{k-2}, y_{k-2}), (x_{k-3}, y_{k-3}), (x_{k-2}, y_{k-2}), (x_{k-1}, y_{k-1})$ 这 4 个点的拉格朗日插值多项式导出 $y(x_k)$ 的近似值 y_k , 并与四阶阿达姆斯内插公式构成一个预报-校正系统, 并利用它来求解例 1.1。

5. 利用四阶龙格-库塔法求解初值问题

$$\begin{cases} y' = 4e^{0.8x} - 0.5y \\ y(0) = 2 \\ x \in [0, 0.5] \end{cases}$$

要求通过对不同的步长的结果进行对比分析而找到具有 6 位有效数字的解。

6. 利用表 8.4~8.5 以及表 7.2, 分别评估当 $h=0.5$, $\text{EPS}=0.4\text{E}-12$, SN 取 0 和 3 时, 利用程序 13.14A、13.14B 和 13.14C 求解例 8.2 的大致计算量(只考虑 $f()$ 和 $f_y()$ 的调用次数, 其余一概不考虑)。

7. 设 A 是 $m \times m$ 的可逆矩阵, $\lambda_1, \lambda_2, \dots, \lambda_m$ 是 A 的 m 个不同的特征值, 又设 h 是大于零的实数。试证明, 如果初 $I - hA$ 可逆, 那么 $(1 - h\lambda_k)^{-1}, k = 1, 2, \dots, m$ 就是 $(I - hA)^{-1}$ 的特征值。

参考文献

1. 王能超. 数值分析简明教程. 武汉: 华中科技大学出版社, 2002
2. 北京大学数学力学系几何与代数教研室代数小组编. 高等代数. 北京: 人民教育出版社, 1978
3. 关治, 陈景良. 数值计算方法. 北京: 清华大学出版社, 1990
4. 张培强主编. MATLAB 语言. 合肥: 中国科学技术大学出版社, 1995
5. 严蔚敏, 吴伟民. 数据结构(C语言版). 北京: 清华大学出版社, 1997
6. 陈礼民, 江礼璋. C语言程序设计及其应用. 北京: 科学出版社, 1993
7. 崔国华主编. 计算方法. 武汉: 华中理工大学出版社, 1996
8. M 阿佛里尔. 非线性规划(上, 下册). 李元熹等译. 上海: 上海科学技术出版社, 1980
9. B L. 范德瓦尔登. 代数学(I). 丁石孙等译. 北京: 科学出版社, 1963
10. ГМ. 菲赫金哥尔茨. 微积分学教程第一卷第一分册. 叶彦谦等译. 北京: 人民教育出版社, 1959 年 8 月第 2 版
11. Г М. 菲赫金哥尔茨. 微积分学教程第二卷第二分册. 北京大学高等数学教研室译. 北京: 人民教育出版社, 1954
12. S C Chapra, R P Canale. Numerical Methods for Engineers(Third Edition)(影印). 北京: 科学出版社, 2000
13. C F Gerald, P O Wheatley. Applied Numerical Analysis(Third Edition). Addison-Wesley Publishing Company, 1984
14. W H Press. Numerical Recipes in C++, The Art of Scientific Computing(Second Edition)(影印). 北京: 电子工业出版社, 2003